Inventory & Point-of-Sale System for a Small Café

Final Project Report — Software Engineering 1

Student	*[Your full name]*
Student number / section	*[Student number] · [Section]*
Instructor	*[Instructor's name]*
Submitted	*[Date of submission]*
System version described	**v0.32.0** (25 October 2026); course submission version **v0.21.2** (23 October 2026)
Source code	https://github.com/JmlAsync/inventory-pos-system
Video presentation	*[Link to the recorded video]*

Contents

About this report 6
Tools and assistance 6
1. Introduction 8
1.1 The problem 8
1.2 What the system does 8
1.3 Objectives 8
1.4 Scope 9
1.5 Who uses it 9
1.6 Technology 10
1.7 Glossary 10
2. A tour of the system 12
3. Software process model (item 1) 22
3.1 The model I chose: incremental development 22
3.2 How each increment was built 22
3.3 Why incremental, and not the other two? 23
4. Feature stories (item 2) 24
F1 — Viewing the product list (v0.3.0) 24
F2 — Adding a new product (v0.4.0) 24
F3 — Editing a product (v0.5.0) 24
F4 — Deleting a product (v0.5.0) 24
F5 — Logging in and out (v0.6.0) 24
F6 — Admin and cashier permissions (v0.7.0) 24
F7 — Processing a sale (v0.8.0, fixed in v0.8.1) 25
F8 — Low-stock warning (v0.9.0, fixed in v0.9.1–v0.9.2) 25
F9 — Sales report (v0.10.0, fixed in v0.10.1–v0.10.2) 25
F10 — Profile picture (v0.12.0) 25
F11 — A modern look with light and dark themes (v0.13.0) 25
F12 — Account safety (v0.13.1–v0.13.5) 26
F13 — Product pictures and a tap-to-add menu (v0.14.0–v0.14.4) 26
F14 — Paying by GCash (v0.15.0) 26
F15 — Hot / Iced and add-ons (v0.16.0–v0.16.1, wording v0.17.2) 26
F16 — Ingredients and recipes (v0.17.0–v0.17.1) 26
F17 — Finding and updating an ingredient quickly (v0.18.0–v0.18.2) 26
F18 — Changing the system safely (v0.19.0) 27

F19 — Serving a queue quickly (v0.20.0–v0.20.1) 27
F20 — A calmer, roomier screen (v0.21.0–v0.21.2) 27
F21 — The shop's own receipt (v0.22.0) 27
F22 — Setting up the café's own copy (v0.24.0) 27
F23 — Making sure the café's email works (v0.26.0) 28
F24 — Everyone with their own account (v0.27.0) 28
F25 — A code that keeps strangers out (v0.28.0) 28
F26 — One tap with Google (v0.29.0) 28
F27 — Taking it back (v0.30.0) 29
F28 — Giving the money back (v0.31.0) 29
F29 — Help on the page (v0.32.0) 29
5. Test cases (item 3) 30
5.1 How the system was tested 30
5.2 F1 — View product list 30
5.3 F2 — Add product 31
5.4 F3 — Edit product 32
5.5 F4 — Delete product 32
5.6 F5 — Log in / log out 33
5.7 F6 — Role-based access (Admin vs. Cashier) 34
5.8 F7 — Process sale (v0.8.0, v0.8.1) 34
5.9 Patches v0.8.2–v0.8.6 — robustness of the sale flow and forms 38
5.10 F8 — Low-stock warning (v0.9.0, v0.9.1, v0.9.2) 41
5.11 F9 — Sales report (v0.10.0, v0.10.1, v0.10.2) 42
5.12 Demo data (v0.11.0, v0.11.1, v0.11.2) 42
5.13 F10 — Profile picture (v0.12.0) 43
5.14 F11 — Modern theme (v0.13.0) 44
5.15 F12 — Security patches (v0.13.1–v0.13.5) 47
5.16 F13 — Product pictures and café menu (v0.14.0–v0.14.3) 49
5.17 F14 — Payment method (v0.15.0) 52
5.18 F15 — Sizes and add-ons (v0.16.0, v0.16.1) 54
5.19 F16 — Ingredients and recipes (v0.17.0, v0.17.1) 56
5.20 F17 — Ingredients page redesign (v0.17.4, v0.18.0, v0.18.1) 58
5.21 F18 — Project rules and automated tests (v0.19.0) 61
5.22 Security audit patches (v0.19.3) 62
5.23 Offline pages (v0.19.4) 63
5.24 F19 — New Sale redesign (v0.20.0) 64

5.25 F20 — Cozy layout and foldable sidebar (v0.21.0) 68
5.26 Last open defects closed (v0.20.2–v0.20.4) 69
5.27 Patches after the submission (v0.21.3–v0.21.8) 72
5.28 F21 — Shop details and receipt notice (v0.22.0) 75
5.29 Database migrations (v0.23.0) 76
5.30 Times in UTC (v0.23.1) 77
5.31 Money in centavos (v0.23.2) 78
5.32 F22 — Setting up and running a shop's copy (v0.24.0) 79
5.33 Blueprints and the store/ layer (v0.25.0) 81
5.34 F23 — Email sending and the login limit in the database (v0.26.0) 81
5.35 F24 — The owner, users, invitations, forgotten passwords and the audit log (v0.27.0) 84
5.36 F25 — A code at every login (v0.28.0) 86
5.37 F26 — Sign in with Google (v0.29.0) 88
5.38 F27 — Undo and a safe count (v0.30.0) 90
5.39 F28 — Voids and refunds (v0.31.0) 91
5.40 F29 — Help on every page (v0.32.0) 93
5.41 Test summary 94
6. Requirements (items 4–10) 97
6.1 Requirements definition: user and system requirements (item 4, Lecture 4 p. 7) 97
6.2 Nonfunctional requirements (item 5, Lecture 4 p. 18) 99
6.3 Nonfunctional requirements metrics (item 6, Lecture 4 p. 21) 100
6.4 Requirements of each part (item 7, Lecture 4 p. 39) 101
6.5 Structured requirements (item 8, Lecture 4 p. 42–43) 103
6.6 Tabular computation (item 9, Lecture 4 p. 45) 105
6.7 Detailed scenarios (item 10, Lecture 4 p. 62–63) 107
7. System models (items 11–25) 109
7.1 Context model (item 12, Lecture 5 p. 10) 109
7.2 Process model (item 13, Lecture 5 p. 12) 109
7.3 Use case diagrams (items 11 and 14, Lecture 4 p. 65 and Lecture 5 p. 15) 110
7.4 Tabular use case descriptions (item 15, Lecture 5 p. 16) 119
7.5 Each actor's use cases (item 16, Lecture 5 p. 17) 126
7.6 Sequence diagrams (item 17, Lecture 5 p. 19–20) 129
7.7 Class associations and class model (items 18 and 19, Lecture 5 p. 23–25) 135
7.8 Generalization hierarchy (item 20, Lecture 5 p. 30–31) 137
7.9 Aggregation (item 21, Lecture 5 p. 33) 137
7.10 Activity model (item 22, Lecture 5 p. 36) 138

7.11 Application process (item 23, Lecture 5 p. 37) 140
7.12 State diagrams (item 24, Lecture 5 p. 40) 141
7.13 Structured forms of states (item 25, Lecture 5 p. 41–42) 143
8. Architecture and design (items 26–30) 146
8.1 Software architecture (item 26, Lecture 6 p. 42 and 49) 146
8.2 System context (item 27, Lecture 7 p. 9) 147
8.3 High-level architecture (item 28, Lecture 7 p. 13) 148
8.4 Object classes (item 29, Lecture 7 p. 19) 149
8.5 Detailed usage scenario (item 30, Lecture 8 p. 53) 150
9. Dependability and security (items 31–39) 151
9.1 Reliability terminology (item 31, Lecture 11 p. 25) 151
9.2 Safety terminology (item 32, Lecture 11 p. 35) 151
9.3 Security terminology (item 33, Lecture 11 p. 42) 152
9.4 Vulnerability avoidance techniques (item 34, Lecture 11 p. 45) 152
9.5 Risk classification (item 35, Lecture 12 p. 15) 154
9.6 Software fault tree (item 36, Lecture 12 p. 18) 155
9.7 Safety requirements (item 37, Lecture 12 p. 23) 155
9.8 Functional reliability requirements (item 38, Lecture 12 p. 39) 156
9.9 Threat and control analysis (item 39, Lecture 12 p. 46) 157
10. Defects, lessons learned and known limitations 159
10.1 Defects found and fixed 159
10.2 Lessons learned 166
10.3 Security audit and security review 167
10.4 Known limitations 169
11. Conclusion 171
Appendix A. Version history 172
Appendix B. Automated test suite 179
Appendix C. Where each Section B item is answered 182
Appendix D. Running the system 184

About this report

This report describes the Inventory & POS System from the first idea to **v0.32.0**, the version I show in the defense. It follows the order of the items in Section B of the final project instructions, so each item can be found quickly. The table in Appendix C lists every item and the section that answers it.

Two versions are named in this report. v0.21.2 (23 October 2026) is the version I submitted for the course. After that, the system was planned as a product for café owners (Chapter 1.1), and I kept building it in the same small increments up to **v0.32.0** (25 October 2026): an owner account and invitations, a code at every login, voids and refunds, help on every page, and the groundwork for a cloud version. Every chapter describes v0.32.0; where something changed after v0.21.2, the text says in which version.

Where the code is. `main` on GitHub holds v0.21.2. Each later version is its own pull request (#74 to #92), stacked on the one before it. Following my own decision DEC-018, they are merged into `main`, in order, after the defense, so that the version the course graded stays on `main` until then.

The chapters build on each other:

- **Chapters 1–2** explain what the system is and show it working.
- **Chapters 3–5** cover how I built it (the process model), what each feature does (stories) and how I tested it.
- **Chapters 6–9** are the engineering models: requirements, UML system models, architecture, and dependability and security.
- **Chapter 10** lists every defect I found and fixed, what I learned from them, and what the system still cannot do.
- **Chapter 11** closes the report. The appendices hold the full version history, the automated test suite and the instructions for running the system.

A few conventions are used throughout. Versions are written as `vX.Y.Z` and match the Git tags in the repository. Features are numbered **F1–F29**, use cases **UC-01–UC-25**, test cases **TC-…** and defects **DEF-01–DEF-71**, so any one of them can be traced from the requirement to the code to the test. Design decisions are numbered **DEC-001–DEC-034** and recorded in `docs/decisions.md`, each with the alternatives that were checked. In the test tables, ✅ means the test passed and ❌→✅ means it failed first, the defect was fixed, and it passed afterwards.

Tools and assistance

I built the first versions (v0.1.0 to v0.8.1) myself. From then on I decided what each version should do, tried each one in the running app, and found several of the defects in Chapter 10 by hand. Because of the time pressure of finals week, I used an AI coding assistant (Claude, by Anthropic) for a large part of the implementation from v0.8.2 onwards, for writing the automated tests and for drawing the UML diagrams from the code. From v0.21.3 onwards I set the plan and the rules for the client version, decided every question the rules left open (each one is recorded in `docs/decisions.md`), and tested the emailed login codes by hand (TC-38.15). Every change still went through the same process as my own code: its own branch, a pull request, the automated checks, my review, and a version tag. The project rules in `AGENTS.md` were written so that anyone changing the code, person or AI assistant, has to follow the same security and data

rules. Where a test or screenshot was produced with the assistant, the result is reported exactly as it came out.

1. Introduction

1.1 The problem

Small cafés in the Philippines often run on a notebook, a calculator and memory. The owner knows roughly how much milk is left, the cashier works out the change in their head, and at closing time someone adds up the day's sales by hand. This works until it doesn't: a drink is promised and the oat milk has run out, the drawer is ₱200 short and nobody can say which sale it was, or two people sell the last slice of cake at the same time.

I started this project as a simple inventory list for a general store. Midway through, a café owner showed interest in it, so from v0.14.0 the system was shaped around what a café actually needs: drinks sold Hot or Iced, add-ons such as oat milk or an extra shot, GCash payments, and ingredient stock that goes down with every cup sold.

On 24 October 2026, after the course submission, the next stage was decided: the system becomes a **product sold to café owners by subscription**. That changed what "finished" means. Every person needs their own account, a password alone is not enough for a system that holds a shop's money, and the counter needs what a real café needs: undo, voids and refunds with an admin's approval, and help for new staff. Versions v0.21.3 to v0.32.0 build exactly that. The full plan, with the order of work, is in `docs/upgrade_plan.md`.

1.2 What the system does

The **Inventory & POS System** is a web application that runs on the café's own laptop. Staff open it in a browser, and it works without an internet connection.

- **At the counter**, a cashier taps drinks and pastries into a basket, adjusts servings and add-ons, takes cash or GCash, and prints a receipt. The change is worked out automatically, and the system refuses to sell anything that is no longer in stock.
- **Behind the counter**, the owner (admin) keeps the menu, prices and pictures up to date, sets up servings and add-ons, records ingredients and recipes, adds deliveries, counts the shelves, and reads the sales report.
- **Throughout the day**, the system warns staff when a product or ingredient is running low, and keeps a history of every ingredient change so a missing amount can be explained.
- **When something goes wrong at the counter**, a cashier taps **Undo**, or asks for a **void** of today's receipt; an admin approves it with their own password, or with a code emailed to them when they are not in the shop. An admin can also **refund** chosen items of any receipt. The receipt itself never changes.
- **For everyone's safety**, the shop has one **owner** account, everyone else joins by **invitation** to their own email, and every login needs the password **and** a 6-digit code (unless that browser is trusted). The **audit log** records who changed which price, recipe, account or receipt.
- **For new staff**, every page has a **?** button with that page's help and a picture of it.

1.3 Objectives

1. Record every sale correctly, with the right total, change and receipt, and never sell more than is in stock.

2. Keep product and ingredient stock accurate without manual counting after each sale.
3. Give the owner a clear daily picture: revenue (cash and GCash), number of sales and best sellers.
4. Keep the data safe: only the right people can change prices or see reports, and passwords are never stored.
5. Be quick and easy at a busy counter, on a laptop or a phone, in a light or dark theme.
6. Give every person their own account, and keep strangers out even when a password is seen or guessed.
7. Let mistakes at the counter be undone and money be given back, without ever changing a receipt.

1.4 Scope

In scope: products, pictures and categories; servings and add-ons; ingredients, recipes and stock history; sales with cash or GCash; receipts with the shop's details; undo; voids and refunds with approval; low-stock warnings; a sales report with net sales; three roles (owner, admin, cashier); invitations; a code at every login, trusted devices, backup codes, an authenticator app and Sign in with Google; email sending; an audit log; help on every page; setting up a shop's own copy.

Out of scope (for now): online ordering, accounting, payroll, senior citizen and PWD discounts, shifts and X/Z readings (the next increments, v0.33.0 and v0.34.0), the cloud package, and anything required of an official BIR-registered cash register. Every receipt says **"This is not an official receipt."**: it is an order slip for the shop's own records, not an official sales invoice (see 10.4).

1.5 Who uses it

An **actor** is anyone outside the system who interacts with it.

Actor	Who they are	What they can do
Visitor	Anyone who opens the site without logging in	See the home page, go to the login page, switch the light or dark theme
User *(general)*	Any logged-in person. Owner, Admin and Cashier are all kinds of User	Log in (password and code) and out, see Home, view products, make sales, undo, open today's receipts, **ask for a void**, read the help, manage their own picture, password, trusted devices, backup codes and authenticator app
Cashier	Counter staff	Everything a User can do. Any admin page answers **403 Forbidden**, even if the address is typed in by hand
Admin	A manager	Everything a User can do, **plus** manage products, servings and add-ons, ingredients and recipes, restocks and counts; view the sales report; **approve or reject voids** and **refund** items; edit the shop details and email settings; read the audit log; invite cashiers if the owner allowed it
Owner	The café owner: the one account made with the subscription's email	Everything an Admin can do, **plus** change roles, make admins, let an admin invite, invite anyone and switch accounts off. The owner can't be switched off or demoted (rule S12)

A visitor can also **join** from an invitation link (choose a username and password, or continue with Google) and ask for a code to choose a new password (**Forgot your password?**). Two other systems take part without being people: the shop's **email server**, which carries the codes and invitations, and **Google**, which confirms a Google sign-in (Chapter 7.1).

Owner, Admin and Cashier are drawn as kinds of User because they share most of their abilities, and the Owner as a kind of Admin. This matches the code: there is one `User` table, and its `role` column (`owner`, `admin` or `cashier`) decides what each person may do; the database allows only one owner.

1.6 Technology

Part	Choice	Why
Language	Python 3.11+ (developed on 3.11, checked on 3.13)	Readable for a beginner, and the language of the course
Web framework	Flask 3.1 with Flask-Login and Flask-SQLAlchemy, in **blueprints** above a `store/` data layer (v0.25.0)	Small, well documented, easy to follow from request to page; the store layer lets the same pages run on another database later
Database	SQLite, one file (`instance/inventory.db`); managed PostgreSQL planned for the cloud package	Nothing to install; enough for one shop. SQLAlchemy speaks PostgreSQL too, so the cloud package is mainly a different address (DEC-014)
Database changes	Alembic through Flask-Migrate (v0.23.0)	Each change is a numbered, tested script, and the database is backed up before one runs
Web server	waitress (v0.24.0)	A real server that runs on Windows; Flask's own server is only for development
Email	Python's own `smtplib` (v0.26.0)	No extra package; always over an encrypted connection
Login codes and Google	`cryptography` and `segno` (v0.28.0), Authlib (v0.29.0)	Proven libraries for encryption, the QR code and Google's sign-in, instead of home-made security code
Pages	Jinja2 templates, Bootstrap 5.3.3, Bootstrap Icons, the Inter font	Clean layout that works on phones; all stored locally so nothing loads from the internet
Version control	Git and GitHub, one branch and pull request per version	Every change can be reviewed, tested and traced
Tests	Python's standard library only (`tests/`)	Anyone can run them with one command; email and Google are tested with fakes, never a real account

Every package is pinned to an exact version and was approved in advance (rule D1, DEC-011); each was added only in the version that first needed it.

1.7 Glossary

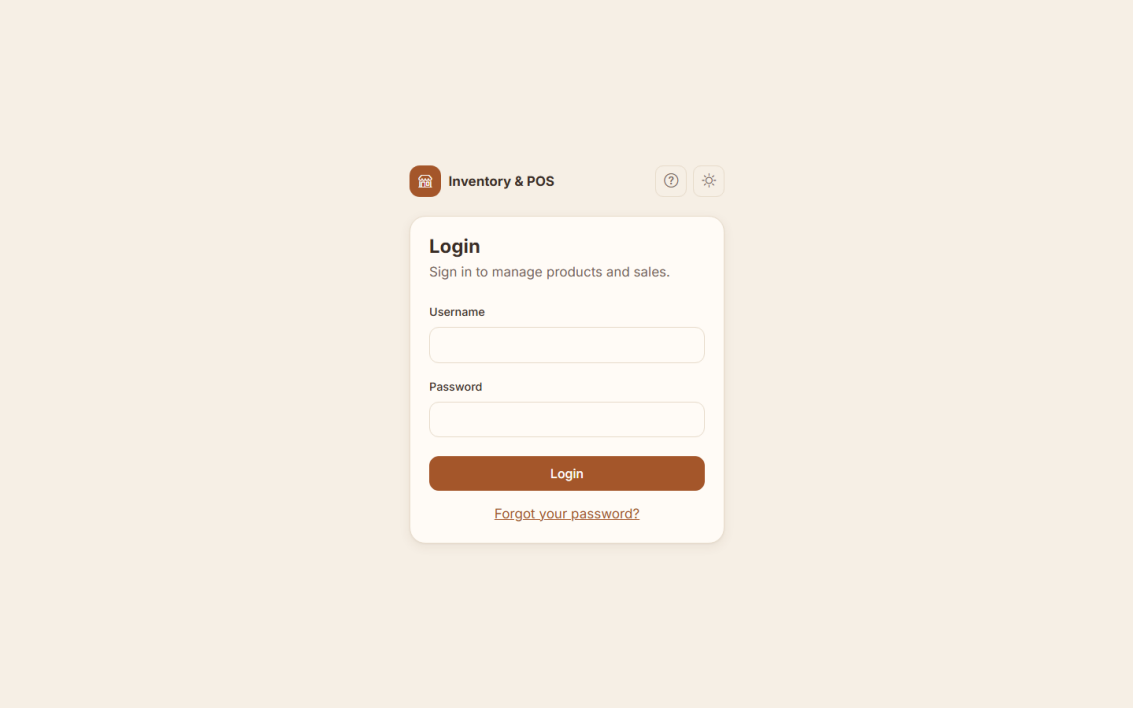
- **POS (Point of Sale):** the counter where a customer pays, and the software used there.
- **SKU (Stock Keeping Unit):** a short unique code for each product, such as `HC-001`.
- **Serving / add-on:** a serving is the one choice every drink needs (Hot or Iced, or a size); an add-on is an optional extra with its own price (Oat Milk +₱40).
- **Recipe:** what one product, serving or add-on uses up (18 g of beans, 160 ml of milk…).
- **CRUD:** Create, Read, Update, Delete, the four basic operations on stored data.
- **Hashing:** turning a password into a scrambled value that cannot be turned back. The system stores only the hash.
- **Role-based access control:** each user has a role (admin or cashier) that decides what they may do.
- **Transaction:** a group of database changes that are saved together or not at all.

- **Owner / invitation:** the owner is the one account made with the subscription's email; everyone else joins through an invitation link sent to their own email, with the role chosen by the person who invited them.
- **Login code / trusted device:** a 6-digit code sent by email (or shown by an authenticator app) that is needed after the password; a browser the user marked as trusted skips only the code, for 30 days (7 for admins).
- **Backup code:** one of ten printed one-time codes that work when there is no internet or no phone.
- **Void / refund / waste:** a void cancels a whole receipt of today; a refund gives back chosen items of any receipt; a refunded item that can't go back on the shelf is recorded as waste.
- **Audit log:** a list of who changed what (prices, recipes, accounts, voids), with the old and the new values.
- **Migration:** a numbered script that changes the shape of the database without losing data.
- **UTC / centavos:** times are stored in world time (UTC) and shown on the shop's clock; money is stored as whole centavos (₱75.50 = 7550), so totals never drift.
- **Keyed hash:** a scrambled value made with a secret key, used to store codes and tokens so that a stolen database doesn't reveal them.
- **Demo mode:** a way to run the system without a mail server: every email is printed in the server window.

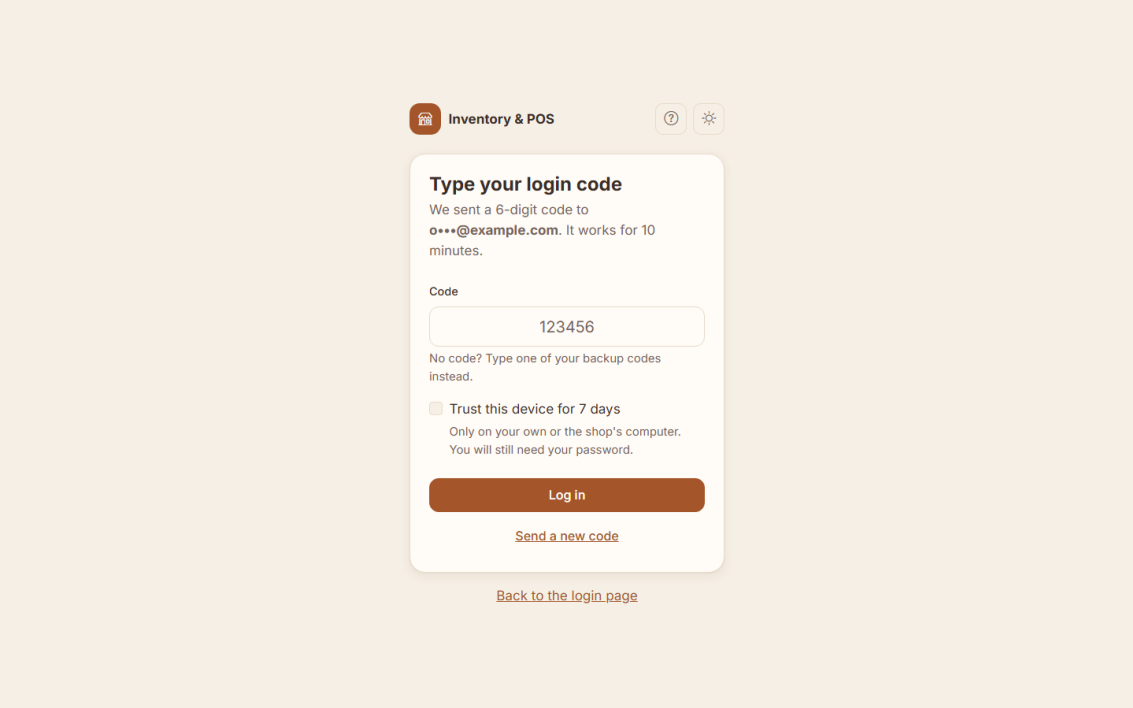
2. A tour of the system

The screenshots below were taken on **v0.32.0** with the generic demo menu (`python seed_demo_data.py`) and a made-up shop called "Demo Café", in **demo mode**, which prints each email in the server window instead of sending it. The café's own menu, prices and photos are used only on my laptop for the live demo and are never published.

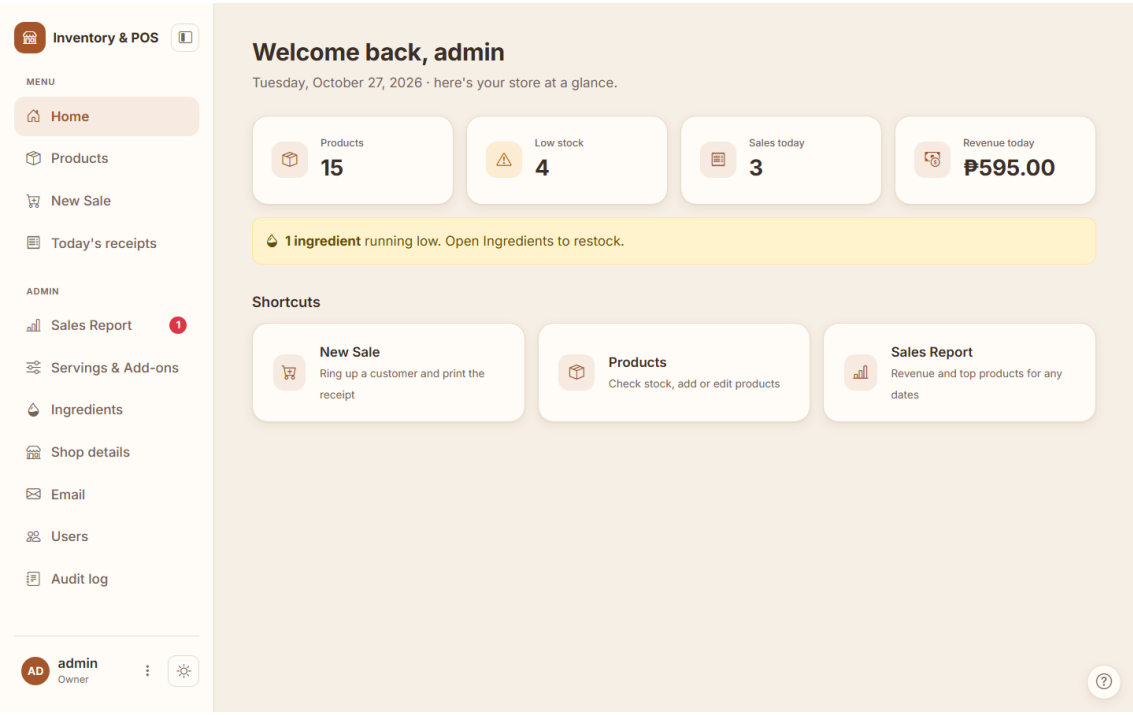
Logging in. Every page except the home page and the login pages needs a login. After five wrong passwords for a username, that computer has to wait five minutes. The **?** in the corner opens the help for this page.



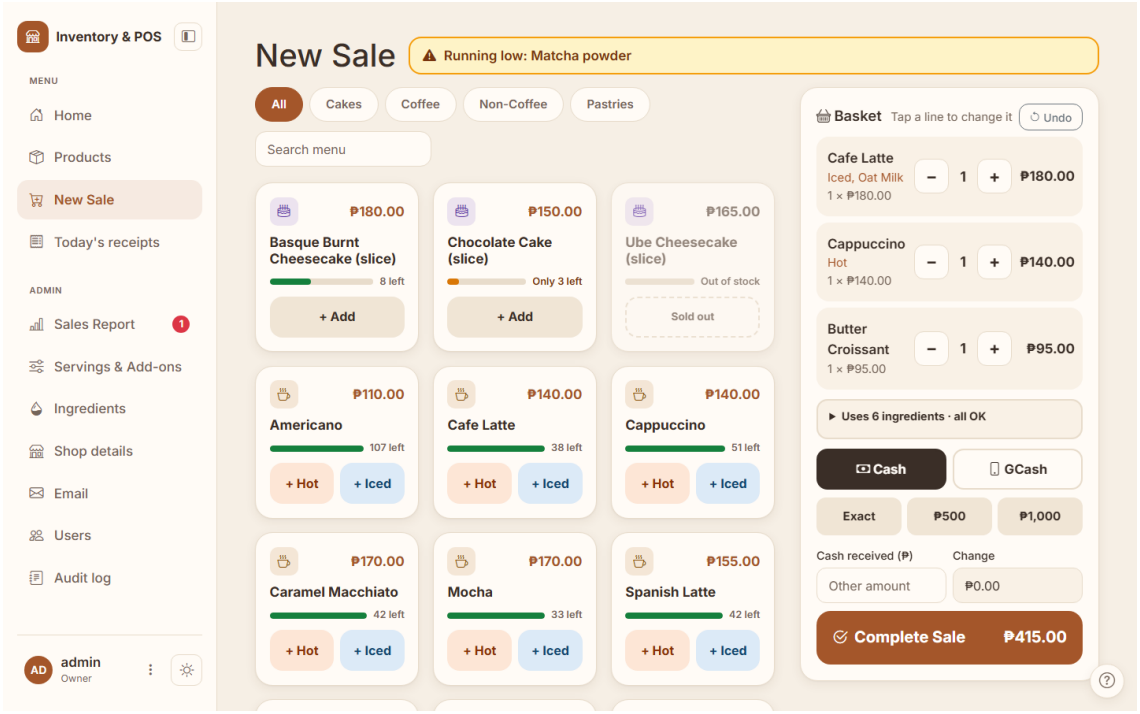
The login code. After the right password, the system sends a 6-digit code to the user's own email (v0.28.0). It works once, for 10 minutes. A browser can be marked as trusted, and then the password alone is enough there for 30 days (7 for admins). Printed backup codes and an authenticator app work even with no internet.



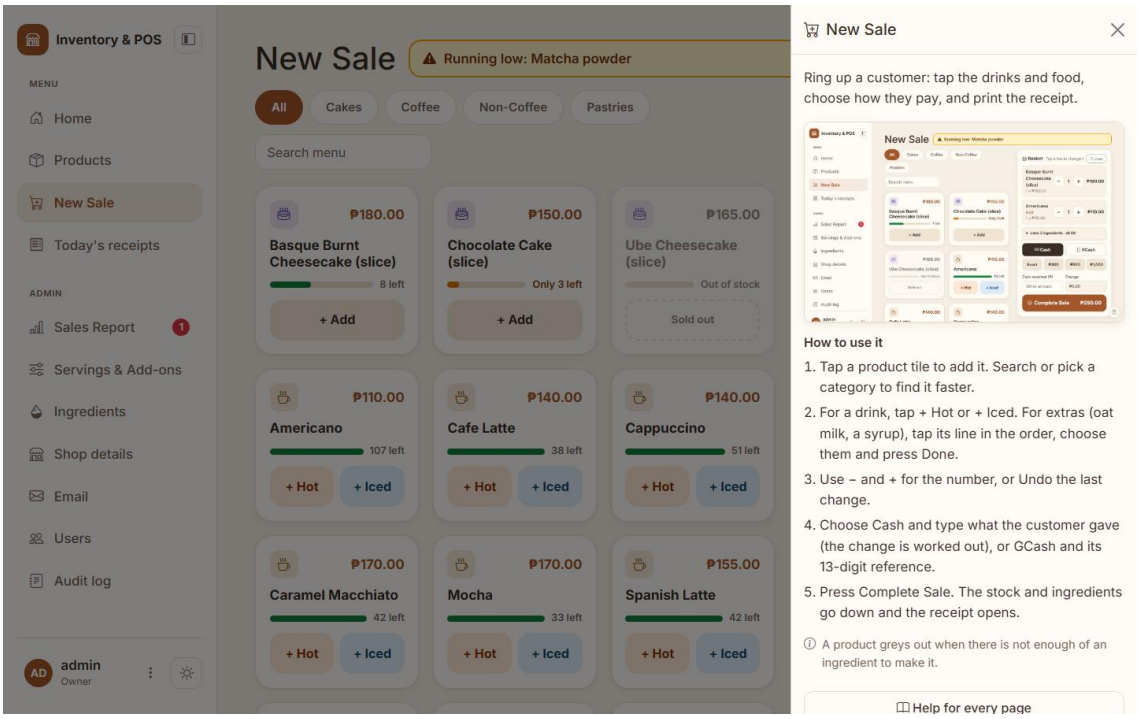
Home. The owner sees today's net sales at a glance and a warning when an ingredient is running low. The sidebar lists only the pages this user may open; the red number beside Sales Report counts the voids waiting for an admin.



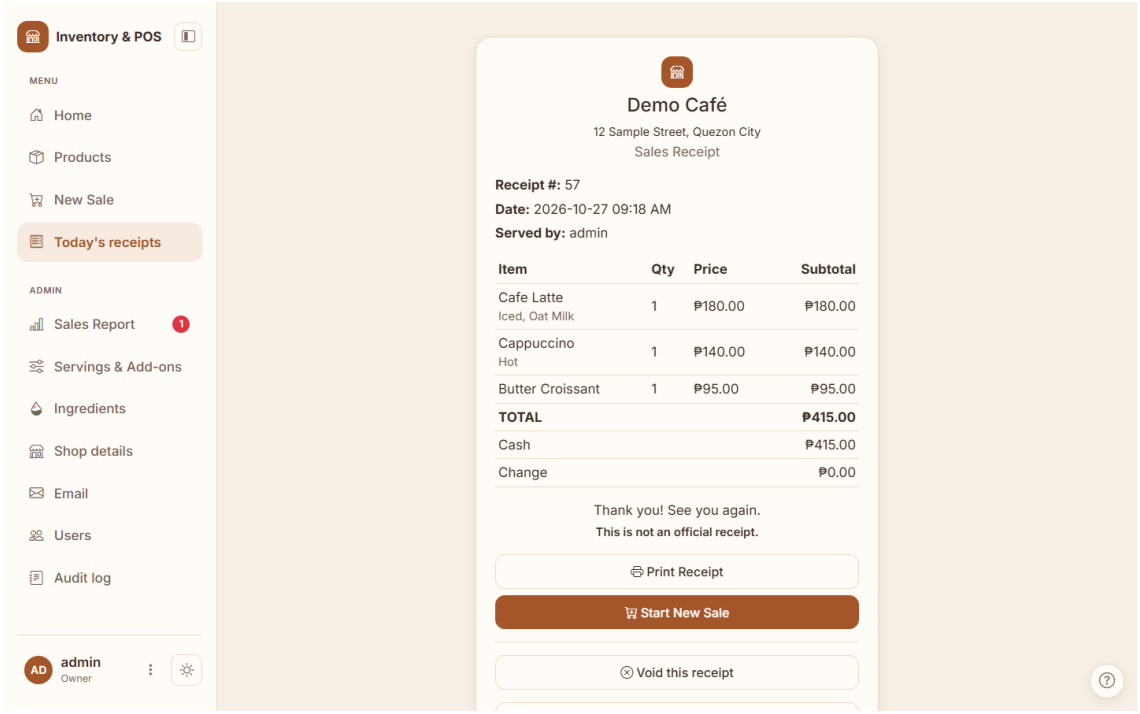
New Sale. This is the screen a cashier uses all day. Each drink tile has **+ Hot** and **+ Iced**, and a bar showing how many more can be made from the ingredients on hand. Tapping a basket line opens it to change the serving and add-ons; here the iced Cafe Latte has Oat Milk. **Undo** takes back the last change (v0.30.0). Quick-cash buttons fill in the amount received, and the change appears immediately.



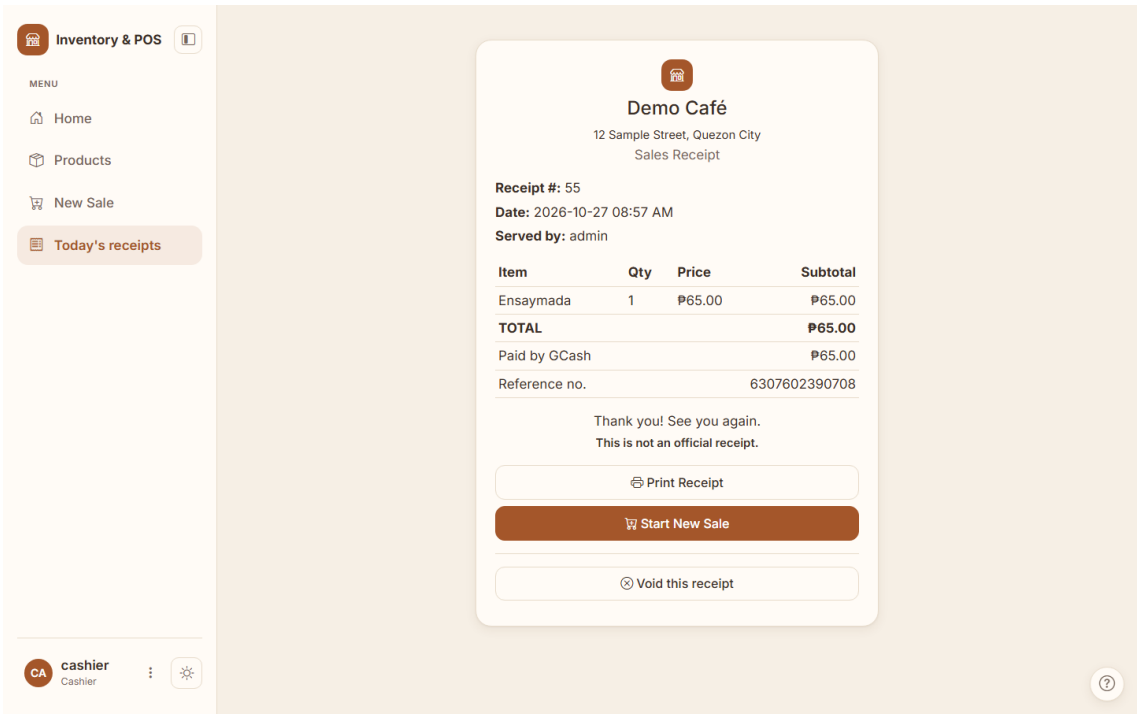
Help on every page. The **?** button opens a side panel with what the page is for, a picture of it and the main steps (v0.32.0). A cashier sees only the steps they can use.



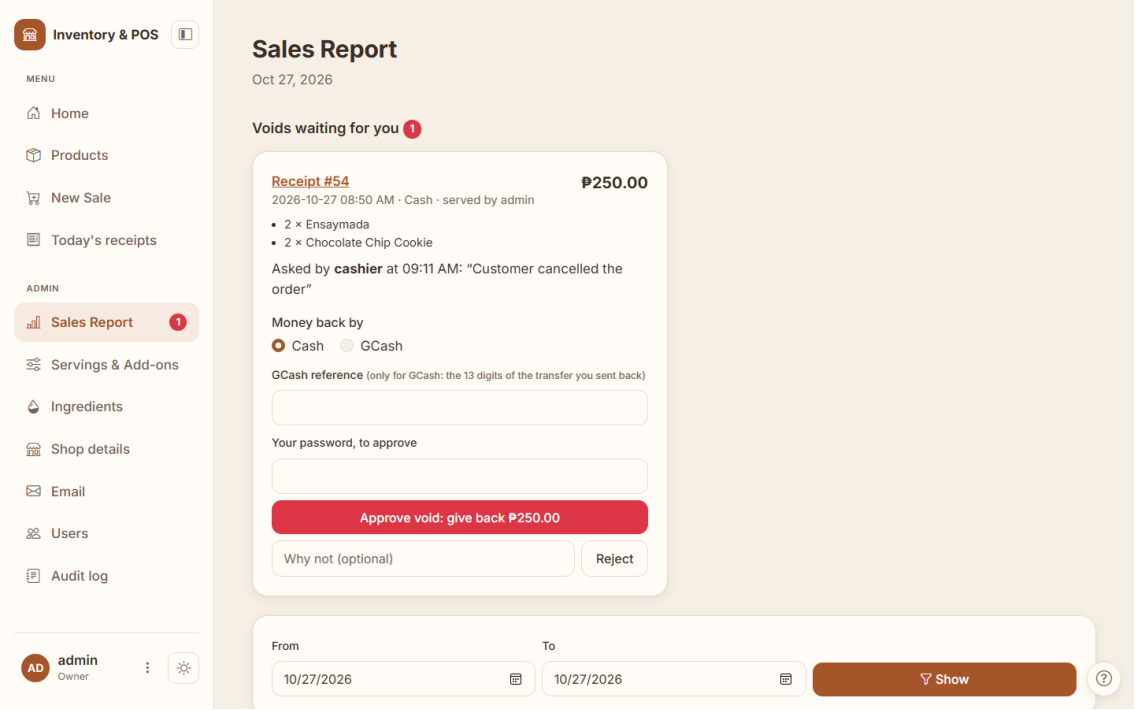
Receipt. The receipt shows the shop's own name and address, keeps its own copy of each item's name, options and price, and always ends with **"This is not an official receipt."** (v0.22.0). Printing hides the sidebar and buttons. An admin sees **Void this receipt** and **Refund items** below it.



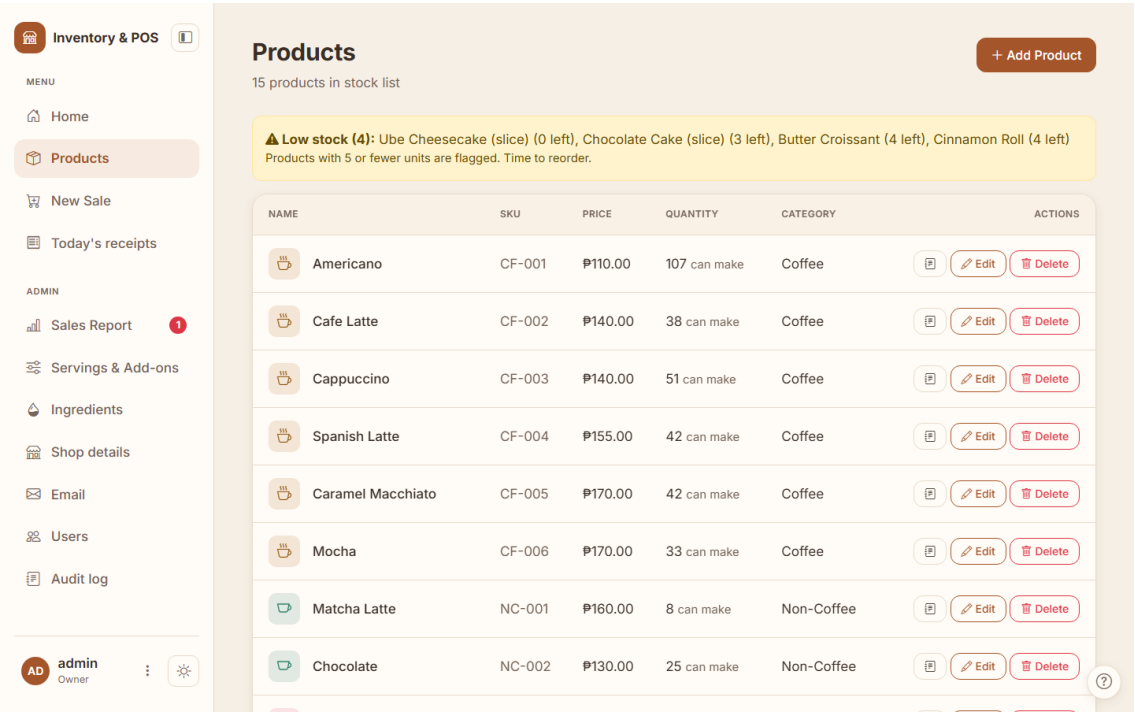
What a cashier sees on a receipt. A cashier can only *ask* for a void of today's receipt, with a reason. Nothing goes back until an admin approves it.



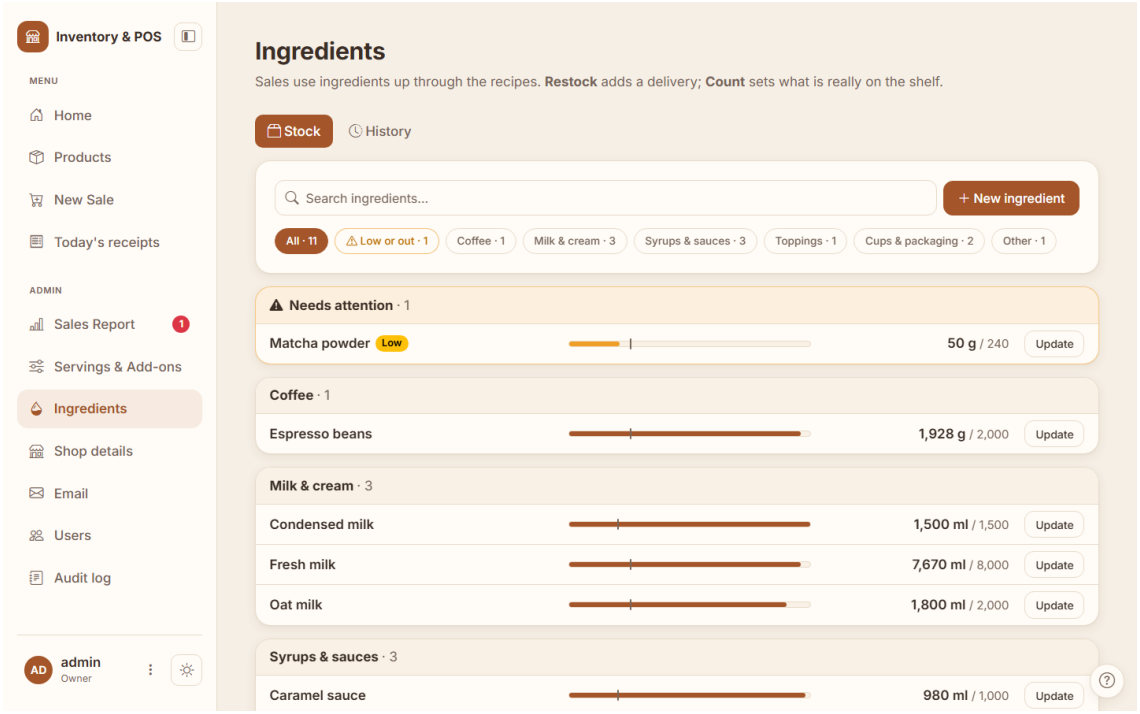
Sales Report. Voids waiting for an admin come first: the admin types their own password to approve (the stock goes back on the shelf) or rejects it with a note. Below it: net sales for any range of dates (gross − voids − refunds), split into cash and GCash, the top five products and a link to every receipt.



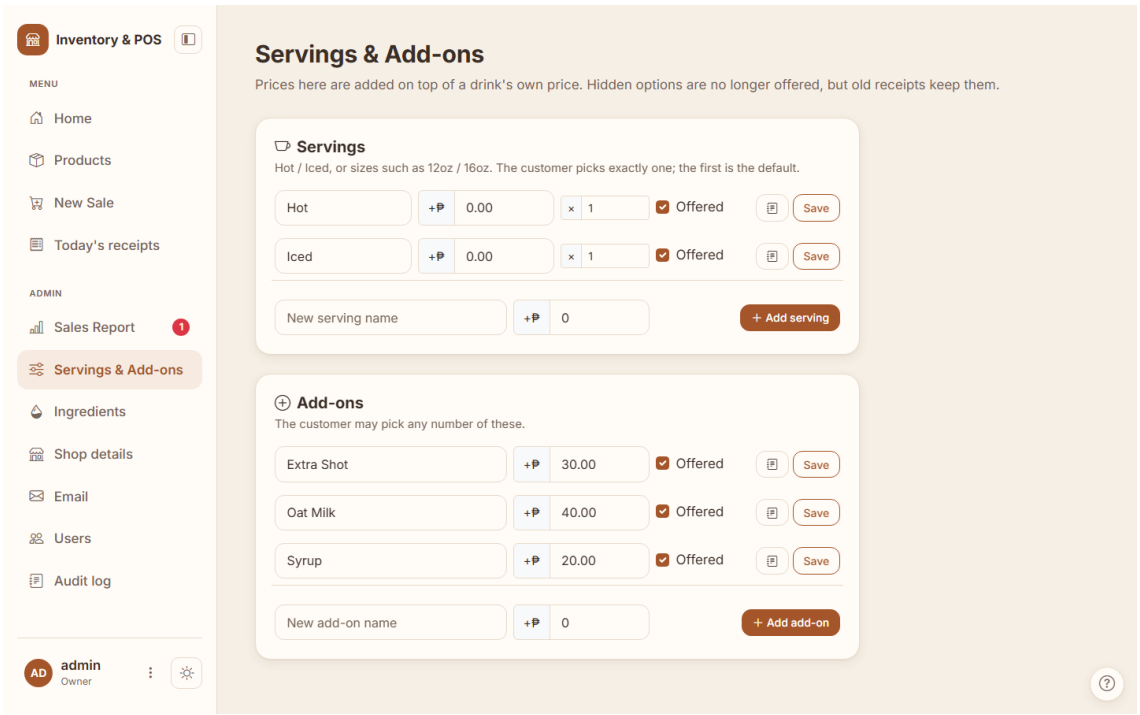
Products. The admin manages the menu here. Drinks show how many can still be made ("can make"), and a box at the top lists everything with five or fewer left.



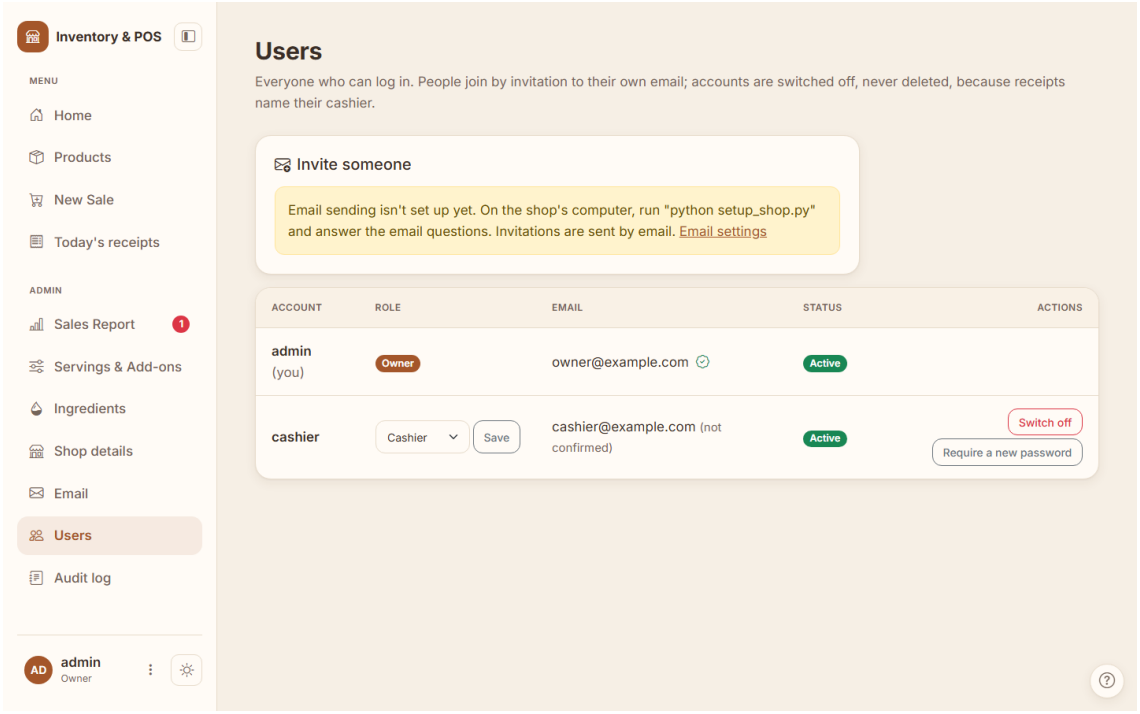
Ingredients. Ingredients are grouped, searchable and shown with a stock bar and a mark at the warning level. *Needs attention* always comes first. **Update** opens a small window for a delivery (Restock) or a shelf count; a count made by mistake can be undone for five minutes.



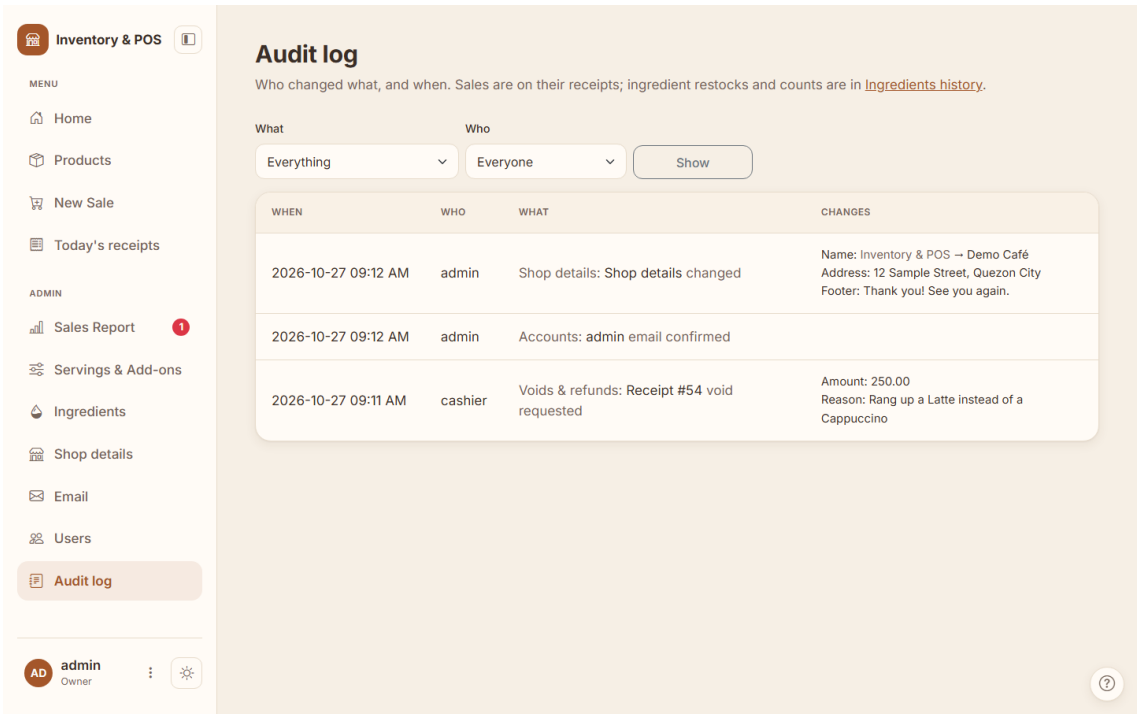
Servings & Add-ons. The admin decides which servings and extras exist, their prices, and whether they are on offer right now.



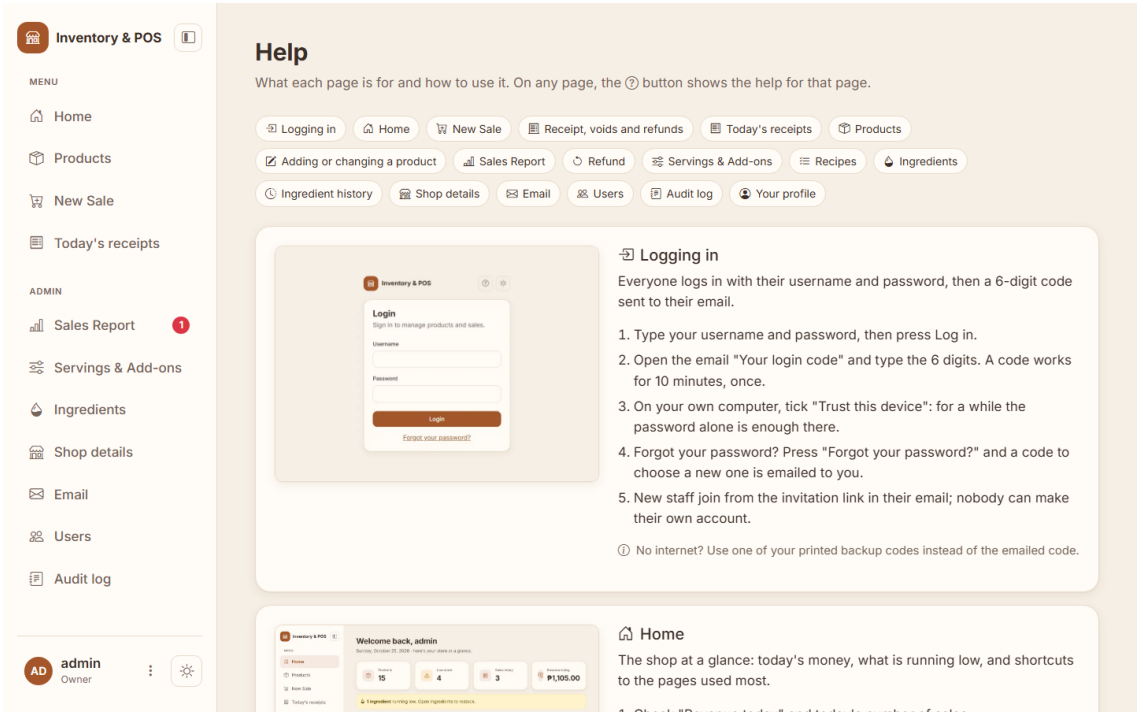
Users. The owner invites people by email and chooses their role; nobody can make their own account (v0.27.0). Accounts are switched off, never deleted, because receipts name their cashier. Here email isn't set up yet, so the page says how to set it up instead of failing.



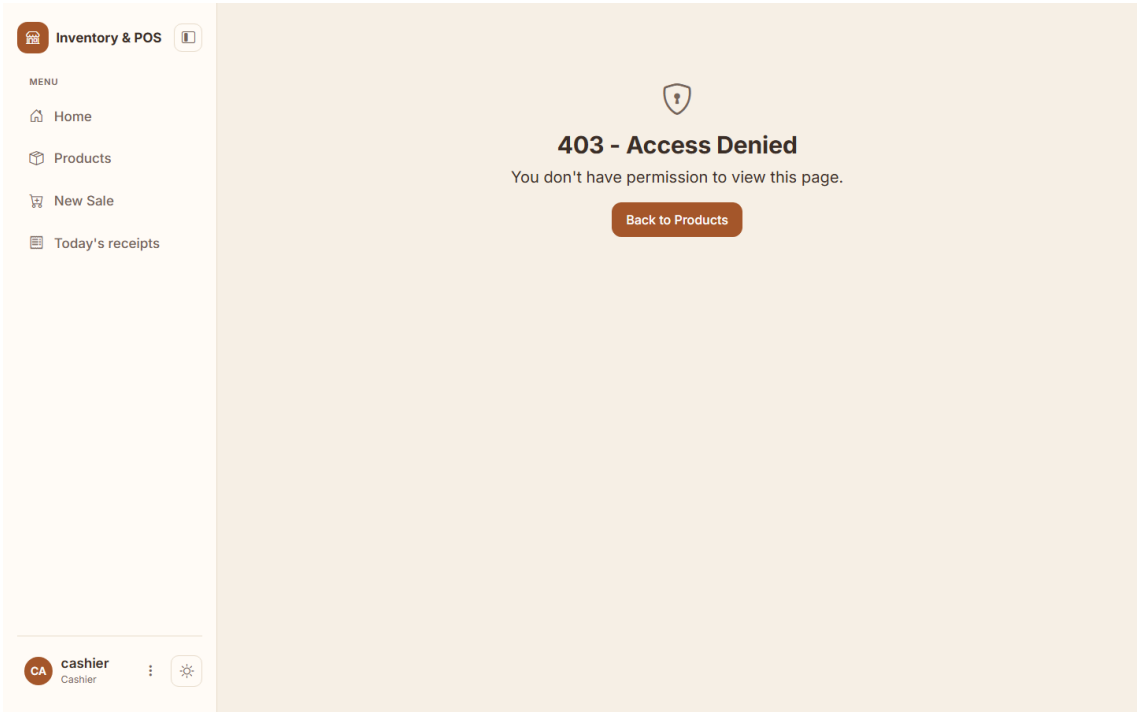
Audit log. Who changed which price, stock, recipe, shop detail, account or receipt, and when, with the old and the new value.



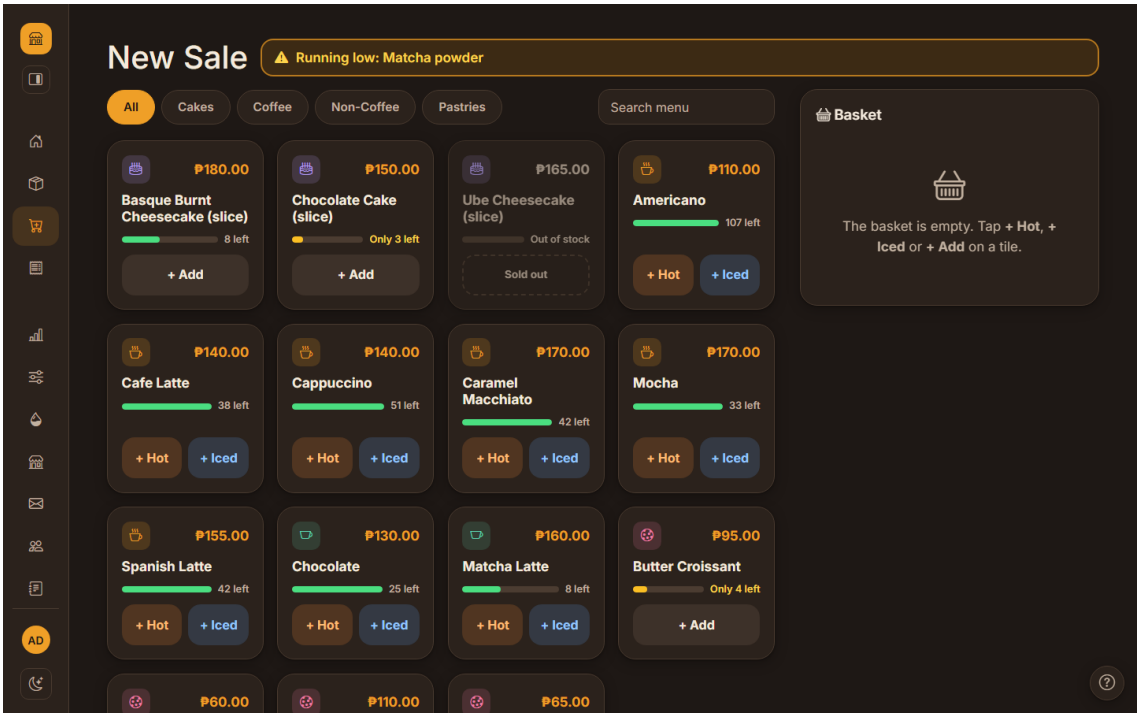
The Help page. Every page's help in one place, with its picture, for a new staff member to read before their first shift.



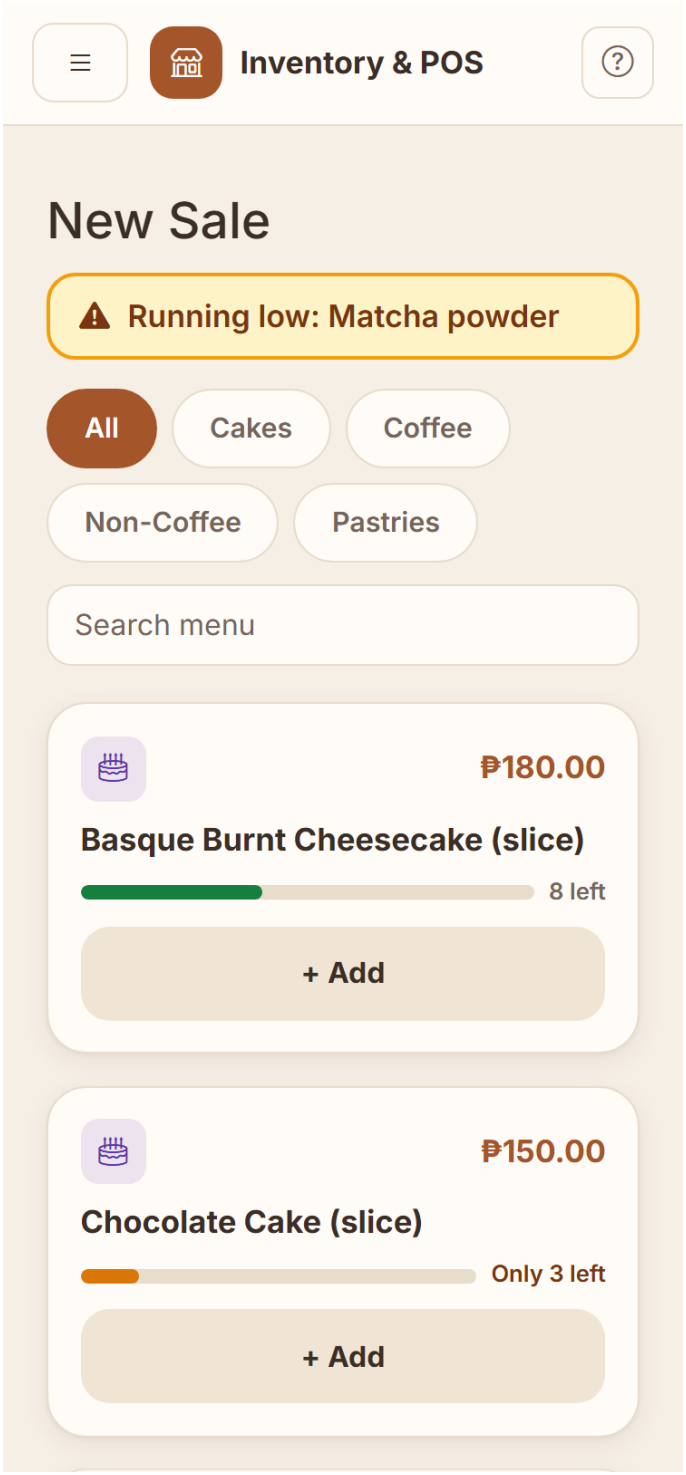
What a cashier sees on an admin page. Hiding a button is not enough, so the server itself refuses.



Dark theme and folded sidebar. For evening shifts there is a dark theme, and on wide screens the sidebar folds into a strip of icons to give the menu more room. Both choices are remembered by the browser.



On a phone. The same pages work at phone width; the sidebar slides in from the ☰ button, and the **?** sits in the top bar.



3. Software process model (item 1)

3.1 The model I chose: incremental development

In **incremental development** a system is built and delivered as a series of small working versions called increments. Each one adds a piece of functionality to the last, and each one can be run, shown and tested. Specification, development and testing overlap instead of happening once, in a fixed order, and what I learned from one increment shaped the next.

That is exactly how this project grew. Every Git tag is one increment, and every increment produced a version that runs:

Increments	What they added
v0.1.0 – v0.7.0	The foundation: a Flask app, the product table, the product list, add / edit / delete, login and logout, then admin and cashier roles
v0.7.1	First fixes found by testing: duplicate SKUs, negative prices, empty names, the peso sign
v0.8.0 – v0.8.6	Sales: basket, cash and change, stock deduction and printable receipts, then five patches that made the sale safe when two tills sell at once and when strange input arrives
v0.9.0 – v0.11.2	Low-stock warnings, the sales report and demo data, each followed by small patches
v0.12.0 – v0.13.5	Profile pictures, the modern look with a light and dark theme, then five security patches (secret key, CSRF tokens, login limit, security headers, password change)
v0.14.0 – v0.18.2	The café: product pictures and a tap-to-add menu, GCash, servings and add-ons, ingredients and recipes, and a redesigned Ingredients page
v0.19.0 – v0.19.4	Project rules and automated tests on every pull request, a security audit and its patches, and pages that work without internet
v0.20.0 – v0.20.6	The counter redesign (one-tap tiles, changes on the basket line, quick cash), then the last open defects: SKU capitals, sessions that can be ended, one receipt per basket, return to the requested page after login
v0.21.0 – v0.21.2	A warmer, roomier look with a foldable sidebar, and a final layout audit of every page *(the course submission)*
v0.21.3 – v0.21.8	Six small patches from the security audit and the known limitations: a friendly error page, login timing, the demo menu sold Hot / Iced, a plain-text requirements file, a tab icon, the dark-theme bar colour
v0.22.0 – v0.25.0	Groundwork for a product: shop details and "not an official receipt" on every receipt, real database migrations, times in UTC, money in whole centavos, a setup program and a real web server, and `app.py` split into blueprints over a `store/` data layer
v0.26.0 – v0.29.0	Accounts for a real shop: email sending, the owner and invitations, an audit log, a code at every login with trusted devices, backup codes and an authenticator app, and Sign in with Google
v0.30.0 – v0.32.0	The counter: undo and a safe stock count, voids and refunds with an admin's approval, and help on every page

The full list, with dates, commits and pull request numbers, is in Appendix A.

3.2 How each increment was built

Each increment followed the same small cycle:

1. **Decide** what the increment should do, written as a user story (Chapter 4).
2. **Branch:** create `feature/<name>` for a feature or `fix/<name>` for a defect.

3. **Build and test** on that branch. From v0.19.0, `python tests/run_tests.py` checks the project rules and runs every test file in a throw-away copy of the project.
4. **Pull request** on GitHub with three headings: Problem, Change, Testing. From v0.21.3 there is a fourth, **Why**: how the change works, why this way, and the other ways that were checked (rule C10). Every decision that changes how the system behaves also gets a numbered entry in `docs/decisions.md` (DEC-001 to DEC-034). GitHub runs the same tests and shows a green tick or a red cross.
5. **Merge and tag** with an annotated tag (`git tag -a v0.20.4 -m "…"`), so every version can be found again.
6. **Update this report**: the version history, the story, the test cases and any defect found.

From v0.21.3 each version's branch starts from the one before it, so the pull requests (#74 to #92) are **stacked** and are merged in order. A database change is now a numbered migration, tested on a copy of an older database before it is merged (rule I5).

I also followed a **stabilization rule**: when a version has a known defect, it is fixed as a patch (v0.9.1, v0.9.2…) before the next feature starts. This is why most features are followed by one or two patch versions. I broke the rule once on purpose: the counter redesign (v0.20.0) went ahead of three small open defects because I wanted the new counter screen finished first, and those defects were fixed straight after it in v0.20.2–v0.20.4.

3.3 Why incremental, and not the other two?

Model	Why it was or wasn't chosen
Waterfall	Waterfall needs every requirement fixed and signed off before coding starts, and nothing works until the end. I started this project as a beginner, so I could not predict every requirement or how hard each part would be, and the café's needs only appeared halfway through. A mistake found late in a waterfall project is also expensive to fix. **Not chosen.**
Reuse-oriented	This model builds a system mainly by configuring existing products, for example adapting an off-the-shelf POS. The project has to be my own work, and a system assembled from an existing product would not show that. **Not chosen as the main model.**
Incremental	It let me learn step by step, always have a working version to show, test each small piece properly, and tag each increment in Git. If time had run out, there would still have been a working system with the most important features. **Chosen.**

Reuse still plays a part. The system builds on well-known components (Flask, Flask-Login, SQLAlchemy, Bootstrap) like almost every modern project. Sommerville points out that real projects usually combine models, so the honest description is *an incremental process that reuses standard framework components*.

Weaknesses I saw. The structure can degrade as increments pile up. Until v0.24.0 every route lived in one `app.py` file, a deliberate choice for the course that had grown to 1,722 lines. v0.25.0 paid that debt back: the routes moved into blueprints and every database change into a `store/` layer, with no visible change and the whole test suite as proof (Chapter 8.1). Progress is also harder to measure against a fixed plan; for the client version I wrote one (`docs/upgrade_plan.md`), with each increment's version number fixed in advance. The project rules (`AGENTS.md`) and the automated contract check stop a later change from quietly removing a protection an earlier increment added.

4. Feature stories (item 2)

Each story follows Sommerville's narrative style: a realistic situation, what the person does, and what the system does in return. The characters are **Maria**, who owns the café (the admin, and from v0.27.0 the **owner**), **Jun**, her cashier, and from F24 **Ana**, her manager. The version in brackets is where the feature was built. Stories F21 to F29 were built after the course submission (v0.21.3 to v0.32.0).

F1 — Viewing the product list (v0.3.0)

Jun arrives for his morning shift. A customer asks whether there are still croissants. Jun logs in on the counter laptop and opens **Products**. The system shows every product with its picture, name, SKU, price, how many are in stock and its category. He finds *Butter Croissant*, sees that 5 are left, and tells the customer.

F2 — Adding a new product (v0.4.0)

The café starts selling cinnamon rolls. Maria logs in and opens Products, where she sees an **Add Product** button that cashiers don't have. She fills in the name, an SKU (`PA-003`), the price ₱110.00, the quantity 12 and the category *Pastries*, and clicks **Add Product**. The roll appears in the list. Had she reused an SKU, left the name empty or typed a negative price, the form would have come back with a message and nothing would have been saved.

F3 — Editing a product (v0.5.0)

The supplier raises the price of cheesecake. Maria clicks **Edit** next to it; the form opens already filled in. She changes the price and, after counting the display case, the quantity, then clicks **Save Changes**. If she changes her mind, **Cancel** takes her back without saving.

F4 — Deleting a product (v0.5.0)

A seasonal drink that never sold is taken off the menu. Maria clicks **Delete**, and the browser asks *"Delete this product?"* to protect against a slip of the finger. She confirms and it disappears. A product that has already been sold cannot be deleted; the system suggests setting its quantity to 0 instead, so old receipts and reports stay correct *(v0.10.2)*.

F5 — Logging in and out (v0.6.0)

Jun types his username and password. The first time he mistypes the password, and the system says *"Invalid username or password"* without saying which of the two was wrong, so a stranger can't find out which usernames exist. On the second try he is in, and his name and role appear in the sidebar. At the end of his shift he clicks **Log out**. Anyone who tries to open a page afterwards is sent to the login page, and after logging in they land on the page they first asked for *(v0.20.6)*.

F6 — Admin and cashier permissions (v0.7.0)

Maria wants Jun to sell and look up products, but not change prices. When Jun logs in, the product list shows no Add, Edit or Delete buttons, and the sidebar has no Sales Report, Servings & Add-ons or Ingredients.

Curious, he types `/products/add` into the address bar. The system answers with a *403 – Access Denied* page. The check happens on the server, so hiding the buttons is not the only protection.

F7 — Processing a sale (v0.8.0, fixed in v0.8.1)

A customer brings two pastries and a bottle of water to the counter. Jun opens **New Sale** and adds them to the basket. The basket shows each line's subtotal and the total, and the menu now shows fewer *available*, because the ones in the basket are spoken for. The customer hands over ₱500; as Jun types it in, the change appears. He clicks **Complete Sale**. The system checks once more that everything is still in stock, saves the sale, lowers the stock, empties the basket and shows a receipt with the receipt number, date and time, who served, every item, the total, the cash and the change. **Print Receipt** prints only the receipt. Had the cash been too little, or had another till just sold the last pastry, the system would have refused with a clear message and saved nothing.

F8 — Low-stock warning (v0.9.0, fixed in v0.9.1–v0.9.2)

When Maria opens **Products** in the morning, a yellow box at the top says *"Low stock (4): Ube Cheesecake (0 left), Chocolate Cake (3 left), Cinnamon Roll (4 left), Butter Croissant (5 left)"*. In the table, the cheesecake has a red **Out of stock** badge and the others a yellow **Low** badge. Jun sees the same box, so he can warn her before a customer asks. On a week with dozens of low items, the box shows the five most urgent and says *"and 30 more"*, so the table stays visible *(v0.9.1)*.

F9 — Sales report (v0.10.0, fixed in v0.10.1–v0.10.2)

At closing time Maria clicks **Sales Report**. Today's report shows the revenue, the number of sales and the items sold, then the top five products and every sale with a link to its receipt. To compare the week she picks a From and To date and clicks **Show**. She renamed a drink on Wednesday, but the report still counts it as one product *(v0.10.1)*.

F10 — Profile picture (v0.12.0)

Jun logs in for the first time and sees a round badge with his initials at the bottom of the sidebar. He clicks it, chooses **Change picture**, and uploads a photo from his phone. The circle now shows his photo on every page. When he tries a GIF, the system says it only accepts PNG, JPG or WebP; a 5 MB photo gets *"That picture is too big. The limit is 2 MB."* **Remove picture** brings back his initials.

F11 — A modern look with light and dark themes (v0.13.0)

Maria opens the system and finds a sidebar on the left with the pages she may use, the current one highlighted. Home shows today's numbers and shortcuts. In the evening the café lights are dim, so she taps the **sun** icon beside her name: it becomes a **moon** and the screen turns dark. Tomorrow it opens dark again, because her browser remembers. A receipt printed from the dark theme is still black on white. On her phone, the sidebar hides behind a ☰ button.

F12 — Account safety (v0.13.1–v0.13.5)

On her first day Maria opens her avatar menu, chooses **Change password**, and replaces the demo password. A password shorter than 8 characters, or the same as her username, is refused. Later someone at the counter tries to guess her password: after five wrong tries the page says *"Too many failed attempts. Please wait 5 minutes and try again."*, even if the sixth guess is right. Behind the scenes every form carries a hidden security token, so another website can't make her browser change anything, and the key that protects her login is no longer in the published code.

F13 — Product pictures and a tap-to-add menu (v0.14.0–v0.14.4)

Maria adds a photo to each pastry; products without one show a coloured icon (a hot cup, an iced cup, a pastry, a cake). At the counter, Jun now sees the menu as tiles grouped by category. He taps **Pastries**, then a croissant twice; each tap adds one, and the Pastries filter stays selected *(v0.14.2)*. On his phone, a bar at the bottom shows *"2 items · ₱190.00 — Pay"* and jumps to the basket *(v0.14.3)*.

F14 — Paying by GCash (v0.15.0)

A customer orders a latte and a croffle (₱340.00) and wants to pay by GCash. Jun taps **GCash**: the cash boxes disappear and a box asks for the reference number. The customer pays in the app and shows the confirmation; Jun types the 13-digit reference, checks the amount and completes the sale. The receipt says *Paid by GCash* with the reference. A reference that was already used is refused and the earlier receipt is named. At closing, the report shows cash (what should be in the drawer) and GCash separately.

F15 — Hot / Iced and add-ons (v0.16.0–v0.16.1, wording v0.17.2)

Maria sets up the servings, **Hot** and **Iced**, and the café's extras (Extra Shot +₱30, Oat Milk +₱40, Syrup +₱20) on the **Servings & Add-ons** page. The Iced serving has its own recipe: 150 g of ice and an iced cup. A latte with Oat Milk appears in the basket as *Cafe Latte — Iced, Oat Milk*, separate from a plain Hot latte, and the receipt keeps the options and the price paid even if the add-on price changes later.

F16 — Ingredients and recipes (v0.17.0–v0.17.1)

Maria enters what is in the stock room (espresso beans 3,000 g, fresh milk 10,000 ml, oat milk 2,000 ml…) and a recipe for each product: one latte uses 18 g of beans and 160 ml of milk; Oat Milk swaps the fresh milk for oat milk. The menu now shows how many of each drink can still be made. When the oat milk runs low, Home warns her. When a delivery arrives she clicks **Restock**, and after counting the fridge at night she uses **Count**. Every change (each sale, delivery and count) appears in the ingredient history with who did it and which receipt it belongs to.

F17 — Finding and updating an ingredient quickly (v0.18.0–v0.18.2)

On Saturday morning the milk delivery arrives. Maria opens **Ingredients**: *Needs attention* comes first, then one card per group (Coffee, Milk & cream, Syrups & sauces, Toppings…). Each row has a bar showing how full the shelf is, with a mark at the warning level. She types "oat", clicks **Update** on Oat milk, taps *Fill up to full* and **Add to stock**. The page comes back to the same row with her search still there *(v0.18.1)*. Later she opens **History** and filters it to see last night's count.

F18 — Changing the system safely (v0.19.0)

A classmate, or an AI coding assistant, is asked to add a new button. Before writing any code they read **AGENTS.md**, which lists the rules already decided: every admin page needs the admin check, every form needs the security token, stock may only be subtracted with a safe conditional update, no new packages without approval, and every change needs a test. When they finish they run `python tests/run_tests.py`; it checks those rules and runs every test in a throw-away copy of the project, so the café's real database is never touched. GitHub runs the same checks on their pull request before anyone merges it.

F19 — Serving a queue quickly (v0.20.0–v0.20.1)

It is 3 PM and six customers are waiting. Jun taps **+ Iced** on the Cafe Latte tile: one iced latte is in the basket, with no window to close. The next customer wants oat milk, so he taps that line in the basket; it opens in place, he taps **Oat Milk** and **Done**. A second croffle is one tap on **+**. Each tile shows a bar and how many are left, so he can see that only three iced drinks are possible (the iced cups are low) and that the Matcha tile is greyed out with *"Out of Matcha powder"*. The customer hands over ₱500: Jun taps **₱500**, sees the change, and taps the big **Complete Sale** button. If the laptop is slow and he taps it twice, only one receipt is made *(v0.20.4)*.

F20 — A calmer, roomier screen (v0.21.0–v0.21.2)

Maria works long shifts at the counter, and the old white-and-teal screen felt cramped and cold. She now sees a warm cream page (espresso brown at night) with bigger text and more space between the tiles. When she wants the whole width for the menu she taps the small fold icon beside the system name: the sidebar shrinks to a strip of icons (hovering shows each name) and stays that way the next time she opens the app. On a phone nothing changes; the menu still slides in from the left.

F21 — The shop's own receipt (v0.22.0)

Maria wants the receipts to carry her café's name, not "Inventory & POS". She opens **Shop details** in the admin part of the sidebar, types the café's name, its address, her TIN and a footer ("Thank you! See you again."), and clicks **Save shop details**. The next receipt Jun prints shows them at the top and bottom, and old receipts show them too when reprinted. At the very bottom every receipt says **"This is not an official receipt."** Maria can't switch that line off: the official invoice still comes from the café's BIR-registered POS, and this slip is for the shop's own records. When she types a TIN with letters in it, the page says it must have 9 to 14 digits. Jun, as a cashier, has no Shop details link, and typing its address gives the 403 page.

F22 — Setting up the café's own copy (v0.24.0)

The system is installed on Maria's counter laptop. She runs `setup_shop.py` once. It asks whether phones in the café will use it (yes: Jun takes orders on a tablet), the café's name and address for the receipts, and her own username and password, typed twice; "maria" or a 6-letter password is refused with a plain reason. It says it's done and how to start. Every morning she double-clicks **start_shop.bat**: a window shows "On this computer: http://127.0.0.1:5000" and "On other devices: http://192.168.1.10:5000", which Jun opens on the tablet. Nobody can log in with the demo password `admin123`, because the demo accounts were never created.

F23 — Making sure the café's email works (v0.26.0)

Soon the system will email Jun his invitation, and a code when he logs in on a new device, so Maria sets up email first. She runs `setup_shop.py` again and says yes to the email questions: Gmail's server, the café's Gmail address and an App Password, hidden as she types. Her shop details and her account are kept. In the app she opens **Email** in the admin menu, sees the server and the address (the password only shows as "Saved"), types her own address and clicks **Send test message**. A minute later "Test message from" the café is in her inbox. On a day the Wi-Fi is down, the same button says it couldn't reach the mail server and that selling still works. And if someone sits down at the counter laptop while Maria is still logged in, they can't guess her password on the Profile page: after 5 wrong tries it says to wait 5 minutes, and closing and restarting the system doesn't reset that count.

F24 — Everyone with their own account (v0.27.0)

Maria ran `setup_shop.py` with her own email, so she is the café's **owner**; on Profile she confirms the address with a code from her inbox. Jun starts on Monday, so she opens **Users**, types Jun's own Gmail address, picks **Cashier** and clicks **Invite**. Jun gets an email with a link, chooses "jun" and a password, types the 6-digit code that arrives, and is logged in as a cashier; nothing he types could have made him an admin. Maria lets her manager Ana invite cashiers too, but only Maria can make someone an admin. When a part-timer leaves, Ana switches the account off: the till they were logged in on is logged out at once, and their old receipts still show their name. A week later Jun forgets his password; **Forgot your password?** on the login page sends a code to his email and he chooses a new one. When the latte's price changes from ₱150 to ₱160, the **Audit log** shows who changed it and when.

F25 — A code that keeps strangers out (v0.28.0)

Jun types his password on the counter laptop and the system asks for a 6-digit code, which arrives in his Gmail. He ticks **Trust this device**, so for the next 30 days that laptop only asks for his password. When someone sees Jun's password over his shoulder and tries it at home, they get stuck at the code. Maria prefers her phone: on Profile she switches to an authenticator app by scanning a QR code, and prints her backup codes. One morning the Wi-Fi is down; the counter laptop is trusted, so Jun logs in and sells as usual, and Maria logs in on her own laptop with the app, which needs no internet. When a part-timer's phone is lost, Ana resets their login code to email from Users, and the part-timer gets an email saying so.

F26 — One tap with Google (v0.29.0)

Maria set up Google sign-in with the café's own Google Cloud client. On her Profile she types her password and the code from her app, taps **Link Google**, and picks her Gmail, the same address as her account. Now on the counter laptop (at `localhost`) she taps **Sign in with Google** and, because the laptop is trusted, she is in. Jun's invitation page offers **Continue with Google**; he chooses "jun", picks the Google account for the invited address, types the code that arrives, and is in, without ever making a password. When Ana tries to link her personal Gmail, which isn't her account's email, the page says the two addresses differ and nothing is linked.

F27 — Taking it back (v0.30.0)

Jun taps Iced Latte twice by mistake while a queue waits; one tap on **Undo** and the order is back as it was, and a second Undo takes back the change before that. At closing, Maria counts the espresso beans and types 75 instead of 750; the Ingredients page offers **Undo count** for the next five minutes and puts the 750 back, with both lines in the history. Another evening she counts the milk while Jun sells a latte; when she saves, the page says the milk changed since she opened it and asks her to count again, so Jun's sale isn't lost.

F28 — Giving the money back (v0.31.0)

Jun rings up a Spanish Latte, but the customer wanted a Matcha. He opens **Today's receipts**, picks the receipt, taps **Void this receipt**, types "wrong drink rung up" and asks for the void. Maria is at the market, so Jun picks her name and taps **Send**; her phone shows the receipt and a code, and because the story matches she texts him the code. Jun types it, the receipt says VOIDED, the milk and beans are back on the shelf, and the slip that prints names Jun as the one who asked and Maria as the one who approved. Two days later a customer returns an unopened croissant from a bigger order; Maria opens that receipt, taps **Refund items**, chooses one croissant **back on the shelf**, types her password and hands back ₱85 from the drawer. The Sales Report shows the void and the refund under Net sales, and the cash total matches the drawer.

F29 — Help on the page (v0.32.0)

Ana joins as a cashier on a busy Saturday. On New Sale she isn't sure how to add oat milk, so she taps the small **?** in the corner; a panel slides in with a picture of the page and five short steps, and the second one tells her to tap the drink's line and choose the extra. Later a customer says they were charged for the wrong drink; the **?** on the receipt tells her to press the Void button and ask for a void, and that no money goes back until an admin approves. Maria, the owner, opens **Help** from her menu the night before a new staff member starts and walks through every page with its picture, so the new cashier knows the screens before touching them.

5. Test cases (item 3)

5.1 How the system was tested

Sommerville's test case description gives, for each feature, the **input**, the **tests** to run and the expected **output**. Each feature below starts with that summary where it helps, followed by a table of individual test cases with the expected and the actual result.

The tests were run in several ways, because no single method catches everything:

1. **Automated tests with Flask's test client.** The test client sends requests to the app the way a browser would, but from a script, on a fresh database every time. Since v0.19.0 these tests live in the `tests/` folder and run with one command (Appendix B).
2. **Attack tests.** Every security weakness was attacked on the old version first (a forged cookie, a form without its token, 200 password guesses) and attacked again after the fix.
3. **Race tests** on a real multi-threaded server: up to 20 "cashiers" press Complete Sale at the same moment.
4. **Browser tests** in Chromium: clicking, typing, the change preview, light and dark themes, printing, and phone width, each time also checking for JavaScript errors.
5. **Hand testing** by me in the running app, including a seven-round attack drill on 13 October 2026. Several defects in Chapter 10 were found this way.
6. **Regression testing.** After every version, all earlier tests were run again, and a version was only published when everything passed.
7. **Fakes for outside services** (from v0.26.0). Email is tested with a fake mailbox and Google sign-in with a fake token checker that signs its own test tokens, so no test needs the internet or a real account (rule T3). Database changes are tested on a copy of an older database, including a copy of the café's real one.

The first features (F1–F6) were tested on 3 October 2026 against v0.7.0, while the shop was still a general store, so their test data uses items such as *Coke 1.5L* and *Instant Noodles*. Later tests use the café menu.

Reading the HTTP codes: `200` = page shown · `302` = sent to another page · `400` = request refused · `403` = forbidden · `404` = not found · `405` = method not allowed · `429` = too many attempts · `500` = the server crashed.

5.2 F1 — View product list

Input: a request to open `/products`, with or without being logged in. **Tests:** 1. Open the page while logged out. 2. Open it while logged in. 3. Check that every saved product appears with all five fields. **Output:** logged out → redirect to the login page; logged in → table of all products.

ID	Precondition	Steps / input	Expected result	Actual result	Pass?
TC-1.1	Not logged in	Open `/products`	Redirect to `/login`	302 → `/login?next=/products`	✅
TC-1.2	Logged in as admin, product "Coke 1.5L" exists	Open `/products`	Table shows Coke 1.5L, BEV-001, price, quantity, category	All fields shown	✅

ID	Precondition	Steps / input	Expected result	Actual result	Pass?
TC-1.3	Not logged in	Open / (home)	Home page loads	200, home page shown	✅

5.3 F2 — Add product

Input: name (text, required), SKU (text, required, must be unique), price (decimal number), quantity (whole number), category (optional text). **Tests:** 1. All fields valid. 2. Category left empty. 3. SKU already used. 4. Price not a number. 5. Negative price or quantity. 6. Empty name. **Output:** valid input → product saved and shown in the list; invalid input → a clear error message and nothing saved.

ID	Precondition	Steps / input	Expected result	Actual result	Pass?
TC-2.1	Logged in as admin	Coke 1.5L, BEV-001, 75.50, 24, Beverages	Saved; redirect to list; shown as 75.50	Saved, redirect, shown	✅
TC-2.2	Admin	Noodles, FD-001, 15, 10, *(no category)*	Saved with empty category	Saved	✅
TC-2.3	Admin, BEV-001 already exists	Dup, **BEV-001**, 1, 1	Error message "SKU already exists"; nothing saved	**500 server crash** (database unique-constraint error not handled) *(also confirmed by hand:* `IntegrityError`*)* · **v0.7.1:** red message, nothing saved *(confirmed by hand)*	❌ → ✅ DEF-01
TC-2.4	Admin	price = `abc` (sent without the browser's check)	Error message; nothing saved	**500 server crash** · **v0.7.1:** error message, nothing saved	❌ → ✅ DEF-02
TC-2.5	Admin	price = **-5**, quantity = **-3**	Rejected: price and quantity cannot be negative	**Accepted and saved** with negative values *(negative price also confirmed by hand)* · **v0.7.1:** rejected by browser (`min="0"`) and by server	❌ → ✅ DEF-03

ID	Precondition	Steps / input	Expected result	Actual result	Pass?
TC-2.6	Admin	name = *(empty)*, sent without the browser's check	Rejected: name required	**Accepted**; product saved with no name · **v0.7.1:** rejected with message	❌ → ✅ DEF-04

> **Why "sent without the browser's check"?** The form's HTML has `required` and `type="number"`, so a normal browser stops you before sending. But anyone can bypass the browser (with developer tools or a script), so the **server** must check again. This is called **server-side validation**, and it is a standard security rule: *never trust input from the client*.

5.4 F3 — Edit product

Input: a product ID in the address (`/products/edit/<id>`) and the same fields as F2. **Tests:** 1. Form opens pre-filled. 2. Valid change is saved. 3. Product ID doesn't exist. 4. SKU changed to one another product uses. **Output:** changes saved and shown; a missing product gives "not found"; a duplicate SKU gives a clear error.

ID	Precondition	Steps / input	Expected result	Actual result	Pass?
TC-3.1	Admin, product 1 exists	Open `/products/edit/1`	Form pre-filled with current values	Pre-filled	✅
TC-3.2	Admin	Change price 75.50 → 80, quantity 24 → 20, save	Saved; list shows new values	Saved (80.0, 20)	✅
TC-3.3	Admin	Open `/products/edit/999`	404 Not Found	404	✅
TC-3.4	Admin, FD-001 belongs to another product	Change product 1's SKU to FD-001	Error "SKU already exists"	**500 server crash** · **v0.7.1:** error message, nothing changed	❌ → ✅ DEF-01

5.5 F4 — Delete product

Input: a product ID, sent with a POST request (the Delete button). **Tests:** 1. Delete an existing product. 2. Delete a product that doesn't exist. 3. Try to delete with a plain link visit (GET) instead of the button. **Output:** product removed; missing ID → 404; GET → refused.

ID	Precondition	Steps / input	Expected result	Actual result	Pass?
TC-4.1	Admin, product 2 exists	Click Delete on product 2, confirm	Removed from database and list	Removed	✅
TC-4.2	Admin	POST `/products/delete/999`	404	404	✅
TC-4.3	Admin	Visit `/products/delete/2` as a normal link (GET)	Refused (deleting through a link is unsafe)	405 Method Not Allowed	✅
TC-4.4	Admin	Click Delete, then Cancel in the pop-up	Product kept	Browser test on v0.21.2 (2026-10-23): pop-up "Delete this product?", Cancel chosen; all 17 products still listed	✅

> **Why does TC-4.3 matter?** If deleting worked through a simple link, a web crawler or a malicious link in an email could delete products just by visiting it. Requiring POST (a form submission) prevents that.

5.6 F5 — Log in / log out

Input: username and password. **Tests:** 1. Correct details. 2. Wrong password. 3. Username that doesn't exist. 4. Username in different capitals.

5. Logout. 6. Protected page after logout. 7. Passwords are not stored in plain text.

Output: correct → logged in and sent to Products; wrong → the same general error message; logout → protected pages locked again.

ID	Precondition	Steps / input	Expected result	Actual result	Pass?
TC-5.1	Users exist	admin / admin123	Redirect to Products; navbar shows "admin (admin)"	As expected	✅
TC-5.2	—	admin / *wrong*	"Invalid username or password"	Shown	✅
TC-5.3	—	ghost / x	Same message as TC-5.2 (doesn't reveal which part was wrong)	Same message	✅
TC-5.4	—	**ADMIN** / admin123	*Design decision:* usernames are case-sensitive → rejected	Rejected	✅
TC-5.5	Logged in	Click Logout	Logged out; redirect to home	302 → `/`	✅
TC-5.6	Just logged out	Open `/products`	Redirect to login	302 → `/login`	✅

ID	Precondition	Steps / input	Expected result	Actual result	Pass?
TC-5.7	Users created	Look at the `password_hash` column in the database	A hash, not the real password	Stored as `scrypt:32768:8:1$…` hash	✅
TC-5.8	Not logged in	Open `/products/add` → log in	After login, return to the page originally requested	**v0.7.1–v0.20.5:** always goes to `/products` (the `next` value is ignored) · **v0.20.6:** returns to the requested page; `test_login_next.py` also checks that other-website links are ignored (TC-24.1–24.4)	❌ → ✅ DEF-06

5.7 F6 — Role-based access (Admin vs. Cashier)

Input: a logged-in user with role `admin` or `cashier`, requesting product-management pages. **Tests:** 1. Cashier sees no management buttons. 2. Cashier types each admin address directly. 3. Cashier sends a hidden form submission. 4. Admin keeps full access. **Output:** cashier → view only, every admin action refused with 403; admin → full access.

ID	Precondition	Steps / input	Expected result	Actual result	Pass?
TC-6.1	Logged in as cashier	Open `/products`	No Add button; "View only" instead of Edit/Delete	As expected *(also checked by hand)*	✅
TC-6.2	Cashier	Type `/products/add` in the address bar	403 Access Denied page	403 page *(also checked by hand)*	✅
TC-6.3	Cashier	Open `/products/edit/1`	403	403	✅
TC-6.4	Cashier	Send a POST to `/products/add` with product data	403; nothing saved	403	✅
TC-6.5	Cashier	Send a POST to `/products/delete/1`	403; product still exists	403; product still exists	✅
TC-6.6	Logged in as admin	Add / edit / delete	All allowed	Allowed *(also checked by hand)*	✅

5.8 F7 — Process sale (v0.8.0, v0.8.1)

Input: products and quantities added to a basket; cash received from the customer. **Tests:** 1. Building the basket (adding, merging, removing). 2. Stock limits, including what is already in the basket. 3. Invalid quantities and products. 4. Cash checks. 5. Completing a sale and its effect on stock. 6. Receipts, including old ones. 7. Who can sell, and basket privacy. **Output:** valid sale → saved, stock reduced, receipt shown; anything invalid → clear message and **nothing saved**.

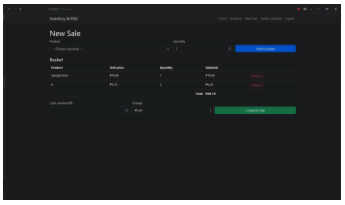
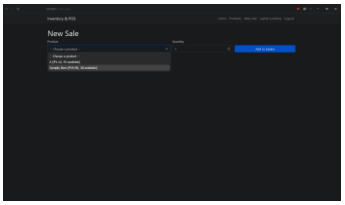
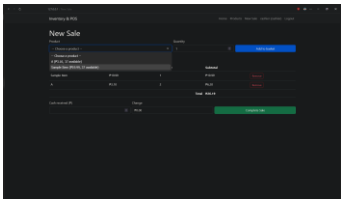
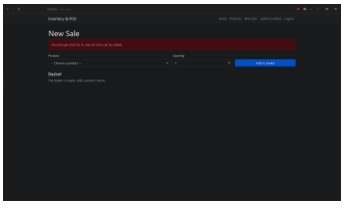
> **How these were run.** On 2026-10-08 against the code tagged **v0.8.0** (`1343236`) using the Flask test client, plus a real browser (Chromium) for the JavaScript change preview; v0.8.1 (`538ce09`) was re-tested the same day, including on a **copy of the real database**. Cases marked *(by hand)* were also confirmed by me in the running app.

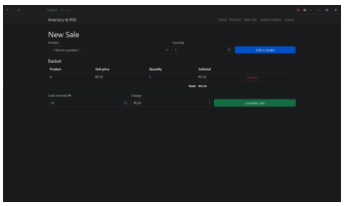
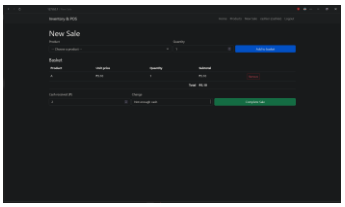
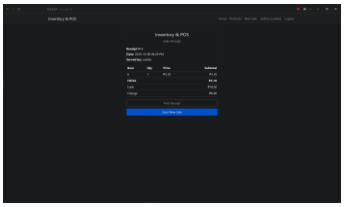
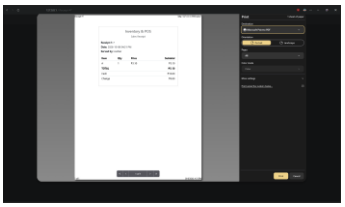
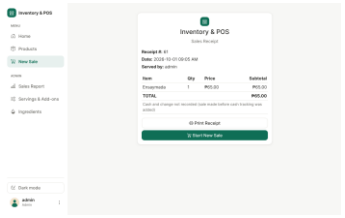
ID	Precondition	Steps / input	Expected result	Actual result	Pass?
TC-7.1	Not logged in	Open `/sales/new`	Redirect to login	302 → `/login`	✓
TC-7.2	Cashier; Coke ₱75.50, Noodles ₱15	Add Coke × 2, Noodles × 3	Two lines; total ₱271.50	As expected *(by hand: Sample Item × 1 + A × 2 = ₱26.19, screenshot S2)*	✓
TC-7.3	Coke × 2 in basket	Add Coke × 1 again	One line, × 3 (not two lines)	One line, × 3	✓
TC-7.4	Noodles: 10 in stock	Add 3 → look at dropdown; add the other 7	"7 available"; then Noodles leaves the dropdown; database stock stays 10 until the sale is completed	As expected *(by hand: A 39 / Sample Item 38 before; 37 / 37 available with A × 2 and Sample Item × 1 in the basket, screenshots S3–S4)*	✓ (DEF-10)
TC-7.5	Coke: 5 in stock, 3 in basket	Add Coke × 3	"only 2 more can be added"; basket unchanged	As expected *(by hand: "Not enough stock for A: only 39 more can be added.", screenshot S5)*	✓
TC-7.6	Cashier	Quantity 0, −2, `abc`	"Quantity must be a whole number of at least 1."	As expected	✓
TC-7.7	Cashier	No product chosen; product ID 999	"Please choose a product."	As expected	✓
TC-7.8	Soap: 0 in stock	Look at dropdown; send a hand-made request for Soap × 1	Not listed; request refused	Not listed; "only 0 more can be added"	✓
TC-7.9	Two lines in basket	**Remove** one	Line gone; total updates	As expected	✓
TC-7.10	Empty basket	**Complete Sale**	"The basket is empty."	As expected	✓
TC-7.11	Basket total ₱226.50	Cash ₱200; `abc`; nothing	"Cash received must be a number of at least ₱226.50."; no sale saved	As expected; 0 sales saved	✓
TC-7.12	Noodles × 3 in basket; stock then lowered to 2 elsewhere	**Complete Sale**	"…no longer has enough stock"; nothing saved, no stock changed	As expected	✓
TC-7.13	Basket Coke × 3 (₱226.50), Coke stock 5	Cash ₱500 → **Complete Sale**	Receipt: number, date, "Served by", items, total ₱226.50, cash ₱500.00, change ₱273.50; Coke stock 5 → 2; basket emptied	As expected *(by hand: receipt #4 ₱12.40, ₱500 cash → ₱487.60 change; receipt #6 ₱3.10, ₱10 cash → ₱6.90 change; A's stock 39 → 37 after receipts #6 and #7, screenshots S8, S1)*	✓

ID	Precondition	Steps / input	Expected result	Actual result	Pass?
TC-7.14	Basket ₱15.00	Cash exactly ₱15	Change ₱0.00	Change ₱0.00	✓
TC-7.15	Logged in as **admin**	Complete a sale	Allowed; "Served by: admin"	As expected	✓
TC-7.16	Receipt for Coke at ₱75.50 exists	Change Coke's price, then delete Coke; reopen receipt	Receipt still shows "Coke 1.5L" at ₱75.50	As expected *(since v0.10.2 a sold product can no longer be deleted; the receipt keeps the name and price from the time of sale after a rename or price change)*	✓
TC-7.17	Items in basket	Logout → log in as another user → New Sale	Basket empty	Empty	✓ (DEF-09)
TC-7.18	Two browsers logged in	Add items in one	Other browser's basket unaffected	Unaffected	✓
TC-7.19	Basket ₱151.00 (browser)	Type cash 100, then 200	Change box: "Not enough cash", then ₱49.00	As expected; no script errors *(by hand: ₱3.10 total, cash 2 → "Not enough cash", cash 10 → ₱6.90, screenshots S6–S7)*	✓
TC-7.20	Receipt open	**Print Receipt** → print preview	Only the receipt; no navbar or buttons	*(by hand, 2026-10-08)* Print preview of receipt #7 shows only the receipt card: no navigation bar, no Print/New Sale buttons (screenshot S9)	✓
TC-7.21	Receipt #1–#3, made before cash was recorded	Open the receipt	No misleading cash amount	**v0.8.0:** "Cash ₱0.00 / Change ₱0.00" · **v0.8.1:** "Cash and change not recorded…"; #4 unchanged *(checked on a copy of the real database)*	✗ → ✓ (DEF-12)
TC-7.22	Logged in	`/sales/999`; visit `/sales/complete` as a link (GET)	404; 405	404; 405	✓

ID	Precondition	Steps / input	Expected result	Actual result	Pass?
TC-7.23	Database from before v0.8.0's cash columns	**Complete Sale**	Sale saved	**Crash** "table sale has no column named cash_received" *(by hand)* → fixed by adding the columns; sale then saved	✗ → ✓ DEF-11

Hand-test screenshots (v0.8.0, 2026-10-08, logged in as cashier)

#	Test	Screenshot
S1	TC-7.13: Products page after receipts #6 and #7, stock of A reduced to 37	
S2	TC-7.2: basket with two lines and total ₱26.19	
S3	TC-7.4: dropdown before adding to the basket (A 39, Sample Item 38 available)	
S4	TC-7.4: dropdown with items in the basket (both 37 available)	
S5	TC-7.5: adding more than is available is refused	

#	Test	Screenshot
S6	TC-7.19: live change preview, ₱10 cash → ₱6.90 change	
S7	TC-7.19: cash lower than the total → "Not enough cash"	
S8	TC-7.13: receipt #6 with cash and change	
S9	TC-7.20: print preview shows only the receipt	
S10	TC-7.21 (v0.8.1): a receipt made before cash tracking says "Cash and change not recorded"	

5.9 Patches v0.8.2–v0.8.6 — robustness of the sale flow and forms

> Found by the v0.8.x stabilization phase (design review, exploratory and fuzz testing). Run on 2026-10-09 with the Flask test client, a real threaded server for the stress test, and the previous version for the "before" column.

ID	Steps / input	Expected result	Before the fix	After the fix	Pass?
TC-7.24	1 left; another sale takes it between this sale's check and save; this sale continues	Refused; stock 0	Both sales saved; DB stock 0 but really −1 (**oversold**)	Refused with message; stock 0	✗ → ✓ v0.8.2
TC-7.25	10 left; another sale of 3 lands mid-sale; this sale buys 5	Both saved; stock 2	DB says 5, really 2 (**lost update**)	Stock 2	✗ → ✓ v0.8.2
TC-7.26	**20 cashiers** press Complete Sale at the same moment; 5 in stock	Exactly 5 sold	**20 sold**, DB showed 2 left	5 sold, 15 refused, stock 0, no errors	✗ → ✓ v0.8.2
TC-7.27	Basket Bread × 2 + Milk × 1; Milk sells out mid-sale	Whole sale refused; Bread stock unchanged	Bread deducted, Milk oversold	Refused; Bread still 5; basket kept	✗ → ✓ v0.8.2
TC-7.28	Cash `nan`, `inf`, `-inf`, `1e308`, `1000000.01`	Refused with message	`nan` crashed (500); `inf`/`1e308` saved	All refused; `1000000` accepted	✗ → ✓ v0.8.3

ID	Steps / input	Expected result	Before the fix	After the fix	Pass?
TC-2.7	Price `nan`, `inf`, `1e400`, `1000000.01`; quantity `1000001` or `99999999999999999999`	Refused with message	`nan` and huge quantity crashed; `inf` saved	All refused; exactly 1,000,000 accepted	✗ → ✓ v0.8.3
TC-7.29	`/sales/99999999999999999999`, `/products/edit/…`, delete, add to basket with that id	404 / "Please choose a product"	Server crash (500)	404 / message; cashier still gets 403 first	✗ → ✓ v0.8.4
TC-7.30	Admin deletes a product that is in a cashier's basket; cashier refreshes or clicks Complete Sale	Product taken out, sale can finish	Basket stuck: hidden line, Complete Sale always failed	Removed with a note; total updated; sale completes	✗ → ✓ v0.8.5
TC-2.8	Name of 101 characters; SKU or category of 51	Refused with message	5,000-character name saved	Refused; exactly 100/50/50 accepted	✗ → ✓ v0.8.6
TC-7.31	Fuzz: 14 odd inputs (empty, spaces, `abc`, `nan`, huge, full-width digits, `<script>`, SQL text, 5,000 characters) on every form and address	No crash; nothing invalid stored	—	0 crashes; `<script>` shown as text; SQL text stored as plain text (no **SQL injection**)	✓

5.10 F8 — Low-stock warning (v0.9.0, v0.9.1, v0.9.2)

ID	Precondition	Steps / input	Expected result	Actual result	Pass?
TC-8.1	Product with 0	Open Products	Red "Out of stock" badge; listed first in the box	As expected	✅
TC-8.2	Product with exactly 5	Open Products	Yellow "Low" badge; in the box (5 counts as low)	As expected	✅
TC-8.3	Product with 6	Open Products	No badge; not in the box	As expected	✅
TC-8.4	Logged in as cashier	Open Products	Same box and badges	As expected	✅
TC-8.5	Product with 6	Sell 1 through New Sale	It appears in the box ("5 left")	As expected	✅
TC-8.6	All low products restocked above 5	Open Products	No box at all	As expected	✅
TC-8.7	Empty product list; product added with 3; low product deleted	Open Products	No crash; new product flagged; deleted one gone	As expected	✅
TC-8.8	40 products, 35 of them low, long names	Open Products	Box stays readable; table visible	**v0.9.0:** ~4,500-character box filled the screen · **v0.9.1:** 5 names (shortened) + "and 30 more"	❌ → ✅ DEF-18
TC-8.9	Product without category	Open Products	Empty cell or dash	**v0.9.0:** word "None" · **v0.9.2:** dash	❌ → ✅ DEF-19

5.11 F9 — Sales report (v0.10.0, v0.10.1, v0.10.2)

ID	Precondition	Steps / input	Expected result	Actual result	Pass?
TC-9.1	Cashier / visitor	Open `/reports/sales`	403 / login page; cashier has no navbar link	As expected	✅
TC-9.2	Sales today at 00:01 and now; one at 23:59 yesterday	Report for today	Only today's two counted (revenue, sales, items)	₱231.00, 2 sales, 10 items	✅
TC-9.3	Same data	Report for yesterday only; 4-day range	Correct totals	₱755.00 / 1 sale; ₱1,061.50 / 4 sales	✅
TC-9.4	Same data	To date before From date	Dates swapped automatically	Same result as correct order	✅
TC-9.5	—	Dates `2026-13-45`, `2026-02-30`, `1999-12-31`, `2101-01-01`, `9999-12-31`	Message; today's report	Message + today *(the 9999 date crashed before release: DEF-24)*	✅
TC-9.6	No sales in period	Report for 2020	₱0.00, 0 sales, "No sales in this period."	As expected	✅
TC-9.7	Bread sold 4; Cola sold 3, renamed "Cola 1.5L", sold 2 more	Top products	Cola 1.5L first with 5 units	**v0.10.0:** "Cola" 3 and "Cola 1.5L" 2 separately; Bread wrongly #1 · **v0.10.1:** one row, 5 units, #1	❌ → ✅ DEF-20
TC-9.8	Product with sales history	Admin clicks Delete	Refused with message; product kept	**v0.10.0:** deleted; its id could be reused by a new product that "inherits" the sales · **v0.10.2:** refused, message shown once	❌ → ✅ DEF-21
TC-9.9	Product never sold	Admin clicks Delete	Deleted	Deleted	✅
TC-9.10	Bread's sales exist; Bread deleted *(before v0.10.2)*	Report	Its sales still counted	₱120.00 kept	✅

5.12 Demo data (v0.11.0, v0.11.1, v0.11.2)

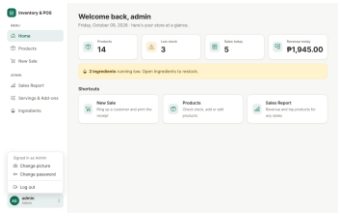
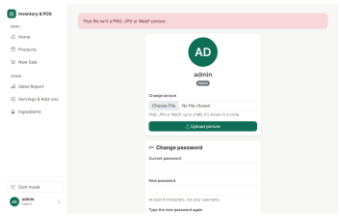
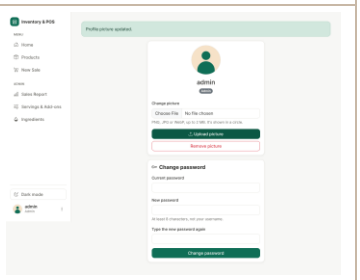
ID	Steps / input	Expected result	Actual result	Pass?
TC-10.1	Fresh database → `python seed_demo_data.py`	15 products, 2 users, past sales; 4 low products	15 products, 27 sales over 7 days (₱6,904.50), 4 low	✅

ID	Steps / input	Expected result	Actual result	Pass?
TC-10.2	Run it a second time	Nothing added twice	"(sales already exist…)"	✅
TC-10.3	Sales report for the demo week	Receipt numbers increase with time	**v0.11.0:** #25 at 9:27 AM after #23 at 6:13 PM · **v0.11.1:** in time order	❌ → ✅ DEF-22
TC-10.4	Any amount ≥ ₱1,000 (report, receipt, products, change preview, messages)	Thousands separator	**v0.11.1:** ₱6904.50 · **v0.11.2:** ₱6,904.50	❌ → ✅ DEF-23

5.13 F10 — Profile picture (v0.12.0)

ID	Precondition	Steps / input	Expected result	Actual result	Pass?
TC-11.1	User without a picture	Open any page	Circle with initials (admin → AD) and the avatar menu	As expected	✅
TC-11.2	Logged in	Upload a PNG	Circle shows the picture; file saved under a random name	`user1_<random>.png` saved and shown	✅
TC-11.3	Picture already set	Upload a JPG	New picture shown; old file deleted	As expected	✅
TC-11.4	Picture set	Click Remove picture	Initials again; file deleted	As expected	✅
TC-11.5	Logged in	Upload a text file renamed to `.png`	Refused: "isn't a PNG, JPG or WebP picture"	Refused (the file's first bytes are checked, not its name)	✅
TC-11.6	Logged in	Upload a GIF	Refused with the same message	Refused	✅
TC-11.7	Logged in	Upload a 3 MB file	"That picture is too big. The limit is 2 MB."; no crash	As expected	✅
TC-11.8	Database from v0.11.2 (no `avatar` column)	Start the app	Column added automatically; old sales still work	Added by `upgrade_database()`; cash sale OK	✅
TC-11.9	Picture file deleted from the server by hand	Open any page	Initials instead of a broken image	As expected	✅
TC-11.10	Logged in	Open `/profile/remove` in the address bar (GET)	405, picture kept	405	✅
TC-11.11	Visitor	Open `/profile`	Login page	As expected	✅

Screenshots: profile picture (v0.12.0)

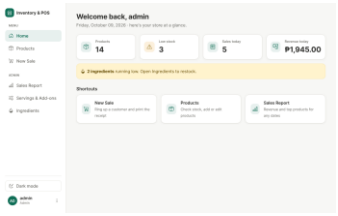
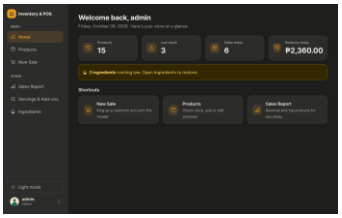
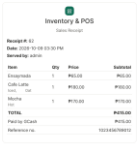
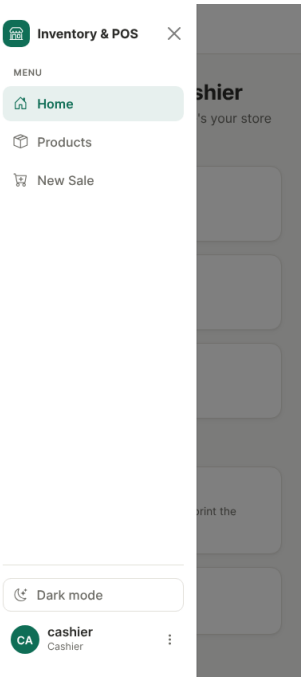
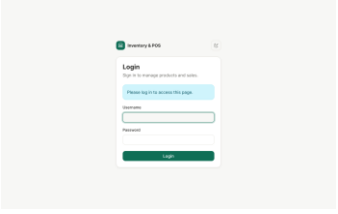
#	Test	Screenshot
S11	TC-11.1: no picture yet, initials "AD"; the avatar menu with Change picture, Change password, Log out	
S12	TC-11.5: a text file named `.png` is refused	
S13	TC-11.2: picture uploaded and shown in a circle (page and sidebar)	

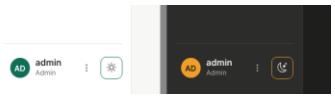
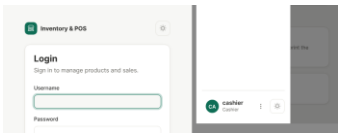
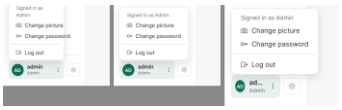
5.14 F11 — Modern theme (v0.13.0)

ID	Precondition	Steps / input	Expected result	Actual result	Pass?
TC-12.1	Admin	Open Products, then Add Product	Sidebar highlights **Products** on both pages	As expected	✓
TC-12.2	Cashier	Look at the sidebar	No Sales Report link	As expected	✓
TC-12.3	Any page	Click **Dark mode**, then reload	Dark theme with amber accent; still dark after reload	As expected	✓
TC-12.4	Dark mode	Print a receipt	Printout in light colours, no sidebar; dark again after printing	As expected	✓
TC-12.5	Window 390 px wide (phone)	Open Home, tap ☰	Top bar with menu button; sidebar slides in	As expected	✓

ID	Precondition	Steps / input	Expected result	Actual result	Pass?
TC-12.6	Font server slow or unreachable	New Sale: type the cash received	Change preview updates	**v0.13.0 draft:** stuck at ₱0.00 · **released v0.13.0:** updates	✗ → ✓ DEF-25
TC-12.7	Demo data	Admin opens Home	Products 15, Low stock 4, today's sales and revenue match the report for today	As expected	✓
TC-12.8	Visitor	Open `/products`	Login page with a styled "Please log in" message	**v0.11.2:** message shown as plain text · **v0.13.0:** blue info box	✗ → ✓ DEF-26
TC-12.9	All earlier tests	Run the full regression suite	Everything still passes	All 10 suites + 17 avatar checks pass	✓
TC-12.10	Light theme, logged in	Look at the bottom of the sidebar; hover the icon; click it; reload	Only a sun icon (no words) beside the user's name, tooltip "Switch to dark mode"; after the click a moon and "Switch to light mode"; still dark after reload	**v0.19.0:** a wide button with the words "Dark mode" · **v0.19.1:** as expected	✓
TC-12.11	Login page and phone (390 px)	Look at the switch; tap it	Same icon rule (sun = light, moon = dark), 40 × 40 px, works on the phone menu	As expected	✓
TC-12.12	Screen reader user	Read the switch's label	"Switch to dark mode" / "Switch to light mode" (the icon alone would say nothing)	As expected	✓
TC-12.13	Logged in (100 %, 110 %, 125 % text size; phone)	Open the avatar menu	Every line fits inside the menu, "Signed in as Admin" on one line	**v0.19.1:** "Change picture" and "Change password" ran past the menu's right edge (menu squeezed to 167 px) · **v0.19.2:** fits at every size (209 px at 100 %)	✗ → ✓ DEF-46

Screenshots: modern theme (v0.13.0)

#	Test	Screenshot
S14	TC-12.3 / TC-12.7: Home in the light theme with today's numbers and the low-ingredient warning	
S15	TC-12.3: the same page in the dark theme	
S16	TC-12.4: printing a receipt from dark mode gives a light page without the sidebar	
S17	TC-12.5: phone (390 px), the sidebar slides in from the menu button	
S18	TC-12.8: "Please log in" as a blue info box	

#	Test	Screenshot
S43	TC-12.10 (v0.19.1): the switch beside the user's name, sun in the light theme (left, with its hover border) and moon in the dark theme (right)	
S44	TC-12.11 (v0.19.1): the same icon on the login page and in the phone menu	
S45	TC-12.13: avatar menu in v0.19.1 (left: words run past the edge), v0.19.2 (middle) and v0.19.2 with 125 % text (right)	

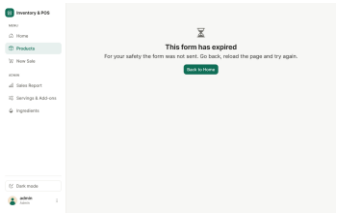
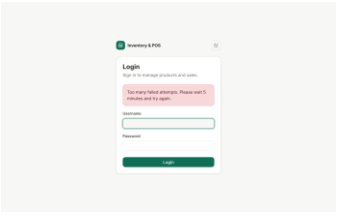
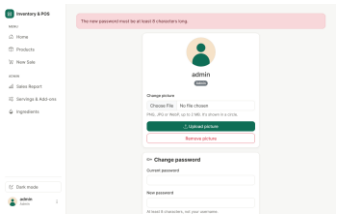
5.15 F12 — Security patches (v0.13.1–v0.13.5)

Each row is an attack or misuse tried on purpose (on the code published as v0.13.0), then again after the fix.

ID	Attack / input	Expected result	Actual result	Pass?
TC-13.1	Build a login cookie with the secret key published on GitHub, open the Sales Report	Refused	**v0.13.0:** Sales Report opened as admin with no password · **v0.13.1:** sent to the login page	❌ → ✅ DEF-07
TC-13.2	Restart the app	Same secret key (users stay logged in); `instance/` ignored by git; `SECRET_KEY` environment variable wins	As expected	✅
TC-13.3	Logged in as admin, send "Add product" without the hidden token / with a wrong token	Refused (400), nothing added	**v0.13.0:** product added · **v0.13.2:** 400 "This form has expired", nothing added	❌ → ✅ DEF-08
TC-13.4	Every POST form on Products, New Sale, Profile, Add, Edit	Each carries a token	All forms counted: tokens = forms	✅
TC-13.5	Open `/logout` as a link (GET)	Not allowed (405); logout only by button	**v0.13.0:** logged out · **v0.13.2:** 405	❌ → ✅ DEF-28
TC-13.6	Session cookie	`HttpOnly` and `SameSite=Lax`	As expected	✅

ID	Attack / input	Expected result	Actual result	Pass?
TC-13.7	200 wrong passwords for admin, then the right one	Locked after 5 tries	**v0.13.0:** right password accepted after 200 guesses · **v0.13.3:** 6th try → 429 "Too many failed attempts"	❌ → ✅ DEF-27
TC-13.8	Locked cashier; admin logs in from elsewhere / 4 wrong then right / wait 5 minutes	Other accounts unaffected; 4 wrong still OK; lock ends	As expected	✅
TC-13.9	Page headers	`X-Frame-Options: DENY`, `X-Content-Type-Options: nosniff`, `Referrer-Policy: same-origin`	**v0.13.0:** none · **v0.13.4:** all three	❌ → ✅ DEF-29
TC-13.10	Start with `python app.py`	Debug console off unless `FLASK_DEBUG=1`	**v0.13.0:** `debug=True` · **v0.13.4:** off	❌ → ✅ DEF-30
TC-13.11	Change password: wrong current / not matching / same as username / shorter than 8	Refused with a message each	As expected	✅
TC-13.12	Change password correctly, then log in with old and new	Old refused, new works	As expected	✅ (DEF-31 fixed)
TC-13.13	Log in with `admin' OR '1'='1` (SQL injection)	Refused	Refused (SQLAlchemy sends input as data)	✅
TC-13.14	Product named `<script>alert(1)</script>` (XSS)	Shown as text	Shown as text (Jinja escapes it)	✅

Screenshots: security patches (v0.13.2–v0.13.5)

#	Test	Screenshot
S19	TC-13.3: a form sent without its security token is refused ("This form has expired")	
S20	TC-13.7: after 5 wrong passwords the login waits 5 minutes	
S21	TC-13.11: a new password shorter than 8 characters is refused by the server (the browser's own check was switched off for this test)	

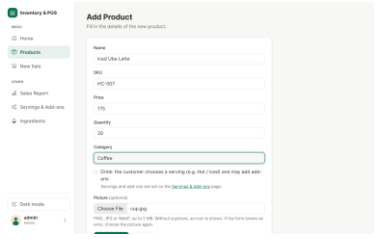
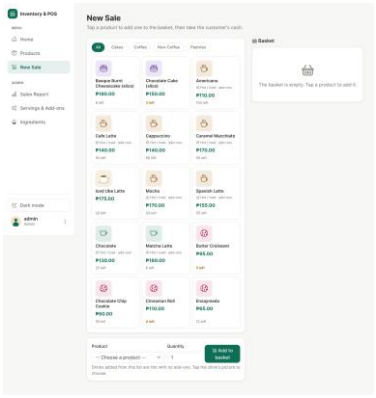
5.16 F13 — Product pictures and café menu (v0.14.0–v0.14.3)

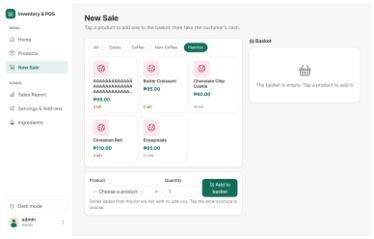
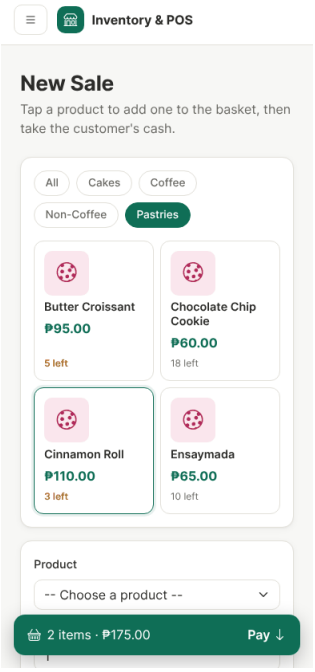
ID	Precondition	Steps / input	Expected result	Actual result	Pass?
TC-14.1	Database from v0.13 (no `image` column)	Start the app	Column added automatically	Added	✅
TC-14.2	Admin	Add a product with a PNG	Saved under a random name in `static/products/`; shown in the list	As expected	✅
TC-14.3	Admin	Add with a text file named `.png` / a GIF on edit	Refused with a message; nothing saved	As expected	✅
TC-14.4	Admin	Add with no file	Allowed; icon tile shown	As expected	✅
TC-14.5	Products named Iced Latte, Butter Croissant, Chocolate Cake, Americano, Hot Chocolate, Bottled Water	Look at the icons	Iced cup, pastry, cake, hot cup, cup, box	As expected	✅
TC-14.6	Product with a picture	Edit without a file / with a JPG / tick "Remove the picture"	Kept / replaced (old file deleted) / removed (file deleted)	As expected	✅

ID	Precondition	Steps / input	Expected result	Actual result	Pass?
TC-14.7	Admin	3 MB picture on Add Product	Back to Add Product with "too big"	**Draft:** sent to the profile page · **v0.14.0:** back to the form	❌ → ✅ (fixed before release)
TC-14.8	Product never sold / with sales	Delete it	Picture deleted / product and picture kept	As expected	✅
TC-14.9	Picture file missing, or name set to `../../app.py`	Show the product; delete it	Icon tile; `app.py` untouched	As expected	✅
TC-14.10	Duplicate SKU + new picture on Edit	Save	Error; no stray picture file	As expected	✅
TC-14.11	Café demo data	New Sale	One tile per product in stock; sold-out Ube Cheesecake not shown; category buttons	As expected	✅
TC-14.12	New Sale	Tap a tile	1 added; "left" count goes down	As expected	✅
TC-14.13	Product name of 100 letters without spaces	New Sale and Products	Wraps inside its tile / cell	**v0.14.0:** spilled over the basket · **v0.14.1:** wraps	❌ → ✅ DEF-32
TC-14.14	Menu with drinks and pastries	Look at the order; tap a pastry while "Pastries" is chosen	Grouped by category; Pastries still chosen	**v0.14.1:** A–Z mixed, filter reset to All after each tap · **v0.14.2:** as expected	❌ → ✅ DEF-33, DEF-34
TC-14.15	Phone (390 px), 2 items in basket	Look at the screen; tap the bar	Bar "2 items · ₱130.00 — Pay"; jumps to Complete Sale	**v0.14.2:** basket only after scrolling past the whole menu · **v0.14.3:** as expected	❌ → ✅ DEF-35

ID	Precondition	Steps / input	Expected result	Actual result	Pass?
TC-14.16	Phone, basket table	Read the names	Normal names on one line	**v0.14.1–v0.14.2:** "Ensaymada" one letter per line · **v0.14.3:** fixed	❌ → ✅ DEF-36
TC-14.17	`seed_demo_data.py --fresh` at 3:30 PM	Run it	Old database renamed as a backup; 22 café items; sales for 7 days + today until 3:30 PM only; receipt numbers in time order	As expected (today: 4 sales, 7:09 AM–2:32 PM)	✅
TC-14.18	All earlier tests	Full regression (10 suites + avatar + security + pictures)	Everything passes	As expected	✅
TC-14.19	Window 992–1360 px wide, 3 items in the basket	Look at the basket	Remove (×) button visible on every line	**v0.14.3:** cut off at the right edge · **v0.14.4:** visible at 390, 768, 992, 1100, 1199, 1200 and 1360 px	❌ → ✅ DEF-37

Screenshots: product pictures and menu (v0.14.0–v0.14.3)

#	Test	Screenshot
S22	TC-14.2: adding a product with a picture	
S23	TC-14.11: New Sale menu: picture or icon tiles, category buttons, "left" counts	

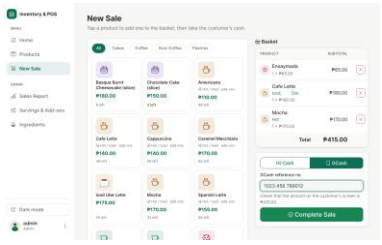
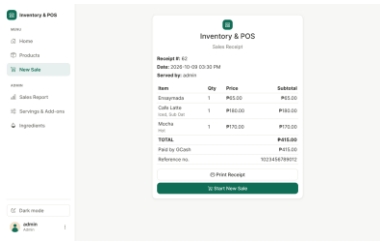
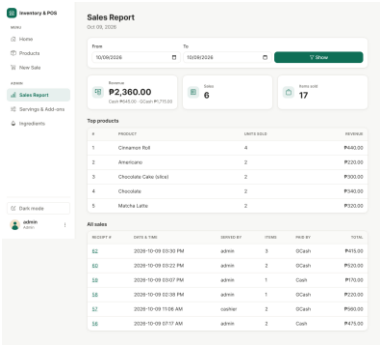
#	Test	Screenshot
S24	TC-14.13 (v0.14.1): a 100-letter name wraps inside its tile	
S25	TC-14.15 (v0.14.3): phone basket bar "2 items · ₱175.00 — Pay"	

5.17 F14 — Payment method (v0.15.0)

ID	Precondition	Steps / input	Expected result	Actual result	Pass?
TC-15.1	Database from v0.14	Start the app	`payment_method`, `payment_reference` and a unique index added; old sales count as cash	As expected	✓
TC-15.2	Basket ₱360	Pay cash ₱400 (old form, no method sent)	Cash, change ₱40	As expected	✓
TC-15.3	Basket ₱360	GCash, reference `1234 567 890123`	Saved as 13 digits, cash = total, change 0, stock deducted	As expected	✓
TC-15.4	GCash sale	Open the receipt	"Paid by GCash" + reference, no Change line	As expected	✓
TC-15.5	Basket	GCash references: 12 digits, 14 digits, letters, Arabic digits, empty	Refused, nothing saved, stock unchanged	As expected (5 cases)	✓

ID	Precondition	Steps / input	Expected result	Actual result	Pass?
TC-15.6	Reference already used	GCash with the same reference	Refused, names the earlier receipt	As expected	✓
TC-15.7	Two sales with the same reference at the same moment	Save both	Database refuses the second (unique index)	As expected	✓
TC-15.8	Basket	Method `card` (not offered)	Refused	As expected	✓
TC-15.9	Sales of both kinds	Sales Report	Revenue split "Cash ₱… · GCash ₱…"; Paid by column	As expected	✓
TC-15.10	Browser	Tap GCash, then Cash	Cash boxes hide/show; empty reference blocked by the browser; change preview still works	As expected	✓

Screenshots: payment method (v0.15.0)

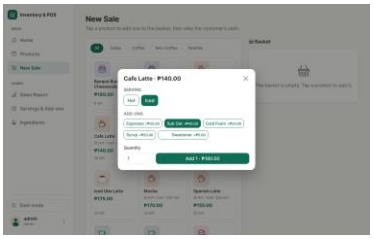
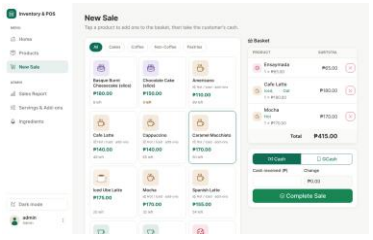
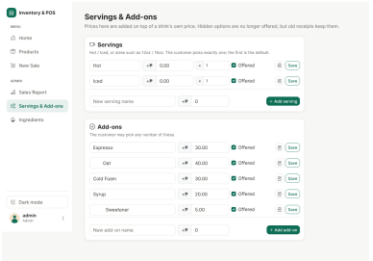
#	Test	Screenshot
S26	TC-15.10: GCash chosen: the cash boxes are replaced by the 13-digit reference	
S27	TC-15.4: receipt "Paid by GCash" with the reference, options under each drink	
S28	TC-15.9: Sales Report revenue split "Cash ₱… · GCash ₱…" and a Paid by column	

5.18 F15 — Sizes and add-ons (v0.16.0, v0.16.1)

ID	Precondition	Steps / input	Expected result	Actual result	Pass?
TC-16.1	Database from v0.15	Start the app	`menu_option` table, `has_options`, `options` columns added	As expected	✅
TC-16.2	Admin	Add sizes and add-ons; duplicate name, negative/nan price, 51-char name, unknown kind	Saved / each refused with a message	As expected	✅
TC-16.3	Cashier	Open or post to Sizes & Add-ons	403	403	✅
TC-16.4	Honey Latte ₱180	2 × 16oz + Oat Milk + Espresso	One line, ₱270.00 each, ₱540.00	As expected	✅
TC-16.5	Same choice ticked in another order	Add	Joins the same line	As expected	✅
TC-16.6	3 in stock, 2 in basket (any sizes)	Add 2 more	Refused: only 1 more	As expected	✅
TC-16.7	—	Size id of an add-on, unknown size, size as add-on, non-number add-on	Refused	As expected	✅
TC-16.8	List form (no size sent) / croffle with options sent	Add	Default size at base price / options ignored	As expected	✅
TC-16.9	Two lines of one product	Remove one line	Only that line removed	As expected	✅
TC-16.10	Basket with options	Complete; then raise the add-on price	Receipt keeps options + price paid; stock reduced by all lines; report counts the product once	As expected	✅
TC-16.11	Line in basket	Admin unticks "sizes and add-ons" on that product	Line taken out with a message	**v0.16.0:** sold as "16oz" at the size price · **v0.16.1:** taken out	❌ → ✅ DEF-38
TC-16.13	Servings Hot and Iced (₱0); Iced recipe: 150 g ice + 1 iced cup	Iced Honey Latte + Oat Milk, and a Hot Mocha; complete	₱220.00 and ₱190.00; ice −150 g, iced cups −1, hot cups −1, oat milk −180 ml, fresh milk unchanged	As expected (v0.17.2)	✅

ID	Precondition	Steps / input	Expected result	Actual result	Pass?
TC-16.14	Café menu	Look at the tiles and the drink window	Tiles say "Hot / Iced · add-ons"; window says "Serving"; admin page "Servings & Add-ons"	**v0.16.0–v0.17.1:** "Size" · **v0.17.2:** as expected	✗ → ✓ DEF-41
TC-16.12	6 add-ons with 50-character names	Complete the sale	Options text fits the 200-character column	**v0.16.0:** 316 characters saved · **v0.16.1:** shortened with "…"	✗ → ✓ DEF-39

Screenshots: servings and add-ons (v0.16.0–v0.17.2)

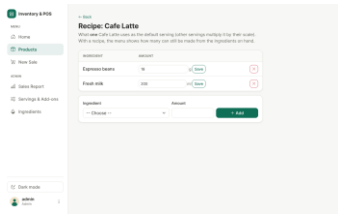
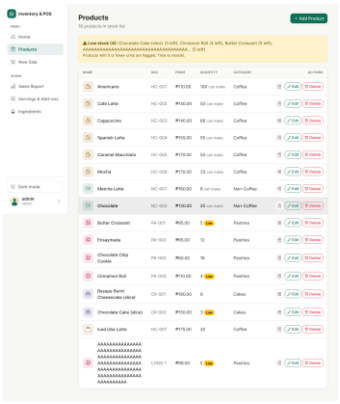
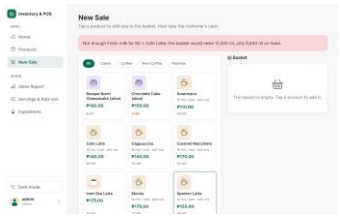
#	Test	Screenshot
S29	TC-16.13: the drink window: Iced + Oat Milk, live price ₱180.00	
S30	TC-16.13: basket lines "Iced, Oat Milk" and "Hot" with their own prices	
S31	TC-16.14 (v0.17.2): the Servings & Add-ons page with Hot / Iced and the upgrades	

5.19 F16 — Ingredients and recipes (v0.17.0, v0.17.1)

ID	Precondition	Steps / input	Expected result	Actual result	Pass?
TC-17.1	Admin	Add ingredients; duplicate name, unit kg, negative warn-at	Saved / refused	As expected	✓
TC-17.2	Ingredient	Restock +1000; restock 0, -5, nan, abc, 1e12	Added and written in the history / refused	As expected	✓
TC-17.3	Cream 400 ml	Count = 380	Set to 380, history shows -20	As expected	✓
TC-17.4	Recipes	Product: negative amount, same ingredient twice / add-on: negative amount	Refused, refused / allowed (replaces milk)	As expected	✓
TC-17.5	Milk 1,000 ml; Honey Latte uses 180 ml	Products list and New Sale	"5 can make", Low badge; tile "5 left"	As expected	✓
TC-17.6	2 × 16oz (scale 1.5) in the basket	New Sale	"2 left" (460 ml ÷ 180)	As expected	✓
TC-17.7	Not enough milk for 2 more	Add	Message names the ingredient and amounts	"Not enough Fresh milk … 1,080 ml, only 1,000 ml on hand"	✓
TC-17.8	Basket: 2 × 16oz, 1 × Oat Milk, 1 croffle	Complete	Milk 460, oat 1,820, beans 928, cream 220; croffle 3 → 2; history lines linked to the receipt	As expected	✓
TC-17.9	Milk drops to 100 ml before checkout	Complete	Refused, nothing saved	As expected	✓
TC-17.10	Another till uses up every ingredient between the check and the save	Complete	Whole sale cancelled, croffle stock restored	As expected	✓
TC-17.11	20 cashiers sell a latte at the same moment, milk for 5	Complete all	Exactly 5 sold, milk 0, never negative, history adds up	5 completed, 15 refused, 0 errors	✓
TC-17.12	Cashier	Ingredients / recipe pages; another recipe's line; unknown kind	403 / 404 / 404	As expected	✓
TC-17.13	Product with a recipe, never sold	Delete it, then add a new product (SQLite reuses the id)	New product has no recipe	**v0.17.0:** new cookie inherited the recipe (could sell 4 instead of 20) · **v0.17.1:** no recipe	✗ → ✓ DEF-40

ID	Precondition	Steps / input	Expected result	Actual result	Pass?
TC-17.14	Café menu in `local_demo/` (every product has a recipe, quantity 0)	`python seed_demo_data.py --fresh`	Real demo sales with items and totals	**v0.17.2:** 56 receipts of ₱0.00 with no items (Home showed 8 sales but only the newest one's revenue) · **v0.17.3:** 52 real sales	✗ → ✓ DEF-42

Screenshots: ingredients and recipes (v0.17.0)

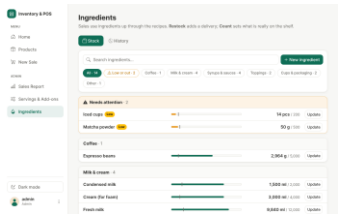
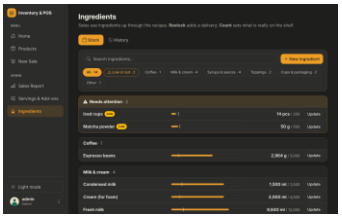
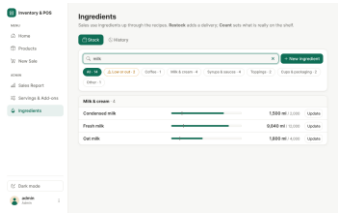
#	Test	Screenshot
S32	TC-17.4: a product's recipe (ingredients grouped in the list since v0.18.0)	
S33	TC-17.5 (+ v0.18.2): Products shows "can make" counts worked out from the ingredients	
S34	TC-17.7: 60 lattes need 12,000 ml of milk, only 9,840 ml on hand: refused with the amounts	

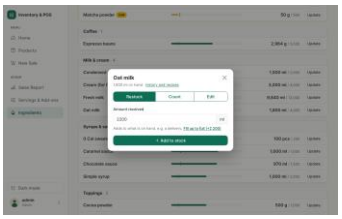
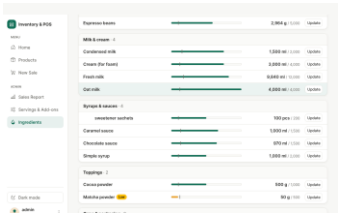
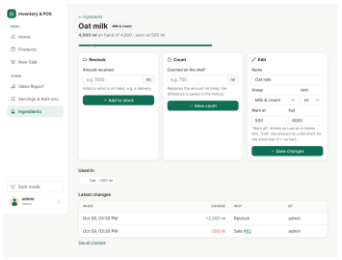
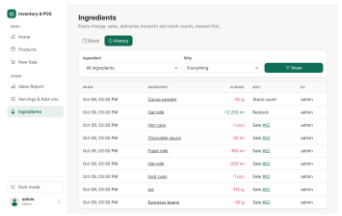
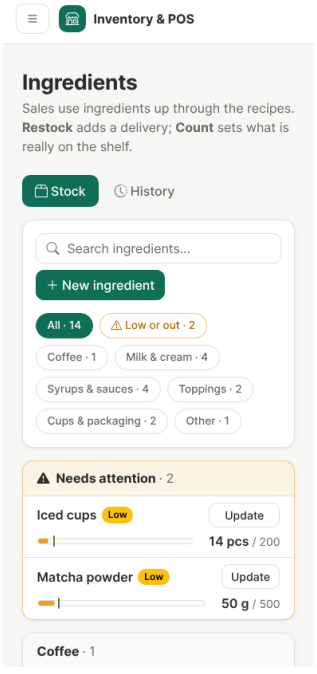
5.20 F17 — Ingredients page redesign (v0.17.4, v0.18.0, v0.18.1)

ID	Precondition	Steps / input	Expected result	Actual result	Pass?
TC-18.1	Fresh milk warn at 2,000 ml; recipe line of 1,500 ml	Open Edit / the recipe	Number boxes show 2000 / 1500	**v0.17.3:** boxes empty ("2,000" can't be read), so the Edit form could not be saved · **v0.17.4:** 2000 / 1500	✗ → ✓ DEF-43
TC-18.2	Database from v0.17 (no group / full columns)	Start the app	Columns added; Oat milk → Milk & cream, Iced cups → Cups & packaging, Ice → Other…; full = on hand (0 if already low)	As expected	✓
TC-18.3	8 ingredients	Open Ingredients	Groups in the fixed order; Needs attention lists Out first, then Low; buttons with counts	As expected	✓
TC-18.4	Oat milk 2,000 of 4,000 ml	Look at the row	Bar 50 %, mark at the warn level	As expected	✓
TC-18.5	Admin	Full ≤ warn at / unknown group / full 0	Refused / refused / allowed with "No full level set"	As expected	✓
TC-18.6	Café menu (28 ingredients)	Search "milk"; filter Toppings; filter Low or out; search "zzz"	3 milks; 9 toppings; 1; "No ingredient matches"	As expected	✓
TC-18.7	Café menu	Update on Oat milk → Fill up to full → Add to stock	Window shows Oat milk / ml; box gets 2000; "Added 500 ml" after typing 500	As expected	✓
TC-18.8	Name `Joe's "special" <b>syrup</b>`	Open Update, save a count	Shown as plain text everywhere	As expected	✓
TC-18.9	Admin	Ingredient page: restock; `return_to` set to a web address	Back to that page; web address ignored	As expected	✓
TC-18.10	Cashier	Open Ingredients, an ingredient page, History	403 each	As expected	✓
TC-18.11	Changes made	History filtered by Cocoa powder + Stock count; junk filter values	Only that count; junk values show everything, no crash	As expected	✓

ID	Precondition	Steps / input	Expected result	Actual result	Pass?
TC-18.12	Filter "Milk & cream"	Restock Whipped cream	Back at the same row, filter still on, row in view	**v0.18.0:** back at the top, filter cleared · **v0.18.1:** as expected	✗ → ✓ DEF-44
TC-18.13	Phone (390 px)	Open Ingredients and Update	No sideways scrolling; window fits	As expected	✓
TC-18.14	All earlier tests	Full regression (19 suites)	Everything passes	As expected	✓
TC-18.15	Products with "can make" counts	Look at the Products table	SKU and count on one line each	**v0.18.1:** "HC-" / "001" and "100 can" / "make" on two lines · **v0.18.2:** one line	✗ → ✓ DEF-45

Screenshots: Ingredients page (v0.18.0–v0.18.1)

#	Test	Screenshot
S35	TC-18.3: groups, Needs attention (Iced cups, Matcha powder low), stock bars with the warn mark	
S36	TC-18.3: the same page in the dark theme	
S37	TC-18.6: search "milk" shows the 3 milks	

#	Test	Screenshot
S38	TC-18.7: Update window on Restock after "Fill up to full" (+2,200 ml)	
S39	TC-18.12 (v0.18.1): after the restock the list comes back to the Oat milk row, highlighted, now full	
S40	TC-18.9: an ingredient's own page: Restock, Count, Edit, Used in, Latest changes	
S41	TC-18.11: History tab: the count, the restock and the ingredients used by sale #62	
S42	TC-18.13: phone (390 px): name and Update on top, bar and amount below, no sideways scrolling	

> **How these were run (F8–F10 and the patches):** on 2026-10-09 with the Flask test client, a threaded server for TC-7.26, and Chromium for the screen checks (TC-8.8, TC-10.3, TC-10.4) and the change preview. Every version was also re-run against all earlier tests.

5.21 F18 — Project rules and automated tests (v0.19.0)

Mutation testing (TC-19.5 to TC-19.13): to prove the tests really catch mistakes, **deliberate bugs** were put into a copy of the code, one at a time, and ran the tests. A test that still passes with a bug in the code is useless; each bug below was caught. "Expected result" here means *the tests fail* (red), which is the correct outcome.

ID	Precondition	Steps / input	Expected result	Actual result	Pass?
TC-19.1	v0.19.0	`python tests/run_tests.py`	Contract check + 15 test files pass (333 checks), about 40 s	"All 16 passed"	✓
TC-19.2	Real project folder	`python tests/test_sales.py` (run a test file directly)	Refused: "Run the tests with: python tests/run_tests.py"; nothing created	As expected	✓
TC-19.3	Real `instance/inventory.db` and an uploaded picture	Run the tests	Database and picture byte-for-byte unchanged (sha1 compared)	Unchanged	✓
TC-19.4	New computer: empty virtual environment	`pip install -r requirements.txt`, then run the tests	Everything passes with only the listed packages	As expected	✓
TC-19.5	Copy of the code	Remove `Product.quantity >= quantity` from the stock update (race guard)	`test_race.py` fails (overselling)	Caught by test_race	✓
TC-19.6	Copy of the code	Remove the same guard for ingredients	Tests fail	Caught by test_race, test_ingredients	✓
TC-19.7	Copy of the code	Remove `@admin_required` from Add Product	Contract + tests fail (cashier could add products)	Caught by check_contract, test_products, test_product_pictures	✓
TC-19.8	Copy of the code	Delete the CSRF token from the login form	Contract + tests fail	Caught by check_contract, test_security	✓
TC-19.9	Copy of the code	Let number boxes show `2,000` again (DEF-43)	Tests fail	Caught by test_number_boxes, test_ingredients_page	✓
TC-19.10	Copy of the code	Login limit 50 tries instead of 5	Tests fail	Caught by test_security	✓
TC-19.11	Copy of the code	Add `requests` to requirements.txt	Contract fails (D1: package not allowed)	Caught by check_contract	✓
TC-19.12	Copy of the code	Receipt shows the product's *current* name instead of the name at the time of sale	Tests fail (old receipts would change)	Caught by test_sales, test_payment, test_options	✓
TC-19.13	Copy of the code	Add a new page `/secret-export` without any login check	Contract fails: "new page is not listed" (S1)	As expected	✓

ID	Precondition	Steps / input	Expected result	Actual result	Pass?
TC-19.14	Pull request #49 on GitHub	Open the pull request	"Checks" runs and shows a green tick	Green tick ("tests" passed), and on every later pull request up to #70	✅

5.22 Security audit patches (v0.19.3)

The security audit of 2026-10-11 (`docs/security_audit_2026-10-11.md`) found these; each attack was run again on v0.19.3 and is now blocked. The automated checks are in `test_limits.py` and `test_security.py`.

ID	Precondition	Steps / input	Expected result	Actual result	Pass?
TC-20.1	A drink made from a recipe	New Sale: quantity of 309 or 400 digits	Message "Quantity can be at most 1,000,000", no crash	**v0.19.2:** *Internal Server Error* (500) · **v0.19.3:** message	❌ → ✅ DEF-47
TC-20.2	Same drink	Quantity 1,000,001; then 3	Refused with the same message; 3 is added normally. Both quantity boxes have `max="1000000"`	As expected	✅
TC-20.3	Login page	50 wrong log-ins with 450 KB usernames	Memory stays flat (each username remembered as at most 50 characters)	**v0.19.2:** 33 MB kept, growing · **v0.19.3:** about 1.5 MB, the same after 200 tries	❌ → ✅ DEF-50
TC-20.4	Wrong usernames older than 5 minutes	Any new log-in attempt	Old entries forgotten for every username, not only the one typed; the 5-try limit still works	**v0.19.2:** 3,000 random usernames stayed forever · **v0.19.3:** cleared	❌ → ✅ DEF-50

ID	Precondition	Steps / input	Expected result	Actual result	Pass?
TC-20.5	`requirements.txt`	`pip-audit`; run all tests on the new version	No known vulnerability; Werkzeug 3.1.9; everything passes	**v0.19.2:** CVE-2026-102598 in Werkzeug 3.1.8 · **v0.19.3:** none	✗ → ✓ DEF-49
TC-20.6	All earlier tests	`python tests/run_tests.py`	All 17 pass	As expected	✓

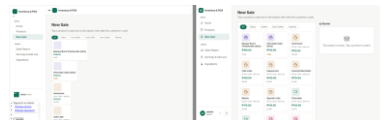
5.23 Offline pages (v0.19.4)

The presentation venue's internet may fail. These tests were run in Chromium with **every request to the internet blocked**; `test_offline.py` checks the same automatically.

ID	Precondition	Steps / input	Expected result	Actual result	Pass?
TC-21.1	Login, Home, Products, New Sale, Ingredients, Report	Look for addresses outside the app in each page	None; every style sheet, script and font comes from `/static/` and exists	**v0.19.3:** 4 files from jsDelivr / Google Fonts · **v0.19.4:** none	✗ → ✓ DEF-53
TC-21.2	Internet blocked	Open New Sale	Menu grid (3 tiles per row), icons and the Inter font as usual	**v0.19.3:** one tile per row, no icons, plain font · **v0.19.4:** as expected	✗ → ✓ DEF-53

ID	Precondition	Steps / input	Expected result	Actual result	Pass?
TC-21.3	Internet blocked	Open the avatar menu; switch the theme; open the phone menu (390 px)	Menu hidden until clicked; sun/moon icon visible; phone menu slides in	**v0.19.3:** avatar menu always open, icon missing · **v0.19.4:** as expected	✗ → ✓ DEF-53
TC-21.4	Internet blocked	Sales Report	~~₱~~ shown in the Inter font; 0 requests to the internet; no JavaScript errors	As expected	✓
TC-21.5	All earlier tests	`python tests/run_tests.py`	All 18 pass	As expected	✓

Screenshot: without internet

#	Test	Screenshot
S46	TC-21.2: New Sale with the internet blocked, v0.19.3 (left) and v0.19.4 (right)	

5.24 F19 — New Sale redesign (v0.20.0)

The design was chosen on the design canvas "POS UI Explorations" (2026-10-11/12): **Rush Hour** (big one-tap buttons) combined with **Stock Aware** (how much is left and why), with add-ons changed **on the basket line** (option 6a; the "+ add-ons on the tile" option 6b was rejected because it makes the tiles crowded). Two new pages were added, both for logged-in users only and both checking the CSRF token: `POST /sales/line-quantity` (the − and + buttons) and `POST /sales/line-options` (change serving and add-ons). Every button is a small form, so the server checks stock, options and prices again; the scripts only open panels, show prices and fill in the cash box. TC-22.1–22.8 are automated in `tests/test_new_sale.py` (49 checks; they fail on v0.19.4 and pass on v0.20.0).

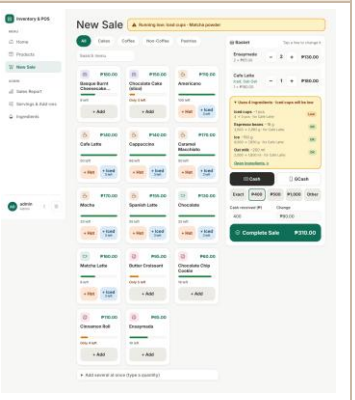
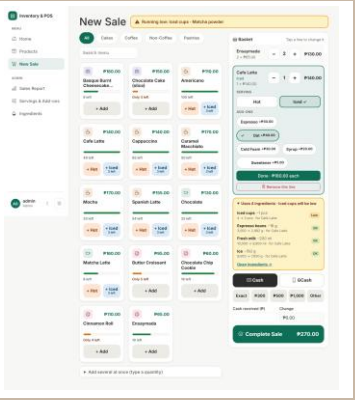
ID	Precondition	Steps / input	Expected result	Actual result	Pass?
TC-22.1	Cafe Latte with 2 iced cups and no hot cups; Matcha with no powder; Croffle 3 in stock	Open New Sale	Latte: "+ Iced" works, "+ Hot" greyed with "no Hot cups", "Only 2 left", bar 10 %; Matcha greyed "Out of Matcha powder", no add button; Croffle "+ Add", "Only 3 left"	As expected	✓
TC-22.2	Iced cups low, Hot cups and Matcha powder at 0	Open New Sale as cashier, then as admin	Banner "Running low: Iced cups · Hot cups, Matcha powder are out"; only the admin's banner links to Ingredients	As expected	✓
TC-22.3	1 Iced latte in the basket	+, + again, then −, −; also send change = 5, −3, abc; an unknown line; + on a line already at 1,000,000	2; refused "Not enough Iced cups" (basket stays 2); 1; line removed. Anything other than −1/+1 changes nothing; unknown line harmless; maximum quantity kept	As expected	✓
TC-22.4	Iced latte in the basket	Tap the line, tick Oat Milk, Done; add a plain Iced latte and change it to Oat Milk (oat milk short, then restocked); switch to Hot (no hot cups); send an add-on as the serving, an unknown serving, a serving as an add-on, "x", a hidden add-on; change a croffle	"Iced, Oat Milk" ₱180.00 and the ingredient panel shows oat milk; merge refused "Not enough Oat milk", then one line "2 × ₱180.00"; Hot refused; every bad input refused with a message; croffle "has no servings or add-ons to change"	As expected	✓
TC-22.5	1 Iced latte in the basket	Look at "Uses N ingredients"; add the last cup	"Uses 4 ingredients", each with − amount, before → after and "for Cafe Latte"; opens by itself with "Iced cups will be low"; Low rows first; link only for the admin; then "Iced cups will run out"	As expected	✓
TC-22.6	Basket ₱310 / ₱600 / ₱1,000 / ₱150	Look at the quick cash buttons; tap one	Exact + the next ₱100, ₱500 and ₱1,000 (310 → 400, 500, 1,000; 600 → 700, 1,000); never above ₱1,000,000; the tap fills Cash received and shows the change	As expected	✓
TC-22.7	Latte + 2 croffles added with the new buttons	Complete Sale with ₱500	Sale ₱440.00, change ₱60.00; iced cups 2 → 1, croffles 3 → 1	As expected	✓

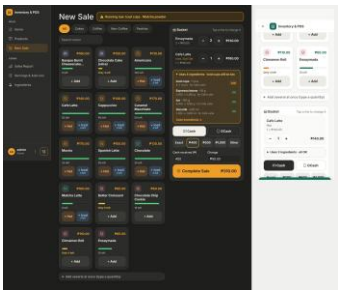
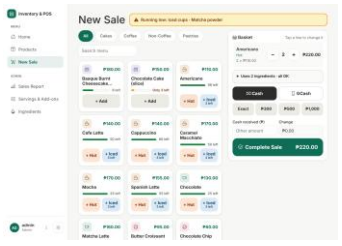
ID	Precondition	Steps / input	Expected result	Actual result	Pass?
TC-22.8	Logged out; then logged in with the token check on	Post to both new pages; count the forms; post without the token	Logged out → login page; every one of the page's forms carries the token; without it → 400 and nothing changes	As expected	✓
TC-22.9	Demo menu, iced cups low	Chromium at 390, 1024, 1280, 1440 and 1920 px, light and dark: tap, change a line, quick cash, search, complete a sale	No sideways scrolling, nothing spills out of a tile or button, every button at least 44 px, 0 requests to the internet, no JavaScript errors	**First run:** names broke in the middle of a word ("Cappuccin/o", DEF-54); "₱1,000" spilled out of its button on phones (DEF-55); basket lines cramped on phones (DEF-56) · **after the fixes:** as expected	✗ → ✓
TC-22.10	All earlier tests	`python tests/run_tests.py`	All 19 pass (7 old checks rewritten because the layout changed on purpose: the drink window is gone and sold-out items are shown greyed)	As expected	✓
TC-22.11	v0.20.1 code	`test_new_sale.py` checks the page and the style sheet	New Sale has no 1180 px limit (other pages keep it); tiles fill each row (auto-fill grid); basket 380–460 px; bar and "N left" on one line	**v0.20.0:** 4 checks fail · **v0.20.1:** pass	✗ → ✓ DEF-57
TC-22.12	Demo menu; Chromium	Open New Sale at 1024, 1280, 1440, 1600, 1920 and 2560 px	No empty bands beside the page; tiles per row grow with the screen (3 at 1280, 4 at 1440–1600, 6+ at 1920); nothing wraps or spills ("+ Iced", Complete Sale, ₱1,000); every button at least 44 px	**v0.20.0 on my wide screen:** about 200 px empty on each side and 3 tiles per row · **v0.20.1:** as expected	✗ → ✓ DEF-57
TC-22.13	v0.20.5 code	`test_new_sale.py` lists every class the New Sale part of `theme.css` styles and looks for it in the other templates	None of them is used on another page; nothing else in `theme.css` styles the New Sale bar names	**v0.20.4:** `stock-bar` also used in `_ingredient.html` · **v0.20.5:** none	✗ → ✓ DEF-58

ID	Precondition	Steps / input	Expected result	Actual result	Pass?
TC-22.14	Demo data; Chromium, light and dark	Open Ingredients; measure the warn-level tick and the bar; compare with v0.19.4	Tick 14 px tall, grey, sticking out above and below the bar; bar not clipped; colours the same as v0.19.4; New Sale still passes TC-22.9 and TC-22.12	**v0.20.0–v0.20.4:** tick 6 px, green, clipped inside the bar · **v0.20.5:** identical to v0.19.4	✗ → ✓ DEF-58

Decision recorded: cashiers can see the ingredient amounts on New Sale (they still can't open or change the Ingredients page). The student chose this because the cashier is the one who warns the customer that something is running out.

Screenshots: New Sale redesign (v0.20.0) (demo menu)

#	Test	Screenshot
S47	TC-22.1 / TC-22.6: tiles with stock bars, basket with − / +, "Uses 4 ingredients", ₱400 tapped (change ₱90.00)	
S48	TC-22.4: the Cafe Latte line opened to change it (Iced ✓, Oat Milk ✓, Done · ₱180.00 each)	

#	Test	Screenshot
S49	TC-22.9: dark theme (left) and phone, 390 px (right)	
S50	TC-22.12 (v0.20.1): New Sale on a 1920 px screen, the whole demo menu in view, no empty bands	
S51	TC-22.12 (v0.20.1): the same on a 1280 px laptop, nothing wraps in the basket	
S52	TC-22.14 (v0.20.5): Ingredients bars, dark theme. Top: my screenshot of v0.20.4 (tick clipped to a speck). Bottom: v0.20.5	

5.25 F20 — Cozy layout and foldable sidebar (v0.21.0)

Colours, sizes and the fold button only; no server rule changed. Automated in `tests/test_layout.py` (12 checks); the look was checked by hand in Chromium on a demo copy (light and dark theme, folded and open, phone width 375 px).

TC	Precondition	Steps	Expected	Actual	Result
TC-25.1	Logged in	Open Products and New Sale	One **Fold menu** button in the sidebar of each page	As expected	✅
TC-25.2	Any page	Read the button	A plain button (not a link, not a form), so it changes nothing on the server	As expected	✅
TC-25.3	Sidebar folded, reload	Reload the page	Still folded, no flash of the open sidebar (choice applied before drawing)	As expected	✅

TC	Precondition	Steps	Expected	Actual	Result
TC-25.4	Sidebar folded	Hover an icon	The tooltip shows the page name	As expected	✅
TC-25.5	Phone width (375 px)	Open any page, tap the ☰ button	Sidebar slides in as before; fold has no effect	As expected	✅
TC-25.7	Sidebar open	Look at the top row	The fold icon is right of "Inventory & POS", above the menu links, with no words (v0.21.1); folded, logo and icon are stacked	v0.21.0: a "Fold menu" row under the links · v0.21.1: as expected	❌ → ✅
TC-25.8	Sidebar folded, logged in	Compare the avatar and the theme icon at the bottom	Both centred in the strip	v0.21.0–v0.21.1: theme icon pushed to the right edge · v0.21.2: centred	❌ → ✅ DEF-59
TC-25.9	Phone 375 px, Ingredients	Tap **New ingredient**	The panel fits the screen with margins on both sides	v0.21.0–v0.21.1: cut off on the right · v0.21.2: fits	❌ → ✅ DEF-60
TC-25.6	Light, then dark theme	Open New Sale	Cream and caramel / espresso and amber; text readable; stock bars, Hot / Iced buttons and the Running-low banner still clear	As expected	✅

5.26 Last open defects closed (v0.20.2–v0.20.4)

The security audit of 2026-10-11 left three defects open (DEF-48, DEF-51, DEF-52). They were fixed one per version, smallest first, each with a test that fails on the old code.

ID	Precondition	Steps / input	Expected result	Actual result	Pass?
TC-23.1	Product `BEV-001` exists	Add a product with SKU `bev-001`, then `Bev-001`	Both refused: "SKU … is already used by Coke 1.5L (BEV-001)"	**v0.20.1:** `bev-001` saved as a second product · **v0.20.2:** refused	✗ → ✓ DEF-52
TC-23.2	`BEV-001` and `FD-001` exist	Edit BEV-001 to `fd-001`; then edit it to `bev-001` (its own SKU, new capitals)	First refused; second allowed	**v0.20.1:** first allowed · **v0.20.2:** as expected	✗ → ✓ DEF-52
TC-23.3	A look-alike pair saved before v0.20.2 (`FD-001`, `fd-001`)	Change the price of `fd-001` without touching its SKU	Saved (old data is not locked)	As expected	✓
TC-23.4	Admin logged in; a copy of the cookie in a second browser	The owner logs out; the copy opens the Sales Report	Copy refused, sent to the log-in page	**v0.20.2:** the copy still opened the report · **v0.20.3:** refused	✗ → ✓ DEF-51
TC-23.5	Two tills logged in as "cashier"	Log out on till 1	Till 2 stays logged in	As expected	✓
TC-23.6	Admin in browser A, a copy of A's cookie, and admin in browser B	Change the password in A	"Password changed … other device(s) logged out"; the copy and B are refused; A stays logged in	**v0.20.2:** copy and B still logged in · **v0.20.3:** as expected	✗ → ✓ DEF-51

ID	Precondition	Steps / input	Expected result	Actual result	Pass?
TC-23.7	The real secret key	Make a cookie that says "user 1" without logging in	Refused (no login record on the server)	**v0.20.2:** accepted as admin · **v0.20.3:** refused	✗ → ✓ DEF-51
TC-23.8	Cashier logged in with items in the basket	Wait 7 idle hours, then 9 (clock moved in the test); then log in again from the log-in page	7 h: still in, and the visit counts as activity; 9 h: logged out and the basket emptied; logging in again works	**v0.20.2:** never timed out · **v0.20.3:** as expected	✗ → ✓ DEF-51
TC-23.9	Several logins	Log out; leave an admin login idle for 30 h, then someone logs in	The log-out removes its row; rows idle more than 8 h are removed	As expected	✓
TC-23.10	A login row	Look at what is stored	A 64-character random token; no password or user data in it	As expected	✓
TC-23.11	One basket (2 × Ensaymada, stock 12); server slowed by 0.3 s per stock update	Send Complete Sale twice at the same moment with the same cookie	One receipt, stock 10; both requests end on that receipt	**v0.20.3:** 2 receipts, stock 8 · **v0.20.4:** one	✗ → ✓ DEF-48
TC-23.12	Same	Five copies at the same moment	Still one receipt, stock 10, no error page	**v0.20.3:** 5 receipts · **v0.20.4:** one	✗ → ✓ DEF-48

ID	Precondition	Steps / input	Expected result	Actual result	Pass?
TC-23.13	A sale just completed	Press Complete Sale again; send an old copy of the cookie (from before the sale)	No new sale; the receipt opens with "This sale was already completed … not charged twice"	**v0.20.3:** the old copy was sold again · **v0.20.4:** as expected	✗ → ✓ DEF-48
TC-23.14	Chromium, demo menu	Double-click Complete Sale	The button turns grey ("Saving…") after the first click; one receipt; Back shows an empty basket and a working page	**v0.20.3:** no greying · **v0.20.4:** as expected	✗ → ✓ DEF-48
TC-23.15	A database made by v0.20.1 (demo data)	Start v0.20.4	`sale.ticket` and the `login_session` table are added; data kept; log-in works (also automated in test_avatar.py)	As expected	✓
TC-23.16	All earlier tests	`python tests/run_tests.py`	All 20 pass	As expected	✓

5.27 Patches after the submission (v0.21.3–v0.21.8)

Small fixes from the security audit and the known limitations in 10.4, one per version, each with a test that fails on the old code. Automated in `tests/test_hardening.py`.

ID	Precondition	Steps / input	Expected result	Actual result	Pass?
TC-26.1	Logged in; a page crashes	Open it	500, the friendly page inside the normal layout	**v0.21.2:** plain "Internal Server Error" · **v0.21.3:** friendly page	✗ → ✓ DEF-61

ID	Precondition	Steps / input	Expected result	Actual result	Pass?
TC-26.2	Same	Read the page	Says nothing was saved and to try again (C6)	**v0.21.2:** no advice · **v0.21.3:** as expected	✗ → ✓ DEF-61
TC-26.3	The crash carries an internal message	Read the page	No error details, code or traceback	As expected (also on v0.21.2)	✓
TC-26.4	After a crash; also as a visitor	Open Products; open the crashing page logged out	Products works; the visitor also gets the friendly page	**v0.21.2:** plain page for the visitor · **v0.21.3:** as expected	✗ → ✓ DEF-61
TC-26.5	No user called `nobody-here`	Log in as `nobody-here`	A password hash is still checked, made the same slow way (scrypt) as real ones	**v0.21.2:** no hash checked (about 1 ms instead of about 100 ms) · **v0.21.4:** one scrypt check	✗ → ✓ DEF-62
TC-26.6	Same; and `admin` with a wrong password	Read the message; count hash checks	Same "Invalid username or password"; a real user still costs exactly one check	As expected	✓
TC-26.7	—	`admin` / right password	Logged in	As expected	✓
TC-26.8	Fresh database	`python seed_demo_data.py`	Servings are exactly Hot and Iced	**v0.21.4:** 12oz, 16oz · **v0.21.5:** Hot, Iced	✗ → ✓ DEF-63

ID	Precondition	Steps / input	Expected result	Actual result	Pass?
TC-26.9	Same	Look at the servings' recipes	Hot: 1 hot cup; Iced: 1 iced cup + 150 g ice	**v0.21.4:** no serving recipes · **v0.21.5:** as expected	✗ → ✓ DEF-63
TC-26.10	Same	List the drinks	Each drink once; no "Iced ..." product beside its Iced serving	**v0.21.4:** Iced Americano, Iced Latte… beside 12oz / 16oz · **v0.21.5:** 8 drinks, each + Hot / + Iced (also checked in Edge: 8 Hot and 8 Iced buttons, an iced latte uses 4 ingredients, no script errors)	✗ → ✓ DEF-63
TC-26.11	`requirements.txt`	`python tests/check_contract.py`	"D1 requirements.txt is plain UTF-8" passes; the file has no byte-order mark	**v0.21.5:** FAIL (UTF-16 with a byte-order mark) · **v0.21.6:** PASS; packages and versions unchanged	✗ → ✓ DEF-64
TC-26.12	Login, Home (visitor), Products (logged in)	Look for the tab icon in each page	Each names an icon file inside the app (`/static/…`, works offline)	**v0.21.6:** none; the browser asked for `/favicon.ico` and got 404 · **v0.21.7:** as expected	✗ → ✓ DEF-65
TC-26.13	—	Open `/static/favicon.svg`	200, an SVG picture with no script in it	**v0.21.6:** 404 · **v0.21.7:** as expected; checked by eye at 64 and 16 px	✗ → ✓ DEF-65

ID	Precondition	Steps / input	Expected result	Actual result	Pass?
TC-26.14	Dark theme	Compare a normal and a Low bar on Ingredients	Different colours; Low stays `#EF9F27`, Out `#DC3545`	**v0.21.7:** both `#EF9F27` · **v0.21.8:** normal green `rgb(74, 222, 128)`, Low amber (measured in Edge)	✗ → ✓ DEF-66
TC-26.15	Light theme	Look at a normal bar	Still the caramel accent	As expected	✓

5.28 F21 — Shop details and receipt notice (v0.22.0)

Input: shop name (required, up to 60 characters), address and footer (optional, up to 120), TIN (optional, 9 to 14 digits, dashes and spaces allowed). **Tests:** 1. The notice on every receipt. 2. Who may change the details. 3. Saving and showing them. 4. Every wrong value. 5. Typed HTML. 6. An older database. **Output:** the receipt shows the details and always the notice; anything wrong gives a plain message and changes nothing. Automated in `tests/test_shop_details.py` (22 checks); checked in Edge on screen and in print preview.

ID	Precondition	Steps / input	Expected result	Actual result	Pass?
TC-27.1	Nothing set yet	Open a receipt	Default name; "This is not an official receipt."	**v0.21.8:** no notice (SR6 not met) · **v0.22.0:** as expected	✗ → ✓ SR6
TC-27.2	Visitor; cashier; admin	Open or send `/settings/shop`; look at the sidebar	Login page; 403 for page and form; admin opens it; only the admin has the link	As expected	✓
TC-27.3	Admin	Save name, address, TIN `123-456-789-00000`, footer	"Shop details saved"	As expected	✓
TC-27.4	Details saved	Open a receipt, also in print preview	Name, address, "TIN: …", footer and the notice; the old fixed name gone	As expected (Edge, print preview)	✓
TC-27.5	Admin	Empty name; name 61, address 121, footer 121 characters; TIN `12345`, `abc-def-ghi`	Each refused with its own message; nothing changed	As expected (6 cases)	✓
TC-27.6	Admin	Name `<script>alert(1)</script>`	Shown as text, never run (S11)	As expected	✓

ID	Precondition	Steps / input	Expected result	Actual result	Pass?
TC-27.7	TIN emptied	Open a receipt	No "TIN:" line	As expected	✅
TC-27.8	A database without `shop_profile`	Start the app; open a receipt	Table added, data kept; default name and the notice	As expected	✅

5.29 Database migrations (v0.23.0)

Automated in `tests/test_migrations.py` (14 checks), which starts from a database in the v0.21.2 shape (no migration record, no `shop_profile`, a `sale` table without the later columns) holding one sale. Also run by hand on a copy of the owner's real database (2026-10-24).

ID	Precondition	Steps / input	Expected result	Actual result	Pass?
TC-28.1	A v0.21.2-shaped database with one sale	Start the app (`upgrade_database()`)	Every table and missing column added; the sale, its item and the product unchanged; at the newest migration	As expected	✅
TC-28.2	Same	Look in `instance/`	One backup taken before the change, holding the old shape and the same data	As expected	✅
TC-28.3	Already up to date	Start again	Nothing changes; no second backup	As expected	✅
TC-28.4	A database built by the migrations	Compare it with `models.py` (Alembic `compare_metadata`)	No differences, so a model change without a migration fails this test	As expected	✅
TC-28.5	Tables dropped, migration record left behind (as tests do)	Start the app	Rebuilt from the migrations; no backup (nothing to protect); works	As expected	✅
TC-28.6	Up to date, with data	Undo migration 0001	Refused: "can't be undone", data kept	As expected	✅
TC-28.7	The app's log exists before the migrations run	Run them; look at the log	Still switched on (Alembic's default logging set-up switches existing logs off)	**Without the `env.py` fix:** log switched off · **v0.23.0:** on	❌ → ✅
TC-28.8	—	List the migrations	One line (no branches), each with a description	As expected	✅
TC-28.9	A copy of the owner's real database (v0.21.2)	Start v0.23.0 on it in a scratch folder	Only `alembic_version` added; row counts unchanged; backup taken; `integrity_check` ok; the real file unchanged (same SHA-1)	As expected	✅

5.30 Times in UTC (v0.23.1)

Automated in `tests/test_times.py` (18 checks); the conversions are checked with fixed dates so the result is the same on every computer (T3). Three older tests were moved to the new helpers because they built times from the computer's clock: `test_report.py` (days on the shop's clock), `test_sessions.py` (idle hours in UTC) and `test_shop_details.py` TC-27.8 (logs in again after its made-up "older database" is upgraded).

ID	Precondition	Steps / input	Expected result	Actual result	Pass?
TC-29.1	A database at migration 0001 with a sale and a login in local time	Start v0.23.1	Migration 0002 turns them into UTC using this computer's time zone	**v0.23.0:** local time kept · **v0.23.1:** as expected	❌ → ✅ DEF-67
TC-29.2	Same	Undo 0002	Exactly the old local times again	As expected	✅
TC-29.3	—	Convert fixed times	Shop = UTC+8; 16:30 UTC Oct 24 = 00:30 Oct 25; the shop's Oct 25 begins 16:00 UTC Oct 24; `utc_now()` and `shop_now()` correct	As expected	✅
TC-29.4	—	Look at what a new sale, login and ingredient change are stamped with	UTC	**v0.23.0:** local time · **v0.23.1:** UTC	❌ → ✅ DEF-67
TC-29.5	Sales at 23:59 Mar 9 and 00:01 Mar 10 (shop time)	Report for Mar 9; for Mar 10	Each sale on its own shop day	As expected	✅
TC-29.6 / 29.8	Same	Read the report list and the receipt	Shop time (2025-03-10 12:01 AM)	**v0.23.0:** would show the stored time · **v0.23.1:** shop time	❌ → ✅ DEF-67
TC-29.7	Sales at 00:05 today and 23:55 yesterday (shop time)	Open Home	"Today" counts only the 00:05 sale; today's shop date shown	As expected	✅

ID	Precondition	Steps / input	Expected result	Actual result	Pass?
TC-29.9	—	Search `app.py`, `models.py`, `seed_demo_data.py`	No local-time stamps left (backup file names excepted)	**v0.23.0:** 10 · **v0.23.1:** none	❌ → ✅ DEF-67
TC-29.10	A copy of the owner's real database	Start v0.23.1 in a scratch folder	Sale #58 stored 15:19 → 07:19 UTC, still shown 3:19 PM; row counts unchanged; backup; integrity ok; real file unchanged	As expected	✅

5.31 Money in centavos (v0.23.2)

Automated in `tests/test_money.py` (19 checks). `test_new_sale.py` TC-22.4 now accepts `data-price="0.00"` (exact money) as well as `0.0`; the page's script reads both the same way.

ID	Precondition	Steps / input	Expected result	Actual result	Pass?
TC-30.1	Admin	Add a product at `75.50`	Stored as 7550 (INTEGER)	**v0.23.1:** 75.5 (REAL) · **v0.23.2:** 7550	❌ → ✅ DEF-68
TC-30.3	Admin	Add a product at `0.125`; sell 3	Stored as 13; receipt ₱0.13 each, ₱0.39 for three	**v0.23.1:** ₱0.12 each but ₱0.38 for three (the receipt didn't add up) · **v0.23.2:** as expected	❌ → ✅ DEF-68

ID	Precondition	Steps / input	Expected result	Actual result	Pass?
TC-30.4	That sale, cash ₱1	Read the sale	Total ₱0.39, cash ₱1.00, change ₱0.61 exactly; total = sum of lines	**v0.23.1:** floats · **v0.23.2:** exact	✗ → ✓ DEF-68
TC-30.5	1,000 sales of ₱0.10	Sum in the database; open the report	Exactly 10039 centavos; report ₱100.39	**v0.23.1:** a float sum · **v0.23.2:** exact	✗ → ✓ DEF-68
TC-30.6	A database at 0002 with 75.5, 19.99, 0.1 and a sale	Start v0.23.2; then undo 0003	7550, 1999, 10 (and the sale, its line and an add-on) as INTEGER; undo gives back 75.5, 19.99, 0.1	As expected	✓
TC-30.7	Same	Read prices in Python	Exact `Decimal` 75.50, 19.99, 0.10	**v0.23.1:** float · **v0.23.2:** Decimal	✗ → ✓ DEF-68
TC-30.8	Admin	₱1,000,000; `1000000.01`, `nan`, `inf`, `-1`, `abc`; an add-on at ₱30.50	Limits unchanged; the add-on stored as 3050	As expected	✓
TC-30.9	A copy of the owner's real database	Start v0.23.2 in a scratch folder	All 58 sales, 120 lines, 12 prices equal to the centavo; every total = sum of its lines; the unique GCash and ticket indexes kept; integrity ok; real file unchanged	As expected	✓
TC-30.10	Demo copy in Edge	Iced latte + Oat Milk; cash ₱200; complete	Live price "₱180.00 each", change ₱20.00, receipt ₱180.00 / ₱200.00 / ₱20.00, no script errors	As expected	✓

5.32 F22 — Setting up and running a shop's copy (v0.24.0)

Automated in `tests/test_setup.py` (30 checks; `setup_shop.py` is run as a program with an answers file, and waitress really answers a request). Also tried end to end in a scratch folder: install, `start_shop.bat`,

and Edge through the computer's network address. `test_security.py` now reads the Werkzeug pin from `requirements/local.txt`.

ID	Precondition	Steps / input	Expected result	Actual result	Pass?
TC-31.1	No `instance/config.toml`	Start the app	The local SQLite database	As expected	✓
TC-31.2	Fresh folder	`setup_shop.py --answers` (other devices, port 5050, shop details, admin "maria")	Exit 0 and how to start; config `0.0.0.0:5050`, local database, no password or secret in it; "maria" only (no demo accounts), password hashed; database at the newest migration; shop details saved	As expected	✓
TC-31.3	Fresh folder	Password `short`, = username, empty username, port 80 or `abc`, empty shop name	Each refused with its reason, exit 1; nothing created	As expected (7 cases)	✓
TC-31.4	Set up	Run again with "this computer only, 5000"	Admin kept (not added twice, same password, "already exists"); new choice saved; old config kept as `config.toml.bak`	As expected	✓
TC-31.5	A config file with a mistake	Start the app	A plain message naming the file and `setup_shop.py`; no traceback	As expected	✓
TC-31.6	`DATABASE_URL` set	Start the app	It wins over the file (for hosting)	As expected	✓
TC-31.7	—	Serve with waitress; open `/login`	200 with the security headers	**v0.23.2:** only Flask's development server · **v0.24.0:** waitress	✗ → ✓
TC-31.8	Config with host `8.8.8.8`, port 99999 or `x`; no `[server]`	`serve.server_settings()`	Each refused with a plain message; defaults 127.0.0.1:5000	As expected	✓
TC-31.9	—	Read `start_shop.bat`	Runs `serve.py` with `venv\Scripts\python.exe` from its own folder	As expected	✓
TC-31.10	An answers file saved by PowerShell 5 (UTF-8 with a byte-order mark)	Run the setup	Accepted	**First try:** refused ("Unexpected UTF-8 BOM"), found in the end-to-end trial · **v0.24.0:** accepted	✗ → ✓
TC-31.11	A fresh scratch install	`start_shop.bat`; Edge at `http://192.168.100.88:5081`	Served by waitress through the network address; `admin`/`admin123` refused; "maria" logs in; shop details there; the start-up window shows both addresses	As expected	✓

ID	Precondition	Steps / input	Expected result	Actual result	Pass?
TC-31.12	A fresh install, typing the answers	`python setup_shop.py` (no answers file)	The questions read clearly; passwords hidden; a wrong answer asked again	**Still to try by hand** (the password prompt reads from the keyboard, so it can't be automated)	

5.33 Blueprints and the store/ layer (v0.25.0)

A change with no visible effect is tested by showing that nothing changed: the whole existing suite, plus a walk through every page.

ID	Precondition	Steps / input	Expected result	Actual result	Pass?
TC-32.1	The split code	`python tests/run_tests.py`	All 28 files pass; the only test edit is one line in `test_hardening.py` (it swaps the password check where `login()` now lives)	As expected, first run	
TC-32.2	—	Count the pages	The same 24 pages and addresses, now in `blueprints/`	24 + static	
TC-32.3	A copy with a page that sets a stock quantity and commits	`python tests/check_contract.py`	Both lines reported ("writes to the database itself")	Both caught	
TC-32.4	Demo copy in Edge, admin and cashier	Open all 14 kinds of page; sell an iced drink; cashier opens Shop details	Every page 200 with the right sidebar link highlighted; the sale saved through `store/` and its receipt shown; 403 for the cashier; no script errors	As expected	

5.34 F23 — Email sending and the login limit in the database (v0.26.0)

Automated in `tests/test_attempts.py` (21 checks) and `tests/test_email.py` (66 checks). A fake mail server stands in for `smtplib`, so no internet or mail account is used (rule T3). `test_security.py` now reads and ages the tries in the table instead of the old list in memory (same 42 checks). The DEF-69 and DEF-70 checks were also run against v0.25.0, where they fail (rule C3). Two older tests were corrected: `test_setup.py` TC-31.2 now reads the newest migration from `migrations/` instead of expecting `0003`, and `test_shop_details.py` TC-27.8 also drops `login_attempt`, because a real database from before v0.23.0 doesn't have it.

ID	Precondition	Steps / input	Expected result	Actual result	Pass?
TC-33.1	A database at migration 0003	Upgrade; undo; upgrade	`login_attempt` added, indexed by key and by time; removed when undone	As expected	✓
TC-33.2	—	5 wrong passwords, then the right one; 3 more tries; 5 minutes later the right one	429 on the 6th; refused tries not kept; then logged in and the count cleared	As expected (the same as since v0.13.3)	✓
TC-33.3	—	A password typed into the username box by mistake	Only a 64-character keyed hash is stored; nothing typed is in the table	As expected	✓
TC-33.4	5 wrong passwords	Restart the app; type the right password	429: the count survives the restart	**v0.25.0:** logged in (the count was in memory) · **v0.26.0:** 429	✗ → ✓ DEF-70
TC-33.5	Cashier logged in	5 wrong current passwords on Change password, then the right one	"Too many wrong current passwords", nothing changed; other users not affected; works again after 5 minutes; the right current password resets the count; the login page isn't locked by it	**v0.25.0:** the 6th try changed the password · **v0.26.0:** as expected	✗ → ✓ DEF-69
TC-33.6	A real server	10 wrong passwords at the same moment	At most 5 are checked, the rest get 429	5 × 200, 5 × 429	✓
TC-34.1	No `[mail]` settings	Send	"isn't set up yet" and how to set it up; nothing sent	As expected	✓
TC-34.2	STARTTLS settings	Send	Encrypted (certificate checked) before the login; from the shop's name; plain text with a date and a message id	As expected	✓
TC-34.3	SSL settings; a server with no login	Send	SMTP_SSL on port 465 with a checked certificate; no login step when no username is set	As expected	✓
TC-34.4	Fake failures	Wrong password, refused address or sender, rejected message, no STARTTLS, TLS error, no internet, timeout, refused or dropped connection	One plain sentence each; offline says selling still works; each in the server log, never the password	As expected (10 cases)	✓
TC-34.5	—	10 malformed addresses, one with a line break and `Bcc:`	Refused; nothing sent	As expected	✓

ID	Precondition	Steps / input	Expected result	Actual result	Pass?
TC-34.6	—	Wrong server name, security, port or sender; a missing or two-line password; a hand-edited config with a mistake	Each refused with its reason; the mistake named; nothing sent	As expected	✓
TC-34.7	Visitor, cashier, admin	Open Email	Login page / 403 (posting too, nothing sent) / the settings without the password; the Email link only for admins	As expected	✓
TC-34.8	Admin	Send a test to a good address, a wrong one, with no internet, with email not set up	"Test message sent …" with the spam-folder hint; asked for an address; the offline sentence; button off and nothing sent	As expected	✓
TC-34.9	Fresh copy	`setup_shop.py --answers` with email answers	`[mail]` written with the usual port filled in; password stored exactly and never printed; the app reads it; `MAIL_PASSWORD` wins	As expected	✓
TC-34.10	Set up with email	A wrong email answer; run again without email; again with new email; again after a hand edit broke the old settings	Refused, config unchanged; settings kept; settings replaced; setup finishes, says the old ones had a mistake and leaves them out	As expected	✓
TC-34.11	Demo copy in the browser, admin	Open Email (not set up, then set up); send a test with no mail server; light, dark, phone width	Readable in both themes and at phone width; the plain sentence after sending; no script errors	As expected: with a server name that doesn't exist, the real `smtplib` answered "Could not reach the mail server … Selling and everything else still works"; the duplicated advice on the not-set-up page was found here and removed; dark colours readable; 375 px wide with no sideways scroll	✓
TC-34.12	A real Gmail account with an App Password	Send a test message	It arrives, from the shop's name	**Tried by hand 2026-10-25, still open:** Gmail closes the connection at the login step on ports 587 and 465, even for a made-up account (a wrong password would get "535"), while the connection, STARTTLS and Google's certificate are fine. So Gmail is refusing logins from the home internet connection, not the App Password or the shop's code. To try again from another connection (a phone hotspot). Meanwhile the email codes were tested in demo mode (TC-38.15)	✗

5.35 F24 — The owner, users, invitations, forgotten passwords and the audit log (v0.27.0)

Automated in `tests/test_users.py` (47 checks), `tests/test_invitations.py` (44) and `tests/test_password_reset.py` (24), with a fake mailbox in `tests/helpers.py` (rule T3). Three older tests changed with the rules: `test_setup.py` expects the **owner** and asks for its email (one more check), `test_email.py`'s setup answers include that email, and `test_shop_details.py` TC-27.8 now builds its "older database" by undoing the migrations to `0001` instead of dropping single tables (a current database already has the new columns).

ID	Precondition	Steps / input	Expected result	Actual result	Pass?
TC-35.1	A database at migration 0004 with an admin	Upgrade; undo; upgrade	New `user` columns and the three tables; the old admin still an admin, switched on, no email; undo removes them, keeping the account	As expected	✓
TC-35.2	An owner	Add a second owner	Refused by the database (unique index)	As expected	✓
TC-35.3	Owner, cashier	Open admin pages; cashier opens Users, Audit log and posts every Users button	Owner 200 with Users and Audit log in the sidebar; cashier 403 everywhere, nothing changed	As expected	✓
TC-35.4	Two admins, one allowed to invite	Invite, change roles, switch admins / the owner off	Only the allowed admin may invite, and only cashiers; everything else 403; nobody can switch the owner off	As expected	✓
TC-35.5	Owner	Make a cashier an admin and back; give "owner"; change the owner; allow an admin to invite	Done, logged; "may invite" cleared when demoted; "owner" and the owner's own row 403	As expected	✓
TC-35.6	Cashier logged in on a till	Admin switches the account off, then on	The till is logged out at once; the right password says "switched off" (403); the account still exists; switched on, logs in again	As expected	✓
TC-35.7	Owner requires a new password for a cashier	Cashier opens pages	Every page goes to Choose a new password (log out still works); current password and the usual rules needed; then the flag is cleared	As expected	✓
TC-35.8	Owner and admin change products, the shop details, accounts	Read the Audit log; filter by area and person; 60 more lines	Who, old → new (price 150.00 → 160.00, stock 20 → 18); no line for a save that changes nothing; filters work; 50 a page with "Older"	As expected	✓
TC-35.9	An older database (admin and cashier, no owner)	`setup_shop.py` with the cashier, then the admin, then a new name	Cashier refused; the admin becomes the owner with the email (to be confirmed); a later run leaves the owner; the log says "Setup"	As expected	✓

ID	Precondition	Steps / input	Expected result	Actual result	Pass?
TC-35.10	Demo copy in the browser, owner	Users, invite with no mail server, Audit log, Join with a bad link, Forgot password, Profile; light, dark, 375 px	Invitation saved with the plain mail sentence and "Send again"; the log line; "This link doesn't work any more" (404); the same answer page; owner's email "Not confirmed" with its button; no sideways scroll (tables scroll inside their card); no script errors	As expected	✓
TC-36.1	Owner, mail set up	Invite `Jun@Example.com` as cashier	One email naming the inviter, role and 48 hours; only a 64-character keyed hash of the token stored; listed with Send again / Cancel	As expected	✓
TC-36.2	Owner on 127.0.0.1	Invite, with other devices allowed and not	The link uses the network address (192.168.1.20) only when other devices are allowed	As expected	✓
TC-36.3	The link	Bad usernames and passwords; good details with a forged `role=admin`; wrong code; right code with a space	Each refused with its reason; no account before the code; the right code makes a **cashier** with the confirmed email and logs in; the link then answers 404	As expected	✓
TC-36.4	Owner, admin allowed to invite	Invite cashiers and an admin; an existing address; an address already invited; not an address	Allowed ones sent; each other case refused with its message; an admin can't resend or cancel the owner's invitation (403)	As expected	✓
TC-36.5	An invitation	Send again; Cancel	A new link, the old one 404; a cancelled link 404	As expected	✓
TC-36.6	An invitation older than 48 hours	Open the link	404; Users says "expired"	As expected	✓
TC-36.7	No internet; email not set up	Invite	"saved, but the email could not be sent … Send again"; not set up: the form is replaced by the reason	As expected	✓
TC-36.8	Signup codes	5 wrong tries; a new code within a minute; a minute later; an older code; after 10 minutes; 7 codes in an hour; a reset code used for signup; a code used twice	Counts down 4–1 then cancelled; refused; sent; only the newest is open; expired; 5 then refused; wrong purpose refused; works once	As expected	✓
TC-36.9	One invitation	Finish the signup twice at once	One account; the second attempt makes nothing	As expected	✓
TC-37.1	Accounts with a confirmed, an unconfirmed, a switched-off and no email	Forgot password with each address	The same message every time; a code only for the switched-on, confirmed one; the email says what, when, from where and what to do	As expected	✓

ID	Precondition	Steps / input	Expected result	Actual result	Pass?
TC-37.2	Cashier logged in on a till	Code and a new password	A rule-breaking password refused before the code is used; a wrong code refused; the right one saves it, the old password stops, the till is logged out; the code works once	As expected	✓
TC-37.3	A wrong code	New password = the username	"not right", nothing about the username	As expected	✓
TC-37.4	One computer; email not set up	12 requests; ask with no mail settings	10 answered, then 429; "ask the owner" for every address	As expected	✓
TC-37.5	An account from before v0.27.0	Add an email: wrong password, another account's address, then right	Refused, refused, a code to the new address; saved and confirmed only after the code	As expected	✓
TC-37.6	Owner with an unconfirmed email	Profile; post another address anyway; enter the code	No box to change it; the code goes to the owner's own address; confirmed, address unchanged	As expected	✓

5.36 F25 — A code at every login (v0.28.0)

Automated in `tests/test_login_codes.py` (69 checks), the one test file that logs in the real way. The 32 older test files now say `app.config['TEST_PLAIN_LOGIN'] = True` (one line each: they test other features), and TC-38.13 shows the contract check refuses that line anywhere outside `tests/`. While building, TC-36.1 (v0.27.0) showed it read the clock twice; the invitation now reads it once.

ID	Precondition	Steps / input	Expected result	Actual result	Pass?
TC-38.1	—	Work out codes for RFC 6238's example times; a code two steps away	RFC 6238's codes exactly; one step either side accepted, two refused	As expected	✓
TC-38.2	A database at migration 0005	Upgrade; undo; upgrade	`trusted_device` and `backup_code` added; existing accounts get emailed codes; undo removes them	As expected	✓
TC-38.3	Cashier with a confirmed email	Right password; wrong code; right code; log out and in again within a minute; wait 5 minutes on the code page	/verify, not logged in yet, the address hidden (c•••@example.com); refused; logged in; a new code is sent at once; after 5 minutes back to the login page	As expected	✓

ID	Precondition	Steps / input	Expected result	Actual result	Pass?
TC-38.4	Same	6 wrong codes; a new code within a minute	429 "Too many"; "less than a minute ago"	As expected	✓
TC-38.5	Same	Tick "Trust this device"; log in again; wrong password; a second cashier on that browser; a copied or made-up cookie; an admin	Only a keyed hash stored; HttpOnly, SameSite=Lax cookie; password alone logs in, no email; the password is still needed; the second cashier needs a code and can trust too (two tokens); copies and made-up tokens ignored; 7 days for an admin	As expected	✓
TC-38.6	A trusted browser	Remove; Log out everywhere; change password; switch off; change role; let it expire	Each one ends trust (Log out everywhere also ends every login); another person's device can't be removed (404)	As expected	✓
TC-38.7	Cashier	Make backup codes (wrong, then right password); log in with one; again; make new ones	Password needed; 10 codes `xxxx-xxxx`, page not kept (`no-store`), only hashes stored; logs in (capitals and spaces fine); refused the second time; Profile says 9 left; old ones stop working	As expected	✓
TC-38.8	No internet	Log in	/verify says the code couldn't be sent and offers a backup code, which logs in (rule D2)	As expected	✓
TC-38.9	Cashier	Set up the app (password); wrong code; right code; log in; the set-up step's code; the next step's; the same again elsewhere; turn off without and with a code	QR as a `data:` SVG and the key as text, `no-store`; key stored encrypted, key file in `instance/`; not switched on a wrong code; switched, new backup codes, an email; no email at login; the used step refused (no replay); next step logs in; refused again; off needs a current code or backup code, then emails	As expected	✓
TC-38.10	Admin; cashier and owner using the app	Reset to email	The cashier's reset, logged and emailed; the owner's refused (403)	As expected	✓
TC-38.11	An account with no email	Log in; a cashier adds an email for it; the owner does; log in	"Ask the owner" (403); 403; saved unconfirmed; the code goes there, logs in and confirms it	As expected	✓

ID	Precondition	Steps / input	Expected result	Actual result	Pass?
TC-38.12	A `cashier123` account	Log in; again in demo mode; start in demo mode with real mail settings	Refused with what to do (403); the code printed in the window, not emailed, and it logs in; start-up refused with a plain message	As expected	✓
TC-38.13	A file outside `tests/` with `TEST_PLAIN_LOGIN = True`	`check_contract.py`	Fails, naming the file and line	As expected	✓
TC-38.14	Demo copy in the browser, demo mode	Log in as the owner with the printed code and trust; log out and in again; set up the app with its current code; Profile at 375 px, dark	Logged in; trusted for 7 days, listed under Trusted devices; owner's email now confirmed; the QR drawn; switched, backup codes, the change email printed; no sideways scroll; no script errors	**First try:** logging in again within a minute sent no code ("less than a minute ago") though the first code was used · **Fixed:** the minute only applies while a code is open	✗ → ✓
TC-38.15	v0.31.0 test copy in the browser, demo mode (no `[mail]` settings), demo accounts with emails	*(By hand, 2026-10-25)* Log in as admin and as cashier with the code printed in the server window; wrong codes; resend; reuse an old code after logging out; tick "Trust this browser"; Forgot password	The window shows the email that would be sent (the account's address, subject, 6-digit code, "works for 10 minutes, once"); wrong codes refused with a plain message, the code stops working after 5 wrong tries; after a resend only the new code works; a used code is refused; the cashier's code confirms its email (Users page); a trusted browser needs only the password; the Forgot password code is printed the same way	As expected. The 10-minute expiry was not waited out by hand. Stands in for real Gmail until TC-34.12 passes	✓

5.37 F26 — Sign in with Google (v0.29.0)

Automated in `tests/test_google.py` (55 checks), with no internet and no Google account (rule T3): the test makes its own RSA key, signs ID tokens with it, and replaces only the two calls that talk to Google. Every check on the answer runs for real. The real exchange with Google needs the owner's own OAuth client and is tried by hand (TC-39.10).

ID	Precondition	Steps / input	Expected result	Actual result	Pass?
TC-39.1	A database at migration 0006	Upgrade; undo; upgrade	`user.google_sub` (unique) and `invitation.pending_google_sub` added and removed	As expected	✓
TC-39.2	Not set up; set up, on `127.0.0.1`	Open the login page; `/login/google`	No button and "not set up"; on `127.0.0.1` a note naming `http://localhost:5000`, no button that would fail	As expected	✓
TC-39.3	Set up, `localhost`	Sign in with Google	To Google's address, scopes `openid email profile` only, random state and nonce, PKCE S256, return to `/auth/google/callback`; the code exchanged with the matching verifier	As expected	✓
TC-39.4	A trip started	Answers with another state; a token signed by another key; another audience or issuer; expired; another nonce; no `sub`; email not verified; the same answer twice; no trip started; Cancel; older than 10 minutes; no internet	Each refused with a plain sentence, not logged in ("Could not reach Google" when offline)	As expected (13 cases)	✓
TC-39.5	Accounts with and without Google linked	Google login: unlinked; linked; trusted browser; Google email changed; switched off	Told how to link; `/verify` with the code; straight in when trusted; found by Google's id, not the email; refused	As expected	✓
TC-39.6	Cashier with a confirmed email	Link: wrong code; wrong password; code sent; another-email Google; a Google account linked elsewhere; the same email	Refused; refused; the code mail says what it is for; refused naming both addresses; refused; linked by id, emailed and logged	As expected	✓
TC-39.7	Unconfirmed email; a linked account	Open Profile, post anyway; unlink with a wrong then the right password	"Confirm your email first"; unlink needs the password, then Google no longer opens it	As expected	✓
TC-39.8	An invitation	Continue with Google: bad username; a Google account for another address; the invited address; the code	Refused; refused; the code step follows; a **cashier** with Google linked (a forged `role` changed nothing), logged in; its password unknown; Google then logs in via the code	As expected	✓
TC-39.9	—	`setup_shop.py` with a wrong client ID, a good one, again without; `GOOGLE_CLIENT_ID`	Refused; `[google]` written and the redirect address printed (not the secret); kept; the variable wins	As expected	✓
TC-39.10	The owner's own Google Cloud client	Sign in with a real Google account at `http://localhost:5000`	Google's page, back to `/verify`, logged in	**To try by hand** (needs the owner's OAuth client; the tests never use a real account, rule T3)	⏳

ID	Precondition	Steps / input	Expected result	Actual result	Pass?
TC-39.11	Demo copy in the browser with Google settings	Login on `localhost` and `127.0.0.1`; Profile at 375 px, dark	The button on `localhost`; the note on `127.0.0.1`; the Google card with "Send me a code" and "Link Google"; no sideways scroll; no script errors	As expected	✓

5.38 F27 — Undo and a safe count (v0.30.0)

Automated in `tests/test_undo.py` (36 checks). The race cases put "another till" exactly between reading and writing, as `test_race.py` does. Two older tests now send the amount seen with a count (`test_ingredients.py`, `test_ingredients_page.py`).

ID	Precondition	Steps / input	Expected result	Actual result	Pass?
TC-40.1	Milk 400 ml	Count with the amount seen 400; with 400 after it became 380; without the amount seen; 999.7 on screen for 999.6999999 stored; a sale exactly between reading and writing	Saved with -20 in the history; refused "changed since you opened the page, now 380, count again"; refused; accepted; refused and the sale kept (I1)	As expected	✓
TC-40.2	A count just made	Undo count (cashier, then admin); again; after 5 minutes; after a delivery; with a sale between reading and writing	403; back to the old amount with a `count-undo` line, "Count undone" in the histories; once only; refused; refused, the delivery kept; refused, the sale kept	As expected	✓
TC-40.3	Cashier	Add, add, +, remove line; Undo 4 times; once more; remove product and undo; a tap that changes nothing	Each Undo steps back one change to empty; "nothing to undo"; the product back; no extra step	As expected	✓
TC-40.4	Cashier	7 changes; 5 undos; a 40-line order	Only the last 5 kept; back 5 steps then nothing; the copies stay under about 1,500 characters and the cashier stays logged in	As expected	✓
TC-40.5	An order	Complete the sale; Undo	No undo list, no button; nothing comes back	As expected	✓
TC-40.6	Item 10: 2 in the order, then removed; another till sells one	Undo	The line comes back with "Undone, but: not enough stock"; the sale is refused until fixed	As expected	✓
TC-40.7	An undo list	Log out and in	Empty	As expected	✓

ID	Precondition	Steps / input	Expected result	Actual result	Pass?
TC-40.8	Demo copy in the browser, demo mode	Count from the Update window (a row showing 2,000); Undo count; New Sale at 375 px, dark: add, Undo	The window sends 2000; the banner; back to 2,000 g; Undo empties the order; no sideways scroll	**First try:** at 375 px the Undo button wrapped onto two lines beside "Tap a line to change it" · **Fixed:** the hint hides on phones while Undo shows	✗ → ✓

5.39 F28 — Voids and refunds (v0.31.0)

Automated in `tests/test_voids.py` (81 checks). The race cases put "another admin" exactly between reading and writing, as `test_race.py` does. `seed_demo_data.py` now marks the demo drinks as recipe lines (demo sales take no ingredients, so they are refunded as waste only).

ID	Precondition	Steps / input	Expected result	Actual result	Pass?
TC-41.1	A v0.30.0 database with a latte (recipe) and a cookie sold	Upgrade; undo migration 0008	Tables added; the latte line marked "used a recipe", the cookie not; undo removes the tables	As expected	✓
TC-41.2	Cookie (own count), Latte (180 ml milk, 18 g beans)	Sell 2 cookies and 2 lattes	The latte line keeps 360 ml and 36 g; stock 18 cookies, 1640 ml, 464 g	As expected	✓
TC-41.3	Today's receipt, cashier	Ask for a void with no reason; 201 characters; "Wrong drink rung up"; again	Refused; refused; "Void requested", nothing changed, email and code forms shown; "already waiting"; Today's receipts and the admin menu (1) show it	As expected	✓
TC-41.4	A waiting void	Cashier opens the Sales Report or the refund page, or posts to approve, reject, void now, refund	403 each; still waiting (S15)	As expected	✓
TC-41.5	Same, admin	Approve with a wrong password; "card"; the right password	Refused; refused; voided: all stock back, `void` history lines, the receipt unchanged and marked VOIDED, slip names both people, audit line	As expected	✓
TC-41.6	An approved void; another waiting	Approve again; another admin approves exactly between reading and writing	"already decided", stock back once (I1)	As expected	✓
TC-41.7	A waiting void	Reject with a note; reject again; ask again	Rejected, nothing changed; "already decided"; a new request is allowed	As expected	✓

ID	Precondition	Steps / input	Expected result	Actual result	Pass?
TC-41.8	A request left from yesterday; yesterday's receipt	Approve; ask for a void	Refused "earlier day" (reject, then refund); refused "Only today's", no button	As expected	✓
TC-41.9	A GCash sale	Void now without a reference; with one; the same reference on another void	Refused; voided by the admin alone with the reference kept; refused, nothing saved	**First try:** the repeated reference crashed the page (the unique rule fired on the update, not at the commit) · **Fixed** before commit: caught and explained	✗ → ✓
TC-41.10	Two admins with a confirmed email; a waiting void	Ask a cashier's id; ask admin2; a wrong code; the right one; the code on another void; 5 wrong tries then the right one; again within a minute; mail down	Refused; email with receipt, items, reason, code; "4 tries left"; voided in admin2's name "by emailed code"; refused; refused; "Wait a minute"; the error shown	As expected	✓
TC-41.11	3 cookies and 2 lattes sold	Refund: wrong password; 4 cookies; "two"; nothing; 1 cookie to the shelf + 1 latte as waste; 1 latte to the shelf; more; void; the rest; a voided receipt	Refused ×4; ₱200 back, 1 cookie back, milk unchanged with `refund` +180 / `waste` −180; exactly one latte's milk and beans back; "At most 0"; "already refunded"; refused when all refunded; "is voided"	As expected	✓
TC-41.12	2 cookies sold	Another refund lands between reading and writing (the money; then the line)	Refused, nothing saved; the money step undone too (I2)	As expected	✓
TC-41.13	A sale from before v0.31.0	Refund its latte "to the shelf"; void it	Waste only (the form and the server); the void returns what its history lines took	As expected	✓
TC-41.14	The day's voids and refunds; a refund today of a 3-day-old receipt	Sales Report today and 3 days ago; Home	Net = gross − voids − refunds; Cash and GCash after money given back; the old refund counts today; badges in the list; Home shows net; a negative day shows −₱130.00	As expected (the negative format: see TC-41.17)	✓
TC-41.15	—	Open a slip (cashier), a missing slip, ingredient history, the audit log's new area, the Sales Report, `/voids`	Prints; 404; "Void slip #…", "Waste"; void requested / voided / void rejected / refunded; the day's voids and refunds with both names; `/voids` is gone (404) and a rejection returns to the report	As expected	✓

ID	Precondition	Steps / input	Expected result	Actual result	Pass?
TC-41.16	Demo mode, a server window that can't show "₱"	Send the void email	Shown with "?" for "₱"	**v0.30.0:** the page crashed (500) · **v0.31.0:** as expected	✗ → ✓ DEF-71
TC-41.17	Demo copy in the browser, demo mode (Edge)	Cashier: sale, ask for a void, email the admin, type the code; admin: refund an older receipt as waste by GCash; Sales Report; 375 px, light and dark	All work; codes and the email in the server window	**First try:** the void email crashed the page (DEF-71); at 375 px the refund lines hid the number and shelf / waste boxes off to the side; the report showed ₱-130.00 · **Fixed:** one stacked row per item; −₱130.00	✗ → ✓

5.40 F29 — Help on every page (v0.32.0)

Automated in `tests/test_help.py` (161 checks). TC-42.1 goes through every page the app can show, so a page added later without help fails the suite.

ID	Precondition	Steps / input	Expected result	Actual result	Pass?
TC-42.1	—	Every page that can be opened; every page a topic names	Each has exactly one topic; no topic names a page that is gone	As expected	✓
TC-42.2	—	Each topic's picture	A WebP file in `static/help/`, at most 150 KB, all together under 1.5 MB; no picture without a topic	As expected (18 pictures, 0.7 MB)	✓
TC-42.3	—	Each topic's words	A title, a summary, at least two steps; no HTML; each step for everyone, admins or cashiers; admin pages only admin steps	As expected	✓
TC-42.4	Admin on Products	Look at the ? buttons and the panel	Only an icon, "Help for this page" for screen readers and as tooltip; opens the Products topic with its picture; links to `/help#products`; left out when printing	As expected	✓
TC-42.5	Visitor	Login page; Forgot password	A ? beside the theme switch; the "Logging in" topic; no link to the Help page	As expected	✓

ID	Precondition	Steps / input	Expected result	Actual result	Pass?
TC-42.6	A receipt; Products	Open the panel as cashier and as admin	Cashier: "Ask for a void", no Void now or Refund items, "View only"; admin: Void now and Refund items	As expected	✅
TC-42.7	—	`/help` as visitor, cashier, admin	Visitor sent to log in; cashier sees 7 topics, none for admin pages; admin sees 18, admin ones marked "Admins only"; every picture; chips to each topic; "Open" links only to pages that need no number	As expected	✅
TC-42.8	Cashier	Type `/admin/users`	403 as before, and no admin help on it	As expected	✅
TC-42.9	A topic whose words look like HTML	Open its page	Shown as text, not run (S11)	As expected	✅
TC-42.10	—	The style sheet	The panel is never wider than the screen; the button and panel use the theme's colours	As expected	✅
TC-42.11	Demo copy in a browser (Edge), demo mode	Open the ? on New Sale (light) and the Sales Report (dark); the Help page; both at 375 px	The panel with picture and steps; readable in both themes; 18 topics, no broken picture; no sideways scroll; one ? visible on a phone	**First try:** the ? sat in the sidebar footer beside the theme switch and squeezed the username to "ad…" · **Fixed:** round button in the bottom right corner on wide screens, in the top bar on phones	❌→✅

5.41 Test summary

Feature	Cases	✅ Pass	❌ Fail	⏳ Open
F1 View list	3	3	0	0
F2 Add	6	2 → **6**	4 → **0**	0
F3 Edit	4	3 → **4**	1 → **0**	0
F4 Delete	4	3 → **4**	0	1 → **0**
F5 Login/out	12	7 → **12**	0	1 → **0**
F6 Roles	6	6	0	0
F7 Process sale	23	20 → **23**	2 → **0**	1 → **0**
Patches v0.8.2–v0.8.6	10	1 → **10**	9 → **0**	0
F8 Low-stock	9	7 → **9**	2 → **0**	0
F9 Sales report	10	8 → **10**	2 → **0**	0
Demo data	4	2 → **4**	2 → **0**	0
F10 Profile picture	11	11	0	0

Feature	Cases	✅ Pass	❌ Fail	⏳ Open
F11 Modern theme	13	10 → **13**	3 → **0**	0
F12 Security	14	8 → **14**	6 → **0**	0
F13 Pictures + café menu	19	13 → **19**	6 → **0**	0
F14 Payment method	10	10	0	0
F15 Servings + add-ons	14	11 → **14**	3 → **0**	0
F16 Ingredients	14	12 → **14**	2 → **0**	0
F17 Ingredients page	15	12 → **15**	3 → **0**	0
F18 Rules + automated tests	14	13 → **14**	0	1 → **0**
Security audit patches (v0.19.3)	6	2 → **6**	4 → **0**	0
Offline pages (v0.19.4)	5	2 → **5**	3 → **0**	0
F19 New Sale redesign (v0.20.0–v0.20.5)	14	9 → **14**	5 → **0**	0
Last open defects (v0.20.2–v0.20.4)	16	6 → **16**	10 → **0**	0
F20 Cozy layout (v0.21.0–v0.21.2)	9	6 → **9**	0	0
Patches after the submission (v0.21.3–v0.21.8)	15	4 → **15**	11 → **0**	0
F21 Shop details (v0.22.0)	8	7 → **8**	1 → **0**	0
Migrations, UTC, centavos (v0.23.0–v0.23.2)	27	17 → **27**	10 → **0**	0
F22 Setup and serve (v0.24.0)	12	9 → **11**	2 → **0**	1 → **1**
Blueprints and store/ (v0.25.0)	4	4	0	0
F23 Email and login limit (v0.26.0)	18	15 → **17**	2 → **0**	1 → **1**
F24 Owner, users, invitations, audit log (v0.27.0)	25	25	0	0
F25 Login codes (v0.28.0)	15	14 → **15**	1 → **0**	0
F26 Sign in with Google (v0.29.0)	11	10	0	1 → **1**
F27 Undo and a safe count (v0.30.0)	8	7 → **8**	1 → **0**	0
F28 Voids and refunds (v0.31.0)	17	14 → **17**	3 → **0**	0
F29 Help on every page (v0.32.0)	11	10 → **11**	1 → **0**	0
Total	**436**	**317 → 433**	**99 → 0**	**7 → 3**

*"→" shows the result when the case was first run and the result on the final version. A failing test is not a bad sign: finding defects is the purpose of testing (Sommerville calls it **defect testing**). Each ❌ became a defect in Chapter 10, was fixed in its own patch version, and the test that found it now passes. On v0.32.0 all 38 automated test files pass (1,203 checks plus 147 rule checks; on v0.21.2 it was 22 files, 470 checks plus 58 rule checks).*

The three cases still open. TC-31.12 (typing the setup answers by hand) and **TC-39.10** (a real Google sign-in, which needs the owner's own Google Cloud client) are waiting for a hand test. **TC-34.12** is sending a real email through Gmail. I tried it on 25 October 2026: Gmail closed the connection at the login step on both of its ports, even for a made-up account. A wrong password would have been refused with a "535" answer instead, so the problem was not my App Password or the shop's code; Gmail was refusing logins from my home internet connection, while the connection, the encryption and Google's certificate were all fine. I then tested the whole code flow in **demo mode**, where each email is printed in the server window instead of sent

(TC-38.15): the codes for the admin and the cashier, wrong codes and the 5-try limit, a new code, an old code refused, the cashier's email confirmed by its first code, a trusted browser, and Forgot password. Everything worked. TC-34.12 stays open until it can be tried from another connection.

6. Requirements (items 4–10)

This chapter states what the system must do, in the forms taught in Lecture 4. The requirements were first written from v0.18.2, checked again against v0.21.2, and extended for v0.32.0 (UR-9 to UR-12 and the parts marked with a version). Each one can be traced to a feature (F…), a use case (UC-…) and a test case (TC-…), so the path from *requirement* to *design* to *test* is visible.

Two words are used carefully. A **user requirement** says what the users need, in everyday language. A **system requirement** says exactly what the software must do, in enough detail to build and test it. "Shall" means "must".

6.1 Requirements definition: user and system requirements (item 4, Lecture 4 p. 7)

User requirement definition	System requirements specification
UR-1 The system shall let a cashier sell products to a customer and give a receipt.	**1.1** A logged-in user (cashier or admin) shall add products to a basket with one tap on a menu tile (+ Hot / + Iced for drinks, + Add for other items; 1 each) or by choosing a product and typing a quantity, and shall change a line's amount with − / + (one at a time). **1.2** The system shall not let the basket hold more of a product than can be sold (stock − basket, or what the ingredients allow). **1.3** On *Complete Sale* the system shall check stock again, save the sale and its items, and lower the stock in one database transaction; if anything is short, nothing is saved. **1.4** The system shall show a printable receipt with receipt number, date and time, cashier, items, options, total and payment. **1.5** One basket shall make at most one receipt: if the same basket is sent twice (double-click, slow network, an old copy of the cookie), the second request shall open the first receipt and charge nothing. **1.6** Every receipt shall show the shop's name, address, TIN and footer as set by an admin, and shall end with "This is not an official receipt.", which can't be turned off (v0.22.0). **1.7 Undo** shall take the basket back one change at a time, for the last 5 changes; a completed sale shall never come back (v0.30.0).
UR-2 The system shall accept cash and GCash payments.	**2.1** For cash, the cashier shall type the amount received; the system shall refuse amounts below the total or above ₱1,000,000 and shall show the change; buttons shall offer the next ₱100, ₱500 and ₱1,000 above the total. **2.2** For GCash, the cashier shall type the 13-digit reference number; the system shall refuse anything else and any reference already used. **2.3** The Sales Report shall show cash and GCash revenue separately.
UR-3 The admin shall keep the list of products up to date.	**3.1** Only an admin shall add, edit or delete products (name, SKU, price, quantity, category, optional picture). **3.2** SKUs shall be unique, without regard to capitals (`hc-001` = `HC-001`); prices and quantities shall be numbers from 0 to 1,000,000. **3.3** A product with sales history shall not be deleted (its quantity can be set to 0). **3.4** Pictures shall be PNG, JPG or WebP up to 2 MB, checked by their content.
UR-4 Staff shall be warned before something runs out.	**4.1** Products with 5 or fewer left shall get a *Low* badge (0: *Out of stock*) and be listed in a warning box. **4.2** Ingredients at or below their *warn at* level shall be marked *Low* (0: *Out*), listed first under *Needs attention*, and counted in a warning on Home.

User requirement definition	System requirements specification
UR-5 A café shall be able to sell drinks Hot or Iced, with upgrades.	**5.1** The admin shall keep a list of *servings* (choose exactly one, e.g. Hot / Iced) and *add-ons* (any number, e.g. Oat Milk +₱40), each with an extra price. **5.2** One tap on a serving button shall add the drink with that serving; tapping the basket line shall let the cashier change the serving and add-ons, showing the new price before saving; the basket shall keep one line per product + serving + add-ons (equal lines are merged). **5.3** Receipts shall keep the options and the price paid even if prices change later.
UR-6 The café shall know how much of each ingredient it has.	**6.1** The admin shall record ingredients (name, unit g/ml/pcs, group, on hand, warn at, full level). **6.2** Each product, serving and add-on may have a recipe; every sale shall use up its ingredients in the same transaction as the sale. **6.3** *Restock* shall add a delivery; *Count* shall set what is really on the shelf; every change shall be written to the ingredient history with user, time and receipt.
UR-7 The owner shall see how the business is doing.	**7.1** An admin shall see revenue, number of sales, items sold, top 5 products and every sale for any dates from 2000 to 2100 (default: today). **7.2** Home shall show today's sales and revenue, products, low stock and low ingredients.
UR-8 Only the right people shall use each part.	**8.1** Users shall log in with a username and password **and** a 6-digit code (UR-10); passwords shall be stored hashed. **8.2** Cashiers shall get *403 Forbidden* on admin pages (products editing, report, servings and add-ons, ingredients, recipes). **8.3** After 5 wrong passwords for a username from one computer, that computer shall wait 5 minutes; the count shall be kept in the database, so a restart can't clear it, and Change password shall have the same limit (v0.26.0). **8.4** Every form shall carry a CSRF token; a form without it shall be refused. **8.5** Users shall be able to change their password (8+ characters, not the username) and their profile picture. **8.6** A login shall end on log-out, on a password change (every other login of that user) and after 8 idle hours; a copied cookie shall then be refused. **8.7** Every change to prices, stock, recipes, options, shop details, accounts and receipts shall be written to the audit log with who, when and the old and new values, in the same transaction as the change (v0.27.0).
UR-9 Every person shall have their own account, and nobody shall be able to give themselves one.	**9.1** There shall be no sign-up page. The owner account shall be made once, by `setup_shop.py`, with the owner's email; the database shall allow only one owner, who can't be switched off or demoted. **9.2** The owner (or an admin the owner allowed, for cashiers only) shall invite a person by email; the link shall work once, for 48 hours, and the role shall be the one the inviter chose. **9.3** An account shall be switched off, never deleted; switching it off shall end all its logins at once. **9.4** *Forgot your password?* shall send a code to a confirmed email and give the same answer for every address.

User requirement definition	System requirements specification
UR-10 A stolen or guessed password alone shall not be enough to log in.	**10.1** After the right password, the system shall ask for a 6-digit code, sent by email or shown by an authenticator app; the code shall work once, for 10 minutes, for one purpose, with at most 5 wrong tries. **10.2** A browser marked as trusted shall skip only the code, for 30 days (7 for admins and the owner); trust shall end on Remove, Log out everywhere, a password change, switching off or a role change. **10.3** Ten printed one-time backup codes shall work with no internet. **10.4** Codes, tokens and backup codes shall be stored only as keyed hashes. **10.5** Sign in with Google shall link an account by Google's permanent id, never by the email, and the code shall still follow.
UR-11 Mistakes and returns shall be corrected without changing a receipt.	**11.1** Anyone may ask for a void of today's receipt, with a reason of 1 to 200 characters. **11.2** Only an admin shall approve it, with their own password at the shop or with a code emailed to them (that void only, 10 minutes, once, 5 tries); everything shall go back on the shelf. **11.3** Only an admin shall refund chosen items of any receipt, each back on the shelf or as waste, never more than was paid. **11.4** A void or refund shall be a new record that names who asked and who approved, with its own slip and history lines; the receipt shall not change. **11.5** The Sales Report shall show net sales = gross − voids − refunds. **11.6** A stock count shall be saved only if the amount is still the one the admin saw, and may be undone for 5 minutes (v0.30.0).
UR-12 A new member of staff shall be able to learn each page on their own.	**12.1** Every page shall have a **?** button that opens that page's help: what it is for, a picture and the main steps. **12.2** A Help page shall show every topic the user may use; a cashier shall not see admin pages or steps. **12.3** A page without help shall make the automated tests fail.

6.2 Nonfunctional requirements (item 5, Lecture 4 p. 18)

Sommerville splits these into **product**, **organizational** and **external** requirements.

Type	ID	Requirement
Product: performance	NFR-1	Every page shall open in under 1 second on the shop's laptop with the demo data (17 products, about 50 sales).
Product: reliability	NFR-2	The system shall never sell more than is in stock, even when several cashiers complete sales at the same moment.
Product: robustness	NFR-3	Wrong or strange input (text instead of numbers, nan, huge numbers, huge ids, wrong files) shall give a message, never a server crash (HTTP 500).
Product: usability	NFR-4	A new cashier shall be able to complete a sale after a 15-minute introduction; a one-item sale shall take at most 5 taps.
Product: usability	NFR-5	The system shall work on a laptop and on a phone screen (390 px wide) without sideways scrolling, in a light and a dark theme.
Product: security	NFR-6	Passwords shall be stored only as hashes, and codes and tokens only as keyed hashes; the authenticator key shall be stored encrypted; the secret key shall not be in the source code; pages shall send anti-framing headers.
Product: space	NFR-7	The database for a week of café sales shall stay under 1 MB.

Type	ID	Requirement
Organizational: development	NFR-8	The code shall be managed in Git/GitHub with one branch and pull request per version, semantic version tags (`MAJOR.MINOR.PATCH`) and a test run before every merge. Every decision shall be recorded with the alternatives checked (`docs/decisions.md`).
Organizational: environment	NFR-9	The system shall run with Python 3.11+ and Flask on Windows, macOS or Linux, in Chrome, Edge or Firefox.
External: legal	NFR-10	Personal data (user names, email addresses, profile pictures) shall be handled in line with the Philippine Data Privacy Act of 2012 (RA 10173): collect only what is needed, keep it on the shop's own computer (local package), and never put a real person's email in the repository.
External: business	NFR-11	Money shall be shown in Philippine pesos with a ₱ sign and thousands separators (₱1,234.50).
Product: availability	NFR-12	Every page shall look and work the same without internet: no file, font or script may be loaded from outside the shop's computer (v0.19.4). The local package shall keep selling with no internet on trusted devices and with backup codes; features that need the internet (emailed codes, Google) shall say so on screen instead of failing silently (v0.28.0).
Product: accuracy	NFR-13	Money shall be stored as whole centavos and handled as exact decimals, so every receipt adds up; times shall be stored in UTC and shown on the shop's clock (UTC+8) (v0.23.1–v0.23.2).
Product: maintainability	NFR-14	Every database change shall be a numbered migration that upgrades an older database without losing data, with a backup taken first (v0.23.0).

6.3 Nonfunctional requirements metrics (item 6, Lecture 4 p. 21)

"Measured" values were measured on **2026-10-14 on v0.20.5** (cloud test machine, 2 CPUs, Python 3.13), with the generic demo data (17 products, 52 sales); each page was opened 30 times through a real local server. The first measurement (2026-10-09, v0.18.2) is kept in brackets for comparison.

Property	Measure	Target	Measured / evidence
Speed	Time to show a page	< 1 s	Median 3.5–18 ms, slowest 53 ms (New Sale, first opening) *(v0.18.2: 2–15 ms, slowest 41 ms)*
Speed	Time to complete a sale	< 1 s	Median 14 ms, slowest 22 ms (10 sales) *(v0.18.2: 5 ms, slowest 8 ms; the extra time is the sale-ticket check and login record added in v0.20.3–v0.20.4)*
Size	Database size after a week of sales	< 1 MB	72 KB *(v0.18.2: 56 KB; now also holds login records and sale tickets)*
Size	Page size sent to the browser	< 100 KB (without Bootstrap)	9.5–58 KB; the largest is New Sale (tiles, basket and ingredient panel in one page)
Ease of use	Taps for a one-item sale	≤ 5	**3 taps, no typing**: + Hot (or + Add), a quick-cash button, Complete Sale. With one add-on: 6 taps (+ Iced, the basket line, the add-on, Done, quick cash, Complete Sale) *(v0.18.2: 3 taps + typing the cash)*
Ease of use	Training time for a cashier	≤ 15 min	⏳ to measure with the café staff (hand test)
Reliability	Oversold items with 20 cashiers selling the last 5 at once	0	0 (5 sold, 15 refused; TC-7.26, TC-17.11)

Property	Measure	Target	Measured / evidence
Reliability	Receipts made when one basket is sent 2 or 5 times at once	1	1 (TC-23.11–23.12)
Robustness	Server crashes (HTTP 500) on bad input	0	0 in the 31-request regression, the number limits (TC-2.7, TC-7.28), the 14-input fuzz test (TC-7.31) and the audit's 6,417 fuzzing requests (2026-10-11)
Robustness	Time to restart after a failure	< 1 min	A few seconds (`start_shop.bat`, or `python app.py` for the demo); no data lost, because each sale is saved in one transaction
Portability	Screen widths without sideways scrolling	390 px and up	390, 1024, 1280, 1440, 1600, 1920 and 2560 px checked (TC-12.5, TC-18.13, TC-22.9, TC-22.12)
Portability	Operating systems	Windows, macOS, Linux	Windows 11 (my laptop), Linux (tests and GitHub Checks)
Availability	Outside requests when the internet is off	0	0 (TC-21.1–21.5; my Wi-Fi-off check on 2026-10-13)
Availability	Ways to log in with no internet	≥ 1	3 on v0.32.0: a trusted browser, a backup code, an authenticator app (TC-38.5, TC-38.8, TC-38.9)
Accuracy	Centavos lost over 1,000 small sales	0	0: the report total equals the sum to the centavo (TC-30.5, v0.32.0)
Security	Logins with the right password but no code (untrusted browser)	0	0 (TC-38.3–38.5, v0.32.0)
Maintainability	Pages without help	0	0 of 18 topics missing (TC-42.1, v0.32.0)
Security	Known attacks that still work	0	0 of 6 (TC-13.1–13.14), 0 of the audit's attacks after v0.20.4 (TC-20, TC-23), and 0 in my 7-round attack drill (2026-10-13); the audit's last open findings A5, A7, A8 and A11 closed in v0.21.3–v0.27.0

6.4 Requirements of each part (item 7, Lecture 4 p. 39)

Like the insulin pump example, the requirements are listed per part of the system.

Products (F1–F4, F13)

- P1. The system shall list every product with picture or icon, SKU, price, quantity (or "can make") and category.
- P2. The system shall refuse a product with an empty name or SKU, a duplicate SKU (ignoring capitals), a price or quantity below 0 or above 1,000,000, or text longer than the database column.
- P3. The system shall keep a product that has been sold; deleting it shall be refused with a message.

Sales (F7, F13)

- S1. The basket shall be kept per logged-in browser and emptied on logout.
- S2. Adding to the basket shall be refused when not enough can be sold, with a message naming the product.
- S3. Completing a sale shall lower the stock with an *atomic* (all-at-once) database update that only succeeds if enough is left.

- S4. A deleted product in a basket shall be taken out with a message.
- S5. − / + on a basket line shall change it by exactly one; − on the last one takes the line out (v0.20.0).
- S6. Each basket shall get a one-time sale ticket when its first item goes in; the database shall accept each ticket once (v0.20.4).

Payment (F7, F14)

- M1. Change shall be cash received − total, shown live while typing and calculated again on the server.
- M2. A GCash reference shall be exactly 13 digits (spaces allowed while typing) and unique.

Drink options (F15)

- O1. A drink shall have exactly one serving (the first is the default) and any number of add-ons.
- O2. A hidden option shall disappear from New Sale (tiles and basket-line panels) and from baskets, with a message; it shall never be deleted.

Ingredients (F16, F17)

- I1. A recipe product's "can make" count shall be the smallest of (on hand − basket needs) ÷ recipe amount over its ingredients.
- I2. Restock shall add; Count shall replace; both shall be written to the history.
- I3. *Full* shall be more than *warn at*, or 0 for no stock bar.

Reports (F9)

- R1. The report shall count sales from the start of the first day to the end of the last day.
- R2. Top products shall be grouped by product, not by name, so a renamed product counts once.

Users and security (F5, F6, F10, F12, F23–F26)

- U1. Admin-only pages shall return 403 to cashiers and send visitors to the login page.
- U2. Uploaded pictures shall get random file names and be checked by their first bytes.
- U3. Debug mode shall be off unless `FLASK_DEBUG=1`.
- U4. The cookie shall hold only a random login token; each login shall be a row on the server that log-out, a password change or 8 idle hours removes (v0.20.3).
- U5. An unknown username shall take as long to refuse as a wrong password (a dummy hash is checked) (v0.21.4).
- U6. A new code shall be sent at most once a minute and 5 times an hour per address and purpose (v0.27.0).
- U7. An authenticator code's 30-second step shall never be accepted twice (v0.28.0).
- U8. Demo mode shall be refused when real mail settings exist, and the demo passwords shall work only in demo mode (v0.28.0).

Receipts and shop (F21, F22)

- H1. Shop name: 1–60 characters; address and footer up to 120; TIN 9 to 14 digits (dashes and spaces allowed) or empty.
- H2. `setup_shop.py` shall never create the demo accounts, and shall refuse a weak owner password with a reason.

Voids, refunds and undo (F27, F28)

- V1. One void request per receipt may be waiting at a time; an approval shall change the request only if it is still waiting, so two admins can't approve it twice.
- V2. A refund shall be refused if it would give back more of a line than is left unrefunded.
- V3. Each receipt line shall keep the ingredients it used, so a refund puts back exactly that.
- V4. A count undo shall be refused if anything else changed the ingredient after the count.

Help (F29)

- E1. Each topic shall have a title, a summary, at least two steps and a WebP picture of at most 150 KB.
- E2. Help words shall be shown as text, never as HTML (S11).

6.5 Structured requirements (item 8, Lecture 4 p. 42–43)

The same form as Sommerville's insulin pump example.

Field	Complete a sale (UC-08)
Function	Complete sale
Description	Saves the basket as a sale, takes the payment, lowers stock and ingredients, and shows the receipt.
Inputs	Basket (from the session); payment method; cash received *or* GCash reference.
Source	Basket: built by the cashier on New Sale. Payment: typed by the cashier.
Outputs	`Sale` with its `SaleItem`s and the basket's ticket; lower product and ingredient stock; ingredient history lines; receipt page.
Destination	SQLite database; cashier's screen (receipt).
Action	If this basket's sale ticket is already on a receipt, open that receipt and stop (v0.20.4). Check every line (product exists, options still offered, enough stock and ingredients); check the payment; then in one transaction create the sale and items and subtract stock and ingredients with conditional updates. If any update finds too little, roll everything back and show a message. Otherwise empty the basket and show the receipt.
Requires	A logged-in user; a non-empty basket.
Pre-condition	Each product in the basket exists; stock and ingredients are enough for the whole basket.
Post-condition	Stock(after) = stock(before) − quantity sold, for every product and ingredient used; total = Σ (unit price × quantity).
Side effects	Low-stock and low-ingredient warnings may appear; the sale appears in the Sales Report and on Home.

Field	Add to basket (UC-08)
Function	Add product to basket
Description	Adds a quantity of one product (with serving and add-ons for drinks) to the basket.
Inputs	Product id; quantity (default 1); serving id; add-on ids.
Source	A serving button on a menu tile (+ Hot / + Iced / + Add), or the product list on New Sale; serving and add-ons can then be changed on the basket line (v0.20.0).
Outputs	Updated basket; New Sale page.
Destination	Session (the browser's basket).

Field	Add to basket (UC-08)
Action	Refuse unknown products, quantities below 1, servings or add-ons not offered. Work out what can still be sold (stock − basket, or what the ingredients allow). If enough, add to the line with the same product + options; otherwise show "only N more can be added".
Requires	A logged-in user.
Pre-condition	The product exists and is in stock.
Post-condition	Basket quantity of the line increased by the quantity; nothing saved in the database.
Side effects	The "left" count on the tile goes down.

Field	Restock an ingredient (UC-15)
Function	Restock ingredient
Description	Adds a delivery to an ingredient's stock.
Inputs	Ingredient id; amount received.
Source	Admin, from the Update window or the ingredient's page.
Outputs	Higher stock; one history line; message "Added … of …".
Destination	Database; Ingredients page (back at the same row).
Action	Refuse amounts that are not numbers, 0 or less, or above 1,000,000. Otherwise add the amount in one database step (so a sale at the same moment is never overwritten) and write a *restock* history line with the user.
Requires	A logged-in admin; CSRF token.
Pre-condition	The ingredient exists.
Post-condition	Quantity(after) = quantity(before) + amount.
Side effects	The Low/Out badge and the Home warning may disappear.

Field	Log in (UC-02)
Function	Log in
Description	Lets a user into the system.
Inputs	Username; password; CSRF token.
Source	Login form.
Outputs	The code page (`/verify`) with a code emailed to the user, or, on a trusted browser, a new `login_session` row and a cookie holding only its random token (v0.20.3); or an error message.
Destination	Browser (session cookie, HttpOnly, SameSite=Lax); the user's email.
Action	If this username has 5 wrong passwords from this computer in the last 5 minutes (counted in `login_attempt`), refuse (429). Otherwise compare the password with the stored hash (a dummy hash for an unknown username, so both take as long). If wrong, record a failure and show "Invalid username or password". If right and the account is switched off, refuse (403). If right and this browser is trusted for this user, log in. Otherwise remember the user as "half logged in" for 5 minutes and send a 6-digit code (or ask for the authenticator code); only the right code (or a backup code) creates the login record (v0.28.0).
Requires	A switched-on user account with an email (or an authenticator app).
Pre-condition	The user is not locked out.
Post-condition	The user is logged in with exactly one new login record, or is waiting for the code, or the failure count went up by 1.

Field	Log in (UC-02)
Side effects	None (the basket is emptied on logout, UC-03).

Field	Approve a void (UC-20)
Function	Approve void
Description	An admin agrees to cancel a receipt of today that a cashier asked to void.
Inputs	The void request; the admin's own password (or the code emailed to them); how the money goes back (cash, or GCash with the 13-digit reference); CSRF token.
Source	The waiting void at the top of the Sales Report, or on the receipt.
Outputs	The request marked *approved* with who and how; stock and ingredients back on the shelf; `void` history lines; an audit line; a printable slip.
Destination	Database; the admin's screen.
Action	Refuse if the user is not an admin (403), if the password or code is wrong, if the receipt is not from today, or if a GCash reference is missing, wrong or already used. Otherwise, in one transaction: change the request from *waiting* to *approved* only if it is still waiting (0 rows changed = another admin decided first: roll back), add back every product and ingredient the receipt used, write the history and audit lines, and commit.
Requires	A logged-in admin; a waiting void request.
Pre-condition	The receipt was sold today and is not already voided or refunded.
Post-condition	Stock(after) = stock(before) + what the receipt took, for every product and ingredient; net sales for today lower by the receipt's total; the receipt itself unchanged.
Side effects	The red count beside Sales Report goes down; the receipt shows VOIDED.

6.6 Tabular computation (item 9, Lecture 4 p. 45)

Like the insulin pump's *CompDose* table: **condition** on the left, **action** on the right.

Change for a cash payment (`complete_sale`)

Condition	Action
cash is not a number, `nan` or infinity	Refuse: "Cash received must be a number…"
cash < total	Refuse; change preview shows "Not enough cash"
cash > ₱1,000,000	Refuse
total ≤ cash ≤ ₱1,000,000	change = cash − total, exact to the centavo (both are whole centavos since v0.23.2); save the sale

What can still be added to the basket (`show_sale_page`, `can_make`)

Condition	Action
Product has no recipe	available = product quantity − quantity already in the basket
Product has a recipe	available = min over its ingredients of ⌊(on hand − basket needs) ÷ amount per item⌋
requested quantity ≤ available	Add to the basket
requested quantity > available	Refuse: "only *available* more can be added" (or names the missing ingredient)

Status of an ingredient (`stock_status`)

Condition	Action
on hand ≤ 0	Status *Out*: red badge, first in *Needs attention*
0 < on hand ≤ warn at	Status *Low*: amber badge, in *Needs attention*
on hand > warn at	Status *OK*
full > 0	Stock bar = min(100, on hand ÷ full × 100) %
full = 0	No bar ("No full level set")

Price of a drink line (`read_line`)

Condition	Action
Product has no options	unit price = product price
Drink with serving *s* and add-ons $a_1 \ldots a_n$	unit price = product price + price(s) + Σ price(a_i)
Any chosen option hidden or unknown	Take the line out of the basket with a message
Line total	subtotal = unit price × quantity (rounded to centavos)

Login attempt limit (`login`)

Condition	Action
failures for (username, computer) in the last 5 minutes ≥ 5	Refuse with 429 "Too many failed attempts"
password correct, browser trusted for this user	Log in; clear the failures
password correct, browser not trusted	Clear the failures; send a code and open the code page
password wrong	Add a failure row (a keyed hash of username and computer) with the time; show "Invalid username or password"

Checking a login code (`verify`, v0.28.0)

Condition	Action
no code open for this user and purpose, or older than 10 minutes	Refuse: ask for a new code
5 wrong tries on this code already	Refuse and close the code; a new one is needed
keyed hash of the typed code = the stored hash (compared in constant time)	Log in; mark the code used; trust the browser if ticked
otherwise	Count a wrong try; say how many tries are left

Net sales in the Sales Report (`sales_report`, v0.31.0)

Condition	Action
a receipt in the period	gross += its total
a void approved in the period	voids += the voided total
a refund made in the period (of any receipt, even an older one)	refunds += the amount given back

Condition	Action
always	net = gross − voids − refunds; cash and GCash each lowered by the money given back that way

Quick-cash buttons (`quick_cash`, v0.20.0)

Condition	Action
total ≤ 0 (empty basket)	No buttons
total > 0	For each step in ₱100, ₱500, ₱1,000: amount = ⌊total ÷ step⌋ × step + step (310 → 400, 500, 1,000)
two steps give the same amount	Show it once (600 → 700, 1,000)
amount > ₱1,000,000	Leave it out

Stock bar and status on a menu tile (`menu_tile`, v0.20.0)

Condition	Action
Product has a recipe	left = what the ingredients allow for each serving, after what the basket already needs; the ingredient that runs out first is named when left = 0
Product has no recipe	left = quantity − units already in the basket
left ≤ 0	Tile greyed, *can't be made* + the missing ingredient; buttons disabled
0 < left ≤ 5	Tile marked *Low* (amber bar)
left > 5	*OK*
any	Bar = min(100, left ÷ 20 × 100) % (20 or more = a full bar)

6.7 Detailed scenarios (item 10, Lecture 4 p. 62–63)

In the form of Sommerville's "collecting medical history" scenario.

Scenario 1: A morning sale paid by GCash

Initial assumption	The café is open. Juan (cashier) is logged in on the counter laptop. Milk, beans and pastries are in stock.
Normal	A customer orders an iced café latte with oat milk and a butter croissant. Juan opens **New Sale** and taps **+ Iced** on the *Cafe Latte* tile, then **+ Add** on the croissant. He taps the latte's basket line, ticks **Oat Milk** (the button shows the new price) and taps **Done**. The basket shows both lines and the total. The customer pays with GCash; Juan taps **GCash**, types the 13-digit reference from the customer's screen and taps **Complete Sale**. The receipt shows "Paid by GCash" and the reference; Juan prints it.
What can go wrong	The reference has 12 digits or was already used: the sale is refused with a message and Juan asks the customer to show the receipt again. The oat milk ran out between adding and paying (another till): the whole sale is cancelled with "was just used up" and Juan takes the line out. Juan taps Complete Sale twice because the screen seemed slow: the button greys out ("Saving…") and the basket can only be sold once, so there is one receipt (v0.20.4). The Wi-Fi is down: nothing changes, because every page, font and icon is stored on the laptop (v0.19.4).
Other activities	Other cashiers can sell at the same time; stock is protected by atomic updates.
System state on completion	A new Sale with 2 SaleItems; croissant stock −1; beans −18 g, oat milk −200 ml, ice −150 g, iced cups −1; history lines linked to the receipt; the sale appears in today's report and on Home.

Scenario 2: Receiving a milk delivery

Initial assumption	Maria (admin) is logged in. Home shows "2 ingredients running low".
Normal	Maria clicks the warning; Ingredients opens filtered to *Low or out*. She clicks **Update** on Fresh milk, taps *Fill up to full (+2,160)* (or types what was delivered) and **Add to stock**. The page comes back to the Fresh milk row, now green.
What can go wrong	She types letters or 0: refused with a message. She restocks the wrong ingredient: she uses **Count** to set the real amount; both changes stay in the history.
Other activities	Sales continue during the restock; the restock adds in one database step, so no sale is lost.
System state on completion	Fresh milk increased; one *restock* history line with Maria's name; the warning disappears when nothing is low.

Scenario 3: Closing time

Initial assumption	The café is closing; Maria is logged in.
Normal	Maria opens **Sales Report** (today by default) and reads revenue split into cash and GCash, number of sales, items sold and the top 5 products. She counts the cash drawer and compares it with the cash figure. On **Ingredients** she uses **Count** for the milk and cream left in the fridge. She logs out, which empties any basket.
What can go wrong	The cash in the drawer differs from the report: she opens the receipts listed in the report to find the sale. A count differs a lot from the system: the history shows each sale and change to explain it.
Other activities	A cashier may still be finishing a sale; the report includes it after it is saved.
System state on completion	Ingredient counts saved with their differences in the history; no user logged in.

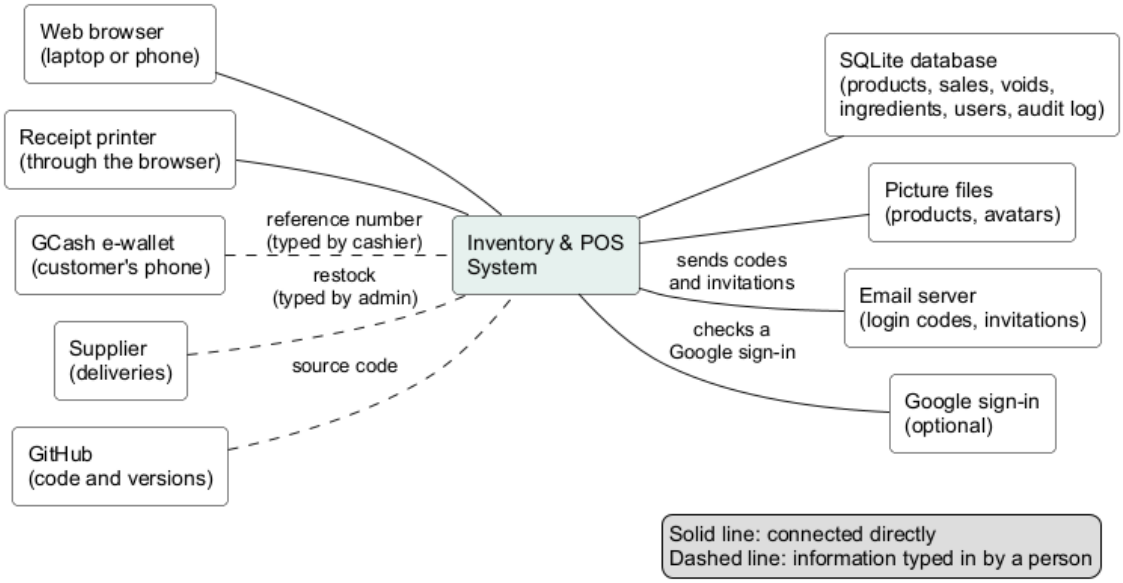
Scenario 4: A void when the owner is away *(v0.31.0)*

Initial assumption	Juan (cashier) sold a Spanish Latte ten minutes ago; the customer wanted a Matcha. Maria (owner) is at the market with her phone.
Normal	Juan opens **Today's receipts**, picks the receipt, taps **Void this receipt**, types "wrong drink rung up" and asks for the void. Because Maria is away, he picks her name and taps **Send**. Maria's email shows the receipt, the reason and a 6-digit code; the story matches, so she texts him the code. Juan types it; the receipt shows VOIDED, the milk and beans go back on the shelf, and a slip prints naming Juan as the one who asked and Maria as the one who approved. He rings up the Matcha as a new sale.
What can go wrong	Juan types the code wrong 5 times: it stops working and he asks for a new one. The email doesn't come because the Wi-Fi is down: the page says it couldn't be sent, and the void waits for an admin at the shop. A second admin approves the same void at the same moment: only the first approval counts; the second is told it was already decided, and the stock goes back once.
Other activities	Sales continue while the void waits; the waiting void shows at the top of the Sales Report with a red count beside the menu link.
System state on completion	A `sale_adjustment` of kind *void*, status *approved*, with both names; stock and ingredients back; `void` history lines; an audit line; today's net sales lowered by the receipt's total; the receipt itself unchanged.

7. System models (items 11–25)

7.1 Context model (item 12, Lecture 5 p. 10)

The context model shows the system (middle) and the other systems and devices around it. It answers *"where does our system end?"*: the database, picture files and (since v0.19.4) the Bootstrap, icon and font files belong to it; the browser, printer, GCash, supplier, GitHub and (since v0.26.0–v0.29.0) the shop's email server and Google's sign-in are outside it. *(Until v0.19.3 a CDN on the internet supplied Bootstrap, the icons and the font; v0.19.4 moved them inside the system so it works offline.)*



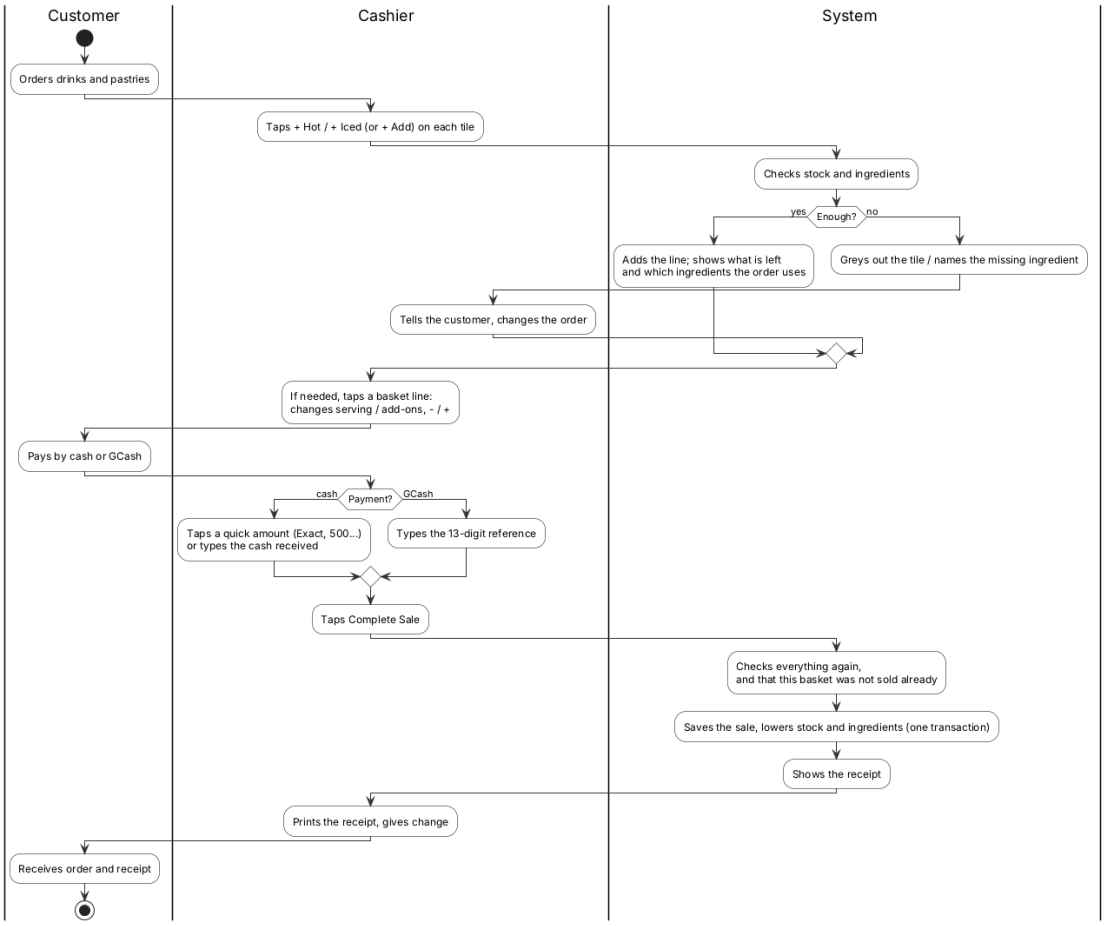
> **Why GCash and the supplier are dashed:** the system does not talk to them directly. A person reads the GCash reference or the delivery note and types it in. A direct GCash connection would need a merchant account (see known limitations). The **email server** and **Google** are solid lines: the system talks to them itself, over encrypted connections. Both are optional: without email the shop still sells (trusted devices, backup codes, an authenticator app), and Google sign-in is only offered when the owner has set it up.

*All UML diagrams in this report were drawn with **PlantUML**, a tool that turns a short text description into a UML picture. They were first drawn from the code of v0.20.4; every diagram that v0.21.3–v0.32.0 changed was redrawn from the v0.32.0 code on 27 October 2026, and the others were checked against it. Each picture is in `diagrams/` as a `.png`, with the `.puml` text it was made from next to it.*

7.2 Process model (item 13, Lecture 5 p. 12)

A **process model** shows the steps of a real-world job and **who does each step**. The columns are called **swimlanes**: each one belongs to one participant (here the Customer, the Cashier and the System). Reading

top to bottom, you follow one sale at the café counter from order to receipt. The diamond shapes are **decisions** (for example "Enough?"), where the flow splits into two paths.



> **What it shows about our design:** the System checks stock twice: once when a line is added to the basket and again when the sale is completed. This is because another till may sell the last item in between (see DEF-13, the race condition fixed with an atomic update). Since v0.20.0 a drink is added with one tap and changed afterwards on its basket line only if needed, so the common order is the shortest path; since v0.20.4 the final check also makes sure this basket was not already sold.

7.3 Use case diagrams (items 11 and 14, Lecture 4 p. 65 and Lecture 5 p. 15)

A **use case** is one thing a person can do with the system, written from their point of view ("Process sale"). An **actor** (the stick figure) is a type of user. Lines join actors to what they can do. Two kinds of dashed arrows appear:

- **«include»**: the use case *always* runs another one. "Add product" always runs "Check permission (role)".
- **«extend»**: the use case *sometimes* adds to another one. "View low-stock warning" only appears inside "View product list" when stock is low.

The use cases

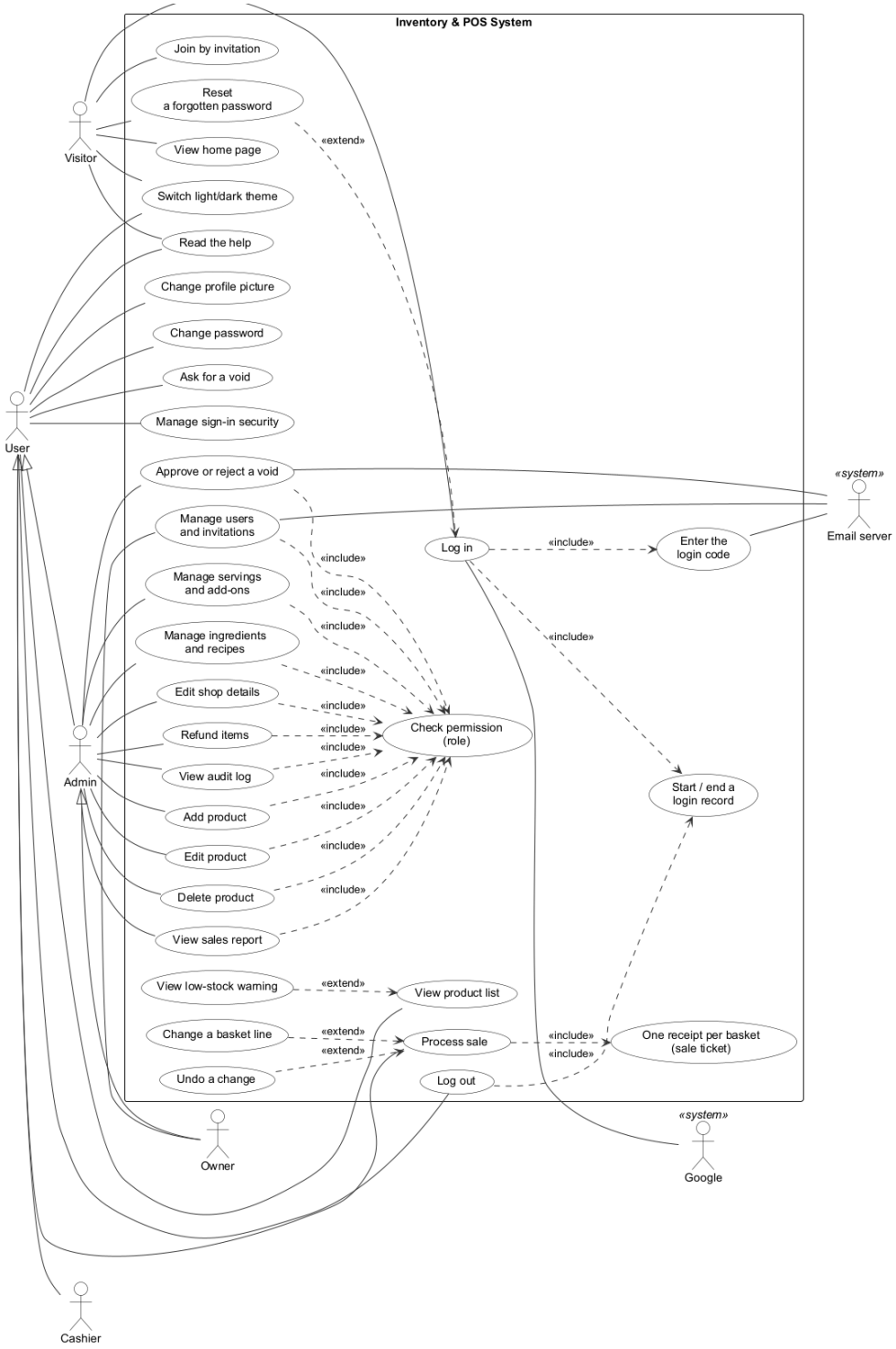
ID	Use case	Primary actor	Status
UC-01	View home page	Visitor	Built (v0.1.0)
UC-02	Log in	Visitor → becomes User	Built (v0.6.0)
UC-03	Log out	User	Built (v0.6.0)
UC-04	View product list	User (Admin and Cashier)	Built (v0.3.0)
UC-05	Add product	Admin	Built (v0.4.0)
UC-06	Edit product	Admin	Built (v0.5.0)
UC-07	Delete product	Admin	Built (v0.5.0)
UC-08	Process sale	Cashier and Admin	Built (v0.8.0, fixed v0.8.1)
UC-09	View low-stock warning	Admin and Cashier	Built (v0.9.0, fixed v0.9.1–v0.9.2)
UC-10	View sales report	Admin	Built (v0.10.0, fixed v0.10.1–v0.10.2)
UC-11	Change profile picture	User (Admin and Cashier)	Built (v0.12.0)
UC-12	Switch light/dark theme	Visitor and User	Built (v0.13.0)
UC-13	Change password	User (Admin and Cashier)	Built (v0.13.5)
UC-14	Manage servings and add-ons	Admin	Built (v0.16.0)
UC-15	Manage ingredients and recipes	Admin	Built (v0.17.0)
UC-16	Edit shop details	Admin	Built (v0.22.0)
UC-17	Manage users and invitations	Owner (Admin: invite cashiers, if allowed)	Built (v0.27.0)
UC-18	Join by invitation	Visitor → becomes User	Built (v0.27.0, Google v0.29.0)
UC-19	Ask for a void	User (Cashier and Admin)	Built (v0.31.0)
UC-20	Approve or reject a void	Admin	Built (v0.31.0)
UC-21	Refund items	Admin	Built (v0.31.0)
UC-22	View audit log	Admin	Built (v0.27.0)
UC-23	Reset a forgotten password	Visitor	Built (v0.27.0)
UC-24	Read the help	Visitor and User	Built (v0.32.0)
UC-25	Manage sign-in security	User	Built (v0.28.0, Google v0.29.0)

Relationships between use cases:

- UC-05, UC-06, UC-07, UC-08 and UC-10 all **«include»** a hidden step, *Check permission (role)*. «include» means "always happens as part of". The `@admin_required` decorator (in `common.py` since v0.25.0) is exactly this.
- UC-09 **«extend»s** UC-04: the low-stock warning appears on the product list *only when* some stock is low. «extend» means "sometimes adds extra behaviour, under a condition". Since v0.20.0 it also extends UC-08 (the "Running low" banner on New Sale).
- *(v0.20.0–v0.20.4)* UC-08 is extended by *Change a basket line* and *See the ingredients the order uses*, and includes *One receipt per basket (sale ticket)*. UC-02, UC-03 and UC-13 include *Start / end a login record*.
- *(v0.27.0–v0.31.0)* UC-02 **«include»s** *Enter the login code* (v0.28.0) and is **«extend»ed** by UC-23 *Reset a forgotten password*. UC-08 is extended by *Undo a change* (v0.30.0). Every admin use case, the new

ones too (UC-16, UC-17, UC-20, UC-21, UC-22), includes *Check permission (role)*. The **Email server** takes part in UC-02, UC-17, UC-18, UC-20 and UC-23, and **Google** in UC-02 and UC-18; they are drawn as actors marked «system».

All use cases together (item 11):



The arrow with a hollow triangle from Cashier and Admin to User means "a Cashier is a kind of User" (generalization, §7.8); the one from Owner to Admin means the owner can do everything an admin can. Whatever a User can do, a Cashier, an Admin and the Owner can do too. To keep the picture readable, the overall diagram leaves out "See the ingredients the order uses" and the low-stock banner on New Sale; they are on the UC-08 and UC-09 diagrams below.

One diagram per use case (item 14). The note on each diagram gives the main rule; the full description is in the tables in §7.4.

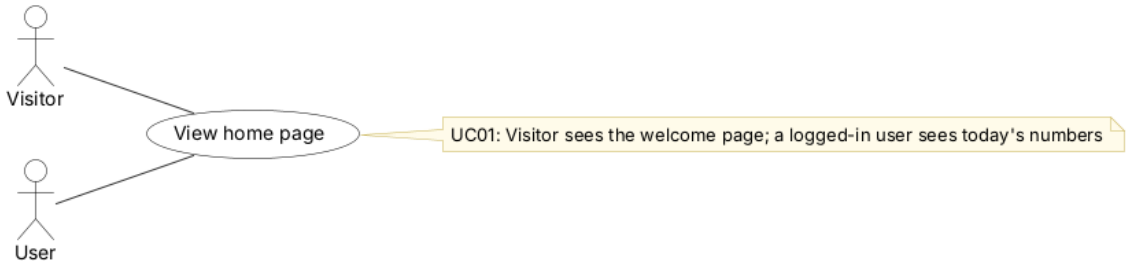


Figure: UC-01 View home page

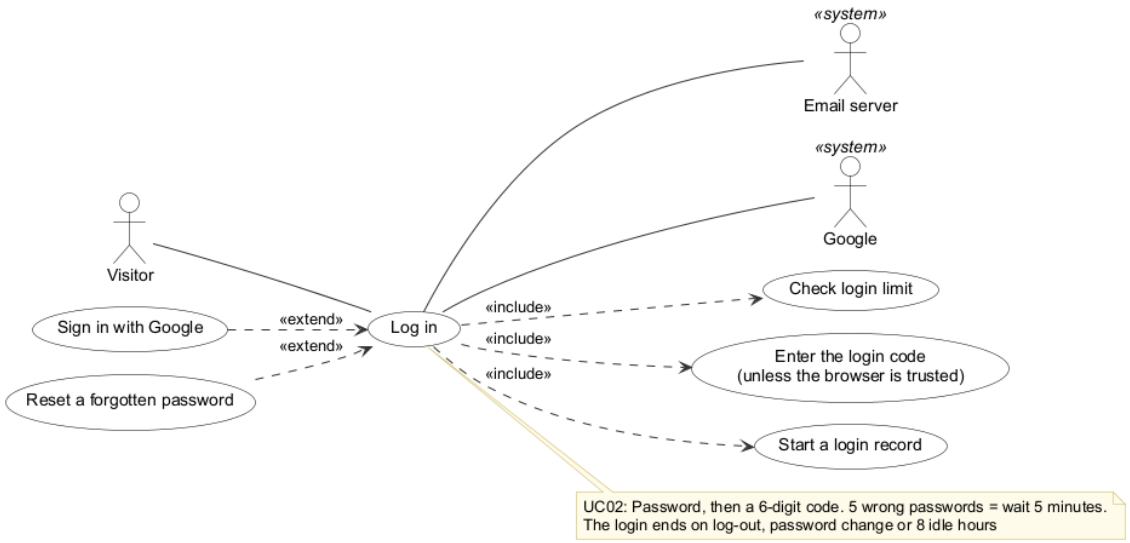


Figure: UC-02 Log in

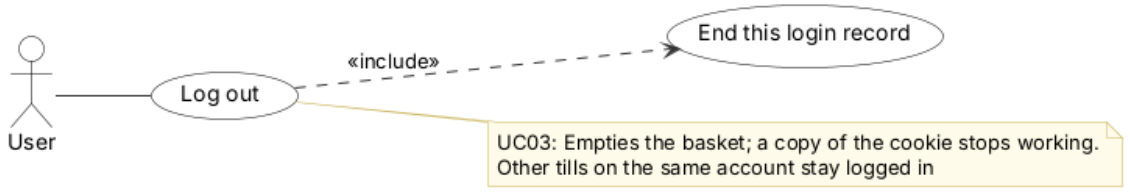


Figure: UC-03 Log out



Figure: UC-04 View product list

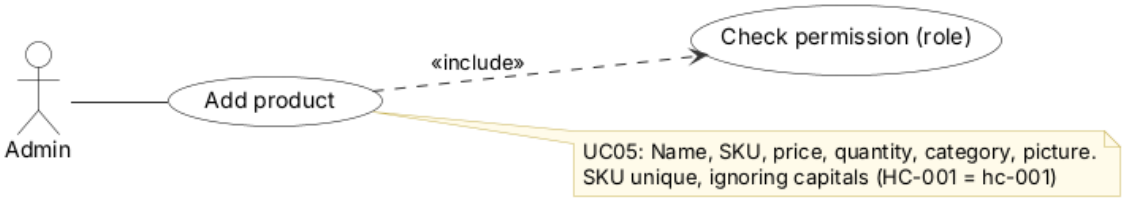


Figure: UC-05 Add product

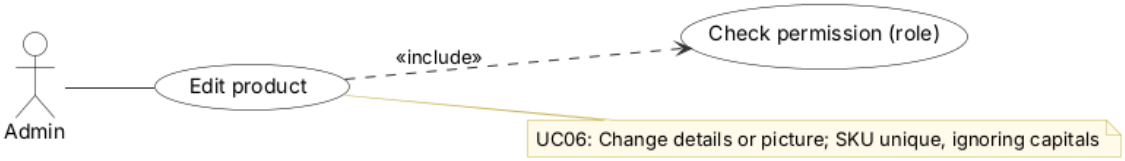


Figure: UC-06 Edit product

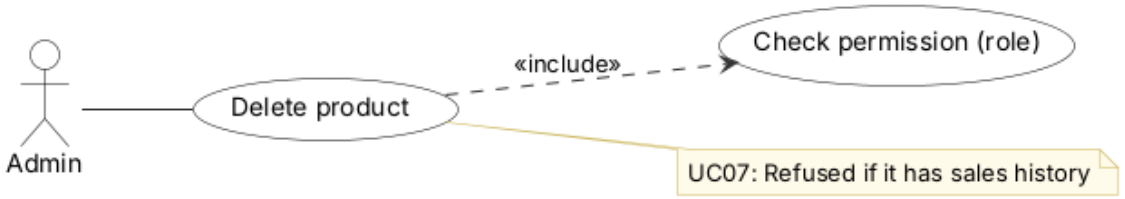


Figure: UC-07 Delete product

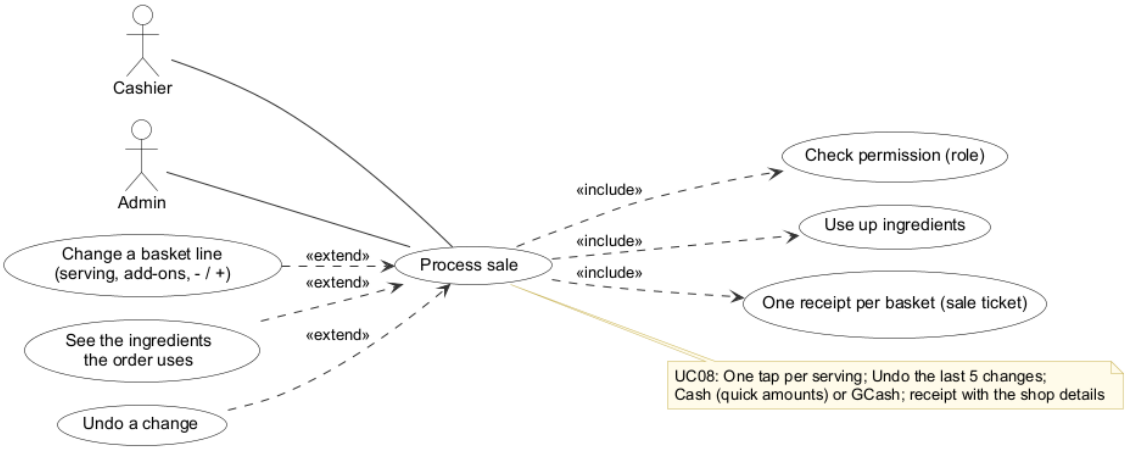


Figure: UC-08 Process sale

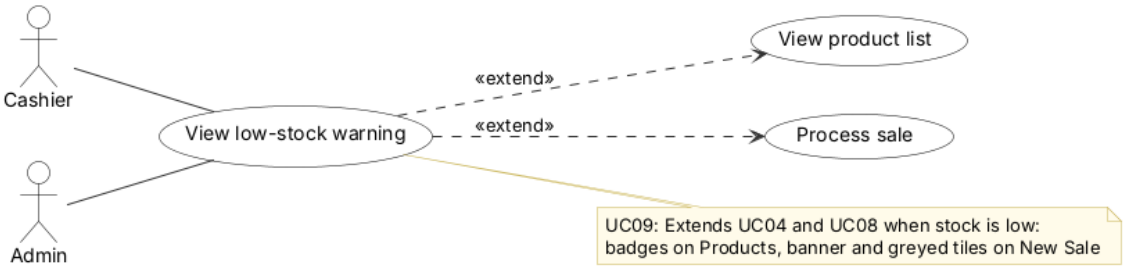


Figure: UC-09 View low-stock warning



Figure: UC-10 View sales report



Figure: UC-11 Change profile picture



Figure: UC-12 Switch light/dark theme

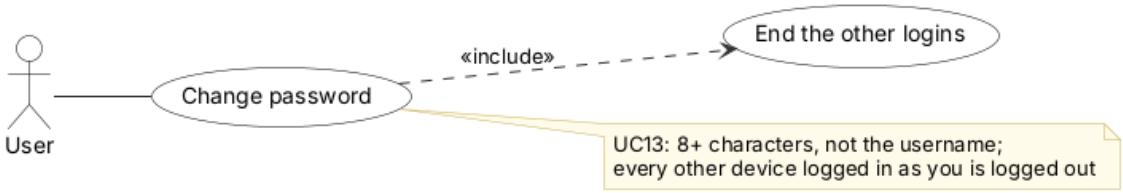


Figure: UC-13 Change password

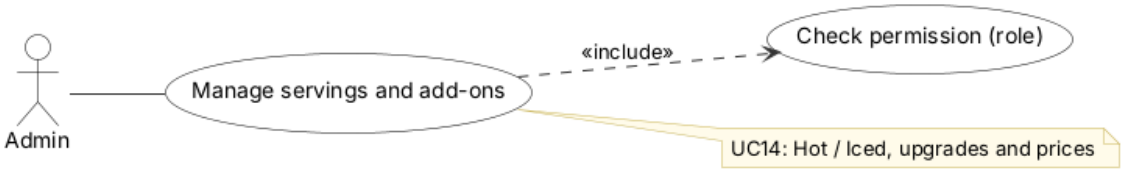


Figure: UC-14 Manage servings and add-ons

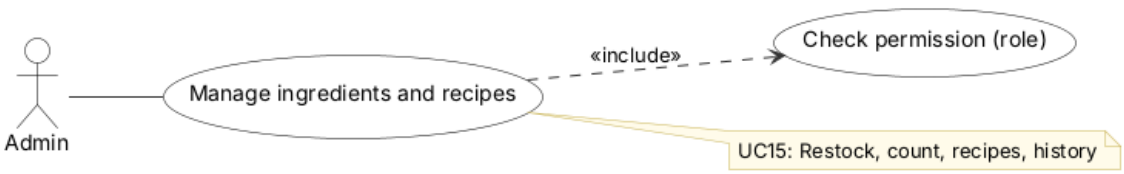


Figure: UC-15 Manage ingredients and recipes

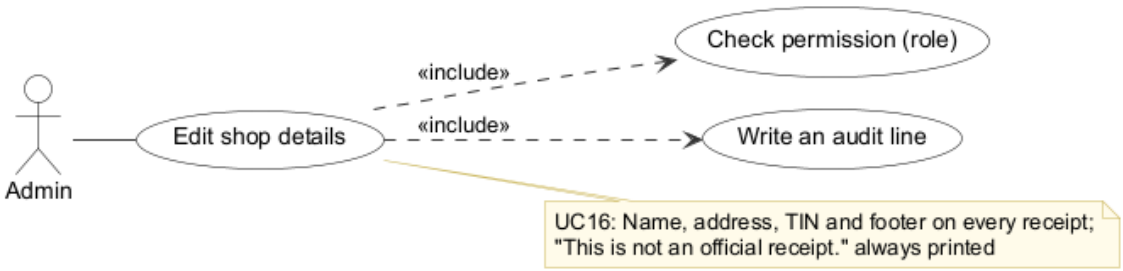


Figure: UC-16 Edit shop details

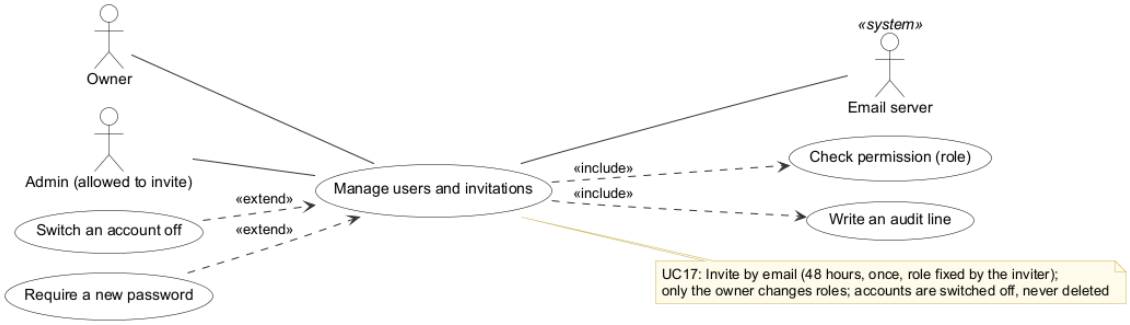


Figure: UC-17 Manage users and invitations

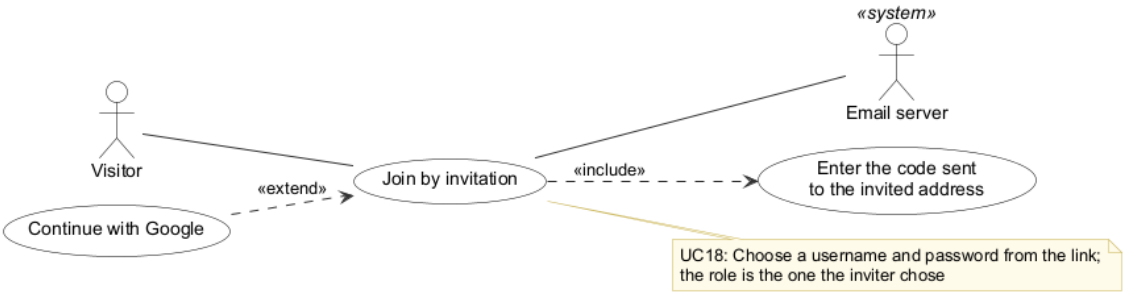


Figure: UC-18 Join by invitation

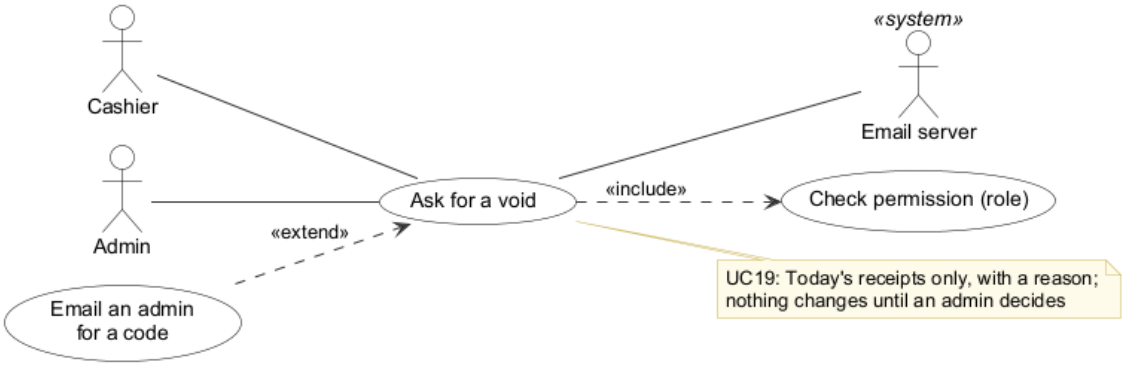


Figure: UC-19 Ask for a void

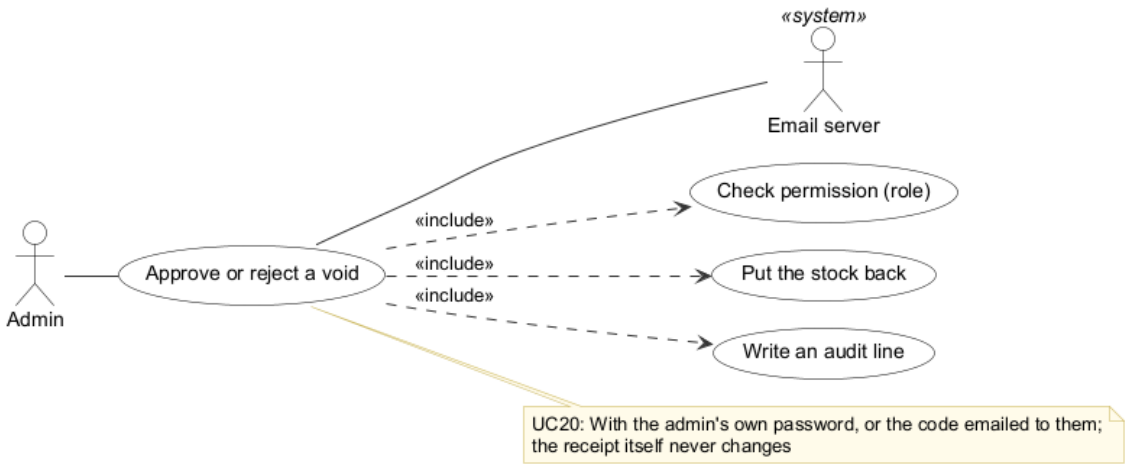


Figure: UC-20 Approve or reject a void

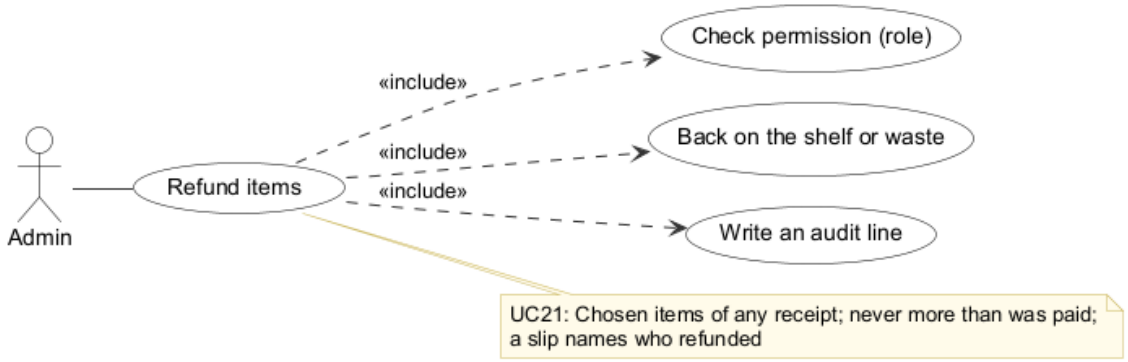


Figure: UC-21 Refund items

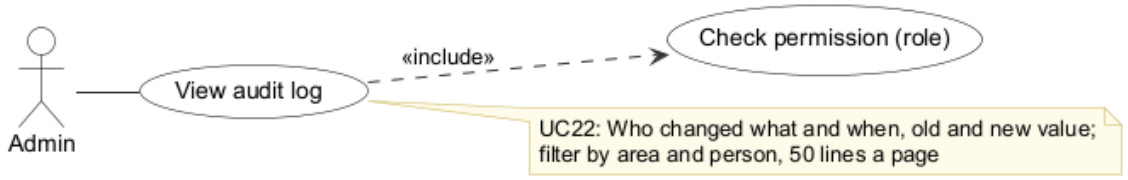


Figure: UC-22 View audit log

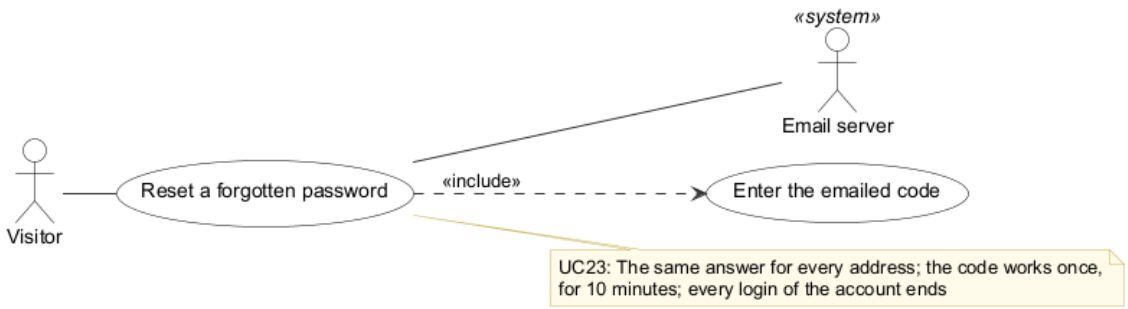


Figure: UC-23 Reset a forgotten password

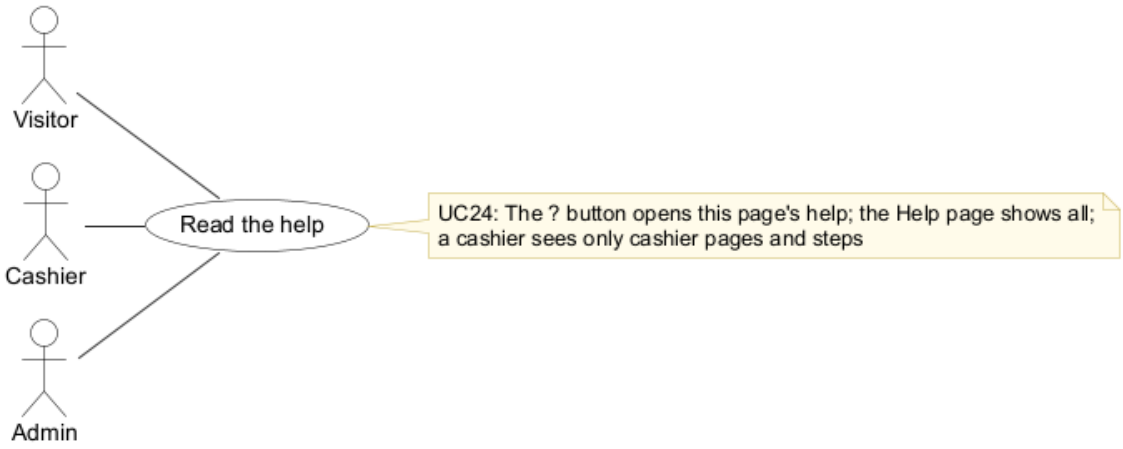


Figure: UC-24 Read the help

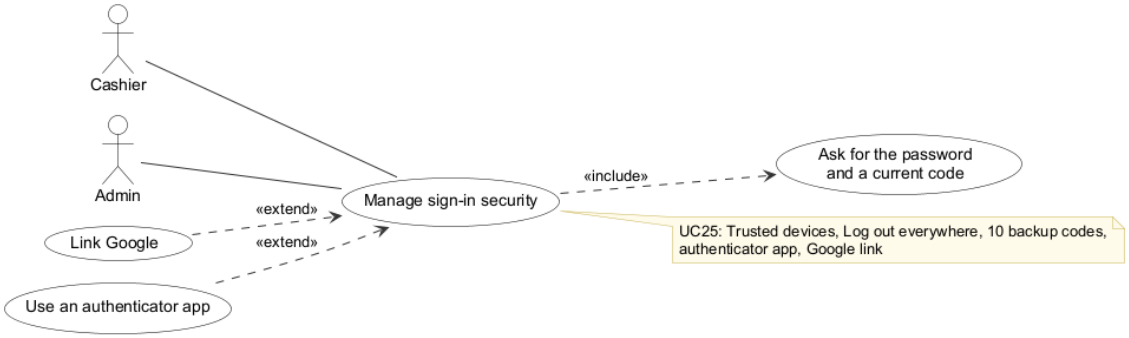


Figure: UC-25 Manage sign-in security

7.4 Tabular use case descriptions (item 15, Lecture 5 p. 16)

Each use case in Sommerville's format: *Actors, Description, Data, Stimulus, Response, Comments*. UC-01 (view the home page) is left out because it has no rules beyond showing the page.

UC-02 Log in

Actors	Visitor (becomes Admin or Cashier)
Description	A person enters a username and password to gain access according to their role
Data	Username, password, 6-digit code; stored User record (username, password hash, role, email, second step); trusted devices; email codes
Stimulus	The visitor submits the login form, then the code
Response	If the password matches and this browser is trusted for the user: a login record is created (one row in `login_session`; the cookie holds only its random token). If the password matches on another browser: a code is emailed (or the authenticator app is asked for) and only the right code creates the login record; "Trust this browser" skips the code there for 30 days (7 for admins). If not: "Invalid username or password" is shown
Comments	The password is checked against a hash and never stored in plain text. The same message, and the same time, for a wrong username or a wrong password, so attackers can't find out which usernames exist (v0.21.4). A backup code can replace the code; Sign in with Google replaces the password, never the code (v0.28.0–v0.29.0)

UC-03 Log out

Actors	User (Admin or Cashier)
Description	The user ends their session
Data	The current session
Stimulus	The user clicks Logout
Response	This browser's login record is deleted and the session (with the basket and its undo copies) is cleared; the user is sent to the home page. A copy of the cookie stops working too (v0.20.3). Other tills logged in with the same account stay logged in; **Log out everywhere** on the Profile page ends them all and every trusted device (v0.28.0)
Comments	Important on a shared counter computer, so the next person can't act under someone else's account. A login also ends by itself after 8 idle hours (v0.20.3)

UC-04 View product list

Actors	Admin, Cashier
Description	Shows every product with its name, SKU, price, quantity and category
Data	All Product records
Stimulus	The user opens the Products page
Response	A table of products. Admins also see Add, Edit and Delete buttons; cashiers see "View only"
Comments	Not available to visitors who haven't logged in (they are sent to the login page)

UC-05 Add product

Actors	Admin
Description	Creates a new product record
Data	Name, SKU (unique, ignoring capitals since v0.20.2), price, quantity, category (optional)
Stimulus	The admin submits the Add Product form
Response	The product is saved and the admin is returned to the updated list
Comments	Includes *Check permission*: cashiers get 403. Defects DEF-01 to DEF-04 (crashes and unchecked values) were fixed in v0.7.1

UC-06 Edit product

Actors	Admin
Description	Changes the details of an existing product
Data	Product ID; new name, SKU, price, quantity, category
Stimulus	The admin clicks Edit, changes the pre-filled form and clicks Save Changes
Response	The record is updated and the admin is returned to the list. An unknown ID gives 404
Comments	Includes *Check permission*. A duplicate SKU (DEF-01) is refused with a message since v0.7.1

UC-07 Delete product

Actors	Admin
Description	Permanently removes a product
Data	Product ID

Stimulus	The admin clicks Delete and confirms the pop-up
Response	The product is removed and the list is shown without it. **Since v0.10.2:** a product with sales history is kept and a message suggests setting its quantity to 0
Comments	Includes *Check permission*. Only works as a POST (form submission), not a link, for safety. Keeping sold products protects receipts and reports (DEF-21)

UC-08 Process sale *(built v0.8.0)*

Actors	Cashier, Admin
Description	Records a customer purchase of one or more products, takes cash, gives change and lowers stock
Data	Basket (in the session): product IDs and quantities. Saved: **Sale** (date and time, user, total, cash received, change due) and one **SaleItem** per line (product, name and unit price *at the time of sale*, quantity)
Stimulus	The user adds items to the basket, enters the cash received and clicks Complete Sale
Response	Stock and cash are checked; the sale and its items are saved and stock reduced in **one transaction**; the basket is emptied; a printable receipt is shown. Invalid input → message, nothing saved
Comments	Includes *Check permission* (must be logged in) and, since v0.20.4, *One receipt per basket* (a one-time sale ticket). Stock can't go negative through normal use; simultaneous sales of the last item are DEF-13 (v0.8.2). Receipts keep a snapshot of names and prices, so old receipts stay correct; each line also keeps the ingredients it used (v0.31.0), so a refund puts back exactly that. Since v0.20.0 each tile has one button per serving (+ Hot / + Iced, or + Add) that adds one with a single tap; three optional steps «extend» this use case: *Change a basket line* (serving, add-ons, − / +), *See the ingredients the order uses* and *Undo a change* (the last 5, v0.30.0). Payment has quick cash buttons. The receipt shows the shop's details and "This is not an official receipt." (v0.22.0)

UC-09 View low-stock warning *(built v0.9.0)*

Actors	Admin, Cashier
Description	Highlights products whose quantity is at or below the threshold
Data	Product quantities; `LOW_STOCK_THRESHOLD = 5` (one value for all products)
Stimulus	Opening the product list when at least one product is low
Response	Yellow box listing the 5 most urgent (lowest first) plus "and N more"; red "Out of stock" badge at 0, yellow "Low" badge at 1–5
Comments	«extend»s UC-04 View product list and, since v0.20.0, UC-08 Process sale (a "Running low" banner and greyed tiles naming the missing ingredient on New Sale). Updates automatically after sales and restocking

UC-10 View sales report *(built v0.10.0)*

Actors	Admin
Description	Summarizes sales over a chosen period
Data	Sale and SaleItem records; From and To dates (default today; allowed 2000–2100)
Stimulus	The admin opens Sales Report and optionally chooses a date range
Response	Voids waiting for an admin first (UC-20); then gross sales, voids, refunds and **net** sales, cash and GCash after money given back, number of sales, items sold, top 5 products by units (grouped per product), and every sale with a link to its receipt and a VOIDED / REFUNDED badge
Comments	Includes *Check permission* (admin only, 403 for cashiers). Depends on UC-08. Uses the price snapshot in each SaleItem, so later price changes don't alter past revenue. Days are counted on the shop's clock, though times are stored in UTC (v0.23.1). A refund counts on the day it was given, even for an older receipt (v0.31.0)

UC-11 Change profile picture *(built v0.12.0)*

Actors	Admin, Cashier
Description	Lets a user replace the initials circle with a round photo, or remove it
Data	An image file (PNG, JPG or WebP, at most 2 MB); stored in `static/avatars/` under a random name; the file name is saved in the user's `avatar` column
Stimulus	The user clicks their avatar in the sidebar and chooses Change picture
Response	The new picture appears on every page; the previous file is deleted. Wrong type or size gives a message instead
Comments	The file's content (first bytes) is checked, not just its name. Random file names stop users from overwriting each other's pictures

UC-12 Switch light/dark theme *(built v0.13.0)*

Actors	Visitor, Admin, Cashier
Description	Switches the look between "Warm cream" (light, caramel; "Clean counter" white and teal before v0.21.0) and "Night shift" (dark, amber)
Data	The choice, saved in the browser's `localStorage`
Stimulus	Clicking the sun / moon icon beside the user's name (or on the login page)
Response	Every page changes colour at once; the choice is kept after reloading. Printing always uses light
Comments	Runs entirely in the browser; the server and database are not involved

UC-13 Change password *(built v0.13.5)*

Actors	Admin, Cashier
Description	Replaces the user's password, e.g. the default `admin123`
Data	Current password, new password typed twice; stored as a hash in `user.password_hash`
Stimulus	Avatar menu → Change password, fill in the form, click Change password
Response	Password changed with a success message saying how many other devices were logged out (v0.20.3: every other login of this user ends and this browser gets a new token); or a message if the current password is wrong, the two don't match, it equals the username or is shorter than 8 characters
Comments	Includes *Check permission* (must be logged in) and the CSRF token check. Since v0.26.0 the current password has the login page's limit (5 wrong tries, then 5 minutes), so a stolen login can't be used to guess it (DEF-69)

UC-14 Manage servings and add-ons *(built v0.16.0, renamed v0.17.2)*

Actors	Admin
Description	Keeps the list of servings (Hot / Iced, or sizes; exactly one per drink) and add-ons (any number) with their extra prices
Data	`menu_option`: kind (size/add-on), name, extra price, offered or hidden, order, recipe scale (sizes)
Stimulus	Admin opens Servings & Add-ons, edits a row or adds one
Response	Saved; hidden options disappear from New Sale (tiles and basket-line panels); baskets holding them lose that line with a message

Comments	Includes *Check permission* (admin only). Options are hidden, never deleted, because receipts and recipes refer to them

UC-15 Manage ingredients and recipes *(built v0.17.0)*

Actors	Admin
Description	Records ingredient stock (restock, count) and what each product, size and add-on uses
Data	`ingredient` (name, unit, on hand, warn at; group and full level since v0.18.0), `recipe_item` (ingredient, amount, product or option), `ingredient_movement` (history)
Stimulus	Admin opens Ingredients or a product's / option's Recipe page
Response	Stock and recipes saved; the menu's "left" counts follow; low ingredients are flagged on Home. Since v0.18.0 the page is grouped and searchable, with stock bars, an Update window and a History tab
Comments	Includes *Check permission*. UC-08 *Process sale* now uses ingredients up, so UC-08 depends on this use case. Since v0.30.0 a count is saved only if the amount is still the one the admin saw, and can be undone for 5 minutes

UC-16 Edit shop details *(built v0.22.0)*

Actors	Admin
Description	Sets the shop's name, address, TIN and receipt footer
Data	`shop_profile` (one row): name (1–60 characters), address and footer (up to 120), TIN (9 to 14 digits, or empty)
Stimulus	Admin opens Shop details, fills in the form and clicks Save shop details
Response	"Shop details saved"; every receipt, old ones too when reprinted, shows them. A wrong value is refused with its own message and nothing changes
Comments	Includes *Check permission* (403 for cashiers). The line "This is not an official receipt." is not part of the form and can't be turned off (SR6). The change is written to the audit log

UC-17 Manage users and invitations *(built v0.27.0)*

Actors	Owner; an Admin the owner allowed (cashiers only); Email server
Description	Invites people, changes roles, switches accounts off or on, requires a new password
Data	`user` (role, email, is_active, must_change_password, can_invite); `invitation` (email, role, keyed hash of the token, 48-hour expiry)
Stimulus	The owner opens Users and types an email and a role, or uses a button on a person's row
Response	Invite: an email with a one-time link, listed with Send again / Cancel. Switch off: every login of that account ends at once. Change role: only the owner; the owner's own row can't be changed
Comments	Includes *Check permission*. There is no sign-up page (S12): an account exists only through the owner or an invitation. Accounts are never deleted, because receipts name their cashier. Every change is in the audit log

UC-18 Join by invitation *(built v0.27.0; Google v0.29.0)*

Actors	Visitor (becomes Cashier or Admin); Email server; Google
Description	A person invited by email makes their account
Data	The link's token; a username and password (or a Google account for the invited address); a 6-digit code
Stimulus	The visitor opens the link and fills in the form, then the code sent to the invited address

Response	The account is made with the role the inviter chose and a confirmed email, and the person is logged in; the link stops working. A wrong, used, cancelled or expired link answers 404
Comments	Nothing typed can change the role (a forged `role=admin` is ignored). Two attempts at once make only one account

UC-19 Ask for a void *(built v0.31.0)*

Actors	Cashier, Admin; Email server
Description	Asks for a receipt of today to be cancelled
Data	The receipt; a reason (1–200 characters); optionally the admin to email
Stimulus	On the receipt (or Today's receipts), the user taps Void this receipt and types the reason
Response	"Void requested"; the request waits at the top of the Sales Report and the menu shows a red count. If no admin is in the shop, the cashier can send a code to an admin's email (UC-20)
Comments	Nothing changes yet: no stock comes back and the receipt is the same. Only today's receipts; one waiting request per receipt; a partly refunded receipt can't be voided

UC-20 Approve or reject a void *(built v0.31.0)*

Actors	Admin; Email server
Description	Decides a waiting void
Data	The request; the admin's password or the emailed code; how the money goes back (cash, or GCash with its reference); a note when rejecting
Stimulus	The admin approves at the top of the Sales Report or on the receipt, or gives the cashier the code from their email
Response	Approved: the receipt shows VOIDED, every product and ingredient goes back, a slip names who asked and who approved. Rejected: nothing changes
Comments	Includes *Check permission* (403 for cashiers, S15). A conditional update makes sure two admins can't approve the same void twice. Only while the receipt is from today. Structured specification in 6.5

UC-21 Refund items *(built v0.31.0)*

Actors	Admin
Description	Gives back money for chosen items of any receipt
Data	For each line: how many, and back on the shelf or waste; the admin's password; cash or GCash
Stimulus	Admin opens a receipt, taps Refund items, chooses the items and confirms with their password
Response	A refund record and slip; the money to give back; items marked "on the shelf" go back to stock, the others are written as waste
Comments	Includes *Check permission*. Never more of a line than is left unrefunded, checked in the same database step as the save. A refund counts in the report on the day it was given

UC-22 View audit log *(built v0.27.0)*

Actors	Admin
Description	Shows who changed what
Data	`audit_log`: time, person, action, target, old → new values

Stimulus	Admin opens Audit log, optionally filters by area or person
Response	The newest 50 lines, with "Older" for more
Comments	Includes *Check permission*. Each line is written in the same transaction as the change it describes, so a change can't happen without its line. A save that changes nothing writes no line

UC-23 Reset a forgotten password *(built v0.27.0)*

Actors	Visitor; Email server
Description	Lets a person with a confirmed email choose a new password
Data	Email address; 6-digit code; new password typed twice
Stimulus	*Forgot your password?* on the login page
Response	Always the same answer, whatever the address; a code only to a switched-on account with that confirmed email. The right code saves the new password and ends every login of the account
Comments	«extend»s UC-02. At most 10 requests per computer an hour; codes as in UR-10

UC-24 Read the help *(built v0.32.0)*

Actors	Visitor, Cashier, Admin
Description	Shows how to use the page the person is on, or every page
Data	`help_topics.py` (title, summary, steps, who each step is for); a picture in `static/help/`
Stimulus	The **?** button, or Help in the avatar menu
Response	A side panel with that page's help, or the Help page with every topic the person may use
Comments	A cashier never sees admin topics or steps. The automated tests fail if a page has no topic

UC-25 Manage sign-in security *(built v0.28.0; Google v0.29.0)*

Actors	Cashier, Admin
Description	The Profile page's sign-in settings
Data	Trusted devices; backup codes; the authenticator key (encrypted); the Google link
Stimulus	Buttons on Profile: Remove a device, Log out everywhere, Make backup codes, Use an authenticator app, Link Google
Response	Each change takes effect at once; making backup codes shows 10 codes once; switching the app on or off sends an email
Comments	The bigger changes need the password and a current code. Linking Google needs the same, confirmed email address on both sides

7.5 Each actor's use cases (item 16, Lecture 5 p. 17)

The same use cases again, grouped by **who** uses them. This makes it easy to check permissions: everything on the Admin diagram that is not on the Cashier diagram must be refused (403 Forbidden) when a cashier tries it. Test cases TC-6.x in §5 check exactly this.

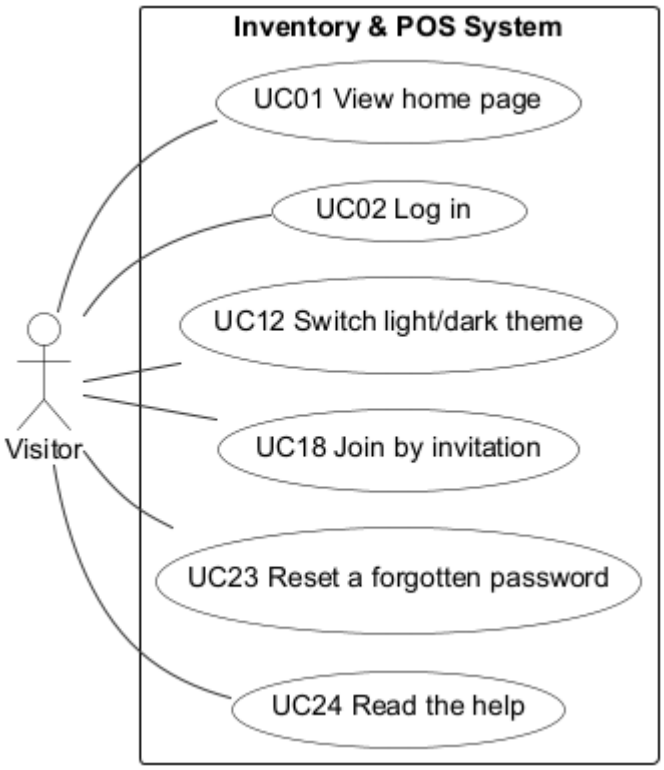


Figure: Visitor

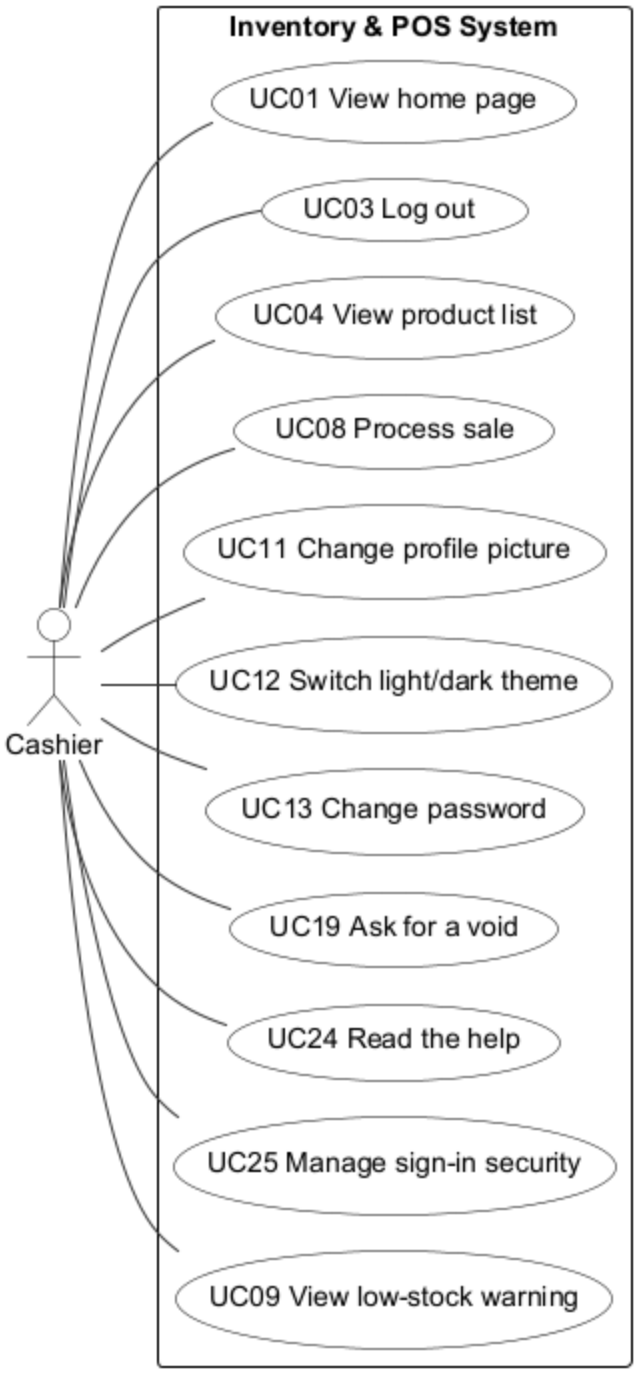


Figure: Cashier

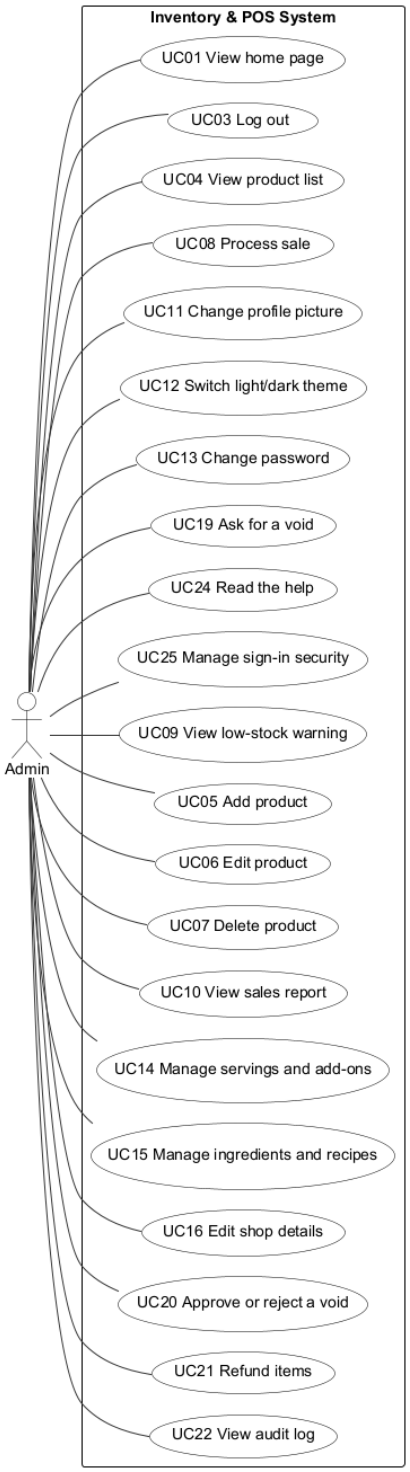


Figure: Admin

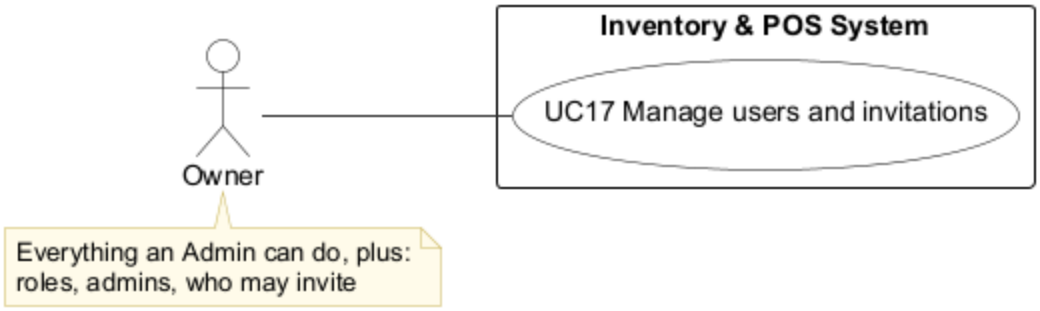


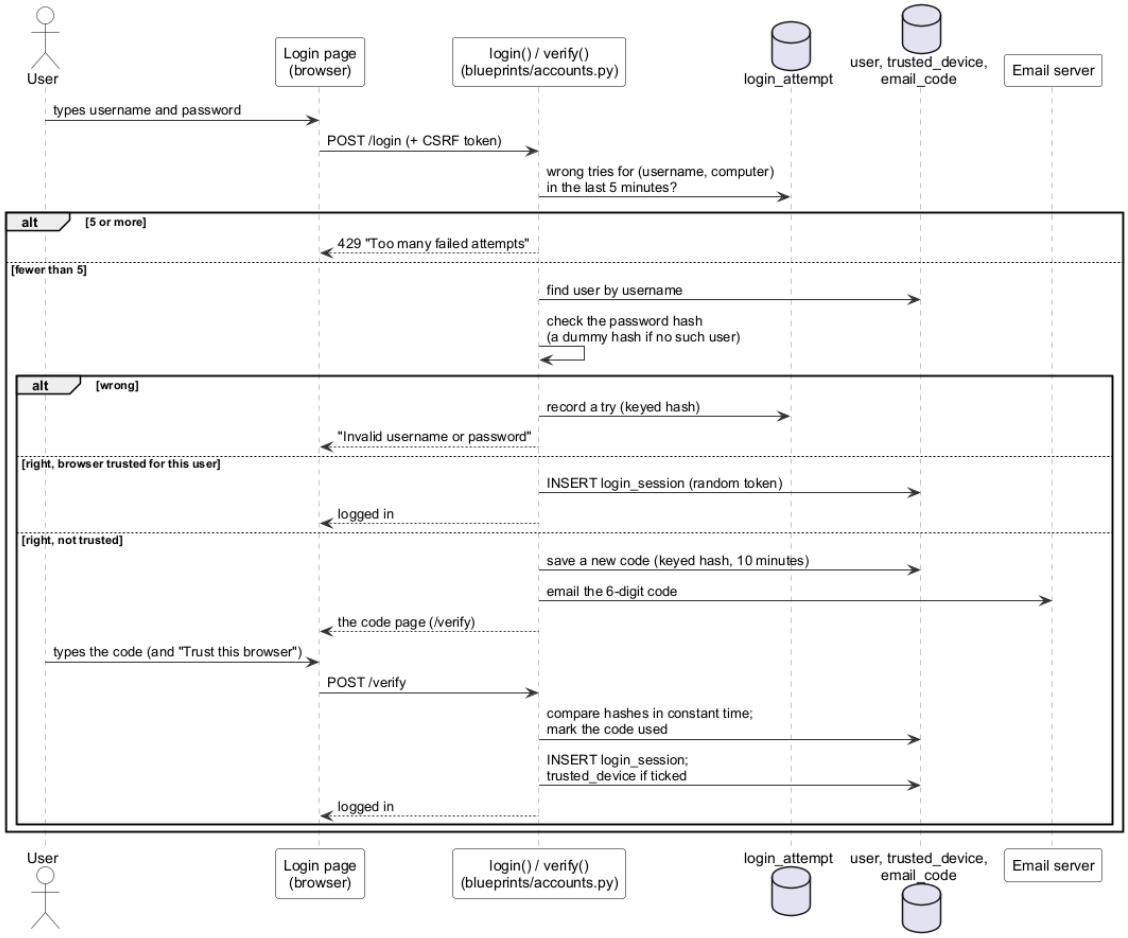
Figure: Owner (everything an Admin can do, plus UC-17)

7.6 Sequence diagrams (item 17, Lecture 5 p. 19–20)

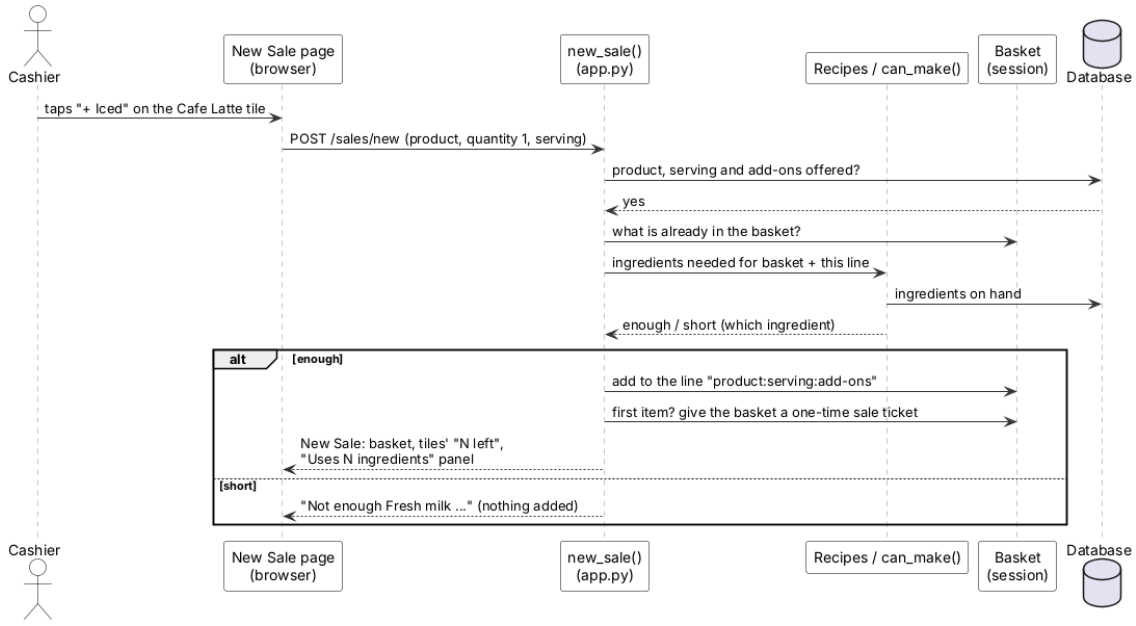
A **sequence diagram** shows the messages passed between the parts of the system, **in time order from top to bottom**. Each part has a dashed vertical line (its **lifeline**). A solid arrow is a request; a dashed arrow is the reply. Boxes marked **alt** mean "only one of these paths happens" (like an if/else), and **loop** means "repeat for each". The names in brackets, like `login()` in `blueprints/accounts.py`, are the real function names, so each step can be found in the code.

#	Action	What to notice
a	Log in (v0.28.0)	The login limit is checked *before* the password, so a guesser is stopped even with the right password after 5 tries. An unknown username still costs one hash check. Unless the browser is trusted, the right password only sends a code; the login record is created after the right code.
b	Add a drink to the basket	One tap on "+ Iced" (v0.20.0). Ingredients are checked for the *whole basket plus the new line*, not just the new line. The first item gives the basket its one-time sale ticket.
c	Complete a sale	A **break** box comes first: if this basket's ticket is already on a receipt, that receipt opens and nothing else happens (v0.20.4). Each stock subtraction only succeeds if enough is still there; if any fails, everything is **rolled back** (undone).
d	Restock an ingredient	One database step adds the amount and writes the history line.
e	View the sales report	Dates are checked to be real and between 2000 and 2100 before the database is asked.
f	Add a product	The picture is checked by its first bytes, not its file name, and saved under a random name.
g	Change password	Four checks; if any fails, nothing changes. Since v0.20.3 every login of the user ends and this browser gets a new token, so a stolen copy of the cookie dies too.
h	Upload a profile picture	The old picture file is deleted so the folder does not fill up.
i	Change a basket line (v0.20.0)	The changed basket is checked as a whole; identical lines are joined. The − / + buttons follow the same path.
j	Every request, and log out (v0.20.3)	Each request looks up the login record; no record or 8 idle hours → log-in page. Log out deletes only this browser's record.
k	Ask for and approve a void (v0.31.0)	The cashier's request changes nothing. The approval changes the request and the receipt only if they are still as the admin saw them; if another admin was first, everything is rolled back.

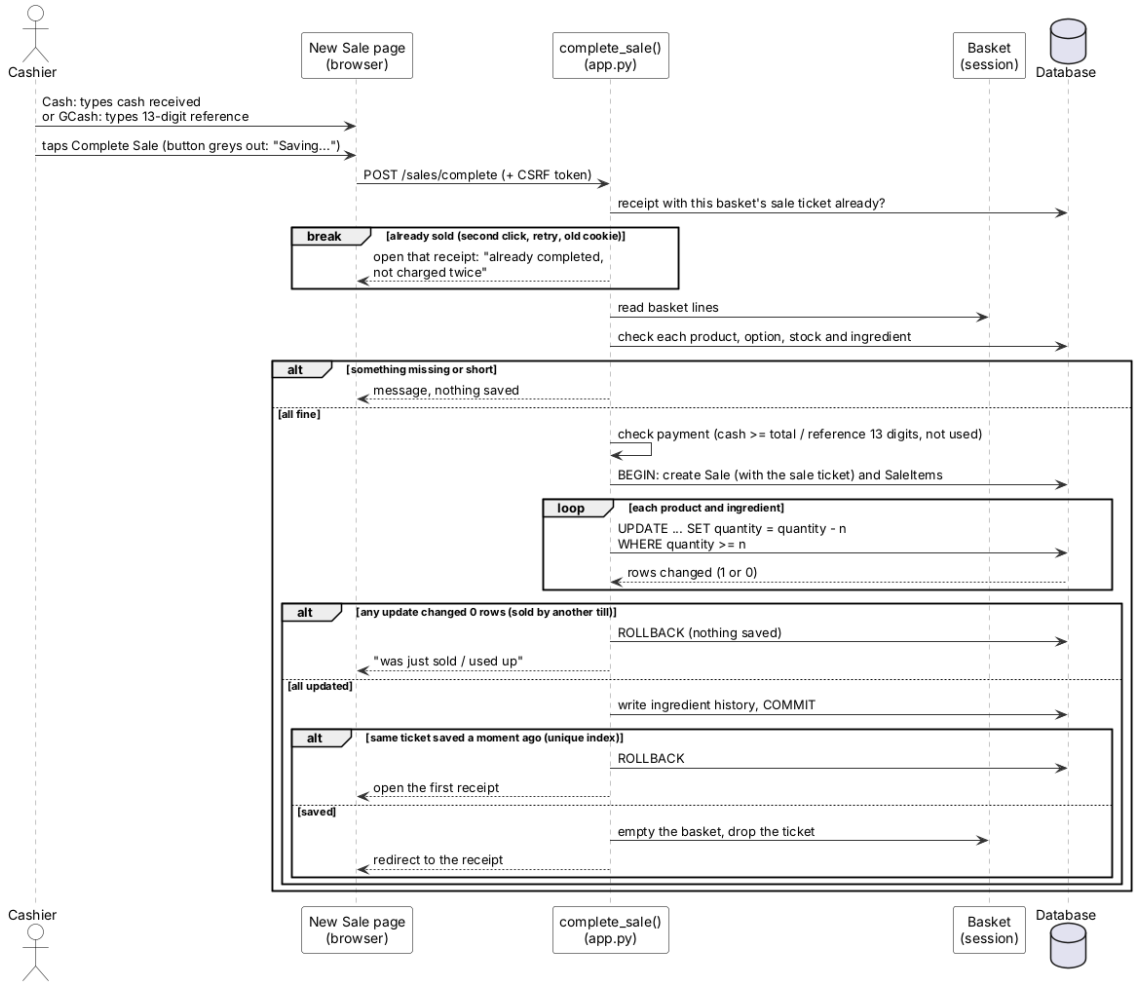
a. Log in



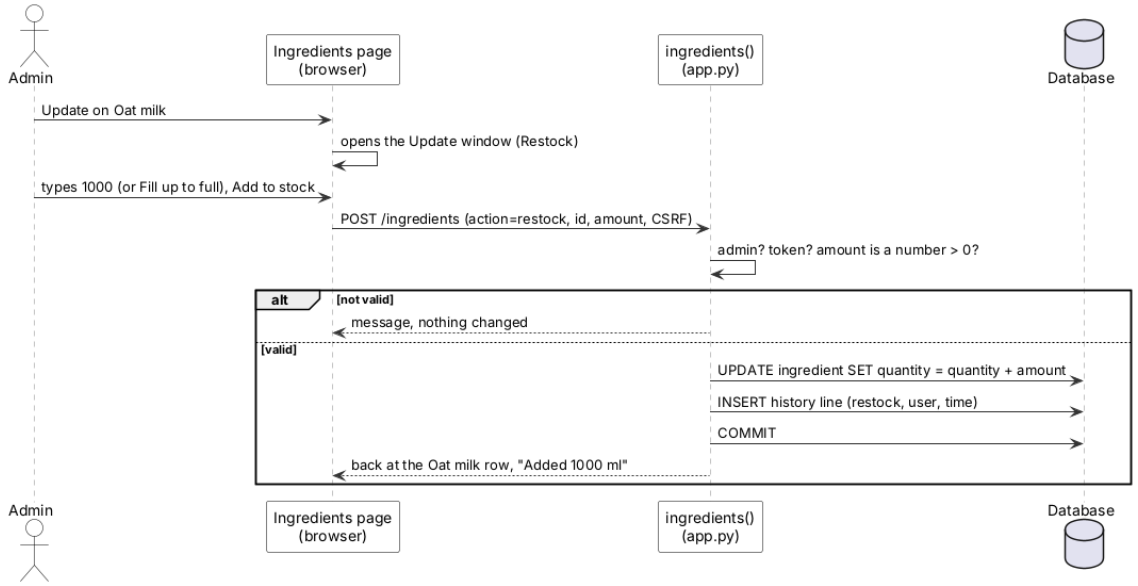
b. Add a drink to the basket



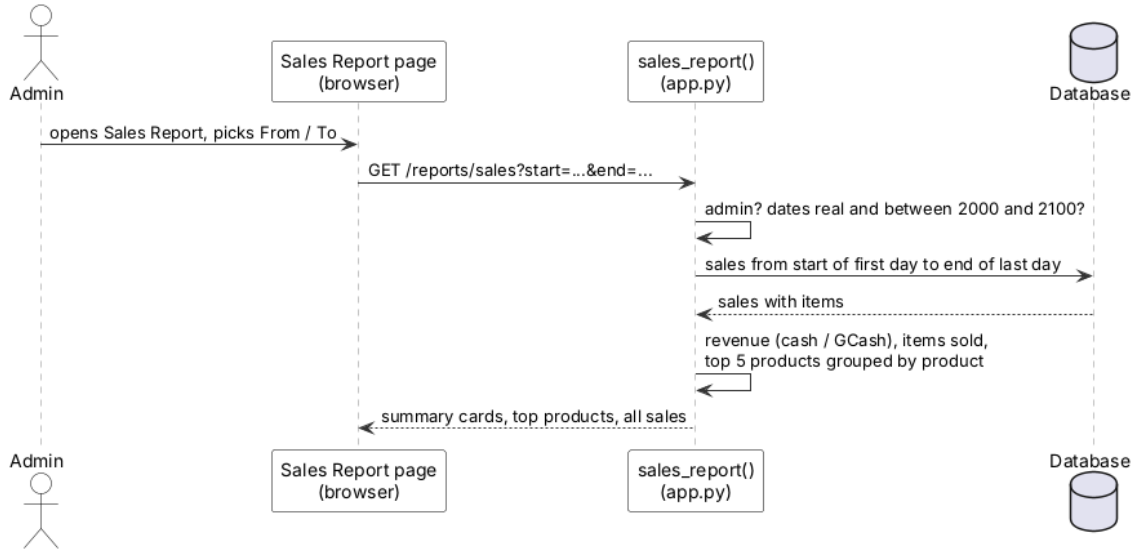
c. Complete a sale



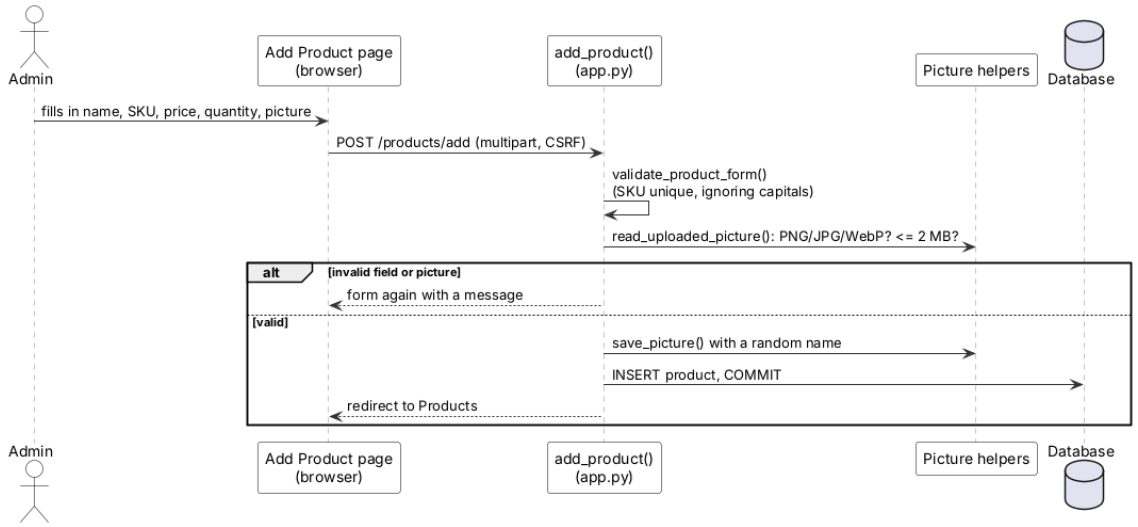
d. Restock an ingredient



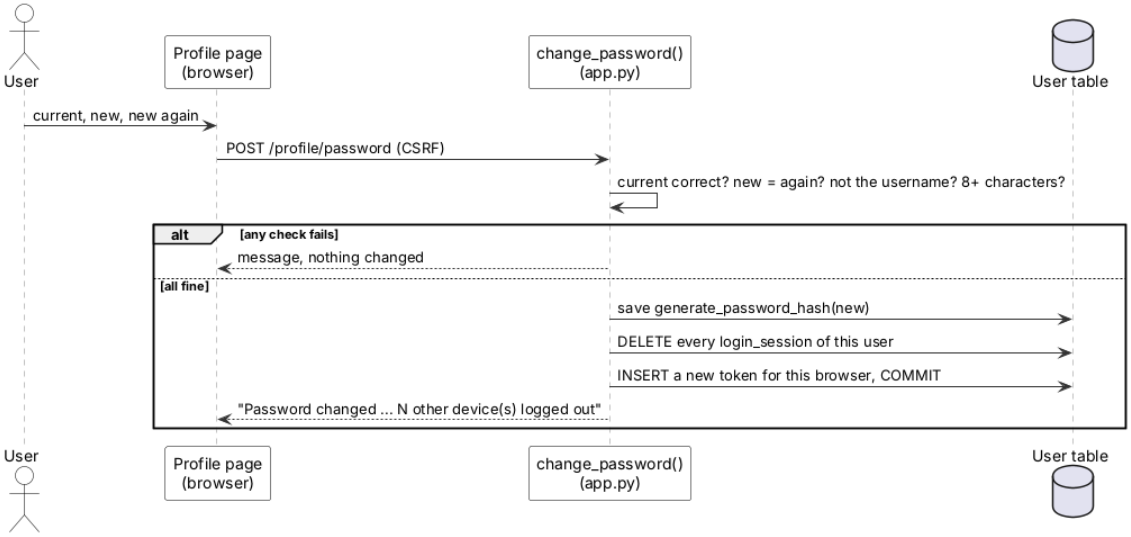
e. View the sales report



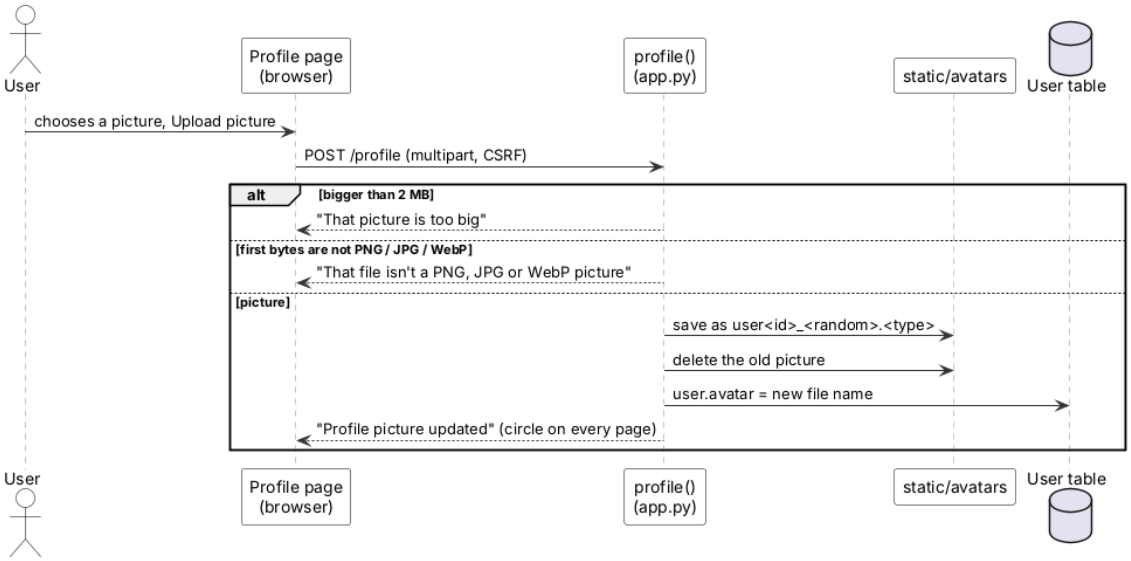
f. Add a product



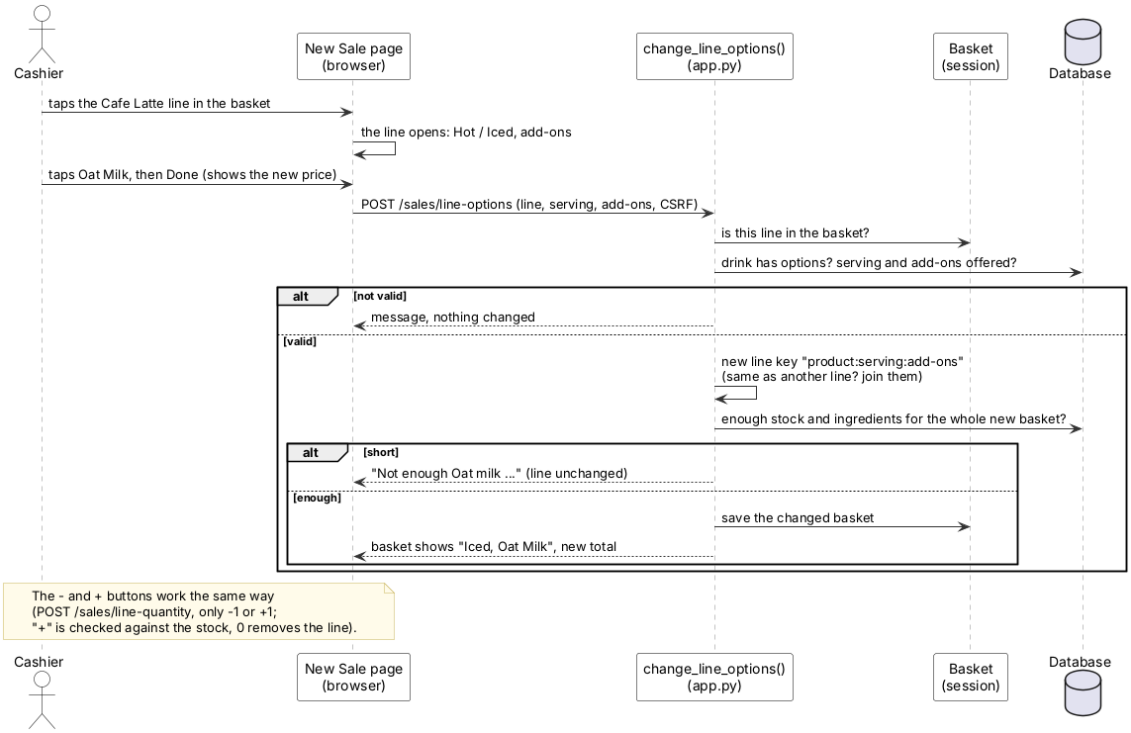
g. Change password



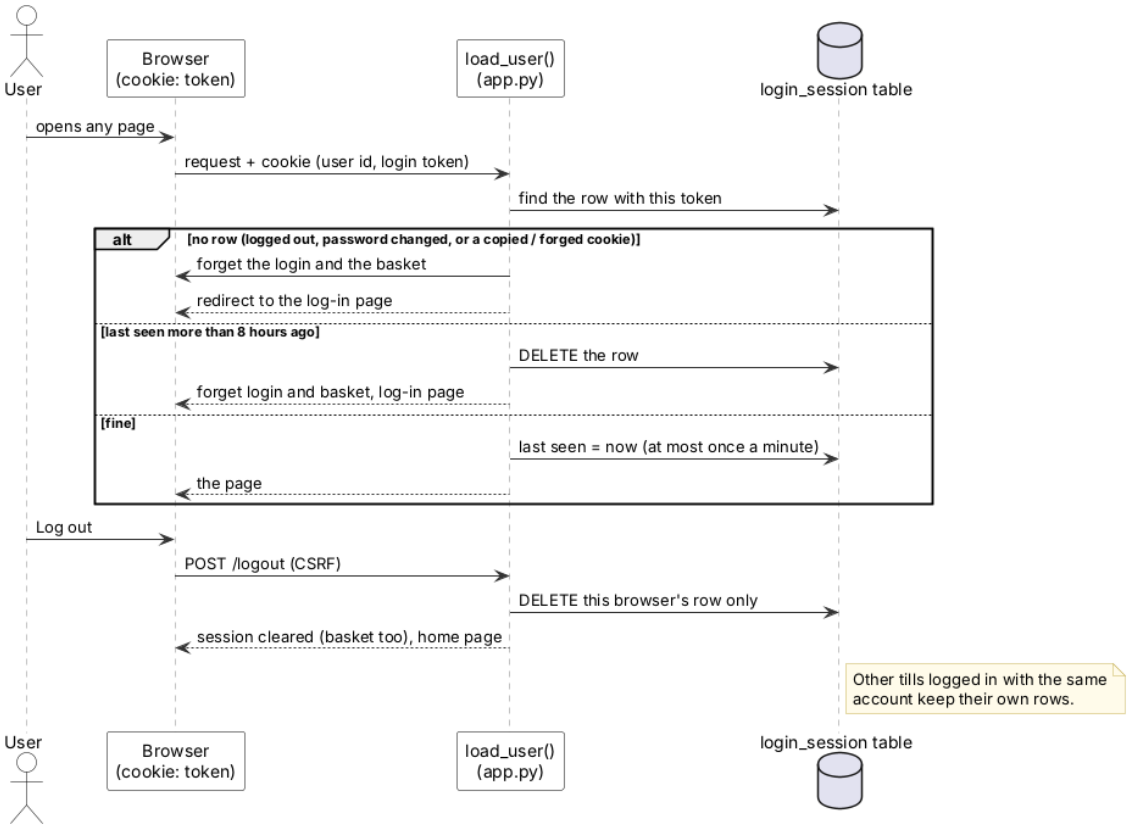
h. Upload a profile picture



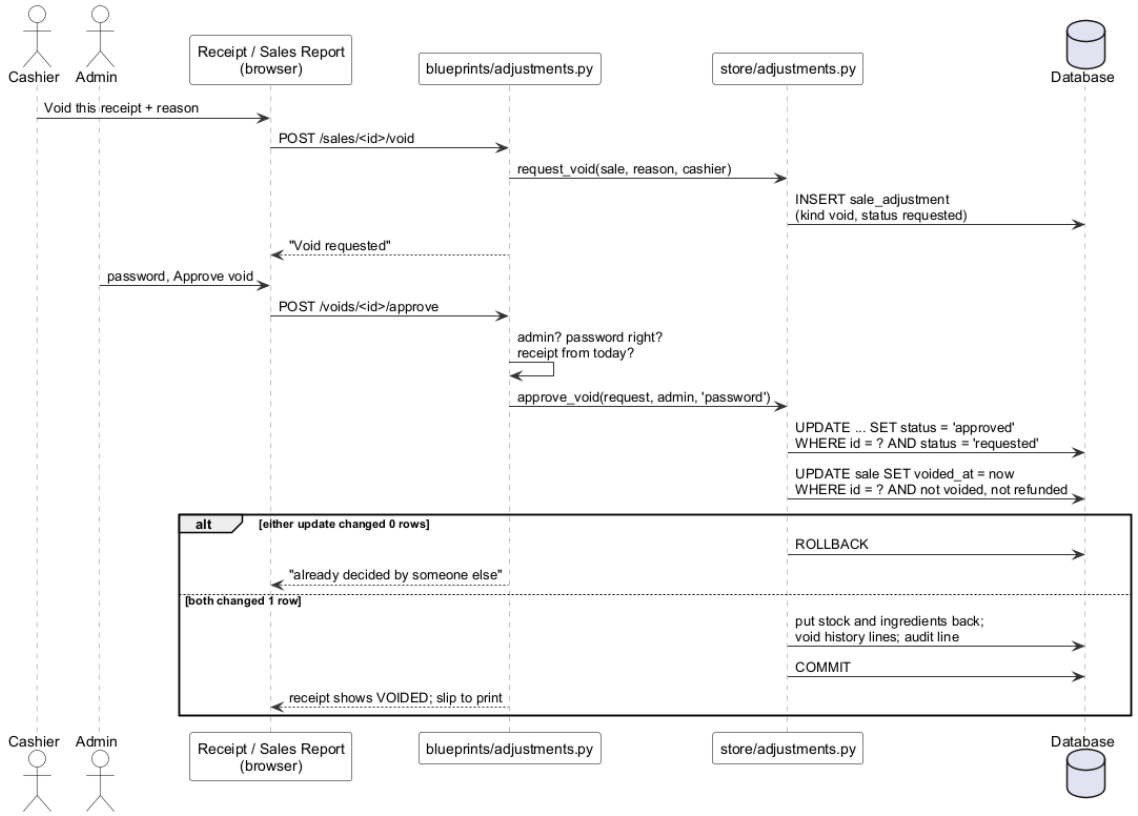
i. Change a basket line (design 6a)



j. Every request of a logged-in browser, and log out



k. Ask for and approve a void



Smaller actions (edit and delete product, switch theme) follow the same pattern as (f) and are not drawn separately: page → function in a blueprint → checks → a `store/` operation → database → redirect with a message. Diagrams b–j were drawn when every route was in `app.py`; since v0.25.0 the same functions are in `blueprints/` and the database steps in `store/`, with no change in what happens.

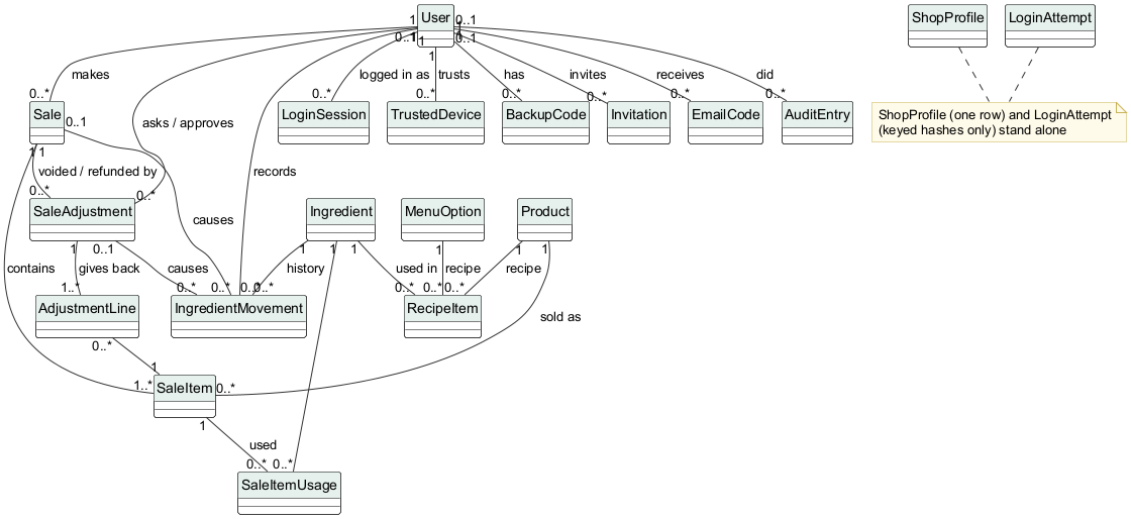
7.7 Class associations and class model (items 18 and 19, Lecture 5 p. 23–25)

A **class** is a type of thing the system stores, for example a Product. In our code each class is also a database table (in `models.py`). An **association** is a line saying two classes are related. The numbers at the ends are the **multiplicity**, meaning "how many":

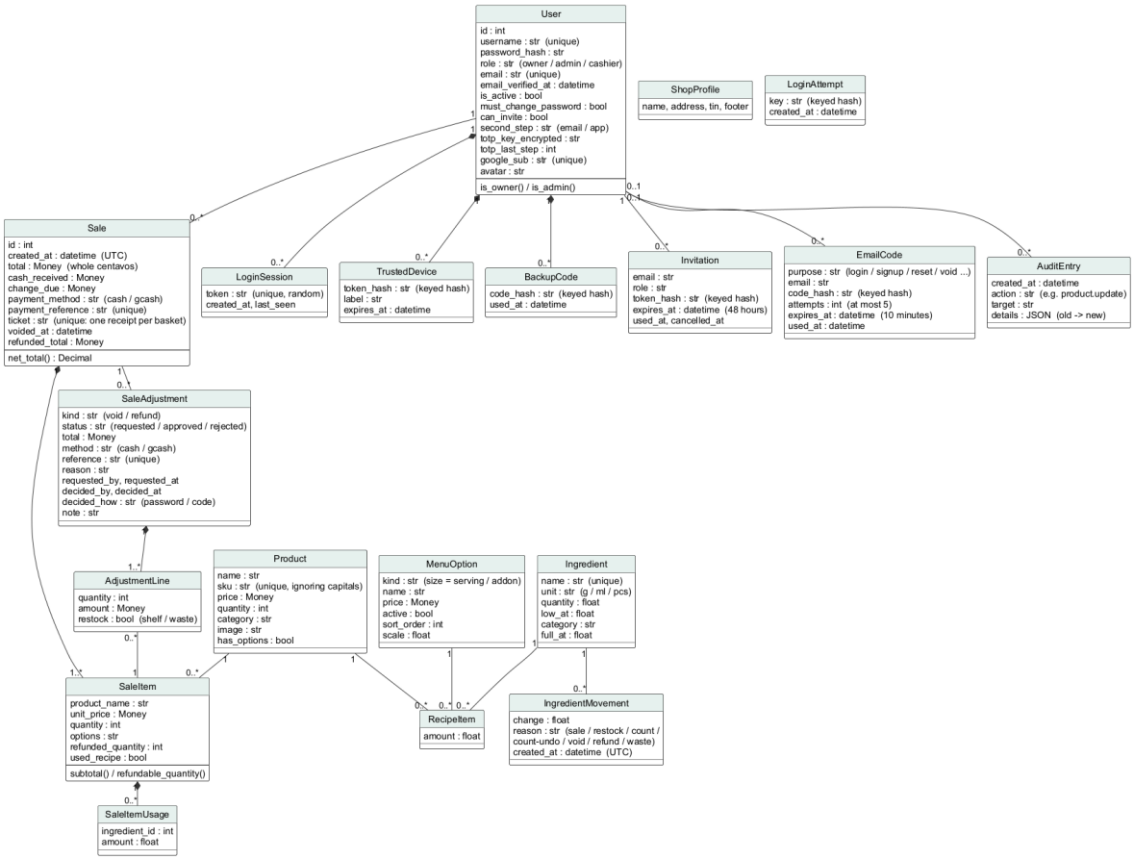
- `1`: exactly one · `0..1`: none or one · `0..*`: any number, including none · `1..*`: at least one

So `User "1" — "0..*" Sale` reads: *one user makes any number of sales, and each sale is made by exactly one user*.

Item 18, associations only (names, no details):



Item 19, the full class model with each class's **attributes** (the data it stores, the columns of its table) and the important **operations** (what it can do):

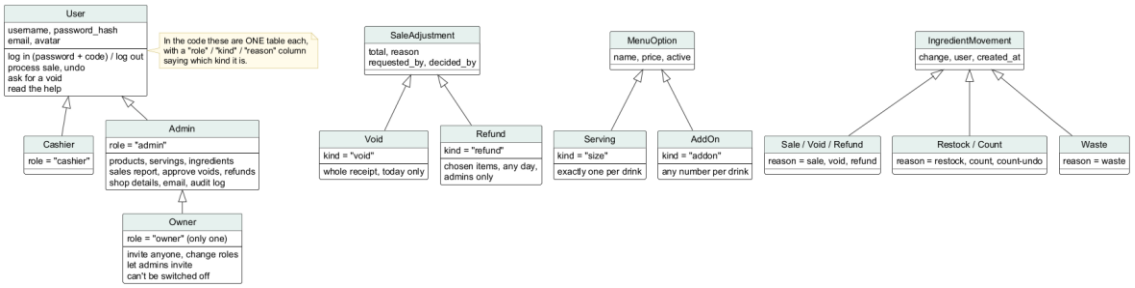


> **New in v0.20.2–v0.20.4:** `LoginSession` (one row per logged-in browser, joined to User by a filled diamond because a login record can't exist without its user), `Sale.ticket` (unique, so one basket makes one receipt) and the SKU rule "unique, ignoring capitals".

> **New in v0.22.0–v0.31.0:** `ShopProfile` (one row), `LoginAttempt` (the login limit, keyed hashes only), `Invitation`, `EmailCode`, `AuditEntry`, `TrustedDevice` and `BackupCode` for accounts; `SaleAdjustment` and `AdjustmentLine` for voids and refunds, and `SaleItemUsage` (what each receipt line used); new User columns (owner role, email, second step, Google id). Money columns are the `Money` type: whole centavos in the database, exact `Decimal` in Python (v0.23.2). Attributes with a note in brackets are marked `{field}` in the `.puml` source so PlantUML keeps them in the attribute box.
>
> **Why SaleItem copies the product name and price:** if the admin later renames a product or changes its price, old receipts must still show what the customer actually paid. Copying them at the time of sale keeps history true (the same reason UC-07 refuses to delete a product with sales history; see DEF-21).

7.8 Generalization hierarchy (item 20, Lecture 5 p. 30–31)

Generalization means "is a kind of". The arrow with a **hollow triangle** points from the specific class to the general one. The specific class **inherits** (automatically gets) everything the general class has and adds its own. Example: a Cashier *is a* User, so a Cashier has a username and password like every user.

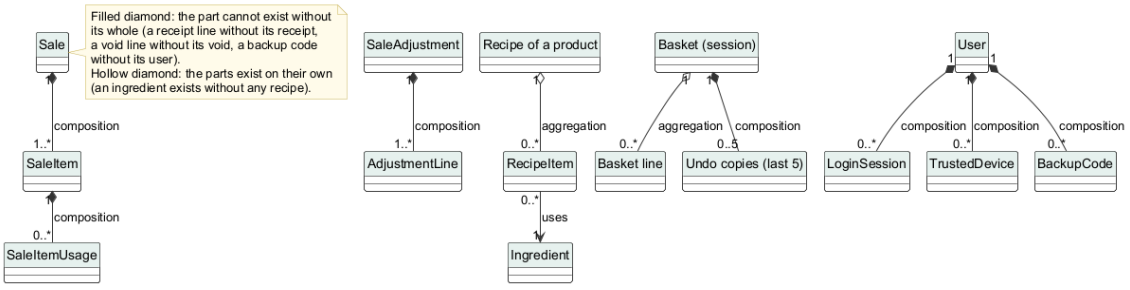


> **Note:** in the code these are not separate classes. Each is **one table** with a column saying which kind it is (`User.role`, `SaleAdjustment.kind`, `MenuOption.kind`, `IngredientMovement.reason`). The Owner is drawn as a kind of Admin: everything an admin may do, the owner may do too. This is a common, simpler way to store a hierarchy in a database (called **single-table inheritance**). The diagram shows the *idea*; the code chooses the simpler storage.

7.9 Aggregation (item 21, Lecture 5 p. 33)

Aggregation is a "has-a / is-part-of" relationship. UML has two strengths of it:

- **Composition** (filled diamond ◆): the part **cannot live without** the whole. A SaleItem only exists inside its Sale; a receipt line without a receipt makes no sense.
- **Aggregation** (hollow diamond ◇): the parts **can exist on their own**. A recipe is made of recipe lines, and each line points to an Ingredient that exists even if no recipe uses it.

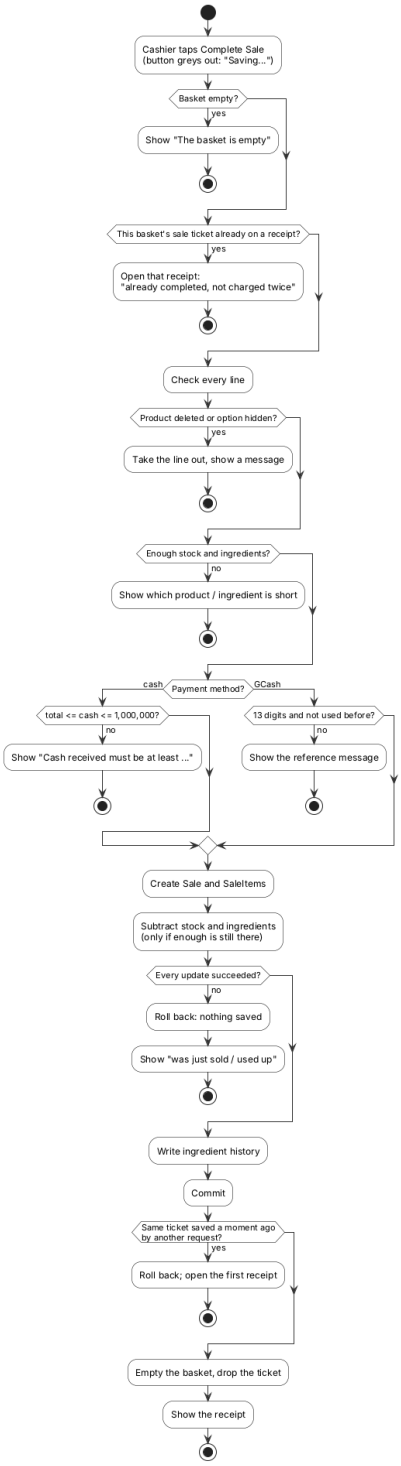


Compositions added since v0.20.3: a user owns their login records, trusted devices and backup codes; a void or refund owns its lines; a receipt line owns the record of the ingredients it used; a basket owns its undo copies.

7.10 Activity model (item 22, Lecture 5 p. 36)

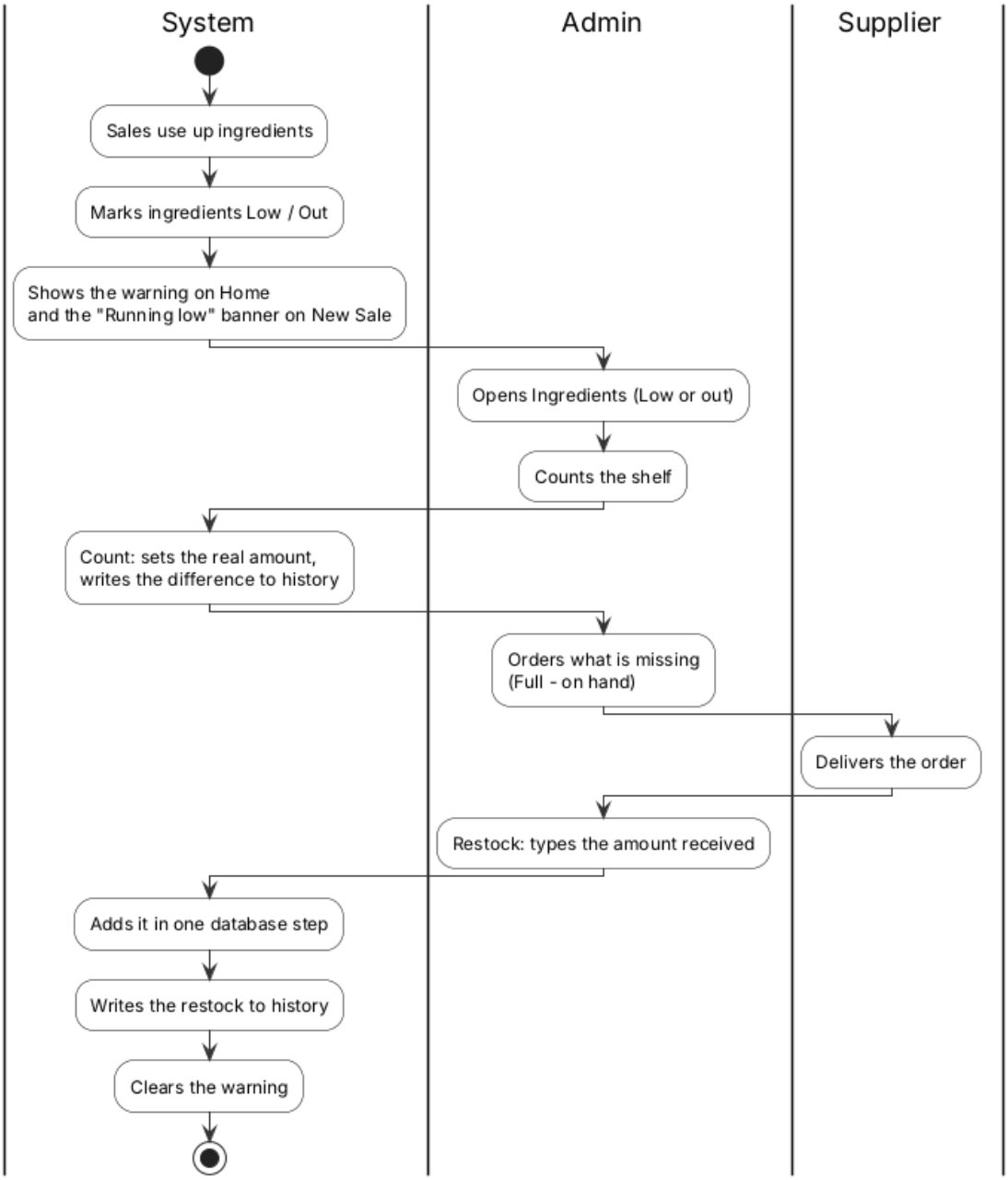
An **activity diagram** is a flowchart of one activity: the rounded boxes are actions, the diamonds are decisions, the black dot is the start and the circled dot is the end. This one shows **every check** the system makes when the cashier taps *Complete Sale*, in the order the code (`complete_sale()` in `blueprints/sales.py`, saving through `store.sales.record_sale()` since v0.25.0) makes them. Every "stop" on the side is an error

message where **nothing is saved**. The two sale-ticket decisions (v0.20.4) are why a double click or a retry can never make a second receipt.



7.11 Application process (item 23, Lecture 5 p. 37)

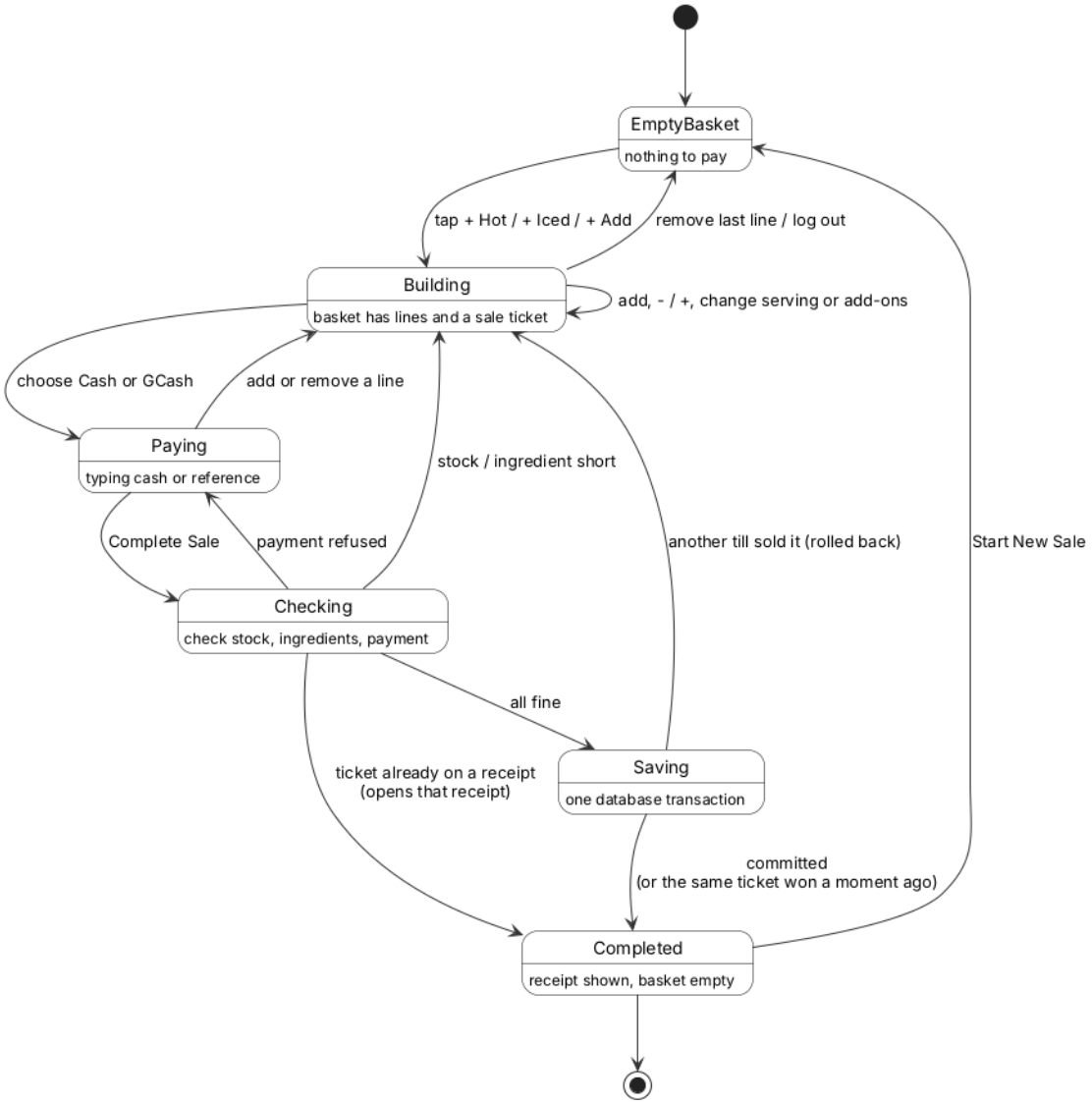
An **application process** model shows how the system fits into a **business process** that goes beyond the software. Here: ingredients run low because of sales, the admin counts and orders, a **supplier** (outside the system) delivers, and the admin records the delivery. The swimlanes show which steps the software does and which a person does.



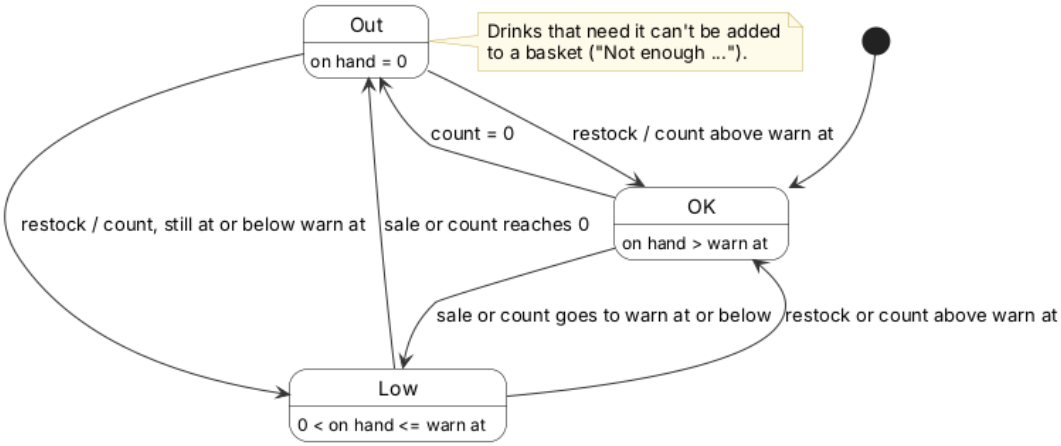
7.12 State diagrams (item 24, Lecture 5 p. 40)

A **state** is a situation something can be in (for example "Low" or "Out"). A **state diagram** shows all the states and the **transitions** (arrows) between them. The label on each arrow is the **stimulus**, the event that causes the change. The black dot is where it starts; the circled dot is where it ends.

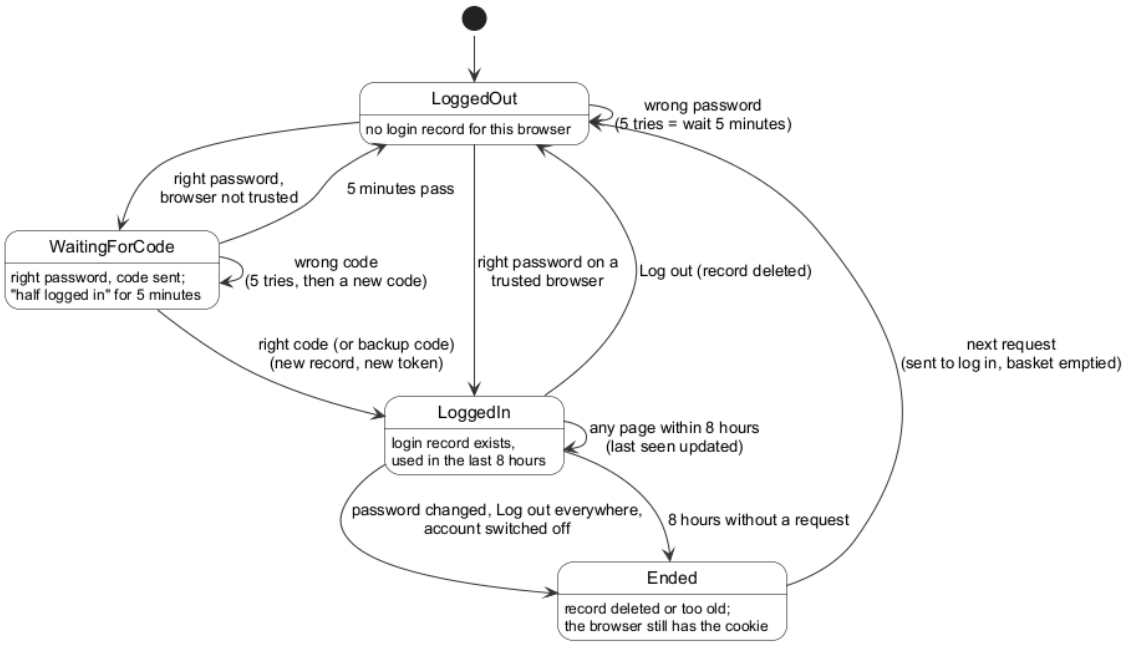
a. A sale, from empty basket to receipt. This shows why a sale is either saved completely or not at all: the *Saving* state can only end in *Completed* (committed) or go back to *Building* (rolled back), never halfway.



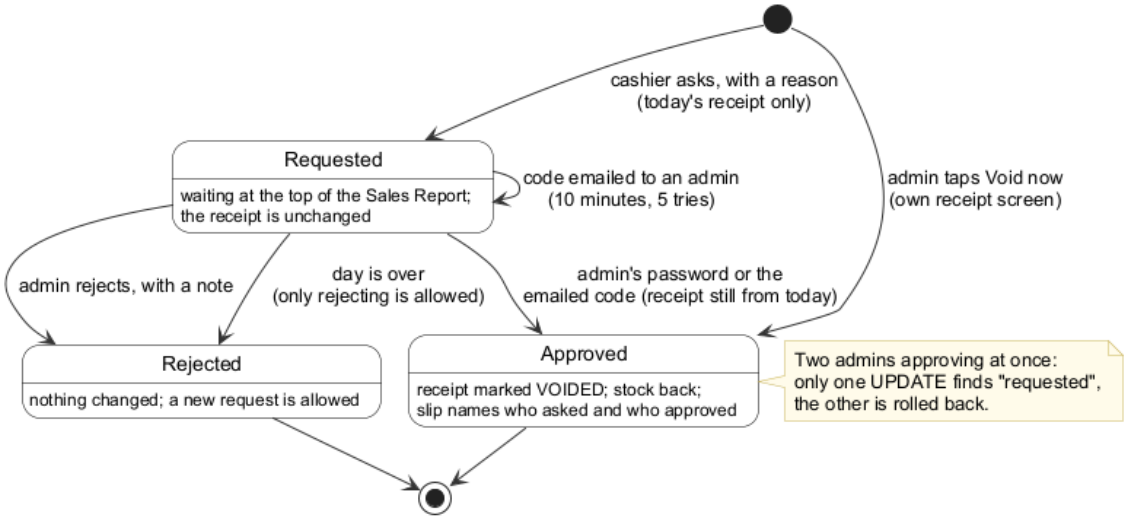
b. An ingredient's stock level. "Warn at" is the low-stock level the admin sets for each ingredient (`low_at` in the code; see `stock_status()` in `store/stock.py`).



c. A login (v0.20.3; code step v0.28.0). "Waiting for code" is the state between the right password and the right code: nothing is open yet. "Ended" is the state a copied cookie falls into as soon as its owner logs out: the browser still has the cookie, but the server no longer has the record, so the next request goes to the log-in page.



d. A void request (v0.31.0). A receipt never changes; the request does. It can end only as *Approved* or *Rejected*, and only an admin moves it there.



7.13 Structured forms of states (item 25, Lecture 5 p. 41–42)

The same state diagrams written as tables, the way Lecture 5 shows them: one table describing each **state**, and one describing each **stimulus**.

a. Sale: states

State	Description
Empty basket	The New Sale page is open and nothing has been added. The total is ₱0 and *Complete Sale* does nothing useful.
Building	The basket has one or more lines and a one-time sale ticket (v0.20.4). The cashier can add lines with one tap, use − / +, change a drink's serving and add-ons on its line (stock and ingredients are checked each time) or remove lines.
Paying	The cashier has chosen Cash and is typing the cash received, or chosen GCash and is typing the 13-digit reference.
Checking	The system re-checks every line (product still exists, option still offered, enough stock and ingredients) and the payment.
Saving	One database **transaction** is open: the Sale and SaleItems are created and stock and ingredients are subtracted.
Completed	The transaction is committed, the basket is emptied and the receipt is shown.

a. Sale: stimuli

Stimulus	Description
Add product	The cashier taps + Hot / + Iced / + Add on a tile, or uses *Add several at once*. Moves Empty basket → Building.
Add, − / +, change serving or add-ons	The cashier changes the basket. Stays in Building (or goes back to Building from Paying).
Remove last line / log out	The basket becomes empty. Building → Empty basket. Logging out always empties the basket.
Choose Cash or GCash	The cashier picks the payment method. Building → Paying.

Stimulus	Description
Complete Sale	The cashier taps *Complete Sale*. Paying → Checking.
Payment refused	Cash is less than the total or over ₱1,000,000, or the GCash reference is not 13 digits or was used before. Checking → Paying.
Stock / ingredient short	A product or ingredient no longer has enough. Checking → Building, with a message naming it.
Ticket already on a receipt	This basket was sold a moment ago (second click, retry, old cookie). Checking → Completed, showing that receipt (v0.20.4).
All fine	Every check passed. Checking → Saving.
Another till sold it	A stock update changed 0 rows because another sale took the last item first. The transaction is **rolled back**. Saving → Building.
Committed	The database confirmed the save (or another request with the same ticket was saved first; this one is rolled back and shows that receipt). Saving → Completed.
Start New Sale	The cashier taps *Start New Sale* on the receipt. Completed → Empty basket.

b. Ingredient: states

State	Description
OK	On hand is more than the "warn at" level. Shown with a green bar.
Low	On hand is above 0 but at or below "warn at". Shown in amber, counted on the Home page and listed under *Low or out*.
Out	On hand is 0. Shown in red. Drinks that need it cannot be added to a basket ("Not enough …").

b. Ingredient: stimuli

Stimulus	Description
Sale	A completed sale subtracts the recipe amounts. Can move OK → Low, Low → Out (or OK → Out in one big sale).
Restock	The admin adds the amount delivered. Can move Out → Low, Out → OK or Low → OK.
Count	The admin types the real amount on the shelf; the system saves the difference in history. Can move to any state.
Edit "warn at"	The admin changes the warning level. The state is recalculated at once (for example OK → Low if the level is raised).

c. Login: states

State	Description
Logged out	This browser has no login record. Only the home page and the log-in pages open.
Waiting for code	The right password was typed on a browser that is not trusted; a code was sent. Only the code page opens, for 5 minutes.
Logged in	A `login_session` row with this browser's token exists and was used in the last 8 hours.
Ended	The row was deleted (log-out elsewhere with a copied cookie, a password change on another device) or is older than 8 idle hours. The browser still holds the cookie, but it no longer works.

c. Login: stimuli

Stimulus	Description
Right password, trusted browser	A new row and a new random token are created. Logged out → Logged in.
Right password, not trusted	A code is emailed (or the app is asked for). Logged out → Waiting for code.
Right code or backup code	A new row and a new random token are created; the browser is trusted if ticked. Waiting for code → Logged in.
Wrong code	Counted; after 5 the code stops working. Stays Waiting for code.
5 minutes pass	Waiting for code → Logged out.
Wrong password	Counted by the login limit (5 tries, then wait 5 minutes). Stays Logged out.
Any page within 8 hours	"Last seen" is updated (at most once a minute). Stays Logged in.
Log out	This browser's row is deleted and the basket emptied. Logged in → Logged out.
Password changed, Log out everywhere, account switched off	Every row of the user is deleted. Logged in → Ended.
8 hours without a request	The row is too old. Logged in → Ended.
Next request	The server finds no valid row: the login and basket are forgotten and the log-in page opens. Ended → Logged out.

d. Void request: states

State	Description
Requested	A cashier (or admin) asked for the void with a reason. It waits at the top of the Sales Report; the receipt and the stock are unchanged.
Approved	An admin approved it with their password or the emailed code, or voided the receipt directly. The receipt is marked VOIDED, every product and ingredient is back, and a slip names who asked and who approved.
Rejected	An admin rejected it, with an optional note. Nothing changed; a new request may be made.

d. Void request: stimuli

Stimulus	Description
Ask for a void	A user taps Void this receipt on a receipt of today and types a reason. Start → Requested.
Void now	An admin voids a receipt of today directly, on its own screen. Start → Approved.
Code emailed	The cashier sends a code to an admin who is not in the shop. Stays Requested.
Approve	The admin's password or the emailed code is right and the receipt is still from today. Requested → Approved. If another admin approved a moment earlier, this one is rolled back.
Reject	An admin rejects it. Requested → Rejected.
Day is over	An approval is refused after midnight on the shop's clock; the admin can only reject it (and give a refund if the customer is owed money).

Source for all eight tables: `stock_status()` *(*`store/stock.py`*),* `complete_sale()` *(*`blueprints/sales.py`*),* `load_user()` *and* `verify()` *(*`blueprints/accounts.py`*) and* `store/adjustments.py` *(v0.32.0), and test cases TC-7.x, TC-15.x, TC-17.x, TC-18.x, TC-23.x, TC-38.x and TC-41.x in §5.*

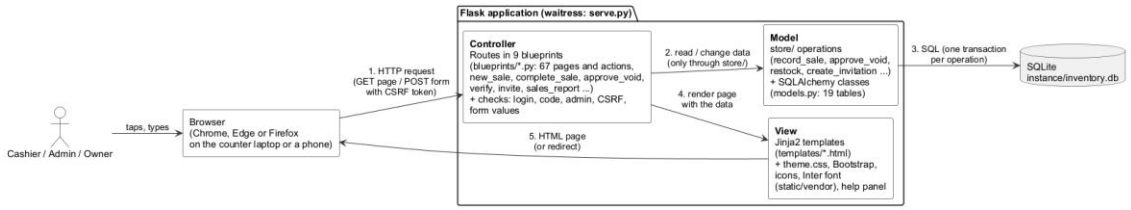
8. Architecture and design (items 26–30)

8.1 Software architecture (item 26, Lecture 6 p. 42 and 49)

Words used: the *architecture* of a system is its overall structure: which big parts it has and how they talk to each other. An *architectural pattern* is a well-known structure that has worked before, like a floor plan that builders reuse.

The system uses two patterns from Lecture 6 at the same time:

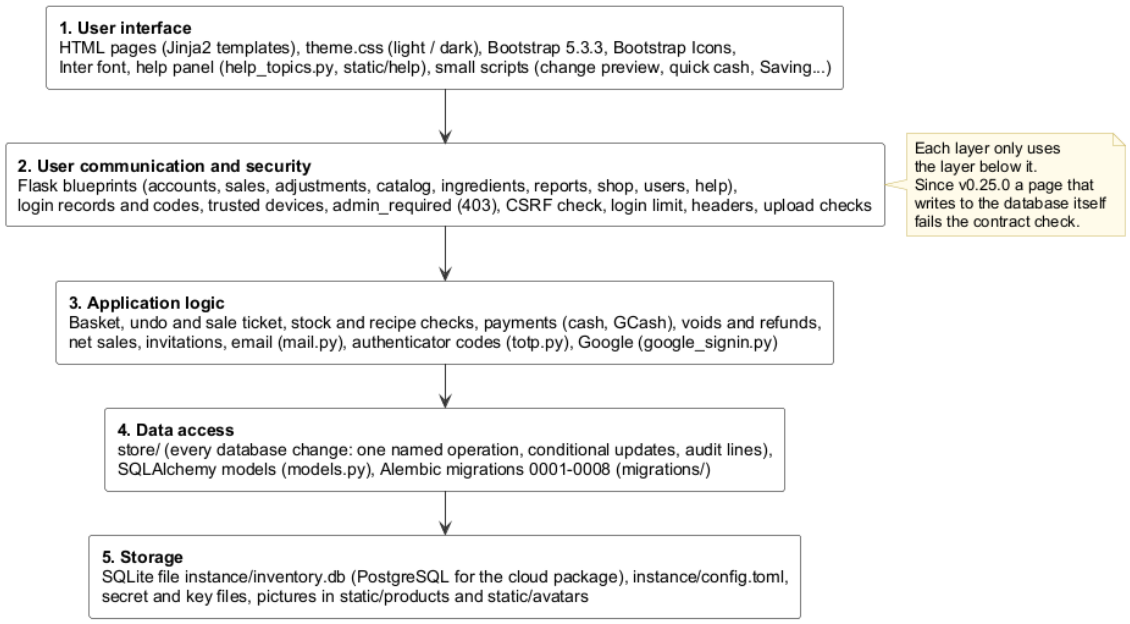
a. Model–View–Controller (MVC), the pattern for web applications. Each request from the browser goes to a *controller* (a route in one of the nine blueprints in `blueprints/`), which reads or changes data through the *model* (the named operations in `store/` and the classes in `models.py`) and then asks a *view* (a template in `templates/`) to build the page. Flask, SQLAlchemy and Jinja2 are built for exactly this split.



MVC part	In this project	Example: completing a sale
Model	`store/` (every database change as one named operation) and `models.py`: 19 classes, one table each, kept in SQLite (`instance/inventory.db`) and changed only by migrations	`store.sales.record_sale()` with `Sale`, `SaleItem`, `SaleItemUsage`, `Product.quantity`, `Ingredient.quantity`, `IngredientMovement`
View	`templates/*.html` (Jinja2) + `static/css/theme.css` + Bootstrap, icons and font in `static/vendor/`, help panel	`new_sale.html` (basket, payment, undo), `receipt.html`
Controller	the 67 routes in `blueprints/` (accounts, sales, adjustments, catalog, ingredients, reports, shop, users, help), with their checks	`complete_sale()`: checks the ticket, stock and payment, calls `record_sale()`, which saves everything in one transaction, then redirects to the receipt

b. Layered architecture. Each layer offers services to the layer above and uses only the layer below. Until v0.24.0 the layers were only *logical*: layers 2 and 3 were both in `app.py`, separated into commented blocks, because the course kept the app in one file. **v0.25.0** made them real (DEC-027): the routes moved into nine blueprints, shared rules into `common.py`, and every database change into a `store/` layer; `app.py` now only starts the app (242 lines at v0.25.0, instead of 1,722). The contract check fails if a page writes to the database itself, so the layering can't quietly erode (rule I1, now *[checked]*). The point of the split is the cloud package:

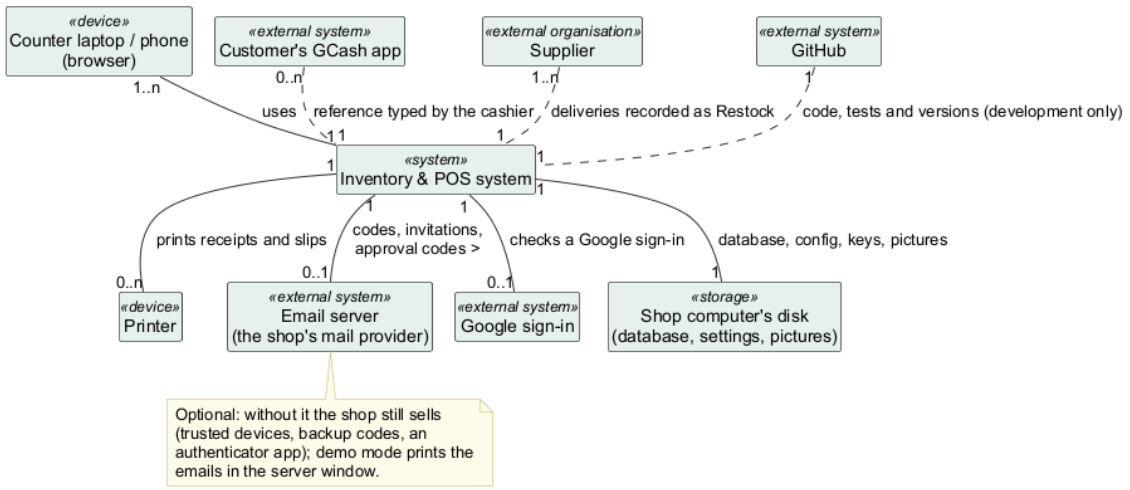
the pages call `store/`, so the same pages can later run on PostgreSQL by changing only the database address (DEC-014).



Why these patterns? MVC keeps the look (templates) apart from the rules (routes), so v0.13.0 and v0.20.0 could change the whole look without touching the database. The layers put every security check in one place (layer 2), before any business rule runs, and every database change in one place (layer 4), where the "check and change in one step" rule (I1) is easy to review. **Client–server** is also present: the browser is the client and the Flask app, served by waitress since v0.24.0, the server; several tills (browsers) can use one server at the same time, and SQLite lets one of them write at a time.

8.2 System context (item 27, Lecture 7 p. 9)

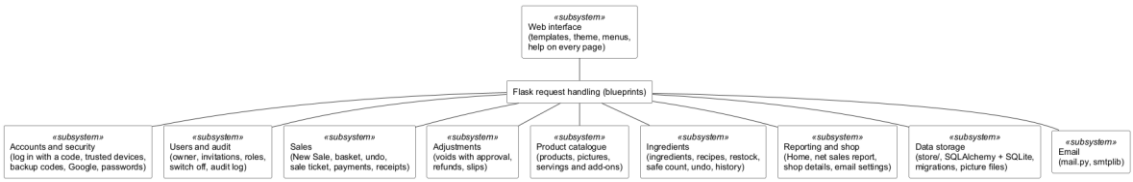
Lecture 7 draws the context as boxes joined by *associations* with *multiplicities* (how many of each: `1`, `0..n`, `1..n`). It complements the Lecture 5 context model in §7.1, which shows the data that flows.



Other system	Multiplicity	Relationship
Counter laptop / phone (browser)	1..n tills use 1 system	Every page is used through a browser; nothing is installed on the till
Printer	0..n	Receipts are printed with the browser's Print; the receipt page hides the menus when printing
Customer's GCash app	0..n	**Not connected** (dashed line): the cashier types the 13-digit reference from the customer's screen
Supplier	1..n	**Not connected**: a delivery is entered by the admin as a *Restock*
Email server	0..1	**Connected** (v0.26.0): the shop's own mail provider sends the login codes, invitations and approval codes, over an encrypted connection. Optional: without it the shop still sells, and demo mode prints each email instead
Google sign-in	0..1	**Connected** (v0.29.0): Google's answer is checked on the server (signature, issuer, audience, expiry, nonce). Optional, and set up by the owner
Shop computer's disk	1	The database, the settings (`instance/config.toml`), the keys and the picture files live here; a backup is taken automatically before every database migration, and backing it up otherwise is the owner's job
GitHub	1	Only for development: code, tests, versions. The running shop does not need GitHub or the internet

8.3 High-level architecture (item 28, Lecture 7 p. 13)

Like the weather station's subsystems joined by a communication link, the system has ten subsystems joined by Flask's request handling (one blueprint per subsystem since v0.25.0).

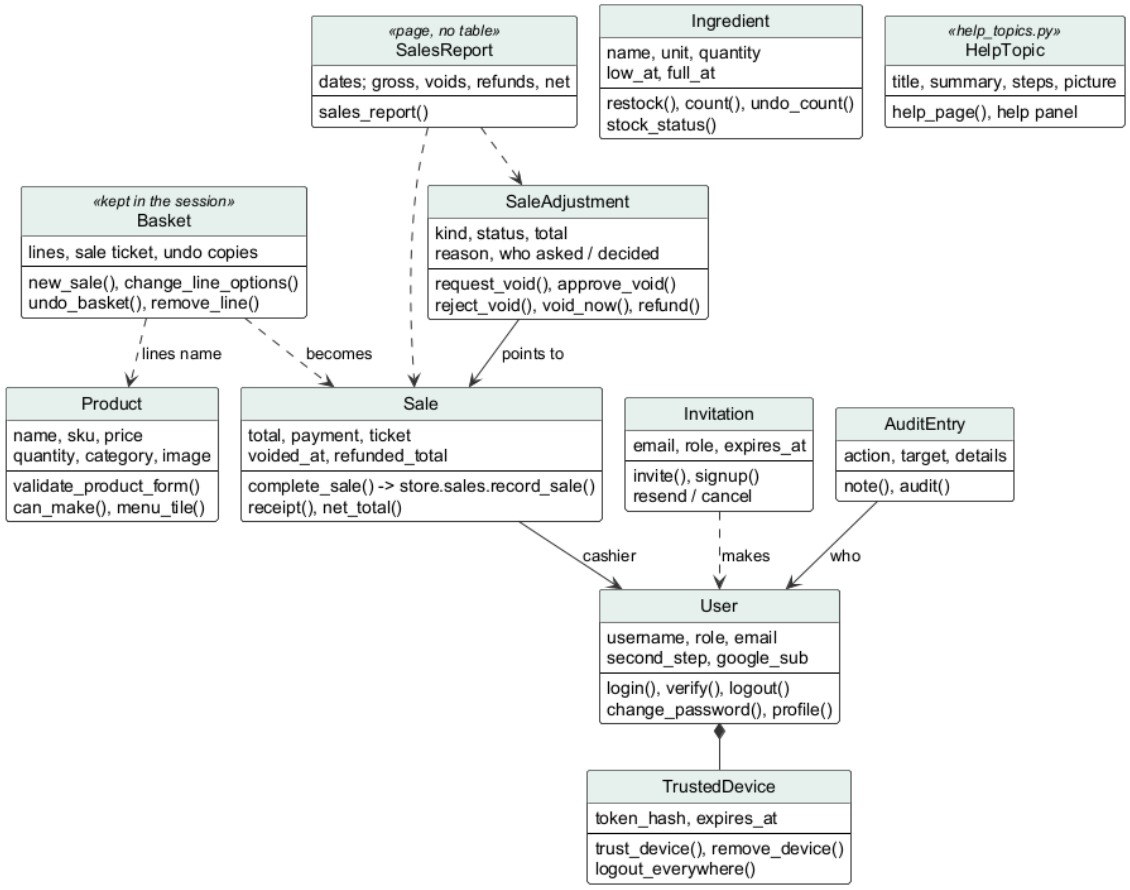


Subsystem	Code (v0.32.0)	Data (`models.py`)
Web interface	`base.html` and the page templates, `theme.css`, `blueprints/help.py` + `help_topics.py`	—
Accounts and security	`blueprints/accounts.py`: `login`, `verify`, `logout`, `profile`, `change_password`, `forgot_password`, trusted devices, backup codes, authenticator, Google; `store/accounts.py`, `store/signin.py`; `totp.py`, `google_signin.py`	`User`, `LoginSession`, `LoginAttempt`, `TrustedDevice`, `BackupCode`, `EmailCode`
Users and audit	`blueprints/users.py`: `users`, `invite`, `signup`, `change_role`, `switch_active`, `audit`; `store/users.py`, `store/audit.py`	`User`, `Invitation`, `AuditEntry`
Sales	`blueprints/sales.py`: `new_sale`, `change_line_options`, `undo_basket`, `complete_sale`, `receipt`; `store/sales.py`	`Sale`, `SaleItem`, `SaleItemUsage` (basket in the session)
Adjustments	`blueprints/adjustments.py`: `todays_receipts`, `request_void`, `approve_void`, `reject_void`, `void_now`, `refund`, `slip`; `store/adjustments.py`	`SaleAdjustment`, `AdjustmentLine`
Product catalogue	`blueprints/catalog.py`: `products`, `add_product`, `edit_product`, `delete_product`, `manage_options`; `store/catalog.py`	`Product`, `MenuOption`

Subsystem	Code (v0.32.0)	Data (`models.py`)
Ingredients	`blueprints/ingredients.py`: `ingredients`, `undo_count`, `ingredient_detail`, `edit_recipe`; `store/ingredients.py`, `store/stock.py`	`Ingredient`, `RecipeItem`, `IngredientMovement`
Reporting and shop	`blueprints/reports.py`: `home`, `sales_report`; `blueprints/shop.py`: `shop_settings`, `email_settings`; `store/shop.py`	reads `Sale`, `SaleAdjustment`, `Product`, `Ingredient`; `ShopProfile`
Email	`mail.py` (Python's `smtplib`, or printed in demo mode)	settings in `instance/config.toml`
Data storage	`upgrade_database()` (backup, then Alembic migrations 0001–0008), `models.Money`, `save_picture`	SQLite file, picture folders

8.4 Object classes (item 29, Lecture 7 p. 19)

Lecture 7's object classes show each class with its **attributes** (what it knows) and **operations** (what it can do). The attributes are the real database columns. The operations are the real functions in `blueprints/` and `store/`, grouped by the object they work on: Flask code is mostly functions, not methods, so this is the *design* view of the code. `Basket`, `SalesReport` and `HelpTopic` have no table: the basket (with its undo copies) lives in the browser's signed session cookie, the report is calculated each time, and the help topics are written in `help_topics.py`.



The full class model with every column and association is in §7.7.

8.5 Detailed usage scenario (item 30, Lecture 8 p. 53)

Lecture 8 uses a *usage scenario* for **scenario testing**: a realistic story is played through the system, and every feature it touches is tested at once. Each step below is covered by the test cases in the table; the security steps were also tried by hand in my attack drill (2026-10-13).

> **Scenario: a busy Saturday afternoon.** Maria owns a small café; Jun is her cashier. At 3 PM Jun logs in on the counter laptop; it is a trusted device, so his password is enough. The New Sale page shows a "Running low: Iced cups" banner. A customer orders two iced lattes, one with oat milk, and an ensaymada. Jun taps **+ Iced** on the Cafe Latte tile, taps the new basket line, ticks **Oat Milk** and taps **Done**; the line now shows the extra ₱40. He taps **+ Iced** again for the plain latte (a second line, because its add-ons differ) and **+ Add** on the ensaymada. The "Uses N ingredients" panel has opened by itself because the iced cups will go low. The customer hands over a ₱500 bill; Jun taps the **₱500** quick-cash button, the change appears, and he taps **Complete Sale**. The laptop is slow, so he taps it again; the button already says "Saving…", and only one receipt is made. He prints it, with the café's name at the top and "This is not an official receipt." at the bottom. The next customer changes their mind after paying; Jun asks for a void of that receipt, and Maria, at the back, approves it with her password from the Sales Report. The milk and beans go back on the shelf.
>
> At 4 PM the Wi-Fi drops. Nothing changes on screen, and Jun keeps selling. At 5 PM Maria's supplier delivers iced cups; on her phone she opens **Ingredients**, filters *Low or out*, and adds the delivery with **Restock**. The banner on Jun's screen disappears at his next tap. At closing, Maria opens the **Sales Report** for today and compares the net cash total with the drawer, then changes her password, which logs out the other phone she had used. Jun logs out; the next morning a new cashier, invited the day before, logs in for the first time with a code from her email, opens the **?** on New Sale, and finds an empty basket.

Feature tested by the scenario	Where it is specified and tested
Log in, login record, log out empties the basket	UC-02, UC-03; TC-5.x, TC-23.4–23.9
Running-low banner, stock bars on tiles	UC-09; TC-22.1, TC-22.2, TC-22.9
One-tap serving buttons, − / + on a line, equal lines merged	UC-08; TC-22.1, TC-22.3
Change add-ons on the basket line, live price	UC-08, UC-14; TC-22.4, TC-16.x
"Uses N ingredients" panel opens when something goes low	TC-22.5
Quick cash, change preview	TC-22.6, TC-22.7, TC-7.x
One receipt per basket, "Saving…"	TC-23.11–23.14
Works without internet	TC-21.1–21.5
Restock adds safely, history line	UC-15; TC-17.x, TC-18.x
Sales Report: cash vs GCash	UC-10; TC-9.x, TC-15.x
Password change ends other logins	UC-13; TC-23.6
Trusted device: password only; a new person's code by email	UC-02, UC-25; TC-38.3, TC-38.5, TC-38.15
Shop details and the notice on the receipt	UC-16; TC-27.1–27.4
Ask for and approve a void; stock back; net sales	UC-19, UC-20; TC-41.3–41.6, TC-41.14
Joining by invitation	UC-18; TC-36.1–36.3
Help panel	UC-24; TC-42.4

9. Dependability and security (items 31–39)

This chapter applies the dependability and security ideas of Lectures 11 and 12 to the system. It draws on the defect list (Chapter 10), the security audit of 11 October 2026, my attack drill of 13 October 2026, and the account and void features added in v0.26.0–v0.31.0, which changed the system's security the most.

To be clear about scope: this is not a safety-critical system like Sommerville's insulin pump. Nobody is physically hurt if it fails; the harm is mostly to **money, stock records and personal data**. One hazard can reach a customer's health, though: if a drink's *Oat Milk* option were lost between the basket and the barista, a customer who cannot have dairy might be served the wrong milk. The tables below use Sommerville's terms with café examples like this one.

9.1 Reliability terminology (item 31, Lecture 11 p. 25)

Term	Description	Example in this system
Human error or mistake	Human behaviour that results in a fault being put into the system	When v0.20.0 was written, the New Sale tile bars were given the class name `stock-bar`, which the Ingredients page already used (DEF-58)
System fault	A characteristic of the software that can lead to a system error	The new style rules for `stock-bar` (`overflow: hidden`) also matched the Ingredients bars
System error	An erroneous system state that can lead to behaviour the user does not expect	The Ingredients page was drawn with the warn-level tick clipped to a 6 px green speck
System failure	An event that occurs when the system does not deliver a service as expected by its users	I could no longer see how close an ingredient was to its warning level (reported 2026-10-13; fixed in v0.20.5)

A second example of the same chain: *mistake* — the stock check and the stock update were written as two steps; *fault* — another sale could run between them; *error* — two sales both saw "1 left"; *failure* — the last item was sold twice and stock showed −1 (DEF-13, fixed in v0.8.2 with an atomic update).

9.2 Safety terminology (item 32, Lecture 11 p. 35)

Term	Definition	Example in this system
Accident (or mishap)	An unplanned event that results in human death or injury, damage to property or to the environment	A customer with a milk allergy is served dairy milk; or the café loses a day's sales records
Hazard	A condition with the potential to cause or contribute to an accident	A drink line loses its *Oat Milk* option between the basket and the receipt; the only copy of the database is on one laptop
Damage	A measure of the loss resulting from an accident	From a refund and an apology (wrong change) to a customer's health (allergen) or a day of lost records
Hazard severity	An assessment of the worst possible damage that could result from a hazard	Allergen: *high* (health). Lost database: *high* (business). Wrong change: *low*
Hazard probability	The probability of the events occurring which create a hazard	Option lost: *very low* (options are copied as text onto each receipt line, TC-16.x). Laptop disk failure: *low*

Term	Definition	Example in this system
Risk	The probability that the system will cause an accident: hazard probability × hazard severity × chance the hazard leads to an accident	Allergen: low (very low probability, high severity, barista also asks the customer). Lost database: **medium**, because there is no automatic daily backup yet (a backup is taken only before each database migration, since v0.23.0)

9.3 Security terminology (item 33, Lecture 11 p. 42)

Term	Definition	Example in this system
Asset	Something of value which has to be protected	The sales and stock records (`inventory.db`), the users' accounts and email addresses, the secret key that signs cookies and hashes codes, the authenticator keys, the GCash reference numbers on receipts
Attack	An exploitation of a system's vulnerability, usually from outside and on purpose	Copying the admin's session cookie into another browser (my attack drill, round 3)
Control	A protective measure that reduces a system's vulnerability	Login records (v0.20.3): log-out or a password change deletes the record, so the copied cookie stops working. A code at every login (v0.28.0): a seen or guessed password alone is not enough
Exposure	Possible loss or harm to a computing system	Someone with a copied admin cookie could read every sale and change prices
Threat	Circumstances that have the potential to cause loss or harm	A cashier, or anyone who uses the counter laptop, tries the admin pages or a copied cookie
Vulnerability	A weakness in a system that may be exploited to cause loss or harm	Before v0.20.3: a cookie kept working after log-out (DEF-51, audit A6)

Full list of attacks tried and their results: §10.3 and the security audit report (`docs/security_audit_2026-10-11.md`).

9.4 Vulnerability avoidance techniques (item 34, Lecture 11 p. 45)

Sommerville names three ways to make a system secure. All three are used:

a. Vulnerability avoidance (design the weakness out, so it can't be attacked):

Technique	How	Version
Passwords never stored	Only scrypt hashes (Werkzeug); minimum 8 characters, not the username	v0.6.0, v0.13.5
Secret key not in the code	Made once and kept in `instance/secret_key.txt` (not in Git), or set with `SECRET_KEY`	v0.13.1
Every value checked on the server	Numbers must be finite and between 0 and 1,000,000; ids too big for the database count as "not found"; text fits the columns	v0.7.1, v0.8.3–v0.8.6, v0.19.3
SQL injection impossible	All database access goes through SQLAlchemy with parameters; no SQL text is built from user input	v0.2.0
Script injection impossible	Jinja2 escapes every value shown on a page	v0.3.0
Uploaded pictures can't be run	Checked by their first bytes (PNG, JPG, WebP), at most 2 MB, saved under a random name, sent with `nosniff`	v0.12.0, v0.14.0

Technique	How	Version
No outside code	Bootstrap, icons and font are stored in the project; nothing loads from the internet	v0.19.4
Only the needed packages, kept up to date	Exact versions in `requirements.txt`; Werkzeug updated for a known weakness	v0.19.3
Changes can't remove a protection by accident	AGENTS.md rules + `tests/check_contract.py` (147 rule checks on v0.32.0) run on every pull request; every decision recorded with its alternatives (`docs/decisions.md`)	v0.19.0, v0.21.3
Nobody gives themselves an account or a role	No sign-up page; the owner made once by setup; everyone else by invitation, with the role fixed by the inviter	v0.27.0
Codes and tokens never stored	Login, signup, reset and void codes, invitation and trusted-device tokens and backup codes are stored only as keyed hashes (HMAC) and compared in constant time	v0.27.0, v0.28.0
Authenticator keys can't be read from the database	Encrypted with `cryptography`; the key file is in `instance/`, not in Git	v0.28.0
Usernames can't be found by timing	An unknown username costs one hash check, like a real one	v0.21.4
Sign in with Google can't be forged	Signature, issuer, audience, expiry and nonce checked on the server; state and PKCE for the trip; accounts linked by Google's permanent id	v0.29.0

b. Attack detection and elimination (notice an attack and stop it):

Technique	How	Version
Wrong-password limit	5 wrong passwords for a username from one computer → that computer waits 5 minutes; kept in the database, so a restart can't clear it; the same limit on Change password	v0.13.3, v0.19.3, v0.26.0
A seen or guessed password	A 6-digit code at every login on a browser that isn't trusted; 5 wrong codes and the code stops working; an authenticator step never accepted twice	v0.28.0
A void approved twice, or a refund above what was paid	Conditional updates: the second approval or the extra refund changes 0 rows and is rolled back	v0.31.0
Forged form requests	Every form carries a random CSRF token; a POST without it is refused (400)	v0.13.2
Forged or copied cookies	The user only counts as logged in if the cookie's token has a login record on the server	v0.20.3
Wrong role	Admin pages answer 403 to cashiers, even when the address is typed	v0.7.0
Same basket sent twice	The sale ticket is unique in the database; the second request opens the first receipt	v0.20.4

c. Exposure limitation and recovery (if an attack works, keep the damage small and recover):

Technique	How	Version
Logins end	Log-out, a password change (ends every other login of the user) or 8 idle hours	v0.20.3
Least privilege	Cashiers can sell, see receipts and *ask* for voids only; voids, refunds, roles and switching off need an admin (S15)	v0.7.0, v0.31.0

Technique	How	Version
Accounts can be stopped at once	Switching an account off, Log out everywhere or a role change ends every login and every trusted device of that person	v0.27.0, v0.28.0
Who did what	The audit log records every change to prices, stock, recipes, shop details, accounts and receipts, old and new value, in the same transaction	v0.27.0
Receipts can't be rewritten	A void or refund is a new record naming who asked and who approved; the receipt never changes (I8)	v0.31.0
A crash leaves nothing half-done	The friendly error page first undoes the unfinished database work	v0.21.3
All-or-nothing sales	A sale and all its stock changes are one transaction; if one part fails, nothing is saved	v0.8.2
History of every ingredient change	Who changed what, when, and on which receipt	v0.17.0
Debug console off	The in-browser Python console of debug mode can't be reached unless `FLASK_DEBUG=1`	v0.13.4
Nothing is deleted when starting over	`--fresh` renames the old database instead of deleting it	v0.14.0
Clickjacking blocked	`X-Frame-Options: DENY`; `Referrer-Policy: same-origin`	v0.13.4

9.5 Risk classification (item 35, Lecture 12 p. 15)

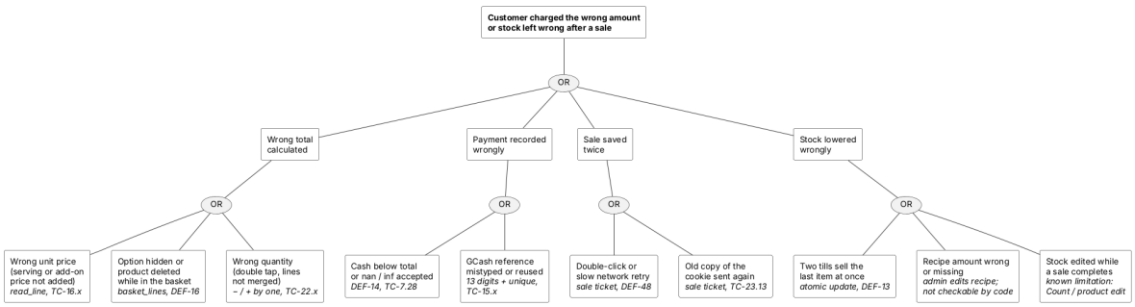
The same columns as Sommerville's insulin pump table. *Acceptability:* **Intolerable** = must be fixed before use; **ALARP** = "as low as reasonably practical", reduce it if the cost is reasonable; **Acceptable** = no action needed.

Identified hazard	Hazard probability	Accident severity	Estimated risk	Acceptability
1. The last item sold twice by two tills (stock −1)	Medium before v0.8.2; now very low (atomic update, TC-7.26)	Medium	Low	Acceptable
2. One basket charged twice (double-click, retry)	Medium before v0.20.4; now very low (sale ticket, TC-23.11–23.13)	Medium	Low	Acceptable
3. Wrong change given (cash below total or `nan` accepted)	Very low (server checks, DEF-14)	Low	Low	Acceptable
4. A drink's milk option lost on the way to the barista (allergen)	Very low (options copied onto each receipt line)	High	Medium	ALARP: the barista should still read the receipt and ask
5. Database lost (disk failure, file deleted)	Low	High	**Medium**	**ALARP**: no automatic backup yet; the owner should copy `instance/` regularly
6. Someone uses an admin login copied from the till, or a password they saw	Low (login records, 8-hour limit, v0.20.3; a code at every login, v0.28.0)	High	Low	Acceptable: everyone has their own account (v0.27.0); the demo passwords work only in demo mode
7. Stock count overwrites a sale made at the same second	Very low since v0.30.0 (the count is saved only if the amount is still the one the admin saw)	Low	Low	Acceptable

Identified hazard	Hazard probability	Accident severity	Estimated risk	Acceptability
8. GCash reference recorded but the money was never received	Low	Medium	Medium	ALARP: the cashier checks the customer's screen; automatic checks would need a GCash merchant account
9. Server error page (HTTP 500) during a sale	Very low (0 crashes in 6,417 fuzzing requests)	Low (nothing is saved half-way)	Low	Acceptable
10. Using this system as the shop's official cash register	Medium (it looks like a POS)	High (tax penalties: not BIR-registered)	**High**	**Intolerable**: receipts are order slips only; since v0.22.0 every receipt says "This is not an official receipt." (SR6), which is the software's part; official invoices must still come from the café's registered POS
11. A cashier voids a sale to keep the cash	Low (a void needs an admin's password or the admin's own emailed code)	Medium	Low	Acceptable: every void names who asked and who approved, and is in the audit log
12. Login codes can't be delivered (mail down, Gmail refusing the connection)	Medium (seen in TC-34.12)	Low (nobody new can log in until it is back)	Low	Acceptable: trusted devices, printed backup codes and the authenticator app need no email; the page says what happened

9.6 Software fault tree (item 36, Lecture 12 p. 18)

A *fault tree* starts from one bad event at the top and asks "what could cause this?" until it reaches basic causes at the bottom. An **OR** gate means any one of the causes below is enough. In italics under each basic cause: the code or test that now prevents it, or why it can't be prevented by code.



Reading the tree: of the ten basic causes, eight are blocked by code and tested; *a wrong recipe amount* can only be prevented by the owner checking the recipes (they are still estimates, §10.4). *A stock edit at the same moment as a sale* is now only half a limitation: since v0.30.0 an ingredient count is refused if the amount changed after the admin opened the page, but the product Edit form still overwrites a product's quantity.

9.7 Safety requirements (item 37, Lecture 12 p. 23)

Written like Sommerville's insulin pump requirements (SR1–SR6), from the hazards in §9.5. ✅ = met by v0.32.0 and tested; ⏳ = not yet met.

ID	Requirement	Status
SR1	The system shall not complete a sale for which there is not enough stock or ingredient at the moment of saving.	✅ atomic updates, TC-7.26, TC-17.11
SR2	The system shall not save the same basket as more than one sale.	✅ sale ticket, TC-23.11–23.13
SR3	The system shall not accept a cash amount below the total, or a value that is not a finite number.	✅ TC-7.28
SR4	The system shall print every chosen serving and add-on on the receipt line of the drink it belongs to, and shall keep it unchanged if options are later renamed or hidden.	✅ TC-16.x
SR5	The system shall save a sale and all its stock and ingredient changes together, or none of them.	✅ one transaction, rollback, TC-7.27
SR6	The receipt shall not be presented as an official sales invoice.	✅ since v0.22.0 every receipt ends with "This is not an official receipt.", which can't be turned off; TC-27.1
SR7	The owner shall be able to restore the sales and stock records after the computer fails.	⏳ partly: a backup is taken before every database migration (v0.23.0); a daily backup and restore is in the upgrade plan
SR8	A receipt shall never change after the sale; a void or refund shall be a separate record that names who asked and who approved.	✅ TC-41.5, TC-41.15
SR9	A void, a refund or a change of role shall need an admin; a cashier shall only be able to ask.	✅ TC-41.4, TC-35.3

9.8 Functional reliability requirements (item 38, Lecture 12 p. 39)

Like Sommerville's RR1–RR4. In brackets, the type of requirement: *checking*, *recovery* or *redundancy*.

ID	Requirement	Status
RR1	A pre-defined range shall be set for every number a user types (prices, quantities, cash: 0 to 1,000,000; GCash reference: exactly 13 digits; dates: 2000 to 2100), and the server shall check every input against it. *(Checking)*	✅ TC-2.7, TC-7.28, TC-15.x, TC-20.1
RR2	The system shall check stock and ingredients again at the moment of saving a sale, even if they were checked when the item was added. *(Checking)*	✅ TC-7.24–7.27
RR3	When the database refuses a change (stock too low, duplicate ticket or GCash reference), the system shall roll back the whole sale and show a message instead of an error page. *(Recovery)*	✅ TC-15.x, TC-23.11
RR4	When a new version starts with a database made by an older version, it shall back the database up and run the missing migrations by itself, without losing data. *(Recovery)*	✅ TC-23.15, TC-28.1–28.9 (also on a copy of the café's real database)
RR5	Starting over with demo data shall keep the old database under a new name instead of deleting it. *(Recovery)*	✅ test_seed.py
RR6	Every change to the code shall pass the automatic tests (38 files, 1,203 checks + 147 rule checks on v0.32.0) before it is merged. *(Checking)*	✅ GitHub Checks on every pull request
RR7	A copy of the database shall be kept on a second device. *(Redundancy)*	⏳ not yet (see SR7)

ID	Requirement	Status
RR8	Money shall be stored and added as whole centavos, so a total never differs from the sum of its lines. *(Checking)*	✅ TC-30.3–30.5
RR9	When two admins decide the same void at the same moment, the stock shall come back once and the second decision shall be refused with a message. *(Recovery)*	✅ TC-41.6
RR10	Logging in shall stay possible without the internet (trusted devices, backup codes, an authenticator app). *(Redundancy)*	✅ TC-38.8

9.9 Threat and control analysis (item 39, Lecture 12 p. 46)

As in Sommerville's preliminary risk assessment for the MHC-PMS: each threat, how likely it is, the control, and whether the control is feasible (cost and effort).

Threat	Probability	Control	Feasibility
Unauthorised user gets in as the admin by guessing the password	Low	Passwords hashed; 5-try limit per username and computer; demo passwords changed	Done (v0.13.3, v0.19.3, 2026-10-12). Low cost
Someone copies a logged-in cookie from the till	Low	Login records; log-out and password change end it; 8 idle hours end it	Done (v0.20.3). Low cost
A cashier opens admin pages by typing the address	Medium	403 on every admin page; contract check makes sure every page is listed as public, user or admin	Done (v0.7.0, v0.19.0)
Another website makes the browser submit a form (CSRF)	Low	Random token on every form; SameSite=Lax cookie	Done (v0.13.2)
Someone guesses the current password on the change-password page with a stolen login	Low	The login attempt limit, counted per user (audit A7)	Done (v0.26.0)
Someone finds which usernames exist by timing the login	Low	A dummy hash is checked for unknown users (audit A8)	Done (v0.21.4)
Someone sees a password over the cashier's shoulder	Medium	A code at every login on an untrusted browser; trusted devices end on a password change	Done (v0.28.0). The code costs the user a few seconds
An invitation link is forwarded or stolen	Low	One use, 48 hours, and the code goes only to the invited address	Done (v0.27.0)
An admin's email account is broken into	Low	An emailed approval code works for one void only, for 10 minutes; the owner can reset an account's second step	Done (v0.31.0, v0.28.0). Admins should protect their email
A malicious file uploaded as a picture	Low	Content check, 2 MB limit, random names, `nosniff`	Done (v0.12.0, v0.14.0)
Script injected through a product name	Low	Jinja2 escaping; *later:* Content-Security-Policy (audit A10)	Escaping done; CSP needs the inline scripts moved to files (in the upgrade plan)
Any logged-in user opens another cashier's receipt by number	Medium	Decision for the owner: normal in a small café, or limit cashiers to their own receipts (audit A12)	Feasible if wanted

Threat	Probability	Control	Feasibility
The database file is copied off the laptop	Low	The file is not on GitHub (.gitignore, rule P1); passwords are hashed	Done; disk encryption is the owner's choice (Windows BitLocker)
Laptop lost or broken: records gone	Low	*Not yet:* automatic backup to a second device (RR7)	Feasible; a copy of `instance/` to a USB drive each week costs nothing

Words used in §9: *atomic update* — a database change that happens completely or not at all; *rollback* — undoing every change of an unfinished transaction; *scrypt* — a slow hashing method that makes guessing passwords expensive; *CSRF* — Cross-Site Request Forgery, another website making your browser send one of our forms; *Content-Security-Policy (CSP)* — a header that tells the browser which scripts it may run.

10. Defects, lessons learned and known limitations

10.1 Defects found and fixed

Testing found **71 defects** over the life of the project: DEF-01 to DEF-60 up to the course submission (v0.21.2) and DEF-61 to DEF-71 after it. Every one of them is fixed in v0.32.0, and from v0.19.0 onwards each fix came with an automated test that fails on the old code and passes on the new one. The table records where each defect was found and how it was fixed.

ID	Severity	Description	Found by	Fix	Status / target
DEF-01	**High**	Adding or editing a product with an SKU that already exists crashes the server (500) instead of showing a message	TC-2.3, TC-3.4	Before saving, check `Product.query.filter_by(sku=...)` (excluding the product being edited); if found, show the form again with an error	✅ Fixed v0.7.1
DEF-02	Medium	Non-number price/quantity crashes the server if the browser check is bypassed	TC-2.4	Wrap `float()` / `int()` in `try/except ValueError` and show an error	✅ Fixed v0.7.1
DEF-03	**High**	Negative price and negative quantity are accepted	TC-2.5	Server check: `price >= 0`, `quantity >= 0`; also add `min="0"` to the form inputs	✅ Fixed v0.7.1
DEF-04	Medium	Empty name/SKU accepted if the browser check is bypassed	TC-2.6	Server check: `.strip()` the value and reject if empty	✅ Fixed v0.7.1
DEF-05	Low	Prices display with a `$` sign, but the store is in the Philippines	Code review; confirmed by hand	Change the `$` in `products.html` to ₱	✅ Fixed v0.7.1
DEF-06	Low	After logging in, the user is always sent to Products instead of the page they first asked for	TC-5.8	Read `request.args.get('next')` and redirect there (only if it is a page within this site: one leading `/`, never `//` or `/` + backslash)	✅ Fixed v0.20.6
DEF-07	**Critical** (security)	`SECRET_KEY` is written directly in `app.py` and pushed to a public GitHub repository. Proven on 2026-10-09: a cookie built with it opened the Sales Report as admin without a password	Code review; attack test TC-13.1	Key from the `SECRET_KEY` environment variable, else a random key created once in `instance/secret_key.txt` (not uploaded)	✅ Fixed v0.13.1
DEF-08	**High** (security)	Forms have no **CSRF protection** (CSRF = Cross-Site Request Forgery: another website tricking a logged-in user's browser into submitting a form). Proven: a product was added by a request without any token	Code review; TC-13.3	Own small CSRF check: a random token per session in a hidden field of every form, compared with `secrets.compare_digest`; no new library needed	✅ Fixed v0.13.2

ID	Severity	Description	Found by	Fix	Status / target
DEF-09	**High**	Logging out did not empty the basket, so on a shared counter computer the next user saw the previous cashier's items	Testing during development (TC-7.17)	`session.pop('basket', None)` in `logout()`	✅ Fixed before v0.8.0 release
DEF-10	Medium (usability)	The New Sale dropdown showed database stock, ignoring items already in the basket, so it looked like more could be sold	My own hand testing (TC-7.4)	Dropdown shows **available = stock − basket**; products fully in the basket are hidden; message says how many *more* can be added	✅ Fixed before v0.8.0 merge
DEF-11	**High** (deployment)	After upgrading the code, **Complete Sale** crashed: "table sale has no column named cash_received". `db.create_all()` creates missing *tables* but never adds *columns* to existing ones	My own hand testing (TC-7.23)	One-time **migration** script `add_cash_columns_once.py` (SQL `ALTER TABLE … ADD COLUMN`), database backed up first. The crashed sale saved nothing, because the save is one transaction	✅ Fixed 2026-10-08 (database change, no code change)
DEF-12	Low	Receipts for sales made before cash tracking showed "Cash ₱0.00 / Change ₱0.00", which looks like a real but impossible payment	Hand testing (TC-7.21)	`receipt.html` shows "Cash and change not recorded…" when cash is 0; no data invented	✅ Fixed v0.8.1 (PR #3)
DEF-13	Medium (reliability)	**Race condition:** two cashiers completing a sale for the last item at the same moment could both pass the stock check, leaving stock at −1	Design review; reproduced by tests (TC-7.24–7.27)	**Atomic update**: subtract stock only *if enough is left*, in one database step; roll back the sale otherwise	✅ Fixed v0.8.2
DEF-14	**High**	`nan`, `inf` and huge numbers in cash, price or quantity crashed the server or were saved as "infinite"	Exploratory testing (TC-7.28, TC-2.7)	Limits of 1,000,000 + `math.isfinite()`	✅ Fixed v0.8.3
DEF-15	Medium	Ids too big for SQLite in an address or form crashed the server	v0.8.x exit check (TC-7.29)	`find_by_id()` treats them as "not found" (404)	✅ Fixed v0.8.4
DEF-16	**High** (usability)	A product deleted while in a basket left the basket stuck: hidden line, sale could never complete	v0.8.x exit check (TC-7.30)	Deleted products are taken out of the basket with a note	✅ Fixed v0.8.5
DEF-17	Low	Text longer than the database columns (e.g. 5,000-character name) was accepted	Fuzz testing (TC-2.8)	Length check + `maxlength` (100 / 50 / 50)	✅ Fixed v0.8.6
DEF-18	Medium (usability)	With many low products the low-stock box filled the screen	v0.9.x stabilization (TC-8.8)	5 most urgent + "and N more"	✅ Fixed v0.9.1
DEF-19	Low	Empty category displayed as the word "None" (since v0.3.0)	v0.9.x stabilization (TC-8.9)	Dash for empty categories	✅ Fixed v0.9.2

ID	Severity	Description	Found by	Fix	Status / target
DEF-20	Medium	A product renamed after being sold appeared twice in Top products, giving a wrong ranking	v0.10.x stabilization (TC-9.7)	Group by product id; show the latest name	✅ Fixed v0.10.1
DEF-21	Medium (data integrity)	Deleting a sold product: SQLite may give its id to the next new product, which then "inherits" its sales	v0.10.x stabilization (TC-9.8)	Products with sales history can't be deleted (set quantity to 0 instead)	✅ Fixed v0.10.2
DEF-22	Low	Demo data: receipt numbers not in time order	v0.11.x stabilization (TC-10.3)	Sort each day's sale times before creating them	✅ Fixed v0.11.1
DEF-23	Low	Amounts ≥ ₱1,000 shown without thousands separator	v0.11.x stabilization (TC-10.4)	`peso` template filter used everywhere	✅ Fixed v0.11.2
DEF-24	Medium	Sales report with end date 9999-12-31 crashed (no next day exists)	Pre-release testing of v0.10.0 (TC-9.5)	Dates limited to 2000–2100	✅ Fixed before v0.10.0 release
DEF-25	Medium	With the web font loading normally, a slow or blocked font server held up the New Sale script, so the change preview stayed at ₱0.00 (browsers wait for stylesheets before running scripts)	Pre-release testing of v0.13.0 (TC-12.6)	Font stylesheet loads in the background (`media="print"` switched to `all` when loaded)	✅ Fixed before v0.13.0 release
DEF-26	Low	"Please log in to access this page." (from Flask-Login) shown as unstyled text, because its category `message` has no Bootstrap colour	v0.13.0 restyle (TC-12.8)	Category `message` shown as `info`	✅ Fixed v0.13.0
DEF-27	**High** (security)	No limit on wrong passwords: 200 guesses in a row, then the right password still worked	Security review (TC-13.7)	5 wrong passwords per username + IP address → wait 5 minutes (429)	✅ Fixed v0.13.3
DEF-28	Low (security)	Logout worked as a plain link, so any website could log a user out	Security review (TC-13.5)	Logout only as a POST button with token	✅ Fixed v0.13.2
DEF-29	Medium (security)	No browser security headers: pages could be framed by other sites (clickjacking)	Security review (TC-13.9)	`X-Frame-Options`, `X-Content-Type-Options`, `Referrer-Policy` on every response	✅ Fixed v0.13.4
DEF-30	Medium (security)	`app.run(debug=True)`: on a crash, the debugger console can run Python code	Security review (TC-13.10)	Debug only with `FLASK_DEBUG=1`	✅ Fixed v0.13.4
DEF-31	Medium (security)	Default passwords (`admin123`) could not be changed	Security review	Change password on the profile page	✅ Fixed v0.13.5
DEF-32	Low	A long name without spaces spilled out of its menu tile over the basket	v0.14.x stabilization (TC-14.13)	Wrap anywhere, max 3 lines in tiles	✅ Fixed v0.14.1
DEF-33	Low (usability)	Menu tiles sorted A–Z, mixing drinks and pastries	v0.14.x stabilization (TC-14.14)	Order by category, then name	✅ Fixed v0.14.2

ID	Severity	Description	Found by	Fix	Status / target
DEF-34	Medium (usability)	Each tap reloaded the menu back to "All", so the cashier had to choose the category again for every item	v0.14.x stabilization (TC-14.14)	Chosen category remembered for the browser tab	✅ Fixed v0.14.2
DEF-35	Medium (usability)	On phones the basket and Pay button were below the whole menu	v0.14.x stabilization (TC-14.15)	Bottom bar with item count and total that jumps to the basket	✅ Fixed v0.14.3
DEF-36	Medium (regression from v0.14.1)	On phones, basket names were squeezed to one letter per line	v0.14.x stabilization (TC-14.16)	Table cells wrap only long words; names kept 8–22rem wide	✅ Fixed v0.14.3
DEF-37	Medium (usability)	Between about 990 and 1360 px wide, the basket's Remove (×) button was cut off	Screenshot review while testing v0.15.0 (TC-14.19)	Quantity under the name, tighter cells, side-by-side only from 1200 px	✅ Fixed v0.14.4
DEF-38	Medium	Unticking "sizes and add-ons" on a product left basket lines that were then sold with the size price	v0.16.x stabilization (TC-16.11)	Such lines are taken out with a message	✅ Fixed v0.16.1
DEF-39	Low	Many long add-on names gave options text longer than the 200-character column	v0.16.x stabilization (TC-16.12)	Shortened with "…"	✅ Fixed v0.16.1
DEF-40	**High** (data integrity)	Deleting a never-sold product left its recipe; SQLite gave the id to the next product, which inherited the recipe	v0.17.x stabilization (TC-17.13)	Recipe deleted with the product; start-up removes leftovers	✅ Fixed v0.17.1
DEF-41	Low (usability)	The café sells drinks Hot or Iced, but the screens said "Size", so "Size: Iced" read wrongly	My decision for the café (TC-16.14)	The choose-one group is called a *serving* on every screen; tiles list the servings ("Hot / Iced")	✅ Fixed v0.17.2
DEF-42	Medium (demo data)	With the café's menu every product gets its stock from ingredients (quantity 0), so the demo sales found no product to sell and created only empty ₱0.00 receipts	Screenshot review of v0.17.2 (Home: 8 sales, ₱570.00) (TC-17.14)	Demo sales pick products that can be made (`stock_levels()`); no sales are created if nothing can be sold; new `test_seed.py` checks both menus	✅ Fixed v0.17.3
DEF-43	Medium	Number boxes were filled with "2,000", which a number box can't read, so they showed up empty; the Edit form of Fresh milk (warn at 2,000 ml) could not be saved	Screenshot review while redesigning (TC-18.1)	`plain_amount` filter (no separator) for every number box	✅ Fixed v0.17.4
DEF-44	Low (usability)	After each Restock / Count / Edit the list reloaded at the top with the search and filter cleared	v0.18.x stabilization (TC-18.12)	Return to the row (`#ing-<id>`), remember search and filter for the tab	✅ Fixed v0.18.1
DEF-45	Low (cosmetic)	In the Products table SKUs broke after the dash ("HC-" / "001") and "100 can make" took two lines	Screenshot review for the documentation (TC-18.15)	SKU and count cells keep their text on one line	✅ Fixed v0.18.2

ID	Severity	Description	Found by	Fix	Status / target
DEF-46	Low (cosmetic)	Avatar menu: "Change picture" and "Change password" ran past the menu's right edge and "Signed in as Admin" took two lines. Side effect of v0.19.1: the menu was set to the name button's width (`w-100`), and the button became narrower when the theme icon moved into its row	My own hand testing (screenshot), confirmed by measuring in Chromium at 100 %, 110 %, 125 % and phone width (TC-12.13)	Menu is at least the button's width but grows to fit its longest line (`min-width: 100%; width: max-content`), lines never wrap	✅ Fixed v0.19.2
DEF-47	Medium	A 309-digit quantity for a drink made from a recipe crashed New Sale (500, `OverflowError` in `Recipes.usage`); neither quantity box has a maximum	Security audit 2026-10-11 (fuzzing, audit A1)	Refuse quantities above `MAX_QUANTITY` before calculating	✅ Fixed v0.19.3
DEF-48	Medium	Two *Complete Sale* requests sent at the same moment with the same basket both succeed: two receipts, stock taken twice (the basket is in the cookie both carry)	Security audit 2026-10-11 (audit A2; a real double-click in Chromium made only 1 sale)	One-time sale ticket per basket, stored on the receipt with a unique index (same idea as the GCash reference); the second request opens the first receipt with a message. Complete Sale greys out after the first click	✅ Fixed v0.20.4
DEF-49	Medium (Windows)	Werkzeug 3.1.8 has CVE-2026-102598: on Windows a static-file request for a device name like `NUL:` hangs	`pip-audit`, security audit 2026-10-11 (audit A3)	`Werkzeug==3.1.9` (all 17 test files pass with it)	✅ Fixed v0.19.3
DEF-50	Medium	The login limit keeps every wrong username in memory, at full length, and never forgets untried ones: 50 tries with 450 KB usernames kept 33 MB	Security audit 2026-10-11 (audit A4)	Cut usernames to 80 characters before counting; clear old entries for everyone	✅ Fixed v0.19.3
DEF-51	Medium	A copied session cookie still works after log-out and after a password change; sessions never time out while the browser is open	Security audit 2026-10-11 (audit A6)	New `login_session` table (one row per logged-in browser, the cookie holds only its random token): log-out deletes the row, a password change ends all of the user's logins and gives this browser a new token, 8 idle hours end a login. Chosen over a per-user "session version" so that logging out on one till doesn't log out another till sharing the same account	✅ Fixed v0.20.3

ID	Severity	Description	Found by	Fix	Status / target
DEF-52	Low	SKUs that differ only in capitals (`HC-001`, `hc-001`) are both accepted	Security audit 2026-10-11 (audit A9)	Compare SKUs ignoring capitals (`lower()` on both sides); a product keeping its own SKU is not re-checked, so pairs saved earlier stay editable	✅ Fixed v0.20.2
DEF-53	Medium (demo)	Without internet the pages lost their layout, icons and font, and the avatar and phone menus stayed open, because Bootstrap, its icons and Inter loaded from jsDelivr and Google Fonts	Offline test in Chromium with the internet blocked (2026-10-10); security audit R4	Store the three in `static/vendor/` with their licences; AGENTS.md rule D2 changed	✅ Fixed v0.19.4
DEF-54	Low	New Sale v0.20.0 (before release): product names broke in the middle of a word ("Cappuccin / o") because the picture took space beside the name	Screenshot review, TC-22.9	Picture and price on the top row, the name across the whole tile, wrapping only at spaces	✅ Fixed before v0.20.0 was published
DEF-55	Low	New Sale v0.20.0 (before release): on phones "₱1,000" spilled out of its quick-cash button	Measured in Chromium at 390 px, TC-22.9	Buttons at least 3.9 rem wide; a fifth button wraps to the next row instead	✅ Fixed before v0.20.0 was published
DEF-56	Low	New Sale v0.20.0 (before release): on phones the basket line squeezed the name beside the − / + buttons ("1 ×" and "₱140.00" on two lines)	Screenshot review at 390 px, TC-22.9	On phones the name gets its own row; − / + and the subtotal go below it	✅ Fixed before v0.20.0 was published
DEF-57	Low	New Sale v0.20.0 kept the 1180 px page width of the other pages, so on a wide screen about 200 px stayed empty on each side, the menu stopped at 3 tiles per row, and each tile kept a blank line under one-line names	My own check, on my laptop with the café's menu (2026-10-12)	New Sale only: no width limit; auto-fill tile grid; basket 380–460 px; bar and "N left" on one line	✅ Fixed v0.20.1
DEF-58	Low	v0.20.0 named the New Sale tile bars `stock-bar`, a class the Ingredients page already used. The new rules clipped the Ingredients warn-level tick to a 6 px green speck, so it was hard to see how close a full shelf was to its warning level (side effect of v0.20.0, like DEF-46 was of v0.19.1)	My own check, on my laptop (2026-10-13); measured in Chromium against v0.19.4	New Sale bars renamed (`tile-stock`, `tile-bar`, `tile-note`); TC-22.13 now fails if any New Sale style name is used on another page. Lesson: the v0.20.0 browser check measured only New Sale; a style added for one page must be checked on the pages that share the style sheet	✅ Fixed v0.20.5

ID	Severity	Description	Found by	Fix	Status / target
DEF-59	Low	Folded sidebar: the light/dark icon sat on the right edge, not under the avatar (the row's `margin-left: auto` still applied after the row became a column in v0.21.0)	My screenshot (2026-10-23); measured in Chromium	`margin-left: 0` for the folded sidebar; TC-25.8. Lesson: a rule written for one layout (a row) must be checked when the layout changes (a column)	✅ Fixed v0.21.2
DEF-60	Medium	Phone: the "New ingredient" panel was 100 % of the screen wide but started at the button, so its right part was cut off (made worse by v0.21.0's bigger padding)	Layout audit of all pages at 375 px (2026-10-23)	On phones the panel floats under the top bar with 1 rem margins and scrolls if tall; TC-25.9	✅ Fixed v0.21.2
DEF-61	Low	A crash showed the plain *Internal Server Error* page: no layout, no advice, and the database session was left half-done until the next request (audit A11)	Security audit 2026-10-11 (A11), seen with A1	`server_error()` undoes the unfinished database work, writes the details to the server log and shows `500.html`; if even that fails, a plain friendly message (DEC-019); TC-26.1–26.4	✅ Fixed v0.21.3
DEF-62	Low (security)	Usernames could be found by timing: a wrong password for a real user took about 100 ms (the password hash is checked), an unknown username about 1 ms (audit A8)	Security audit 2026-10-11 (A8, measured)	`login()` checks `DUMMY_PASSWORD_HASH` (a random password hashed with scrypt at start-up) when the username doesn't exist (DEC-020); TC-26.5–26.7	✅ Fixed v0.21.4
DEF-63	Low (demo)	The built-in demo menu sold drinks as 12oz / 16oz, with separate "Iced …" products, while the app, the stories (F15, F19) and the café use Hot / Iced, so the demo didn't show the feature as designed (audit R10)	Security audit 2026-10-11 (R10)	`seed_demo_data.py`: one product per drink, servings Hot and Iced, cup and ice recipes on the servings (DEC-021); TC-26.8–26.10	✅ Fixed v0.21.5
DEF-64	Low (tooling)	`requirements.txt` was saved as UTF-16 (PowerShell's `pip freeze >` writes UTF-16). pip reads it, but many tools (GitHub's dependency graph, some editors and security scanners) don't, and Git showed it as a binary file (audit R11)	Security audit 2026-10-11 (R11)	Saved as UTF-8 with the same 13 packages; new contract rule "requirements.txt is plain UTF-8" with the command to fix it; TC-26.11	✅ Fixed v0.21.6
DEF-65	Low (cosmetic)	No browser tab icon: every page load ended with "404 Not Found" for `/favicon.ico` in the browser's console, and the tab showed a blank page icon (known limitation since the report)	Report §10.4 known limitations	`static/favicon.svg` and a `<link rel="icon">` in `base.html`; TC-26.12–26.13	✅ Fixed v0.21.7

ID	Severity	Description	Found by	Fix	Status / target
DEF-66	Low (cosmetic)	Ingredients page, dark theme: a normal stock bar used the amber accent `#EF9F27`, exactly the colour of a Low bar, so a full shelf and a low one looked alike (the report's known limitation called them "tile bars"; measuring showed it is the Ingredients page; the New Sale tiles were already green and amber)	Report §10.4 known limitations; checked against the CSS (rule C10)	`[data-bs-theme="dark"] .stock-fill.is-ok` uses `--stock-ok` (green); TC-26.14–26.15	✅ Fixed v0.21.8
DEF-67	Medium (future)	Times were stored in the computer's local time with no zone. On a cloud server (UTC) "today", the report's days and receipt times would have been 8 hours off (audit R1)	Security audit 2026-10-11 (R1)	Store UTC (`models.utc_now`), show and count days in the shop's time (`SHOP_TIMEZONE` UTC+8, `shop_time` filter); migration 0002 converts old times (DEC-024); TC-29.1–29.10	✅ Fixed v0.23.1
DEF-68	Medium	Money was stored as decimal numbers (REAL/float). Sums of many amounts can drift by a centavo, and a price typed with three decimals (₱0.125) showed as ₱0.12 per item but ₱0.38 for three, so the receipt didn't add up (audit R5)	Security audit 2026-10-11 (R5); the receipt case found by TC-30.3	`Money` column type: whole centavos in the database, exact `Decimal` pesos in Python, rounded once, half up, on saving; migration 0003 (DEC-025); TC-30.1–30.10	✅ Fixed v0.23.2
DEF-69	Low	Change password checked the current password with no limit, so someone using a stolen login could guess it there as fast as they liked (audit A7)	Security audit 2026-10-11 (A7)	The login page's 5-tries limit, counted per user (DEC-028); TC-33.5	✅ Fixed v0.26.0
DEF-70	Low (Medium once online)	The login limit was kept in memory: a restart forgot it, and each server process had its own count (audit R2)	Security audit 2026-10-11 (R2)	Tries are rows in `login_attempt` (migration 0004), recorded before they are counted, keys stored as keyed hashes (DEC-028); TC-33.1–33.4, TC-33.6	✅ Fixed v0.26.0
DEF-71	Medium (demo)	Demo mode crashed the page (500) when an email held a character the server window's text encoding can't show: Windows writes a redirected window or log in cp1252, which has no "₱", and the void email has prices (the earlier emails had none)	Browser check of v0.31.0 (TC-41.17)	`mail.py` prints the demo email with "?" for any such character; TC-41.16 fails before and passes after	✅ Fixed v0.31.0

10.2 Lessons learned

Looking back over the 71 defects, a few patterns stand out:

- **Never trust the browser.** The first four defects (DEF-01 to DEF-04) were all values the browser would normally stop but the server accepted. Every form value is now checked again on the server.
- **"Check, then write" is not safe when two people work at once.** Two tills could both see "1 left" and both sell it (DEF-13). The fix was to let the database do the check and the subtraction in one step.
- **A change to one page can break another.** DEF-46, DEF-58 and DEF-59 were side effects of a style change made for a different page. Since v0.20.5 a test makes sure New Sale's style names are not shared with other pages, and the last version ended with an audit of every page.
- **Screenshots find bugs that tests miss.** Looking carefully at each screenshot for this report found DEF-25, DEF-32, DEF-36, DEF-42, DEF-43 and DEF-45.
- **Real users find the rest.** Seeing the system on my own laptop with the café's real menu found DEF-57, DEF-58 and DEF-59, which no automated test had caught.
- **The browser check is still worth doing after 1,200 automated checks.** DEF-71 (demo mode crashed on a "₱" in an email) and the layout problems in TC-38.14, TC-40.8, TC-41.17 and TC-42.11 were all found by opening the real page, not by a test. Each one then got a test.
- **A big change is safest when nothing is supposed to change.** Splitting `app.py` into blueprints (v0.25.0) moved 1,500 lines, and the whole existing suite passing with one changed line was the proof that nothing broke.
- **Some failures are outside the code.** Gmail refused the login from my home connection (TC-34.12). Telling that apart from a wrong password (no "535" answer) saved hours of looking for a bug that wasn't there, and is why the system has backup codes and an authenticator app that need no email.

> **Note on the test tools (v0.16.0):** two race tests (TC-7.24–7.26) found the "other cashier" moment by counting database look-ups, which changed when the basket code changed, so they briefly reported false failures. They now step in right before the stock update itself, and pass on both the old and the new code. Lesson: a test should depend on *what* the code does, not on *how* it is written.
>
> **How the v0.7.1 fix works.** All checks live in one function, `validate_product_form()` (in `app.py` then, in `blueprints/catalog.py` since v0.25.0), used by both Add and Edit (the **DRY** principle: Don't Repeat Yourself). It returns either clean data or an error message; on error the form is shown again with a red Bootstrap alert and nothing touches the database. The browser checks (`required`, `min="0"`) are a convenience; the server check is the real protection.
>
> **Lesson from DEF-11:** changing the *shape* of an existing database (adding a column) needs a **migration**: a small, deliberate script run once, after a backup. Since v0.23.0 the project does exactly this with Flask-Migrate (Alembic): migrations 0001 to 0008, each tested on a copy of an older database, with a backup taken automatically before one runs.

10.3 Security audit and security review

On 11 October 2026 the whole of v0.19.2 went through a vulnerability and debugging review; the full report is in `docs/security_audit_2026-10-11.md`. In short:

- **Method:** dependency check (`pip-audit`), code scanner (`bandit`), manual review of all 21 routes, fuzzing of 39 forms with 6,417 hostile requests, 11 targeted attacks and live server probes.
- **What held up:** no SQL injection, no script injection, no way past the login or admin checks, no crash from the 6,417 fuzzing requests.

- **What it found:** 1 crash (DEF-47), a duplicate-sale risk (DEF-48), an outdated package (DEF-49), memory growth in the login limit (DEF-50), sessions that can't be cancelled (DEF-51), case-different SKUs (DEF-52), public demo passwords, and 11 risks for future goals (time zone, money as decimals, migrations, splitting `app.py`, user

management).

- **After the submission:** the open findings A5 (shared demo passwords), A7 (no limit on Change password), A8 (login timing) and A11 (plain error page), and the risks R1 (time zone), R2 (login limit in memory), R3 (development server), R5 (money as decimals), R6 (no migrations), R7 (one big `app.py`), R9 (no audit trail), R10 (demo menu) and R11 (UTF-16 requirements file), were all closed in v0.21.3–v0.27.0. A10 (Content-Security-Policy) and A12 (which receipts a cashier may open) are still open (§9.9).

Security review (2026-10-09): threats and controls

Each threat below was **tested as an attack**, not just assumed. Chapter 9 builds on this table.

Threat (what an attacker tries)	Asset at risk	Control in the system	Tested by	Status
Steal the database and read passwords	User accounts	Passwords stored as scrypt **hashes**	Code review	✅ since v0.6.0
SQL injection in the login form	Whole database	SQLAlchemy sends input as data, never as SQL	TC-13.13	✅
Cross-site scripting (XSS) through a product name	Other users' browsers	Jinja escapes everything shown	TC-13.14	✅
Forge a login cookie	Admin access	Secret key not in the code	TC-13.1	✅ v0.13.1
Cross-site request forgery (CSRF)	Products, sales, passwords	Token in every form; SameSite cookie	TC-13.3–13.6	✅ v0.13.2
Guess passwords (brute force)	User accounts	5 tries then 5 minutes wait; change default passwords	TC-13.7, 13.8, 13.11	✅ v0.13.3, v0.13.5
Clickjacking / content sniffing	User actions	Security headers	TC-13.9	✅ v0.13.4
Run code through the debugger console	The computer	Debug off by default	TC-13.10	✅ v0.13.4
Cashier uses admin pages	Products, reports	`@admin_required` → 403	TC-6.x	✅ since v0.7.0
Upload a harmful file as a "picture"	Server, other users	Content check (PNG/JPG/WebP bytes), 2 MB limit, random names, `nosniff`	TC-11.5–11.7, TC-14.3, TC-14.9	✅
Two sales take the last item at once	Stock accuracy	Atomic conditional update	TC-7.24–7.26	✅ since v0.8.2
Two tills use the same milk at once	Ingredient stock	Same atomic update for every ingredient; whole sale cancelled otherwise	TC-17.10, 17.11	✅ v0.17.0
Reuse one GCash payment for two purchases	Café's money	Reference number must be unique (checked + unique index)	TC-15.6, 15.7	✅ v0.15.0
Untraceable stock changes	Ingredient stock	Ingredient history: every sale, restock and count with user and time	TC-17.2, 17.3, 17.8	✅ v0.17.0

Threat (what an attacker tries)	Asset at risk	Control in the system	Tested by	Status
Log in with a seen or guessed password	User accounts	A code at every login; trusted devices; backup codes	TC-38.3–38.9	✅ v0.28.0
Make yourself an admin	Admin access	No sign-up page; invitations with a fixed role; only the owner changes roles	TC-35.3–35.5, TC-36.3	✅ v0.27.0
Read codes or tokens from a copied database	User accounts	Keyed hashes only; authenticator keys encrypted	TC-33.3, TC-36.1, TC-38.5, TC-38.9	✅ v0.27.0, v0.28.0
Forge a Google sign-in	User accounts	Every check on Google's answer done on the server	TC-39.4 (13 cases)	✅ v0.29.0
A cashier voids a sale or refunds money	Café's money	An admin must approve; 403 on every approval page	TC-41.4	✅ v0.31.0

Words used: **threat** (something that could cause harm), **vulnerability** (a weakness it can use), **control** (the protection), **attack** (an attempt to exploit a vulnerability).

10.4 Known limitations

These are things the system does not do yet, or does in a simple way on purpose. They are candidates for future increments; the ones already planned are in `docs/upgrade_plan.md`.

- **Not a BIR-registered POS.** In the Philippines a machine that issues official sales invoices must use software from a BIR-accredited supplier (RMO 24-2023), each machine needs a Permit to Use, and invoices must follow RR 7-2024. Every receipt says "This is not an official receipt." (v0.22.0); it is an order slip for the shop's own records.
- **No senior citizen / PWD discount and no X- or Z-reading** (end-of-shift and end-of-day summaries), which Philippine cafés are expected to have. Shifts and X/Z readings are the next increment (v0.33.0; voids will then end at the Z reading instead of midnight); discounts follow in v0.34.0, once the café's written rules are in hand.
- **No automatic daily backup** (SR7, RR7): a backup is taken before every migration, but otherwise the owner must copy `instance/` and the picture folders. Product pictures and avatars are files on disk, so the database alone does not back them up.
- **Real email depends on the mail provider accepting the login.** Gmail refused it from my home connection (TC-34.12); demo mode, backup codes and the authenticator app work without it.
- **Sign in with Google works only on `localhost` or HTTPS** (Google's rule), and needs the owner's own Google Cloud client (TC-39.10 still open).
- **The help pictures must be retaken when a page changes**; the test checks that each page has a picture, not that it is up to date.
- **Browser tests are not automatic yet:** `tests/` covers the server; clicking through pages in a real browser is still done before each release (automated with Playwright from v1.0.0, rule T4).
- **No Content-Security-Policy header** yet, because the pages use inline scripts (audit A10).
- **The cloud package isn't built yet.** The `store/` layer is ready for PostgreSQL; the package itself, and the licence check against the subscription website, come later in the plan.

- **GCash is recorded, not verified:** the cashier checks the amount on the customer's screen. Automatic checking needs a GCash merchant account.
- **Recipe amounts are estimates** (e.g. 18 g beans per shot, 150 g ice for Iced) until the café owner confirms them; the admin can edit them.
- **Editing a product's stock overwrites it with the number typed in** (products without a recipe). Ingredients have Restock-by-amount, a safe count and a history; products could get the same (planned: product restock).
- **Sold products can't be deleted, only set to 0**, so they stay in the list with an "Out of stock" badge (planned: archive).
- **One low-stock threshold (5) for every product**, and **ingredient groups are a fixed list**.
- **The light/dark choice is saved per browser**, not per user account.
- **SQLite allows one writer at a time.** Fine for a small shop; many tills would use the cloud package's PostgreSQL.

Limitations of v0.21.2 that are now fixed: the fixed shop name on receipts (v0.22.0), money stored as `Float` (v0.23.2), the login limit kept in memory and no way to reset a forgotten password (v0.26.0, v0.27.0), no undo for a tap and a count that could overwrite a sale (v0.30.0), no voids or refunds (v0.31.0), the dark-theme bar colour (v0.21.8) and the missing tab icon (v0.21.7).

11. Conclusion

The project set out to replace a café's notebook and calculator with a system that records every sale correctly, keeps stock accurate on its own, and tells the owner how the day went. Version 0.21.2, the course submission, did that, and version 0.32.0 turns it into something a café could be given:

- A cashier can ring up a typical order in a few taps, take cash or GCash, and print a receipt that never changes.
- Stock and ingredients go down with every sale, inside the same transaction, so the system never sells what it does not have, even with several tills selling at the same moment.
- The owner sees low stock as it happens, can explain every ingredient change from the history, and reads revenue split into cash and GCash for any range of dates.
- Cashiers cannot reach admin pages, passwords are stored only as hashes, forms are protected against forgery, logins can be ended, and the pages work without the internet.
- Every person has their own account, made only by the owner or an invitation, and a code at every login keeps out someone who only knows the password.
- A mistake at the counter can be undone, and a sale can be voided or refunded with an admin's approval, while the receipt itself never changes and the audit log shows who did what.
- Money is exact to the centavo, times are stored in world time, the database upgrades itself safely, and the code is layered so the same pages can run on a cloud database.

Building it in small increments meant there was always a working version to show and test. It also meant that every defect had a version it was found in and a version it was fixed in, which made the 436 test cases and 71 defects in this report possible to trace one by one. The same process carried the project past the course: nineteen more versions in three days, each with its own decision record, tests and browser check.

What I learned. The biggest lesson was that most problems were not where I expected them. The features themselves were rarely the hard part; the hard parts were two cashiers pressing a button at the same moment, a customer typing something strange, a style meant for one page changing another, and the gap between "the test passes" and "it looks right on a real screen". Testing on purpose for those cases, writing the rules down in `AGENTS.md` and letting the automated checks enforce them is what made the later versions stable.

What comes next. The plan (`docs/upgrade_plan.md`) continues with **v0.33.0** shifts and X/Z readings, **v0.34.0** senior citizen and PWD discounts, product restock and archive, backup and restore (SR7, RR7) and the Content-Security-Policy header, then the **cloud package** on managed PostgreSQL and the licence check against the subscription website. Release **v1.0.0** adds automated browser tests and a trial at the café. Using the system as a café's official register would also require BIR accreditation, which stays outside the scope of this project.

Appendix A. Version history

Each version is marked with an **annotated tag** in Git, which stores a message, a date and who made it. The tag message summarizes what the increment added. From v0.8.2 the Commit column shows the commit with the change; the tag itself sits on the GitHub merge commit of that pull request. Versions v0.21.3 to v0.32.0 are open pull requests, stacked in order (#74 to #92); each gets its tag when it is merged into `main` after the defense (DEC-018).

Version numbers follow **semantic versioning**, `MAJOR.MINOR.PATCH`: MINOR goes up for a new feature (v0.20.0 → v0.21.0), PATCH goes up for a fix (v0.21.1 → v0.21.2), and MAJOR goes from 0 to 1 when the system is declared complete. Documentation-only changes get no tag.

Tag	Date	Commit	Tag message (what was added)
—	2026-08-12	`2cf8b49`	*(no tag)* Initial commit: add README
v0.1.0	2026-08-12	`cb95116`	Walking skeleton: Flask running, GitHub connected
v0.2.0	2026-08-12	`d3ca52b`	Walking skeleton plus working database layer (Product model)
v0.3.0	2026-08-12	`e6ceedb`	Product list view with Bootstrap-styled templates
v0.4.0	2026-08-12	`e2f3947`	Add product creation via form
v0.5.0	2026-08-12	`51f0ecc`	Full product CRUD: create, read, update, delete
v0.6.0	2026-08-12	`2d1a6ae`	User authentication (login/logout, no role restrictions yet)
v0.7.0	2026-08-13	`6f552f9`	Role-based access: Admin can manage products, Cashier is view-only
—	2026-10-03	`e76f4a1`	*(no tag)* Add documentation draft, test results and use case diagram
v0.7.1	2026-10-04	`6043cdb`	Product validation fixes: duplicate SKU, negative values, peso sign *(first feature branch + pull request #1)*
—	2026-10-04	`1d1dd0e`	*(no tag)* Update documentation for v0.7.1: test results and fixed defects
v0.8.0	2026-10-08	`1343236`	Sale processing: basket checkout, cash and change, stock deduction, printable receipt *(pull request #2)*
v0.8.1	2026-10-08	`538ce09`	Receipt fix: show 'not recorded' for sales made before cash tracking *(pull request #3)*
—	2026-10-08	`bf7dc7d`	*(no tag)* Add v0.8.0 hand-test screenshots to documentation
v0.8.2	2026-10-09	`caf360f`	Safe stock: atomic deduction prevents overselling when sales overlap *(pull request #4)*
v0.8.3	2026-10-09	`1a15679`	Number limits: reject nan, infinity and unrealistic prices, quantities and cash *(pull request #5)*
v0.8.4	2026-10-09	`2f94325`	Not found instead of crash for ids too big for the database *(pull request #6)*
v0.8.5	2026-10-09	`9dbb8b0`	Deleted products are taken out of the basket instead of blocking the sale *(pull request #7)*
v0.8.6	2026-10-09	`decb8f0`	Text limits: name 100, SKU and category 50 characters *(pull request #8)*
v0.9.0	2026-10-09	`d27efd5`	Low-stock warning: badges and reorder alert for 5 units or fewer *(pull request #9)*
v0.9.1	2026-10-09	`403bd7b`	Low-stock box: 5 most urgent shown, count of the rest *(pull request #10)*
v0.9.2	2026-10-09	`a7b4c9c`	Empty category shows a dash instead of 'None' *(pull request #11)*
v0.10.0	2026-10-09	`aa07bb2`	Sales report: revenue, sales count, items sold and top products by date range *(pull request #12)*
v0.10.1	2026-10-09	`96b6682`	Top products: a renamed product counts once *(pull request #13)*
v0.10.2	2026-10-09	`d095cd9`	Products with sales history can't be deleted *(pull request #14)*

Tag	Date	Commit	Tag message (what was added)
v0.11.0	2026-10-09	`9c0bbac`	Demo data: seed script with shop products and a week of sales *(pull request #15)*
v0.11.1	2026-10-09	`e02353c`	Demo data: receipt numbers follow the clock *(pull request #16)*
v0.11.2	2026-10-09	`16c6c7a`	Money shown with thousands separators (₱6,904.50) *(pull request #17)*
—	2026-10-09	—	Documentation update for v0.8.2–v0.11.2 *(pull request #18, no tag)*
v0.12.0	2026-10-09	`5f3648e`	Profile pictures: round avatar (initials until a picture is uploaded), avatar menu, automatic database upgrade *(pull request #19)*
v0.13.0	2026-10-09	`ea368ff`	Modern theme: left sidebar with icons, light "Clean counter" / dark "Night shift" switch, dashboard home, card layout *(pull request #20)*
—	2026-10-09	—	Documentation update for v0.12.0–v0.13.0 *(pull request #21, no tag)*
v0.13.1	2026-10-09	`b9e44a8`	Secret key out of the code (environment variable or `instance/secret_key.txt`) *(pull request #22)*
v0.13.2	2026-10-09	`9da53af`	CSRF tokens on every form; logout is a button; SameSite cookie *(pull request #23)*
v0.13.3	2026-10-09	`9134e6a`	Login attempt limit: 5 wrong passwords → wait 5 minutes *(pull request #24)*
v0.13.4	2026-10-09	`aed2017`	Debug mode off by default; browser security headers *(pull request #25)*
v0.13.5	2026-10-09	`7893486`	Change password on the profile page *(pull request #26)*
v0.14.0	2026-10-09	`59c6d54`	Product pictures with icon tiles, tap-to-add menu on New Sale, café demo menu (`--fresh`) *(pull request #27)*
v0.14.1	2026-10-09	`2795482`	Very long names wrap on the menu and in tables *(pull request #28)*
v0.14.2	2026-10-09	`afea681`	Menu in category order; chosen category kept after each tap *(pull request #29)*
v0.14.3	2026-10-09	`e598e88`	Phone basket bar; table columns stay readable *(pull request #30)*
—	2026-10-09	—	Documentation update for v0.13.1–v0.14.3 *(pull request #31, no tag)*
v0.14.4	2026-10-09	`8a34326`	Basket Remove button always visible; menu and basket side by side from 1200 px *(pull request #32)*
v0.15.0	2026-10-09	`aa44988`	Cash or GCash payment with a 13-digit reference; receipt and report show the method *(pull request #33)*
v0.16.0	2026-10-09	`40555c0`	Sizes and add-ons: admin page, drink window on New Sale, options on receipts *(pull request #34)*
v0.16.1	2026-10-09	`2f97c8d`	Sizes/add-ons edge cases: unticked product, very long add-on list *(pull request #35)*
v0.17.0	2026-10-09	`4492d92`	Ingredients, recipes, can-make counts, ingredient history; sales use ingredients up *(pull request #36)*
v0.17.1	2026-10-09	`63f9d53`	A deleted product's recipe is deleted too (no inherited recipes) *(pull request #37)*
v0.17.2	2026-10-09	`da4a932`	The one-of choice is called a *serving*, so the café's Hot / Iced reads naturally (sizes still possible) *(pull request #38)*
v0.17.3	2026-10-09	`a7cd2e5`	Demo sales include products whose stock comes from ingredients (no more ₱0 demo receipts) *(pull request #39)*
—	2026-10-09	—	Documentation update for v0.14.4–v0.17.3 *(pull request #40, no tag)*
v0.17.4	2026-10-09	`c985356`	Number boxes show `2000`, not `2,000` (which they can't read) *(pull request #41)*
v0.18.0	2026-10-09	`9689751`	Ingredients page: groups, Full level and stock bars, search and filters, Needs attention, Update window, ingredient page, History tab *(pull request #42)*
v0.18.1	2026-10-09	`11e33e7`	Ingredients page keeps the search, filter and row after an update *(pull request #43)*
—	2026-10-09	—	Documentation update for v0.17.4–v0.18.1 *(pull request #44, no tag)*
v0.18.2	2026-10-09	`4f1ca90`	Products table keeps SKUs and stock counts on one line *(pull request #45)*

Tag	Date	Commit	Tag message (what was added)
—	2026-10-09	—	Documentation: screenshots and how each version was tested *(pull request #46, no tag)*
—	2026-10-09	—	Documentation: requirements chapter (Lecture 4) and context model *(pull request #47, no tag)*
—	2026-10-09	—	Documentation: Lecture 5 system models, 35 UML diagrams *(pull request #48, no tag)*
v0.19.0	2026-10-09	`cf2894e`	AGENTS.md rules, automated tests in `tests/` and GitHub checks on every pull request *(pull request #49)*
v0.19.1	2026-10-09	`b6b9090`	Light/dark switch is an icon only: sun in the light theme, moon in the dark theme, beside the user's name *(pull request #50)*
v0.19.2	2026-10-09	`57527bb`	Avatar menu grows to fit its words; each line on one line (DEF-46) *(pull request #51)*
—	2026-10-11	—	Documentation: security audit of v0.19.2 (`docs/security_audit_2026-10-11.md`, DEF-47–DEF-52) *(pull request #52, no tag)*
v0.19.3	2026-10-11	`1318ec3`	Security audit patch: quantity limit on New Sale, login limit remembers at most 50 characters and forgets old entries, Werkzeug 3.1.9 (DEF-47, DEF-49, DEF-50) *(pull request #53)*
v0.19.4	2026-10-11	`4821094`	Pages work without internet: Bootstrap 5.3.3, Bootstrap Icons 1.11.3 and Inter stored in `static/vendor/` (DEF-53) *(pull request #54)*
v0.20.0	2026-10-12	`6f42f1e`	New Sale redesign "Rush Hour + Stock Aware": + Hot / + Iced on each tile, stock bar and the missing ingredient, running-low banner, − / + and add-ons changed on the basket line (design 6a), "Uses N ingredients" panel, quick cash *(pull request #55)*. A feature chosen by the student before the open patches DEF-48/51/52: a recorded exception to the stabilization rule
v0.20.1	2026-10-12	`9e183f2`	New Sale uses the whole screen: no 1180 px limit on that page, as many tiles per row as fit, basket 380–460 px, bar and "N left" on one line, "Other" cash button replaced by an "Other amount" box (DEF-57) *(pull request #56)*
v0.20.2	2026-10-12	`6b2b4f5`	SKUs are compared without regard to capitals, so `hc-001` can't be added next to `HC-001`; an unchanged SKU stays editable (DEF-52) *(pull request #57)*
v0.20.3	2026-10-12	`6a726b9`	Logins can be ended: new `login_session` table, the cookie carries only a random token; log-out deletes it, a password change ends every login of that user, 8 idle hours end a login. Everyone logs in once more after the upgrade (DEF-51) *(pull request #58)*
v0.20.4	2026-10-12	`f2eda4d`	One sale per basket: each basket gets a one-time sale ticket, the receipt stores it and the database allows it once; a second click, a retry or an old cookie opens the first receipt instead; Complete Sale greys out after the first click (DEF-48) *(pull request #59)*
—	2026-10-12	—	Documentation: README.md written (what the app does, quick start, demo accounts, tests, folders, security, contributing); Python version in Section 1 corrected *(pull request #61, no tag)*
v0.20.5	2026-10-13	`efb87ec`	Ingredients page: the warn-level tick on each stock bar is a full line again; the New Sale tile bars got their own class names (`tile-bar`) instead of reusing `stock-bar`, and a test stops New Sale style names from being shared again (DEF-58) *(pull request #62)*
—	2026-10-13	—	Documentation: new guide `docs/run_on_another_computer.md` (set the app up on another computer such as a school lab: with or without Git, without activating the venv, with an optional USB copy of the shop's own menu, no-internet install, troubleshooting, cleaning up a shared computer); README wording fixes and a link to the guide; security audit A5 closed (demo passwords changed on the demo laptop) *(pull request #63, no tag)*

Tag	Date	Commit	Tag message (what was added)
—	2026-10-14	—	Documentation check against v0.20.5: actors, requirements (UR-1, 2, 3, 5, 8; NFR-12 offline; parts S5, S6, U4), structured specifications (sale ticket, login record), two new computation tables (quick cash, tile stock bar), scenario 1 with the one-tap flow, metrics measured again on v0.20.5, known limitations (BIR registration, senior/PWD discount, X/Z reading, dark-mode bar colour, tab icon), checklist and AGENTS.md folder list *(pull request #64, no tag)*
—	2026-10-14	—	Report items 26–39 written from the v0.20.5 code: §8 architecture (MVC, layers, system context, subsystems, object classes, usage scenario) and §9 dependability and security (reliability, safety and security terminology, vulnerability avoidance, risk classification, fault tree, safety and reliability requirements, threat and control analysis); 6 new diagrams *(pull request #65, no tag)*
—	2026-10-14	—	Generic names: the café's own add-on and drink names that had slipped into the code comments, tests, test menu, report and one diagram were replaced with generic ones (Oat Milk, Sweetener, Honey Latte, Mocha), and erased from 8 screenshots, because the owners' consent covers only their photos and prices *(pull request #66, no tag)*
v0.20.6	2026-10-23	—	Login goes back to the page the user first asked for, but only a page on this site (`page_after_login()`); a link to another website, `//host` or `/` + backslash is ignored. Last open defect closed (DEF-06). Short comments added above the 11 older routes (v0.3–v0.20) that had none; new `docs/defense_preparation.md` (demo script and question list for the one-to-one session) *(finishing touches, branch `docs/finishing-touches`)*
v0.21.0	2026-10-23	—	Cozy layout (F20): new warm palette (cream and caramel in the light theme, espresso and amber in the dark theme), bigger text, softer corners and wider gaps on every page, roomier New Sale tiles and basket, and a **Fold menu** button that shrinks the sidebar to an icon strip on wide screens (choice saved in the browser, like the theme). Only `theme.css`, `base.html` and a new test; no database or server change *(branch `feature/cozy-layout`)*
v0.21.1	2026-10-23	—	The fold button moved to the top row, as an icon beside the system name (requested after seeing v0.21.0); folded, the logo and the icon sit one above the other. No "Fold menu" words any more; the tooltip and screen-reader label stay *(branch `fix/fold-button-beside-brand`)*
v0.21.2	2026-10-23	—	Layout audit of every page (14 pages, light and dark theme, phone 375 px and desktop, open and folded sidebar). Two defects fixed: the theme icon of the folded sidebar sat to the right of the avatar (DEF-59), and the "New ingredient" panel ran off the right edge of a phone (DEF-60). No page had sideways page scrolling *(branch `fix/rail-alignment-audit`)*

—	**2026-10-23**	**—**	**Final report: `final_report.md`, `final_report.docx` and `final_report.pdf`, written from the documentation draft, with a tour of v0.21.2, cleaned-up tables and the last two open hand tests closed (TC-4.4, TC-19.14); plus `docs/system_overview.canvas`, a one-page map of the system *(pull request #71, no tag)***
v0.21.3	2026-10-24	—	A crash shows a friendly "Something went wrong" page in the normal layout instead of the plain *Internal Server Error*; the half-done database work is undone and the details go only to the server log (DEF-61, audit A11, DEC-019) *(pull request #74, branch `fix/friendly-error-page`)*
v0.21.4	2026-10-24	—	An unknown username now takes as long to refuse as a wrong password: a dummy password hash is checked, so the time taken no longer tells anyone which usernames exist (DEF-62, audit A8, DEC-020) *(pull request #75, branch `fix/login-timing`)*
v0.21.5	2026-10-24	—	The built-in demo menu sells drinks **Hot or Iced**, like the café: one product per drink (8 drinks instead of 4 hot + 4 "Iced ..."), servings Hot and Iced at the same price, and the cup and ice in each serving's own recipe (new demo ingredients Hot cups, Iced cups, Ice). Was 12oz / 16oz (DEF-63, audit R10, DEC-021) *(pull request #76, branch `fix/demo-hot-iced`)*

—	**2026-10-23**	—	**Final report: `final_report.md`, `final_report.docx` and `final_report.pdf`, written from the documentation draft, with a tour of v0.21.2, cleaned-up tables and the last two open hand tests closed (TC-4.4, TC-19.14); plus `docs/system_overview.canvas`, a one-page map of the system *(pull request #71, no tag)***
v0.21.6	2026-10-24	—	`requirements.txt` saved as plain UTF-8 instead of UTF-16 (the result of PowerShell's `pip freeze >`), and a new contract rule keeps it that way (DEF-64, audit R11) *(pull request #77, branch `fix/requirements-utf8`)*
v0.21.7	2026-10-24	—	A browser tab icon (a white cup on the caramel brand colour, `static/favicon.svg`), named on every page, so the browser's request no longer ends in "404 Not Found" (DEF-65) *(pull request #78, branch `fix/favicon`)*
v0.21.8	2026-10-24	—	Ingredients page, dark theme: a normal stock bar is green (the same green as the New Sale tiles) instead of the amber accent, which was exactly the colour of a Low bar; light theme, Low and Out unchanged (DEF-66) *(pull request #79, branch `fix/dark-low-bar`)*
v0.22.0	2026-10-24	—	Shop details on receipts (F21): the admin sets the shop's name, address, TIN and a footer on a new **Shop details** page, and every receipt now says **"This is not an official receipt."**, which can't be turned off (SR6 met; closes the report's only *Intolerable* hazard as far as the software can). New `shop_profile` table, created at start-up on older databases (DEC-022) *(pull request #80, branch `feature/receipt-shop-details`)*
v0.23.0	2026-10-24	—	Database migrations with Alembic (Flask-Migrate): every database change is now a numbered script in `migrations/versions/`. Migration `0001` brings any older or empty database to the v0.22.0 shape (it took over the `NEW_COLUMNS` list) and can't be undone; start-up backs the database up (`instance/inventory_backup_<date>_before_<number>.db`) before running any missing migration. New packages Flask-Migrate 4.1.0, alembic 1.20.0, Mako 1.4.3 (approved in DEC-011). Tested on a copy of the owner's real database: only `alembic_version` added, every row count unchanged, integrity ok (DEC-023) *(pull request #81, branch `feature/alembic-migrations`)*
v0.23.1	2026-10-24	—	Times stored in **UTC** and shown, and counted into days, on the shop's clock (UTC+8): receipts, the report, Home's "today", ingredient history, logins and the login limit. Migration `0002` turns every stored time from this computer's local time into UTC (and back when undone). Needed before any cloud server, which runs on UTC (DEF-67, audit R1, DEC-024). Tested on a copy of the owner's database: times 8 hours earlier in storage, the same on screen *(pull request #82, branch `fix/utc-times`)*
v0.23.2	2026-10-24	—	Money stored as **whole centavos** (INTEGER: ₱75.50 = 7550) through a new `Money` column type, and handled in Python as exact `Decimal` pesos, so totals can't drift and a receipt always adds up. A price typed with more than two decimals is rounded once, half up, when saved. Migration `0003` converts every price and total (and back when undone). Tested on a copy of the owner's database: all 58 receipts, 120 lines and 12 prices identical to the centavo (DEF-68, audit R5, DEC-025) *(pull request #83, branch `fix/money-centavos`)*
v0.24.0	2026-10-24	—	Setting up and running a shop's own copy (F22): `setup_shop.py` asks once for the network (this computer only, or also phones on the shop's network), the shop's details and the owner's own admin account, writes `instance/config.toml`, brings the database up to date and never creates the demo accounts; `serve.py` / `start_shop.bat` run it with **waitress** instead of Flask's development server (audit R3) and show the address for phones. Packages split into `requirements/local.txt` (rule D1); waitress 3.0.2. Tried end to end: a fresh install, started by `start_shop.bat`, used from the network address in Edge (DEC-026) *(pull request #84, branch `feature/serve-and-setup`)*
v0.25.0	2026-10-24	—	`app.py` (1,722 lines) split into six **blueprints** (`accounts`, `catalog`, `ingredients`, `sales`, `reports`, `shop`), shared rules in `common.py`, and a **`store/`** data layer through which every database change goes (selling, restock and count, products, options, recipes, logins, passwords, pictures, shop details). `app.py` (242 lines) only starts the app. No visible change: every page, message and rule is the same; 26 of 27 test files unchanged (one line in `test_hardening.py`). The contract check now fails if a page writes to the database itself (rule I1, now checked) (DEC-027) *(pull request #85, branch `feature/blueprints-store`)*

—	**2026-10-23**	**—**	**Final report: `final_report.md`, `final_report.docx` and `final_report.pdf`, written from the documentation draft, with a tour of v0.21.2, cleaned-up tables and the last two open hand tests closed (TC-4.4, TC-19.14); plus `docs/system_overview.canvas`, a one-page map of the system *(pull request #71, no tag)***
v0.26.0	2026-10-24	—	Email sending and a login limit that holds (F23). `mail.py` sends plain-text email with Python's own `smtplib`, always over an encrypted connection, using the `[mail]` settings that `setup_shop.py` now asks for (optional; running it again keeps them). The admin page **Email** shows them (never the password) and sends a test message. Wrong passwords are counted in a new `login_attempt` table (migration `0004`), recorded before they are counted and stored as keyed hashes, so the limit survives a restart and holds for every server process (DEF-70, audit R2); **Change password** now has the same 5-tries limit (DEF-69, audit A7) (DEC-028) *(pull request #86, branch `feature/email-and-attempts`)*
v0.27.0	2026-10-24	—	The owner, users and invitations (F24). Roles **owner**, admin and cashier; the database allows one owner, made by `setup_shop.py` with the owner's email (confirmed with a code on Profile; the purchase email replaces it in v0.38.0, owner's decision). **Users** page: invite by email (48-hour one-time link, role fixed by the inviter), change roles (owner), let admins invite cashiers, switch accounts off (never deleted; every login ends) and **Require a new password** (audit A5). **Emailed 6-digit codes** (keyed hashes, 10 minutes, one use, one purpose, 5 tries, one a minute and 5 an hour) confirm signups and emails and power **Forgot your password?** (the same answer for every address). **Audit log**: who changed which product, price, stock, option, ingredient, recipe, shop detail or account, old → new, written in the same commit as the change (audit R9). Migration `0005`; `add_users.py` removed (rule S12); rule S1 lists the three new public pages (owner's decision) (DEC-029) *(pull request #87, branch `feature/users-and-invitations`)*
v0.28.0	2026-10-25	—	A code at every login (F25, rule S13): after the password, a 6-digit code by email, unless the browser is **trusted** (30 days; 7 for admins and the owner; ended by Remove, **Log out everywhere**, a password change, switching off or a role change). Each user may use an **authenticator app** instead (works with no internet; QR code drawn by `segno`, key encrypted with `cryptography`, a step never accepted twice) and print 10 one-time **backup codes**. Accounts with no email get one from the owner on Users (owner's decision). **Demo mode** (`DEMO_MODE=1`) prints codes and emails in the server window, lets the demo passwords work, and is refused with real mail settings (rule P3). Migration `0006`; rule S1 lists `/verify` (owner's decision); the tests of other features log in with `TEST_PLAIN_LOGIN`, which the contract check allows only in `tests/` (DEC-030) *(pull request #88, branch `feature/login-codes`)*
v0.29.0	2026-10-25	—	Sign in with Google (F26): the OpenID Connect code flow with state, nonce and PKCE (Authlib); Google's answer is checked here: signature against Google's keys, issuer, audience, expiry, nonce and a verified email. Accounts are linked by Google's permanent id, never the email; linking from Profile needs the password and a fresh code, and the Google email must be the account's email (owner's decision). People can join from an invitation with Google (the invited address; the code still follows). The login code still follows Google unless the browser is trusted. Works on `localhost` and HTTPS only (Google's rule), so the login page says where. `setup_shop.py` asks for the client ID and secret (optional). Migration `0007`; packages Authlib, joserfc, requests and its four (D1); rule S1 lists the two Google pages (owner's decision) (DEC-031) *(pull request #89, branch `feature/google-signin`)*
v0.30.0	2026-10-25	—	Undo and a safe count (F27, plan §4.2 and §4.8). **Undo** on New Sale takes the order back one change at a time (the last 5; the copies stay small for the session cookie; emptied when the sale is completed, so a sold order never comes back; the stock is checked again). A **stock count** is saved only if the amount on hand is still the one the admin saw on the page, in one conditional step (I1); otherwise "count again". **Undo count** for 5 minutes adds the opposite history line (`count-undo`), refused if anything else changed the ingredient since. Found in the browser check: the Undo button wrapped on phones; the hint now makes room (DEC-032) *(pull request #90, branch `feature/undo-and-safe-count`)*

—	**2026-10-23**	**—**	**Final report: `final_report.md`, `final_report.docx` and `final_report.pdf`, written from the documentation draft, with a tour of v0.21.2, cleaned-up tables and the last two open hand tests closed (TC-4.4, TC-19.14); plus `docs/system_overview.canvas`, a one-page map of the system *(pull request #71, no tag)***
v0.31.0	2026-10-25	—	Voids and refunds (F28, plan §4.1, rules I8 and S15). A receipt is never changed: a void or refund is a new record that names who asked and who approved, with a printable **slip**. **Void** = the whole receipt, today only: anyone asks (with a reason), an admin approves with their **own password** (at the top of the **Sales Report**, whose menu link shows how many are waiting, or on the receipt; no separate page, owner's decision), or, with no admin at the till, by a **code emailed** to the admin the cashier chooses (that void only, 10 minutes, once, 5 tries, keyed hash). Everything goes back on the shelf. **Refund** = chosen items, any day, admins only, each **back on the shelf** or **waste** (a `refund` + and a `waste` − line). Conditional updates stop a double approval and refunds above what was paid (I1, I2). Each receipt line now keeps its own ingredients, so a refund returns exactly what it used. **Today's receipts** for everyone; the Sales Report and Home show **net** sales (gross − voids − refunds) and cash after money given back. Migration `0008` (DEC-033) *(pull request #91, branch `feature/voids-and-refunds`)*
v0.32.0	2026-10-25	—	Help on every page (F29, DEC-034). A **?** button, only an icon (round, bottom right; in the top bar on phones; beside the theme switch on the login pages), opens a side panel with **that page's help**: what it is for, a **picture** of the page and the main steps. The **Help** page (`/help`, from the avatar menu) shows all 18 topics with chips to jump between them. A cashier sees only their own pages and steps. The words are in `help_topics.py`, the pictures (screenshots of the demo shop, WebP, 0.7 MB) in `static/help/`; a test fails if a page has no help. Later versions in the plan moved down one number (shifts are now v0.33.0) *(pull request #92, branch `feature/help-pages`)*
—	2026-10-25	—	Documentation: Chapter 12 of this report, an addendum on v0.21.3–v0.32.0 for the defense *(pull request #93, no tag)*; version history rows for #72 and #73 in the draft *(pull request #94, no tag)*
—	2026-10-27	—	Documentation: this report rewritten for v0.32.0: every chapter, the tour (new screenshots), stories F21–F29, test cases TC-26 to TC-42, requirements UR-9 to UR-12, use cases UC-16 to UC-25, 31 diagrams redrawn or added, defects DEF-61 to DEF-71; Chapter 12 folded into the chapters *(branch `docs/final-report-rewrite`, no tag)*

From v0.8.2 the Commit column shows the commit with the change; the tag sits on the GitHub merge commit of that pull request. Pull request numbers up to #70 assume the versions were published in order in one session.

Appendix B. Automated test suite

Since v0.19.0 the tests live in the project's `tests/` folder, so anyone can run them with one command. Each file runs in a temporary copy of the project, so the real database is never touched, and GitHub runs the same command on every pull request (`.github/workflows/checks.yml`). Browser tests are not in `tests/` because they need extra software; they are run by hand before each release (automated with Playwright from v1.0.0, rule T4). Email and Google are tested with fakes (a fake mailbox, a test-made signing key), never a real account.

```
python tests/run_tests.py           # the rule check, then every test file
python tests/run_tests.py sales     # only test files whose name contains "sales"
python tests/check_contract.py      # only the project rules marked [checked] in AGENTS.md
```

On v0.32.0 the run ends with **"All 38 passed"**: 1,203 checks + 147 rule checks. (On v0.21.2 it was "All 22 passed", 470 checks + 58 rule checks.)

Test file	Covers	Checks
`check_contract.py`	The AGENTS.md rules marked [checked] (147 rule checks on v0.32.0: every page listed, CSRF tokens, the store-only rule and more): every page protected, CSRF tokens, no debug or secret in code, allowed packages and CDN files, `requirements.txt` in UTF-8, no private files in Git	59 rule checks
`test_products.py`	F1–F6: product forms, login, roles, DEF-01–DEF-04	40
`test_sales.py`	F7: basket, cash and change, receipts, upgrade from v0.7.1, DEF-09, DEF-10, DEF-21	40
`test_race.py`	DEF-13: two tills at once; 20 cashiers on a real server (products and ingredients); DEF-48 (v0.20.4): the same basket sent 2 and 5 times at once, an old cookie	11
`test_limits.py`	DEF-14, DEF-15, DEF-17, DEF-47: nan, infinity, huge numbers and ids, long text, huge drink quantities	23
`test_low_stock.py`	F8, DEF-18	8
`test_report.py`	F9: dates, midnight, totals, top products	11
`test_avatar.py`	F10 / TC-11.x	18
`test_security.py`	F12 / TC-13.x, TC-20.3–20.5: secret key, CSRF, login limit (incl. DEF-50; read from the database since v0.26.0), headers, passwords, Werkzeug version (DEF-49)	42
`test_product_pictures.py`	F13 / TC-14.x	25
`test_payment.py`	F14 / TC-15.x	19
`test_options.py`	F15 / TC-16.x	34
`test_ingredients.py`	F16 / TC-17.x	38
`test_ingredients_page.py`	F17 / TC-18.x	34
`test_number_boxes.py`	DEF-43, 5 pages	5
`test_seed.py`	Demo data: both menus, run twice, photos, `--fresh` backup, DEF-42; Hot / Iced demo servings (DEF-63, v0.21.5)	9

Test file	Covers	Checks
test_theme.py	F11 / TC-12.10–12.13 (v0.19.1–v0.19.2): icon-only switch on 3 pages, label, position; avatar menu width (DEF-46)	16
test_offline.py	DEF-53 / TC-21.1 (v0.19.4): no outside addresses, every asset and font file exists	11
test_new_sale.py	F19 / TC-22.1–22.8, TC-22.11, TC-22.13 (v0.20.0–v0.20.5): tiles, banner, − / +, change on the basket line, ingredient panel, quick cash, login and token on the new pages, full-width layout, no shared style names (DEF-58)	55
test_sessions.py	DEF-51 / TC-23.4–23.10 (v0.20.3): copied cookie after log-out and password change, two tills on one account, cookie made with the real key, 8 idle hours, table clean-up	16
test_layout.py	F20 / TC-25.1–25.9 (v0.21.0–v0.21.2): fold icon beside the name, saved choice before drawing, tooltips, wide screens only, warm palette, centred folded icons, phone panel	12
test_login_next.py	DEF-06 / TC-5.8, TC-24.1–24.4 (v0.20.6): back to the requested page; other-website links ignored	6
test_hardening.py	Phase 1 patches / TC-26.x (v0.21.3 onward): friendly 500 page (DEF-61), login timing (DEF-62), tab icon (DEF-65), dark-theme bar colour (DEF-66)	22
test_shop_details.py	F21 / TC-27.x (v0.22.0): notice on every receipt (SR6), who may change the details, saving, wrong values, HTML as text, older database	22
test_migrations.py	Migrations / TC-28.x (v0.23.0): older database upgraded with data kept, backup first, no second backup, models match migrations, rebuild after drop_all, 0001 can't be undone, log stays on, one line of migrations	14
test_times.py	Times in UTC / TC-29.x (v0.23.1): migration 0002 both ways, conversions with fixed dates, UTC stamps, report and Home days, shop time on pages, no local-time stamps left	18
test_money.py	Money in centavos / TC-30.x (v0.23.2): migration 0003 both ways, INTEGER storage, Decimal in Python, one rounding on save, receipts add up, exact sums, limits unchanged	19
test_setup.py	F22 / TC-31.x (v0.24.0): config file, setup run with good and wrong answers, run twice, broken config, DATABASE_URL, waitress serving, server settings, start_shop.bat, answers file with a byte-order mark; the owner and its email (v0.27.0)	31
test_attempts.py	F23 / TC-33.x (v0.26.0): migration 0004, the limit unchanged for users, hashed keys, a restart (DEF-70), Change password (DEF-69), tries at the same moment	21
test_email.py	F23 / TC-34.x (v0.26.0): fake mail server, STARTTLS and SSL, every failure in plain words, address and settings checks, the Email page and its test message, setup's email answers, MAIL_PASSWORD	66
test_users.py	F24 / TC-35.x (v0.27.0): migration 0005, one owner, who may do what (403s), roles, switching off, Require a new password, the audit log and its page, setup making the owner on an older database	47
test_invitations.py	F24 / TC-36.x (v0.27.0): invitation emails and hashed tokens, the link's address, signup in two steps, the role can't be forged, Send again / Cancel / 48 hours, mail down, every code rule, one account per invitation	44
test_password_reset.py	F24 / TC-37.x (v0.27.0): Forgot password's same answer, code and new password, logins ended, limits, the email on Profile, the owner's fixed email	24

Test file	Covers	Checks
test_login_codes.py	F25 / TC-38.x (v0.28.0): RFC 6238 codes, migration 0006, the code step and its 5 minutes, limits, trusted devices and every way they end, backup codes, offline login, the authenticator app (no replay), admin reset, accounts with no email, demo mode, the test-only switch	69
test_google.py	F26 / TC-39.x (v0.29.0): migration 0007, where it works, the trip (state, nonce, PKCE), every check on Google's answer with a test-made key, login by Google's id and the code after it, linking and unlinking, joining with Google, setup and the environment	55
test_undo.py	F27 / TC-40.x (v0.30.0): the safe count (seen amount, decimals, a sale in between), undo count (5 minutes, nothing since, once, races), basket undo step by step, 5 steps and the cookie size, after a sale, stock checked again, a new login	36
test_voids.py	F28 / TC-41.x (v0.31.0): migration 0008, each line's ingredients, asking for a void, 403s, approving by password and by emailed code, races, only today, GCash references, refunds to the shelf or as waste, older sales, net sales and the drawer, slips, history, audit log, DEF-71	81
test_help.py	F29 / TC-42.x (v0.32.0): a topic for every page, the pictures, the words, the icon-only button and panel, the login pages, steps per role, the Help page per role, 403s, escaping, the look	161
38 files		**1,203 checks + 147 rule checks**

Appendix C. Where each Section B item is answered

#	Item	Lecture	Section
1	Process model and why	L2	3
2	Full story of each feature	L3 p. 21	4
3	Detailed test case of each feature	L3 p. 31	5
4	Requirements definition: user and system requirements	L4 p. 7	6.1
5	Nonfunctional requirements	L4 p. 18	6.2
6	Nonfunctional requirements metrics	L4 p. 21	6.3
7	Requirements of each part	L4 p. 39	6.4
8	Structured requirements	L4 p. 42–43	6.5
9	Tabular computation	L4 p. 45	6.6
10	Detailed scenarios	L4 p. 62–63	6.7
11	Use case diagram	L4 p. 65	7.3
12	Context model	L5 p. 10	7.1
13	Process model	L5 p. 12	7.2
14	Every use case's UML diagram	L5 p. 15	7.3
15	Tabular use case descriptions	L5 p. 16	7.4
16	Each actor's use cases	L5 p. 17	7.5
17	Sequence diagrams	L5 p. 19–20	7.6
18	Class associations	L5 p. 23–24	7.7
19	Class model	L5 p. 25	7.7
20	Generalization hierarchy	L5 p. 30–31	7.8
21	Aggregation	L5 p. 33	7.9
22	Activity model	L5 p. 36	7.10
23	Application process	L5 p. 37	7.11
24	State diagrams	L5 p. 40	7.12
25	Structured forms of states	L5 p. 41–42	7.13
26	Software architecture	L6 p. 42, 49	8.1
27	System context	L7 p. 9	8.2
28	High-level architecture	L7 p. 13	8.3
29	Object classes	L7 p. 19	8.4
30	Detailed usage scenario	L8 p. 53	8.5
31	Reliability terminology	L11 p. 25	9.1
32	Safety terminology	L11 p. 35	9.2
33	Security terminology	L11 p. 42	9.3
34	Vulnerability avoidance techniques	L11 p. 45	9.4
35	Risk classification	L12 p. 15	9.5
36	Software fault tree	L12 p. 18	9.6

#	Item	Lecture	Section
37	Safety requirements	L12 p. 23	9.7
38	Functional reliability requirements	L12 p. 39	9.8
39	Threat and control analysis	L12 p. 46	9.9

Appendix D. Running the system

You need **Python 3.11 or newer**. On Windows, from the project folder (for v0.32.0, use the branch of pull request #92, `feature/help-pages`, until it is merged into `main`):

```
python -m venv venv
.\venv\Scripts\python -m pip install -r requirements.txt
.\venv\Scripts\python seed_demo_data.py
$env:DEMO_MODE = "1"
.\venv\Scripts\python app.py
```

Then open **http://127.0.0.1:5000** in a browser. The demo accounts are `admin` / `admin123` (the demo shop's owner) and `cashier` / `cashier123`. They work only in **demo mode**: after the password, type the 6-digit login code that is printed in the window where the app runs (nothing is emailed in demo mode). Tick **Trust this device** to skip the code next time.

For a real shop (v0.24.0): run `.\venv\Scripts\python setup_shop.py` once. It asks whether phones may use the system, the shop's details, the owner's own username, password and email, and (optionally) the email settings and Google sign-in. Then start the system every day with **`start_shop.bat`**, which runs it with the waitress server and shows the address to open on phones. The demo accounts are never created in a shop's own copy. `python seed_demo_data.py --fresh` starts again with only the demo data, after renaming the old database so nothing is lost. Step-by-step instructions for another computer, including one without internet, are in `docs/run_on_another_computer.md`.